



ViewPoint 2.0 Web SDK Reference Manual

Version 1

June 2007

ViewPoint 2.0 Web SDK Reference Manual

Document Version 1

June 2007

© 2005 Olive Software Incorporation. All rights reserved. No part of this publication may be reproduced, transmitted, stored in a retrieval system, nor translated into any human or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of the copyright owner, Olive Software, 2953 Bunker Hill Lane, Suite 203 Santa Clara, 95054, U.S.A. The copyrighted software that accompanies this manual is licensed to the User for use only in strict accordance with the License Agreement, which the Licensee should read carefully before commencing use of the software.

Olive Software may change this manual, correct inaccurate current information, or change or improve the application at any time without prior notice. Some of these changes may be incorporated into new versions of this manual.

Olive Software, the Olive Software logo, ViewPoint, Content Administrator, XML Distiller, ViewPoint Warehouse, PipeX, and XMD, ActivePaper, the ActivePaper logo, ActivePaper Publisher, ActivePaper Daily, ActivePaper Archive, ActiveMagazine, AdLauncher, ActiveTearsheet and all its modules are trademarks and/or registered trademarks of Olive Software Incorporated. ABBYY and FineReader are trademarks and/or registered trademarks of ABBYY Software House. Adobe Acrobat, Acrobat, Portable Document Format, and PDF are trademarks and/or registered trademarks of Adobe Systems Inc. Neevia and Document Converter Pro are trademarks and/or registered trademarks of Neevia Technology, Inc.

All other trademarks are the property of their respective owners.

Olive Software Inc.
2953 Bunker Hill Lane
Suite 203
Santa Clara
95054

Tel: 408.200.1780
Fax: 408-200-1790

E-mail: info@olivesoftware.com

Table of Contents

ViewPoint 2.0 Web SDK Reference Manual	1
Introduction	23
The Need for a Web SDK	23
Document Structure	23
ViewPoint Web SDK Architecture	24
<i>BACKGROUND</i>	24
ViewPoint Architecture	25
<i>CONCEPTS</i>	25
ViewPoint Web SDK Client Side Architecture	28
ViewPoint Web SDK Server Side architecture	31
Guidelines to Building Web Applications with the Web SDK	33
Building a Simple Web Application.....	37
<i>ACCESSING THE SDK AND PLACING CONTROLS ON A PAGE</i>	37
SDK Events	40
<i>EVENT BUBBLING</i>	42
Communicating Between Controls with Event-Handlers.....	42
Server-side Controls.....	44
Event Handlers for Dynamic Data (Content Items)	46
What's Next?	48
ViewPoint Web SDK Reference Guide	49
Application Overview	53
WebAppBase Class Overview	53
Application Class Events.....	53
<i>APPLICATION APPLICATIONREADY EVENT</i>	53
Application Class Methods.....	54
<i>GETPREFERENCE METHOD</i>	55
<i>GETPREFERENCEBOOLEAN METHOD</i>	55
<i>GETPREFERENCENUMBER METHOD</i>	56
<i>OPENWINDOW METHOD</i>	56
Application Class Properties.....	56
<i>DATAPROVIDERURL PROPERTY</i>	57
<i>M_ARRPREFERENCES PROPERTY</i>	58
Command Control Overview	58
Command HTML Attributes	59
<i>OWC:COMMAND HTML ATTRIBUTE</i>	59
<i>OWC:COMMANDPARAMS HTML ATTRIBUTE</i>	59
Command Class Overview	60
OwcCommand Class Properties	60
<i>COMMAND NAME</i>	61
Command Control Example	62
CommandButton Control Overview	63

CommandButton Class Overview.....	64
CommandButton Class Properties.....	65
CommandButton Control CSS Classes.....	65
CommandButton Control Example	65
ContentItem Overview	66
ContentItem Class Overview	67
ContentItem Class Methods.....	68
buildQueryString Method	68
CONTENTCANLOAD METHOD	68
ContentItem Class Properties.....	69
DATAOBJECTTYPE PROPERTY	69
ContentItem Example	69
ContentItem.Document Overview	70
ContentItem.Document HTML Attributes.....	70
DOC-CID HTML ATTRIBUTE.....	71
DOC-HREF HTML ATTRIBUTE	71
DOC-UID HTML ATTRIBUTE	72
DOC-VER HTML ATTRIBUTE	72
TYPE HTML ATTRIBUTE	73
ContentItem.Document Class Overview.....	73
ContentItem.Document Class Methods	74
ContentItem.Document Class Properties	74
ContentItem.Document Example.....	74
ContentItem.Entity Overview	75
ContentItem.Entity HTML Attributes.....	76
ENTITYID HTML ATTRIBUTE	76
TYPE HTML ATTRIBUTE	77
ContentItem.Entity Class Overview.....	77
ContentItem.Entity Class Methods	78
ContentItem.Entity Class Properties	78
ContentItem.Entity Example.....	78
ContentItem.Folder Overview	79
ContentItem.Folder HTML Attributes	80
FOLDER-UID HTML ATTRIBUTE	80
TITLE HTML ATTRIBUTE.....	81
TYPE HTML ATTRIBUTE	81
ContentItem.Folder Class Overview	82
ContentItem.Folder Class Methods	82
ContentItem.Folder Class Properties	82
ContentItem.Folder Example	83
ContentItem.FolderTree Overview	84
ContentItem.FolderTree HTML Attributes	84
TREE-NAME HTML ATTRIBUTE	85
TREE-UID HTML ATTRIBUTE.....	85
TYPE HTML ATTRIBUTE	86

ContentItem.FolderTree Class Overview	87
ContentItem.FolderTree Class Methods.....	87
ContentItem.FolderTree Class Properties.....	87
ContentItem.FolderTree Example	87
ContentItem.Mail Overview.....	88
ContentItem.Mail HTML Attributes.....	88
<i>TYPE HTML ATTRIBUTE</i>	88
ContentItem.Mail Class Overview.....	89
ContentItem.Mail Class Methods.....	89
ContentItem.Mail Class Properties	90
ContentItem.Mail Example.....	90
ContentItem.Page Overview	90
ContentItem.Page HTML Attributes	91
<i>LABEL HTML ATTRIBUTE</i>	91
<i>PAGE-NO HTML ATTRIBUTE</i>	92
<i>TYPE HTML ATTRIBUTE</i>	92
ContentItem.Page Class Overview	93
ContentItem.Page Class Methods.....	93
ContentItem.Page Class Properties.....	94
ContentItem.Print Overview	94
ContentItem.Print HTML Attributes.....	94
<i>TYPE HTML ATTRIBUTE</i>	95
ContentItem.Print Class Overview.....	95
ContentItem.Print Class Methods.....	95
ContentItem.Print Class Properties	96
ContentItem.Print Example	96
ContentItem.SearchQuery Overview	96
ContentItem.SearchQuery Class Overview	97
ContentItem.SearchQuery Class Methods.....	97
ContentItem.SearchQuery Class Properties.....	97
ContentItem.SearchQuery Example	97
ContentItem.SearchResults Overview	98
ContentItem.SearchResult HTML Attributes	98
<i>COLLECTION HTML ATTRIBUTE</i>	99
<i>PAGE-NO HTML ATTRIBUTE</i>	99
<i>PRIMID HTML ATTRIBUTE</i>	100
<i>SCORE HTML ATTRIBUTE</i>	101
<i>TYPE HTML ATTRIBUTE</i>	101
ContentItem.SearchResult Class Overview	102
ContentItem.SearchResult Class Methods	102
ContentItem.SearchResult Class Properties.....	102
ContentItem.SearchResult Example	103
ContentItem.TocEntry Overview.....	104
ContentItem.TocEntry HTML Attributes	104
<i>TYPE HTML ATTRIBUTE</i>	105

ContentItem.TocEntry Class Overview	105
ContentItem.TocEntry Class Methods	106
ContentItem.TocEntry Class Properties	106
ContentItem.TocEntry Example	106
Control Overview	110
Control HTML Attributes	111
OWC:CONTROL HTML ATTRIBUTE	111
OWC:INIT-STATE HTML ATTRIBUTE	112
Control Class Overview	113
Control Class Events.....	113
CONTROLCREATED EVENT.....	113
CONTROLCREATING EVENT	114
CONTROLINITIALIZED EVENT	115
Control Methods.....	115
ENABLECONTEXTMENU METHOD.....	116
GETCONTEXTMENU METHOD.....	116
OMENU = OCONTROL.GETCONTEXTMENU(BSCANANCESTORS)	117
GETCONTROLTYPE METHOD.....	117
GETOWNERPAGE METHOD.....	117
SETCONTEXTMENU METHOD.....	118
OwcControl Class Properties.....	118
HTMLELEMENT.....	119
M_BCTXMENUENABLED	119
M_OCTXMENU	119
M_SID.....	120
PARENT.....	120
WEBAPPLICATION	120
WEBPAGE.....	121
Data.SearchOptions Class Overview.....	121
Data.SearchOptions Class Methods.....	122
CLEAR METHOD	123
COPYOPTIONS METHOD.....	124
GETFUZZYLEVEL METHOD	124
GETOPERATORFLAGS METHOD	125
GETOPERATORMASK METHOD	125
GETOPTIONSSTRING METHOD.....	126
GETPHONICMODE METHOD	126
GETSTEMMINGMODE METHOD	127
GETSYNONYMSMODE METHOD	128
PARSEOPERATORS METHOD	129
SETFUZZYLEVEL METHOD	129
SETPHONICMODE METHOD	130
SETSTEMMINGMODE METHOD	130
SETSYNONYMSMODE METHOD	131
Data.SearchQuery Class Overview	132
Data.SearchQuery Class Methods.....	132
ADDSEARCHINCONTENTFIELD METHOD	134
ADDSEARCHINDOCUMENT METHOD.....	134
ADDSEARCHINFOLDER METHOD	135
ADDSEARCHINTERM METHOD.....	135
CLEAR METHOD	136
CONTENTBUILDPARAMS METHOD.....	136

<i>COPYFROM METHOD</i>	137
<i>COPYSEARCHINTERM METHOD</i>	137
<i>FINDSEARCHINTERM METHOD</i>	138
<i>GETHITSONLY METHOD</i>	138
<i>GETPAGE SIZE METHOD</i>	139
<i>GETPROPAGATEHITS METHOD</i>	140
<i>GETSEARCHINFIELD METHOD</i>	140
<i>GETSORTBY METHOD</i>	141
<i>SETHITSONLY METHOD</i>	141
<i>SETPAGE SIZE METHOD</i>	142
<i>SETPROPAGATEHITS METHOD</i>	142
<i>SETSORTBY METHOD</i>	143
<i>TOSTRING METHOD</i>	144
Data.SearchTerm Class Overview	144
Data.SearchTerm Class Methods.....	144
<i>CONTENTBUILDPARAMS METHOD</i>	145
<i>COPYFROM METHOD</i>	145
<i>GETVALUE METHOD</i>	146
<i>MATCHOPTIONS METHOD</i>	146
DataType Overview	146
DataType Methods.....	147
<i>COMPAREVALUE METHOD</i>	148
<i>CREATENEWVALUE METHOD</i>	148
<i>FORMATVALUE METHOD</i>	149
<i>GETDEFAULTFORMAT METHOD</i>	149
<i>PARSEVALUE METHOD</i>	150
<i>VALIDATEVALUE METHOD</i>	151
DataType.Boolean Overview	151
DataType.Boolean Methods.....	152
<i>FORMATVALUE METHOD</i>	152
<i>PARSEVALUE METHOD</i>	153
<i>VALIDATEVALUE METHOD</i>	154
DataType.Date Overview	155
DataType.Date Methods.....	155
<i>FORMATVALUE METHOD</i>	156
<i>GETDEFAULTFORMAT METHOD</i>	157
<i>PARSEVALUE METHOD</i>	157
<i>VALIDATEVALUE METHOD</i>	158
DataType.Day Overview	158
DataType.Day Methods.....	159
<i>FORMATVALUE METHOD</i>	159
<i>GETDEFAULTFORMAT METHOD</i>	160
<i>PARSEVALUE METHOD</i>	161
<i>VALIDATEVALUE METHOD</i>	161
DataType.Email Overview	162
DataType.Email Methods.....	162
<i>VALIDATEVALUE METHOD</i>	163
DataType.EmailList Overview	164

DataType.EmailList Methods	164
VALIDATEVALUE METHOD	165
DataType.Month Overview	165
DataType.Month Methods	166
FORMATVALUE METHOD	166
GETDEFAULTFORMAT METHOD	167
PARSEVALUE METHOD	168
VALIDATEVALUE METHOD	168
DataType.Number Overview	169
DataType.Number Methods	170
FORMATVALUE METHOD	170
PARSEVALUE METHOD	171
VALIDATEVALUE METHOD	171
DataType.Object Overview	172
DataType.Object Methods	172
CREATENEWVALUE METHOD	173
DataType.SortBy Overview	173
DataType.SortBy Methods	174
CREATENEWVALUE METHOD	174
FORMATVALUE METHOD	175
PARSEVALUE METHOD	175
VALIDATEVALUE METHOD	176
DataType.String Overview	176
DataType.String Methods	177
FORMATVALUE METHOD	177
PARSEVALUE METHOD	178
VALIDATEVALUE METHOD	179
DataType.Year Overview	179
DataType.Year Methods	180
FORMATVALUE METHOD	180
GETDEFAULTFORMAT METHOD	181
PARSEVALUE METHOD	182
VALIDATEVALUE METHOD	182
Date Overview	183
PUBLIC API	184
Date Class Overview	184
Date Commands	185
Today Command	185
Date Class Methods	186
SETTODAY	187
Day Overview	187
DocList Overview	188
DocList HTML Attributes	188
DocList Class Overview	189

OwcDocList Class Methods	189
<i>DISPLAYFOLDERCONTENT METHOD</i>	190
<i>NAVIGATETOPAGE METHOD</i>	190
DocList Example.....	191
DocViewer Control Overview.....	191
Attributes DocViewer HTML	192
OWC:HIGHLIGHTSEARCHENTITIES HTML ATTRIBUTE.....	193
OWC:HIGHLIGHTSEARCHTYPE HTML ATTRIBUTE.....	194
OWC:ENTITYSEARCHHIGHLIGHTCOLOR HTML ATTRIBUTE.....	194
OWC:HIGHLIGHTMOUSEENTITIES HTML ATTRIBUTE	195
OWC:HIGHLIGHTMOUSETYPE HTML ATTRIBUTE.....	195
OWC:ENTITYMOUSEHIGHLIGHTCOLOR HTML ATTRIBUTE.....	196
OWC:SHOWHITS HTML ATTRIBUTE.....	197
OWC:ARROWURL HTML ATTRIBUTE.....	197
OWC:PRIMITIVES HTML ATTRIBUTE.....	198
TYPE	198
BOOLEAN.....	198
USE	198
OPTIONAL.....	198
DEFAULT VALUE	198
FALSE.....	198
OWC:CORNERS HTML ATTRIBUTE	198
OWC:USEPAGESRANGE HTML ATTRIBUTE	199
OWC:DOUBLEPAGEMODE HTML ATTRIBUTE.....	199
DocViewer Class Overview	200
DocViewer Events	201
DocViewer viewerCurrentHitChanged Event	201
OwcDocViewer Class Methods	201
ISNEXTPAGEAVAIL METHOD.....	202
ISPREVPAGEAVAIL METHOD	202
NEXTPAGE METHOD.....	202
PREVPAGE METHOD	203
SETLEGALPAGERANGE METHOD	203
GETCONTENTHTMLSTRING METHOD	204
VIEWERHIGHLIGHTENTITY METHOD.....	204
DocViewer Class Properties.....	205
LAYOUTSET	206
BHIGHLIGHTSEARCHENTITIES	206
HIGHLIGHTSEARCHTYPE.....	206
HIGHLIGHTSEARCHCOLOR	207
BHIGHLIGHTMOUSEENTITIES.....	207
HIGHLIGHTMOUSETYPE	208
HIGHLIGHTMOUSECOLOR.....	208
BDISPLAYHITS.....	208
HITS	209
HITQUADS	209
HITIMGS	209
SELHITINDEX.....	209
SELHITPAGENO.....	209
ARROWTOHIGHLIGHTURL.....	210
LEFTFORARROW.....	210
TOPFORARROW.....	210
DIVFORARROW.....	210

	211
bEntityPrims.....	211
EntitiesOnPage.....	211
PrimitivesOnPage.....	211
EmbeddedPictures.....	211
HighlightedSearchDivs.....	212
PrimDivs.....	212
ImagesCache.....	212
DBLPAGEMODE.....	212
BCORNERs.....	212
BUsePAGESRANGE.....	213
LEGALPAGESAFTER.....	213
LEGALPAGESBEFORE.....	213
DocViewer Control CSS Classes	214
DocViewer Example	214
EntViewer Overview	214
EntViewer HTML Attributes	216
OWC:SELECTEDHitSTYLE HTML ATTRIBUTE	216
OWC:UNSELECTEDHitSTYLE HTML ATTRIBUTE	217
OWC:SELECTEDHitCOLOR HTML ATTRIBUTE	218
OWC:UNSELECTEDHitCOLOR HTML ATTRIBUTE	218
OWC:ARROWURL HTML ATTRIBUTE.....	219
OWC:LAYOUTSET HTML ATTRIBUTE.....	219
OWC:ISFORWARD HTML ATTRIBUTE.....	220
OWC:HIGHLIGHTARROW HTML ATTRIBUTE	221
EntViewer Class Overview	221
EntViewer Class Events	222
ENTVIEWERCALLED EVENT	222
ENTVIEWERLOADED EVENT.....	223
EntViewer Class Methods.....	223
ISNEXTHitAVAILABLE METHOD.....	223
ISPREVHitAVAILABLE METHOD.....	224
NEXTHit METHOD.....	225
PREVHit METHOD	225
GOTONEXTCHUNK METHOD	225
GOTOPREVCHUNK METHOD	226
GETLOADINGSTATUS METHOD	226
EntViewer Class Properties.....	227
CURHitBox PROPERTY	228
HITS PROPERTY.....	228
SEARCHSTR PROPERTY	228
DOCUMENTID PROPERTY	229
CHUNK PROPERTY	229
ENTITYWIDTH PROPERTY	229
ENTITYHEIGHT PROPERTY	230
PRINTWIDTH PROPERTY	230
PRINTHEIGHT PROPERTY	231
HIGHLIGHTDIVS PROPERTY.....	231
SELECTEDHitSTYLE PROPERTY	232
UNSELECTEDHitSTYLE PROPERTY	232
SELECTEDHitCOLOR PROPERTY.....	233
UNSELECTEDHitCOLOR PROPERTY.....	233
ISHIGHLIGHTING PROPERTY.....	234
BADDARROWTOHIGHLIGHT PROPERTY	234
BARROWLOADED PROPERTY	235

<i>ARROWToHIGHLIGHTURL</i> PROPERTY	235
<i>IMGARROW</i> PROPERTY	235
EntViewer Control CSS Classes	236
EntViewer Example	236
EventSource Overview	237
EventSource Class Overview	238
EventSource Class Methods	238
<i>REGISTEROWCEVENTSCLASS2</i>	238
<i>ATTACHOWCEVENTHANDLER</i>	239
<i>DETACHOWCEVENTHANDLER</i>	240
<i>DETACHALLOWCEVENTHANDLERS</i>	241
<i>CREATEOWCEVENTOBJECT</i>	241
<i>FIREOWCEVENT</i>	242
FlashViewer Control Overview	242
Flash Viewer Options	244
FlashViewer HTML Attributes	251
<i>OWC:BG</i> COLOR	251
<i>OWC:FLASHCONTENTOPTIONS</i>	252
FlashViewer Class Overview	253
FlashViewer Class Events	253
<i>FLASHEVENTRECEIVED</i> EVENT	254
<i>PRINT</i> EVENT	255
<i>PAGETURNED</i> EVENT	256
<i>PAGEMODECHANGED</i> EVENT	256
<i>TOGGLEFRAME</i> EVENT	257
FlashViewer Class Methods	257
<i>DISPLAYDOCUMENT</i> METHOD	258
<i>CLOSETOC</i> METHOD	258
<i>GOTOPAGE</i> METHOD	259
<i>PROCESSCUSTOMFLASHEVENT</i> METHOD	259
FolderTree Overview	259
FolderTree HTML Attributes	260
OwcFolderTree Class Overview	261
OwcFolderTree Class Methods	261
<i>DISPLAYTREE</i> METHOD	262
<i>GETDISPLAYNAME</i> METHOD	262
OwcFolderTree Class Properties	263
<i>DISPLAYNAME</i> PROPERTY	263
FolderTree Example	263
Global Functions	264
<i>OLIVE.CMDTARGET.REGISTERCOMMAND</i> METHOD	269
<i>OLIVE.PAGE.GETPAGEFORWINDOW</i> METHOD	269
<i>OLIVE.PAGE.GETPAGEFORDOCUMENT</i> METHOD	270
<i>OLIVE.CONTROLS.IDATABOUND.GETCONTENTITEM</i> METHOD	270
<i>OLIVE.WINDOWBINDER.REGISTERPAGECLASS</i> METHOD	271
<i>OLIVE.WINDOWBINDER.REGISTERPAGECUSTOMPROTOTYPE</i> METHOD	271
<i>OWCGETCONTROLFROMHTMLELEM</i> METHOD	271
<i>JSCRIPT_DERIVECLASS</i> METHOD	272
<i>OWCGETCONTROL</i> METHOD	272
<i>OBJECT_CLONE</i> METHOD	273

<i>STRING_PARSEBOOLEAN METHOD</i>	273
<i>DOM.CREATEFRAGMENTFROMCONTENT METHOD</i>	273
<i>DOM.MOVECONTENTTOFRAGMENT METHOD</i>	274
<i>DOM.MOVECONTENTTOPARENT METHOD</i>	274
<i>DOM.CLEARCONTENT METHOD</i>	275
<i>DOM.PASTECONTENT METHOD</i>	275
<i>OLIVE.RESOURCES.DEFINERESSTRING METHOD</i>	275
<i>OLIVE.RESOURCES.GETRESSTRING METHOD</i>	276
<i>OLIVE.RESOURCES.DEFINEGLOBALRESSTRING METHOD</i>	276
<i>OLIVE.ERRORS.GETERRORINFO METHOD</i>	277
<i>OLIVE.ERRORS.CREATEERROR METHOD</i>	277
<i>OLIVE.ERRORS.DEFINEKNOWNERROR METHOD</i>	278
<i>OLIVE.ERRORS.GETLOCALIZEDFORMATSTRING METHOD</i>	278
<i>OLIVE.ERRORS.GETLOCALIZEDERRORMSG METHOD</i>	279
<i>STRING_PARSEINT METHOD</i>	279
<i>STRING_FORMAT METHOD</i>	279
<i>STRING_FORMAT_ARG METHOD</i>	280
<i>NUMBER.PROTOTYPE.TOPRECISION METHOD</i>	280
<i>FUNCTION.PROTOTYPE.APPLY METHOD</i>	281
<i>OLIVE.CONTROLS.IDATABOUND.GETHTMLCONTENTTARGETNULL METHOD</i>	281
<i>OLIVE.WINDOWBINDER.GETBINDERFORWINDOW METHOD</i>	282
IActiveItem Overview	282
IActiveItem Events.....	283
<i>ITEMACTIVATED EVENT</i>	283
<i>ITEMACTIVATING EVENT</i>	284
IActiveItem Methods	284
<i>GETACTIVEITEM METHOD</i>	285
<i>GETACTIVEITEMINDEX METHOD</i>	285
<i>GETITEM METHOD</i>	286
<i>GETITEMSCOUNT METHOD</i>	286
<i>INDEXOFITEM METHOD</i>	287
<i>SETACTIVEITEM METHOD</i>	287
ICmdItem Overview	288
ICmdItem Methods.....	288
<i>UPDATECMDUISTATE METHOD</i>	288
ICmdSource Overview	289
ICmdSource Methods	289
<i>GETCMDTARGET METHOD</i>	290
<i>SETCMDTARGET METHOD</i>	290
<i>SENDCOMMAND METHOD</i>	291
<i>UPDATECOMMAND METHOD</i>	291
<i>UPDATECMDUISTATE METHOD</i>	292
IComplexValue Overview	292
IComplexValue Methods.....	293
<i>CREATENEWVALUE METHOD</i>	293
<i>UPDATECONTROL METHOD</i>	293
<i>UPDATEDATA METHOD</i>	294
IControlContainer Overview	294
IControlContainer HTML Attributes	295
<i>OWC:CONTROL HTML ATTRIBUTE</i>	295
<i>WC:UI HTML ATTRIBUTE</i>	296

IControlContainer Methods.....	297
<i>BINDCONTROLS METHOD.....</i>	<i>298</i>
<i>PARSEHTMLELEMENT METHOD</i>	<i>298</i>
<i>INITIALIZECONTROLS METHOD.....</i>	<i>298</i>
<i>CLEARCONTROLS METHOD.....</i>	<i>299</i>
<i>CLEARSELECTIONS METHOD</i>	<i>299</i>
<i>ADDCONTROL METHOD.....</i>	<i>300</i>
<i>REMOVECONTROL METHOD.....</i>	<i>300</i>
<i>FINDCONTROLBYID METHOD.....</i>	<i>301</i>
<i>FINDCONTROLBYTYPE METHOD</i>	<i>301</i>
<i>FINDCONTROLSBYTYPE METHOD.....</i>	<i>301</i>
<i>GETCONTROLSCOUNT METHOD</i>	<i>302</i>
<i>GETCONTROL METHOD.....</i>	<i>302</i>
<i>GETCONTROLREV METHOD.....</i>	<i>303</i>
<i>INDEXOFCONTROL METHOD.....</i>	<i>303</i>
<i>ISFIRSTCONTROL METHOD.....</i>	<i>304</i>
<i>ISLASTCONTROL METHOD.....</i>	<i>304</i>
<i>SETACTIVEMASK METHOD.....</i>	<i>304</i>
IControlContainer Properties.....	305
<i>CONTROLS PROPERTY</i>	<i>305</i>
IControlFactory Overview.....	306
IControlFactory Methods	306
<i>REGISTERCONTROLTYPE METHOD.....</i>	<i>306</i>
<i>GETREGISTEREDCONTROLTYPE METHOD.....</i>	<i>307</i>
<i>CREATECONTROLINSTANCE METHOD.....</i>	<i>307</i>
<i>CREATECONTROL METHOD.....</i>	<i>308</i>
IDataBound Overview	308
IDataBound Events.....	309
<i>SENDINGREQUEST EVENT.....</i>	<i>309</i>
<i>DATARECIEVED EVENT</i>	<i>310</i>
<i>DATAPROCESSED EVENT</i>	<i>310</i>
<i>CONTENTITEMCLICKED EVENT.....</i>	<i>311</i>
<i>CONTENTITEMDBLCLICKED EVENT</i>	<i>312</i>
<i>CONTENTITEMRIGHTCLICKED EVENT.....</i>	<i>312</i>
<i>CONTENTITEMACTIVATESTATECHANGED EVENT</i>	<i>313</i>
<i>CONTENTSELECTIONCHANGED EVENT.....</i>	<i>313</i>
<i>FOLDERTREENAMERECEIVED EVENT</i>	<i>314</i>
IDataBound Methods	314
<i>CONTENTBUILDPARAMS METHOD.....</i>	<i>315</i>
<i>CONTENTCANLOAD METHOD</i>	<i>316</i>
<i>CONTENTLOAD METHOD</i>	<i>316</i>
<i>GETCONTENTITEM METHOD.....</i>	<i>317</i>
<i>BUILDREQUESTURL METHOD.....</i>	<i>317</i>
<i>GETDATAPROVIDERURL METHOD.....</i>	<i>317</i>
<i>SENDREQUESTRAW METHOD</i>	<i>318</i>
<i>SETLOADINGDATAMESSAGE METHOD</i>	<i>318</i>
<i>REQUESTDATA METHOD.....</i>	<i>319</i>
<i>PROCESSRESPONSE METHOD.....</i>	<i>319</i>
<i>PROCESSRESPONSETEXT METHOD</i>	<i>320</i>
<i>PROCESSRESPONSEDATA METHOD.....</i>	<i>320</i>
<i>PROCESSRESPONSEHTML METHOD</i>	<i>321</i>
<i>GETHTMLCONTENTTARGET METHOD</i>	<i>321</i>
<i>POSTHTMLCONTENTPASTE METHOD.....</i>	<i>322</i>
IDataBound Properties	322

<i>DATABOUNDURL</i> PROPERTY	322
IForm Overview	323
IForm Events	323
IForm Methods	324
<i>FINDFIELDCONTROLS</i> METHOD	324
<i>GETFIELDVALUE</i> METHOD	325
<i>SETFIELDVALUE</i> METHOD	325
IFormField Overview	325
IFormField HTML Attributes	326
<i>OWC:REQUIRED</i> HTML ATTRIBUTE	327
IFORMFIELD METHODS	328
<i>GETFIELDNAME</i> METHOD	328
<i>ISFIELDREQUIRED</i> METHOD	328
IPane Overview	329
IPane HTML Attributes	329
<i>OWC:RESIZABLE</i> HTML ATTRIBUTE	330
<i>OWC:PANEMAXIMIZETOOLTIP</i> HTML ATTRIBUTE	331
<i>OWC:PANEMINIMIZETOOLTIP</i> HTML ATTRIBUTE	331
IPane Commands	332
MINIMIZE COMMAND	332
RESTORE COMMAND	333
TOGGLE COMMAND	334
SHOW COMMAND	335
HIDE COMMAND	336
IPane CSS Classes	337
IPane Methods	337
<i>SETPANEITEMSIZE</i> METHOD	337
<i>GETTOOLTIP</i> METHOD	338
IPane UIElements	338
HEADER UIELEMENT	339
CONTENT UIELEMENT	339
TOGGLE-COMMAND UIELEMENT	340
IPanelList Overview	341
IPanelList HTML Attributes	341
<i>OWC:KIND</i> HTML ATTRIBUTE	341
IPANELLIST METHODS	342
<i>ISHORIZONTAL</i> METHOD	342
IPanelList Properties	343
KIND PROPERTY	343
IPanelListItem Overview	343
IPanelListItem HTML Attributes	344
<i>OWC:PANE-SIZE</i> HTML ATTRIBUTE	344
IPANELLISTITEM METHODS	345
<i>ISVISIBLE</i> METHOD	345
SHOW METHOD	346
HIDE METHOD	346
<i>ISMINIMIZED</i> METHOD	346
MINIMIZE METHOD	347
RESTORE METHOD	347
TOGGLE METHOD	348

<i>TOGGLEVISIBLE METHOD</i>	348
<i>ISCOLUMNITEM METHOD</i>	349
<i>GETPANEITEMSIZEFACTOR METHOD</i>	349
IPanelItem Properties	350
<i>PANECONTAINER PROPERTY</i>	350
<i>RESIZABLEPANE PROPERTY</i>	350
IPaneSplitter Overview	351
IPaneSplitter Methods	351
<i>ONDRAgend METHOD</i>	351
<i>ONDRAgMOVE METHOD</i>	352
<i>ONDRAgSTART METHOD</i>	352
ISearchFormField Overview	353
ISearchFormField Methods	353
<i>READFROMSEARCHDATA METHOD</i>	354
<i>WRITETOSEARCHDATA METHOD</i>	354
ISelection Overview	355
ISelection Events	355
<i>SELECTIONCHANGED EVENT</i>	355
<i>SELECTIONCHANGING EVENT</i>	356
IState Overview	357
IState HTML Attributes	358
<i>OWC:INIT-STATE HTML ATTRIBUTE</i>	358
IState Events	359
<i>STATECHANGED EVENT</i>	359
IState Methods	359
IUIElements Overview	360
IUIElements Methods	361
IUIElemTypes Overview	361
IUIElemTypes Methods	362
<i>GETUIELEMTYPE METHOD</i>	362
<i>REGISTERUIELEMTYPE METHOD</i>	362
WebAppBase Example	363
WebAppBase Class Methods	366
WebAppBase Class Properties	368
IValue Overview	368
IValue HTML Attributes	368
<i>OWC:DATATYPE HTML ATTRIBUTE</i>	369
<i>OWC:FORMAT HTML ATTRIBUTE</i>	370
<i>OWC:HTMLVALUE HTML ATTRIBUTE</i>	371
IValue Class Events	372
<i>VALIDATIONERROR EVENT</i>	372
<i>VALUECHANGED EVENT</i>	373
IValue Class Methods	374
LangList Control Overview	375
OwcLangList Class Overview	375

LangList Example	375
List Overview	376
List HTML Attributes	376
OWC:PAGESIZE HTML ATTRIBUTE	377
OWC:USEPAGINATION HTML ATTRIBUTE	377
List Class Overview	378
List Class Methods	378
List Class Methods	379
List CSS Classes.....	379
List Control Example.....	380
ListItem Overview.....	380
ListItem HTML Attributes	380
OWC:ITEMTYPE HTML ATTRIBUTE.....	381
ListItem Class Overview	381
Requirements	382
ListItem Class Methods	382
ITEMSELECT METHOD.....	382
ListItem Control Example.....	383
Magnifier Control Overview.....	383
Magnifier HTML Attributes	383
Magnifier Class Overview	384
Menu Class Methods	394
Menu Example	395
MenuItem Control Overview	398
MenuItem HTML Attributes.....	399
OWC:COMMAND HTML ATTRIBUTE	399
MenuItem Class Overview	400
MenuItem Class Methods	401
GETPARENTMENU METHOD.....	401
MenuItem Example	401
Metadata Control Overview.....	403
Metadata Class Overview.....	404
Metadata Example.....	404
Value Control Overview.....	408
OWC:SEPARATOR HTML ATTRIBUTE.....	409
MetadataFieldValue Class Overview	410
MetaField Control Overview.....	413
MetaField HTML Attributes	414
OWC:DISPLAYNAME HTML ATTRIBUTE	414
OWC:FIELD HTML ATTRIBUTE.....	415
OWC:SHOWIF HTML ATTRIBUTE	416
MetaField Class Overview	416
MetaField Example	417

Month Overview.....	420
Page Overview	421
OwcPage Class Overview.....	421
Page Events.....	422
PAGELOADED EVENT.....	422
PAGEUNLOADED EVENT.....	422
Page Class Methods.....	423
Page Class Properties	424
PageItem Control Overview	424
PageItem Class Overview.....	425
PageItem Class Events	425
PageItem Control CSS Classes.....	425
PageSwitcher Control Overview	426
PageSwitcher Class Overview	426
PageSwitcher Class Events	427
PageSwitcher Class Methods.....	427
PageSwitcher Example	428
Pane Control Overview	428
Pane HTML Attributes.....	429
Pane Class Overview	429
Pane Class Methods	430
pane Example	430
pane-list Control Overview.....	432
pane-list HTML Attributes	433
PaneList Class Overview.....	433
PaneList Class Methods.....	433
PaneList Class Properties.....	434
pane-list Example	434
pane-splitter Control Overview	436
PaneSplitter Class Overview.....	437
PaneSplitter Class Methods	437
PaneSplitter Class Properties	437
pane-splitter Example.....	438
PrintList Overview	439
PrintList Class Events	440
READYTOPRINT EVENT	440
PrintList Control Example	441
ScoreBar Overview	441
ScoreBar HTML Attributes	442
OwcScoreBar Class Overview	443
OwcScoreBar Class Properties.....	443
M_DISABLECONTENTAUTOLOAD PROPERTY	445

<i>M_SCOREVALUEPROPERTY PROPERTY</i>	445
ScoreBar Example	445
SearchForm Overview	446
SearchForm Class Events	450
<i>SEARCHREQUEST EVENT</i>	450
SearchForm Class Methods	451
<i>SEARCHFORMSUBMIT METHOD</i>	451
SearchForm Example	451
SearchInFolder Class Overview	457
SearchInFolderLookup control Overview	457
SearchInFolderLookup Class Overview	459
SearchInFolderLookup Methods	460
<i>ONSEARCHINALLFOLDERSCHANGED METHOD</i>	460
SearchInFolderLookup UiElements	461
<i>SETFOLDERTREE METHOD</i>	465
SearchRes Overview	469
SearchRes HTML Attributes	470
SearchRes Class Overview	470
SearchRes Class Methods	471
<i>NAVIGATETOPAGE METHOD</i>	472
OwcSearchRes Class Properties	472
SearchRes Control Example	472
SearchTerm Overview	473
SearchTerm HTML Attributes	473
SearchTerm Class Overview	474
SearchTerm Methods	474
<i>PARSESEARCHFLAGS METHOD</i>	475
<i>PARSESEARCHOPERATORS METHOD</i>	475
<i>READFROMSEARCHDATA METHOD</i>	476
<i>WRITETOSEARCHDATA METHOD</i>	476
SortBy Overview	477
SortBy Class Overview	478
SortBy Class Methods	478
<i>GETLISTCONTROL</i>	479
<i>SETLISTCONTROL</i>	479
<i>UPDATELISTFROMSELECTION</i>	479
<i>UPDATESELECTIONFROMLIST</i>	480
TabItem Control Overview	480
TabItem Class Overview	481
TabItem Class Events	482
TabItem Example	482
TabStrip Control Overview	485
TabStrip HTML Attributes	486
<i>OWC:DECORATE HTML ATTRIBUTE</i>	486

TabStrip Class Overview	487
TabStrip Control CSS Classes	488
Thumbnail Class Overview.....	488
Implemented Interfaces	489
TOC Overview	489
TOC Class Overview	492
TOC Example	492
Tree Overview	493
Tree HTML Attributes and Sub-elements.....	493
CSSCLASS HTML ATTRIBUTE OF THE OWC:NODETYPE HTML ELEMENT	495
CSSCLASSSELECTED HTML ATTRIBUTE OF THE OWC:NODETYPE HTML ELEMENT	495
ICONURL HTML ATTRIBUTE OF THE OWC:NODETYPE HTML ELEMENT.....	495
ICONURLSELECTED HTML ATTRIBUTE OF THE OWC:NODETYPE HTML ELEMENT	496
NAME HTML ATTRIBUTE OF THE OWC:NODETYPE HTML ELEMENT	496
NOICON HTML ATTRIBUTE OF THE OWC:NODETYPE HTML ELEMENT	497
OWC:MULTISELECT HTML ATTRIBUTE	497
OWC:DATA HTML SUB-ELEMENT.....	498
OWC:NODETYPE HTML SUB-ELEMENT.....	498
Tree Class Overview.....	499
Tree Class Events	499
Tree Class Methods	500
ONNODEMOUSEDOWN	500
REGISTERNODETYPE METHOD	501
TreeNode Overview.....	501
TreeNode Class Overview	503
TreeNode Class Commands	503
EXPAND COMMAND	503
TreeNode Class Events	504
NODEEXPANDED EVENT.....	505
NODEEXPANDING EVENT	505
TREEBOXMOUSEDOWN EVENT.....	506
OwcTreeNode Class Methods.....	507
GETNODEICONSELECTEDURL METHOD.....	507
GETNODEICONURL METHOD.....	508
GETOUTLINEMASK METHOD	508
ISNODEEXPANDABLE METHOD.....	509
ISNODEEXPANDABLE METHOD.....	509
ISNODEFIRSTCHILD METHOD	509
ISNODELASTCHILD METHOD.....	510
ISNODESELECTED METHOD.....	510
NODEEXPAND METHOD.....	510
NODESELECT METHOD.....	511
UiElemType Overview.....	511
UiElemType Methods	512
BINDUIELEMENT METHOD.....	512
BINDUIELEMENTCOMMON METHOD	513
BINDUIELEMENTCUSTOM METHOD.....	513
UPDATEUIELEMSTATE METHOD.....	514
UiElemType Properties.....	514

<i>CLASSNAME PROPERTY</i>	515
<i>OWCUIPARENT PROPERTY</i>	515
UiElemType.TabItemPos Overview	516
UiElemType.TabItemPos Methods.....	516
<i>UPDATEUIELEMSTATE METHOD</i>	516
UiElemType.TabItemPos Properties.....	517
UiElemType.TreeBox Overview	517
UiElemType.TreeBox Methods	518
<i>BINDUIELEMENTCUSTOM METHOD</i>	519
<i>UPDATEUIELEMSTATE METHOD</i>	519
UiElemType.TreeBox Properties.....	520
<i>CLASSNAME PROPERTY</i>	520
UiElemType.TreeIcon Overview	521
UiElemType.TreeIcon Methods.....	521
<i>UPDATEUIELEMSTATE METHOD</i>	521
UiElemType.TreeIcon Properties.....	522
UiElemType.TreeNodeSelect Overview	522
UiElemType.TreeNodeSelect Methods	523
<i>BINDUIELEMENTCUSTOM METHOD</i>	523
<i>UPDATEUIELEMSTATE METHOD</i>	524
UiElemType.TreeNodeSelect Properties.....	525
<i>CLASSNAME PROPERTY</i>	525
UiElemType.TreeOutline Overview	525
UiElemType.TreeOutline Methods.....	526
<i>UPDATEUIELEMSTATE METHOD</i>	526
UiElemType.TreeOutline Properties.....	527
<i>CLASSNAME PROPERTY</i>	527
UiElemType.TreeSubNodes Overview	528
UiElemType.TreeSubNodes Methods	528
<i>UPDATEUIELEMSTATE METHOD</i>	529
UiElemType.TreeSubNodes Properties.....	529
Value Overview	530
PUBLIC API.....	530
Value Class Overview.....	531
Viewer Overview	531
OwcViewer Class Overview	532
OwcViewer Class Methods	532
OwcViewer Methods.....	532
<i>DISPLAYDOCUMENTBYPAGENUMBER METHOD</i>	534
<i>DISPLAYDOCUMENTBYPAGENUMBERFROMSEARCH METHOD</i>	535
<i>DISPLAYDOCUMENTBYPAGENUMBERLABEL METHOD</i>	535
<i>DISPLAYDOCUMENTBYPAGENUMBERLABELFROMSEARCH METHOD</i>	536
<i>VIEWERGOTOENTITY METHOD</i>	536
<i>VIEWERGOTONEXTHIT METHOD</i>	537
<i>VIEWERGOTOPAGEBYLABEL METHOD</i>	537

VIEWERGOTOPAGEBYNUMBER METHOD.....	537
VIEWERGOTOPREVHIT METHOD.....	538
VIEWERRELOADCURPAGEBYLABEL METHOD.....	538
VIEWERRELOADCURPAGEBYNUMBER METHOD.....	539
OwcViewer Class Properties	539
OwcViewer properties	539
BASELEGALPAGE.....	540
BRETRIEVEHITS.....	540
COLLNAME	541
CURPAGE	541
ENTITYID.....	541
FIRSTLEGALPAGE	541
FIRSTPAGE.....	541
FIRSTPAGEWIDTH.....	541
IMAGESRES.....	542
LANGDIRECTION	542
LANGUAGE.....	542
LASTLEGALPAGE	542
LASTPAGE	543
LOADREQPARAMS.....	543
NUMPAGESDISPLAYED.....	543
PAGEHEIGHT.....	543
PAGERES.....	543
SEARCHSTR.....	544
SECONDPAGEWIDTH	544
TOTALPAGEWIDTH.....	544
WebAppBase Class Overview	544
WebAppBase Class Events.....	545
APPLICATIONREADY EVENT.....	545
Year Overview	546
Layout XML Overview.....	547
File Structure	547
XML Elements and Attributes.....	547
Attributes.....	557
Syntax	558
Layout.xml File	558
CLIENT SIDE CODING STANDARDS.....	575

Introduction

The Olive ViewPoint Web Software Development Kit (SDK) is a toolkit for developing Web applications that implement Olive's unique document publishing technology.

ViewPoint uses advanced software to create to identify and segment a document to its basic entities. Once processed, or segmented, the information in these entities is archived in a standard XML format (Olive's PrXML) in the ViewPoint Warehouse (repository), which maintains the original document's presentation.

The documents in the ViewPoint Warehouse, in PrXML format, can then be re-purposed in a variety of formats, including on-screen and hardcopy presentation, HTML, plain text, and many other industry-standard formats.

ViewPoint enables document viewing in any standard Internet browser (such as Microsoft Internet Explorer, Mozilla Firefox, and Apple Safari), on all platforms, as well as via the FlashViewer, which maintains the document's original look and feel.

The ViewPoint Web SDK is a software development framework and toolkit that provides Web developers with the tools to develop custom Web applications or implement ViewPoint technology into existing enterprise applications.

The Need for a Web SDK

Web application developers and HTML designers typically have a limited set of GUI components from which they can design, usually HTML form elements. To develop a richer application, the developer often needs to augment his limited tool set with additional GUI elements and data retrieval mechanisms that interface with the application's business logic. With every new Web application, the developer must spend time and repeat the process of building a new development environment with new UI blocks and flow controls. As much as 80% of the development effort is dedicated to these peripheral tasks, instead of focusing on the application's business logic. To save time, many developers copy and modify GUI elements from other applications, resulting in difficult to maintain and upgrade duplicated code. At Olive we identified a need for a Web development framework that would address this need. We investigated available frameworks and component libraries and found only limited solutions. For this reason we decided to build an internal framework for our environment, the Olive Enterprise Platform Web SDK.

The ViewPoint Web SDK is a framework for building smart Web applications for ViewPoint, providing an out-of-the-box set of plug-and-play customizable Web controls that you can easily assemble to build cross browser and cross-platform Web applications. The SDK includes data bound controls, with an internal mechanism that queries and presents relevant information, automatically generating HTML, CSS and JavaScript content as they render. Customizable event driven mechanisms control data and flow operations, and can be integrated in a variety of host environments and applications.

Document Structure

This document is presented in the form of online help where the Table of Contents appears in the left pane.

It is divided into the following major sections:

Web SDK Architecture - This section describes the basic architecture of the Web SDK. It would be very useful to read through this section before starting to program using the SDK.

Building a Simple Web Application - This section instructs how to begin building a simple Web-based application. This section may be very useful to programmers who have little or no experience developing client-server web-based applications.

Olive Web SDK Reference - This is the primary section this document. It includes descriptions, explanations, and instructions for each of the components and modules of the Olive Web SDK. This section is organized according to categories, as listed in the [Web SDK Overview](#) page.

Client-Side Coding Standards - This section is a checklist for designing client-side applications that are to communicate with Olive servers and application servers that use Olive technologies.

ViewPoint Web SDK Architecture

Olive ViewPoint Web SDK brings structure and reusability to Web development by providing a framework by which all components can fit. These reusable components can then be mixed and matched in other combinations, without needing to design entire Web pages or sites from scratch.

Background

The traditional approach to adding content to a Web application is to follow a certain procedure:

1. Paint a bitmap picture
2. Create an image map that responds to the click events
3. Write scripts, event handles that handle the click events

This procedure is then repeated for every page in the application. Even if the application functions properly, it is difficult to maintain for several reasons:

- The various parts of the Web application are not clearly organized. Document links in Web applications that are not based on a solid framework tend to become scattered throughout the Web pages, like spaghetti.
- Much of the code is duplicated between and within the Web applications.
- Localization - Implementing the same application in different languages requires redesigning each page for each language.
- Inflexible Application design or graphic layout is difficult to change. Upgrading the user interface usually requires rewriting major parts of the application.

The ViewPoint Web SDK is designed to provide a framework for Web application design using controls with built-in presentation and functionality, all in an out-of-the-box package.

- Print like ePublishing
- Browser based unified viewing interface.

- Fast access to complete documents and document components (parts of documents).
- Component level usage statistics

ViewPoint Architecture

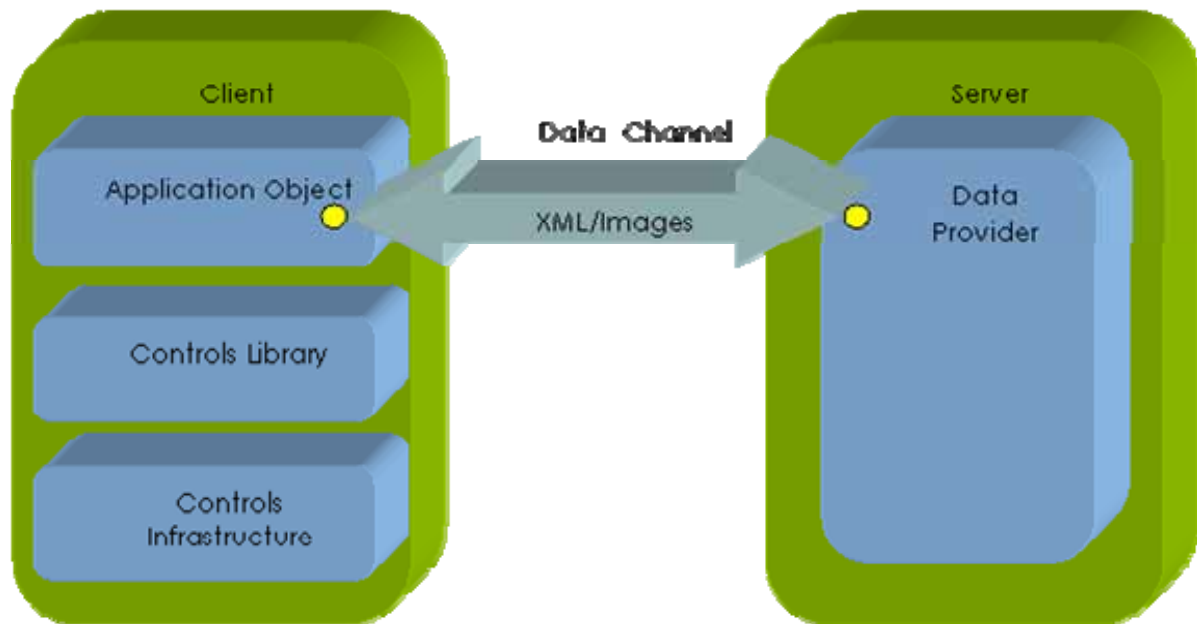


Figure 1: High-Level VP Web SDK Client-Server Architecture

Olive ViewPoint Web SDK simplifies the process of developing and implementing business logic by providing a framework on which to build rich Web applications, thus liberating the developer from the infrastructure tasks to focus on the business logic. It provides an out-of-the-box set of visual presentation and infrastructure controls (Olive Web Controls SDK and Olive Controls library), which the flow between the Web controls and customization of style and localization.

Application Object is a customizable helper class to implement the business logic. It is centrally installed, is available to all HTML pages and controls, and functions by centralizing client-server data exchange and providing access to global settings and resources.

Olive Controls Library is a collection of visual building blocks that provides visual controls and an extension to the browser DHTML model. It is built on and based on the Web Controls SDK.

Olive Web Controls SDK Infrastructure is a framework that enables building general purpose Web controls with built-in data exchange capabilities. Based on a rich event model Object Oriented Programming (OOP) architecture, the Web Controls SDK is extensible, customizable, compatible with all major browsers, and is easily integrated into any hosting environment with no client-side installation.

Concepts

The ViewPoint Web SDK is based on two basic concepts; **dynamic content**, which enables reading and updating constantly changing content, and **HTML templates**,

which enables fast and easy development of rich Web sites and pages. These concepts are summarized below:

Controls with dynamic content

- Ready made client-side visual objects are available to present the various document data types provided by the server
- Data that is not presented is not transmitted to the client side

HTML Templates

- HTML pages include fields with XML tags that are then dynamically filled by content from the server
- Provides a good solution for varied document types

Controls with dynamic contents

The design process of the Web SDK reviews different data presentation methods, and gives special focus to flexibility readiness. The process asks specific questions regarding the application engineers work, such as the data source (from where is it taken) and presentation (how it viewed).

The visual controls design process asked more questions:

1. Visual presentation of buttons and rendered code (DIV or SPAN)
2. Thumbnail presentations and pages (none, first, first and last)
3. Present TOC upon opening and to what level

The following guidelines answer the above questions:

- Content driven presentation: The content type, such as metadata and TOC, define how the data is presented.
- Data that is not presented is not transmitted to the client side.
- Diverse data dictates different presentation:
 - Journals and magazines
 - Grey literature short research papers and report documents without a defined structure or pictures

These documents are completely different and must be presented differently. HTML templates can provide a good solution for the varied document types.

HTML Templates

HTML pages can hold fields with XML tags that are then filled by content that is retrieved from the server. Designing HTML templates raises the following questions:

- What placeholders are used?
- What are the placeholder formats?
- What capabilities do they have?
- How are they presented? Can they present XML, markup data (HTML entity)?

An additional client-side presentation mechanism is added to the VP Web SDK. This new mechanism can integrate additional data. It works by interfering with the data transformation during client-side template rendering.

The client-site templates include the following components:

- **Instructions:** A predefined set of XSL-like operations, including the following instructions:
- **Flow:** if, for, choose
- **Constructors:** Elements attributes, text and notes (like in XSL)
- **Engine** extension
- **JavaScript** infrastructure support of test functions. It provides access to HTML DOM and the external data that the application object retrieves, such as localized messages. It also provides access to the controls data object and all the other objects and functions on the page.
- **HTML elements** are copied as they are

Each HTML template configuration has its advantages and disadvantages as described below:

Template type	Benefits	Disadvantages
Client side	* Flexible * Accessible to the current application state * Can integrate additional data	* Can potentially stress the client
Server side	* Accessible to sensitive data not sent to client.	* Not flexible * Difficult to maintain * No access to current application state, unless server transmits state data to the client

Table 1: Template types pros and cons

Performance Issues

The following solutions may be used when addressing the potential performance issues that may arise when using client side templates:

- **Data Pagination** sends only the relevant data to the client on demand. For example, when presenting a set of 100 documents, ViewPoint can be configured to present 10 documents at a time. The portion size is configurable.
- **Progressive Download** immediately presents partial information on all relevant documents, and the retrieves additional data upon demand. For example, when presenting 100 documents in a tree structure, only partial information is presented

initially (title) while retrieving more detailed information on each document on demand.

- **Combined Data Pagination and Progressive Download** combines both of the above solutions. For example, Data Pagination is implemented first, selecting the initial data that is retrieved, and then progressive download is implemented on the initial data. This solution is commonly used when there are many documents that should be presented with minimal information on each. This method enables us to get only a portion of the documents at a time, a chunk that could fit into one presentation page (Data Pagination). The user will be able to switch between portions at will and get more detailed information that is relevant to a specific document (Progressive Download) upon demand.

When to use templates

Templates should only be used with data that repeats itself. For example, do not use a template in text or Flash view of an entity.

ViewPoint Web SDK Client Side Architecture

The ViewPoint client side architectures primary responsibility is to provide presentation, navigation and data retrieval services. The server side communicates with the client side provides data retrieval services. The client side architecture is based on Web SDK client building blocks, some of which, such as the application object, are optional and may be omitted in simple implementations.

Application and Controls Architecture

A typical web application is built out of a single application object that references a set of HTML files, as indicated by the * (asterisk) in Figure 2. Each HTML page contains a page control and set of web controls. Some of the web controls can contain other controls. A simple web application, such as a web application that has only one page, can support a workflow with no application object.

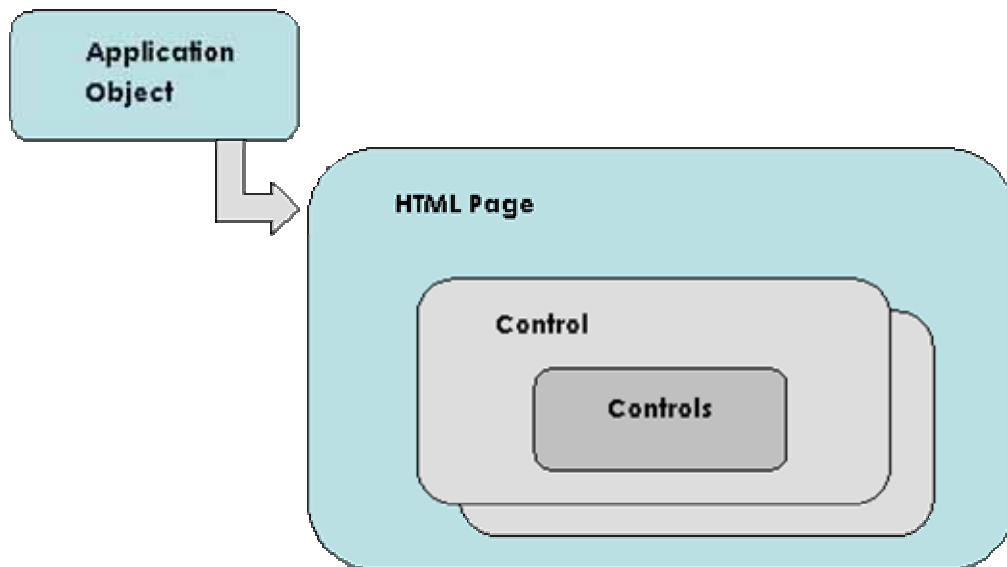


Figure 2: Control Hierarchy - Containment and Reference Relationships

A single Application Object (OwcAppBase) must reside in a static frame, which is always operational and its content cannot be replaced. This Application Object manages all application HTML pages. One page control (OwcPage) is added to each HTML page and references its HTML container, so that it knows its DOM. The page control links to all the Web controls on the same page, which in turn link to the controls they contain and to their parent containers.

This architecture relies heavily on event propagation, where each control fires an event. The same control may catch the fired event or it may propagate along its reference model (container hierarchy). For example, if a control fires an event but does not handle it, the event continues on to its controls container, which may be another control or a page. Each recipient tries to handle the event by checking for an appropriate handler. If it does not find an appropriate handler, it will continue propagating the event up the hierarchy until a page object or an application object handles it and stops the propagation. The handling control may stop the propagation or continue to the next level.

Application Initialization and Event Propagation

A web application that is based on ViewPoint Web SDK performs the following initialization procedure:

1. Creates the applicationobject and calls its initialize function.
2. The application object loads and parses some configuration settings from the server:
 - A. **Settings** as name and value pairs
 - B. **Localization** string labels per language
3. The application object sends an ApplicationLoadedevent when application loading is complete.
4. The page control on each HTML page waits for the ApplicationLoadedevent and then starts loading itself and all its controls.
5. While the page control is loading, it traverses the entire HTML DOM and maps all its controls.
6. The PageLoaded event is sent upon completion of page loading, which may also be caught by the application object (the page objects parent). The application object may choose to interfere in the pages loading process and change its content, such as checking the setting and deciding what type of tree and controls to present.
7. The PageUnloaded event is sent upon unloading the page.

Implementation Notes

- Web pages should be differentiated by using a unique ID in the **Body** HTML tag.
- The JavaScript in each page is not directly connected to the JavaScript on another page. Neighbors (controls) should not communicate, but may be implemented through a common parent in the hierarchy using the event propagation mechanism.

The Application Object

Roles:

The application object implements the applications business logic (BL).

Responsibilities:

- **Manage the applications pages and windows** provides support for application flow operations, such as:
 - Opening and closing pages and windows
 - Data loading, transmission and retrieval

This includes managing the hidden/shown pages loaded in memory, loading new pages, or reloading frames.

- **Response** to user actions
- **Manage APIs** The SDKs only client side gateway to the outside. It is also responsible to provide tools for URL analysis and parsing.
- **Localization support** Provides access to external resources that contain language specific labels and message definitions. A specific application API is responsible for acquiring the relevant resource table, which reside on the server side.
- **Customization (configuration) support** Provides access to server side external configuration files using a dedicated configuration retrieval application API.
- **Log support** lists debug and error information log support.
- **Data filtering** Restricts, limits or minimizes the data retrieval using server side configuration. Some common data filtering uses are listed below:
 - Limit search to specific collections. Documents can be indexed to different collections, so only the configured collections will be searched when searching for documents.
 - Restrict search to a specific tree or folder
 - Mask out specific metadata fields

The client is unaware of these limitations or restrictions, and client operations during runtime do not influence these restrictions.

Implementation Notes:

- An application can have no more than one application object. The application object can be omitted in simple Web applications.
- No server side code is required in a typical Web application. The server side must only be configured as required.

The Page Object

The page object (owcPage) serves as a control container and references to all the controls on the same page. It serves as the event handling parent of these controls and as a communications mediator for between controls when necessary.

Implementation Notes:

One page should not manage any other page directly. This should be managed through the application object.

The SDK Controls

Olive Controls Library is a collection of visual building blocks that are based on ViewPoint Web Controls SDK. It provides an extension to the browser DHTML model and provides a set of out of the box visual controls.

ViewPoint Web Controls SDK is a development framework for general purpose Web controls with built-in data exchange capabilities. The SDK was developed with OOP principles and implements rich event model based architecture. It is highly extensible, customizable, can be easily integrated in any hosting environment with no client side installations and is compatible with all current Web browsers.

The control contacts the application object directly when it needs to:

- Get settings
- Connect to the server.

Events, Commands and Event Handlers

Events are tokens that flow throughout the containment hierarchy. When an event is fired, it can be handled by the control that fired it or by its container page or control. The control that handles the event (using an event handler) may decide to stop the event or continue propagating it throughout the containment hierarchy. The application developer can attach an event handler to a UI action using a direct assignment (see Figure 3).

```
someUIElement.onClick=someControl.someEventHandler
```

Figure 3: Assigning an Event Handler Directly (application level)

Assigning a single event handler is usually sufficient for application development and there is no need for multiple bindings of event handlers, using DHTML_attachEvent and DHTML_detachEvent mechanisms. When multiple event handlers are needed, they are activated in a random order, so don't count on the order of their activation.

The command is a lightweight event handler. It contains only one event handler. Multiple event handlers cannot be attached. It only accepts an event handler with only one parameter. There is no need to define a new event handler when defining a new command.

The following mechanisms are available for event handling:

- **Event handlers** - Instance events treat the event at the control level.
- **Event Bubbling** - Class events throw the event throughout the containment hierarchy.

ViewPoint Web SDK Server Side architecture

The VP server side's primary responsibility is to provide dynamic content and transformation services in the required format. For example, search results or settings

are considered as dynamic data, while HTML files, Help files and images are not dynamic content.

The server side part of the VP Web SDK should be regarded as a black box by most applications developers. No changes should be conducted on the server side in most scenarios. The applications server side configuration files are:

- GetContent.asp
- Global.asa
- Configuration
- JavaScript (*.js or *.inc).
- Common styles (*.xsl files)

URL APIs

The URL APIs enables developer to send by e-mail a link to content within a document or to application favorites. The recipient can then open that link and view the same content with the same format as the sender intended him to see. This option enables the developer to work with a portion of the directory content. Security issues are addressed internally, prohibiting the user from doing the following:

- View documents that are not intended
- Search other documents

The URL APIs provide access to URL parameters that are sent to the applications Web pages. The developer can use them in application development. The SDK controls do not use the URL APIs internally or when transmitting information, such as page to page and control to control.

VP Web SDK addressed the security as follows:

- Provides a dedicated GetContent.asp file for each application
- Can add skin level code with security
- The GetContent.asp file should link to an external authentication code when custom authentication is required
- Provides each application with its own set of parameters

When the client does not pass authentication, the server returns an error with relevant information and a link to a configurable page. User authentication information should be kept in cookies or in an IIS Session object. A session ID may also be sent back and forth between the client and the server during the session lifetime.

The Web application may have multiple entry points, but all should connect to the same application object.

The Data Provider

Every Web application must get its data from some data resource. VP Web SDK provides the developer with a single data supply route using URL APIs (see previous section).

Several ways are available to provide data to the client side:

- Direct supply:
 - Raw data and preferences
 - View data in HTML format
- Redirect to another process, such as to a getImage.dll process for image retrieval.

Lets follow the executed server actions during a simple query to the server. The examples query is for a list of HTML files that should appear according to a custom layout. The servers actions are conducted in this order:

- **Get data** The server retrieves the data in XML format.
- **Format the data** The server transforms the data from XML to the custom layout in a few steps; using layout, metadata and locales configuration settings and custom XSL files manipulations. The VP broadcaster.dll provides transformation services from XML to any other format in a set of consecutive transformations. During the transformation procedures, the DOM Object or localized text may be sent as parameters to the XSL files.
- **Send data** to the client.

Guidelines to Building Web Applications with the Web SDK

General

In a typical Web application implementation scenario, the application developers do not need to write any server side code. Only when custom security support is required, code must be written on the server side, as described in the advanced features paragraph

Configuration

All application specific configuration is done in the **OlvApplication.config.xml** file. This file includes these settings:

- Preferences
- Restrictions
- Localization
- Metadata definitions

Preferences

A set of preferences for the application may be defined with key and value entries for client or server access. All preferences are defined on the server side and may be defined with different scopes:

- **General scope** Preferences that are available for all applications. These preferences are defined in the settings.xml file.
- **Application level** Preferences that are application specific. These preferences are defined in the OlvApplication.config.xml file.

Operators syntax was developed to provide a more comfortable way to define the preferences, by enabling writing short and clear statements. Table 2 describes the available operators syntax.

Operator	Description	Example
OlvVar	Provides a reference to another reference that is already defined.	<pre><Prop name="debug-folder"> \$OlvVar(\$LogFilesDir\Sample) </Prop></pre>
ServerVar	Provides a reference to the IIS variables.	<pre><Prop name="server"> \$ServerVar(SERVER_NAME) </Prop></pre>
MapPath	Converts a logical path to a physical path.	<pre><Prop name="resource-file"> \$MapPath(Resources.xml) </Prop></pre>
Prop	Provides a reference to another reference that resides on the same file.	<pre><Prop name="data-provider"> \$Prop(protocol) : //\$Prop(server) \$ServerVar(URL) </Prop></pre>

Table 2: Operator's Syntax References

Client side preferences are transmitted to the client side, where they can influence the behavior and the layout of the application.

Implementation Notes:	Do not use the references operators syntax when writing preferences for the client side.
------------------------------	--

Restrictions

Provides means to filter data during search operations - For example, the application can be configured to limit search to specific collections. The following procedure takes place when no restrictions are enforced (the part of the restrictions is marked out as comment):

- Check the default style in settings.xml
- Search only in the collections that are defined with that style
- The configuration options available to the application developer are:
- Change the default configuration style
- Provide an accurate list of searchable collections

The list of collections to search may be sent to the client side through URL APIs. When restrictions are defined they get higher priority over this list of collections, resulting in intersections in the restrictions and in the list of collections. This ensures that the client cannot search in restricted collections.

Notes: Additional restrictions will be supported in the future, such as:

- Restricting the search to a specific publication.

- Identifying out of specific metadata fields.

Localization

Access to the **locales** mechanism references language specific external resources. The locales contain language specific label and message sets. The resource tables reside on the server side. Locales hierarchy may be defined as follows:

- The parent locale is defined on the server level.
- Define the application locale.

A custom locale for the application may be defined. The custom locale references an external locale file, defines its parent locale and references it using the preferences mechanism. For example:

```
<Localization language="en">
  <Locale name="english-sample" base="english-ep"
location="$MapPath(Resources.xml)" />
</Localization>
```

Figure 4: Localization Definition

Meta data definitions

Application specific Meta data fields may be defined.

Customizations

How to prepare the application for customization is a key issue. The next sections describe all customization possibilities available to the Web developer.

Data and Metadata Customization

The web application can be customized to use its own set of data by indicating the relevant publications, trees or search criteria. A client of such an application can browse only through the relevant data. Metadata fields can be customized to the clients needs. It may also configure the Metadata fields that are presented and sets the appropriate locale when considering left-to-right and right-to-left languages. These settings can be configured in the `OlvApplication.config.xml` file. Changing application localization is done as follows:

- Copy the application
- Translate to the destination language by changing the HTML files and resources (`OlvApplication.config.xml`)

Help Screens, Logo and Welcome Screens Customization

A Web application, such as FileCabinet or Open University, should be prepared with placeholders for customization options. Help pages, the Logo image or the Welcome pages may be changed by changing a link or an image.

Layout and Visual Style Customization

The visual style of Web SDK based applications may be customized using a custom CSS file. The application is easily customizable with different graphical designs, layouts, colors, fonts, and backgrounds. The developer should never write any styles that are hard coded in the application pages.

Layout and HTML Customization

Using HTML templates, as described in section 1.4.2, provides the developer with out-of-the-box customization capabilities. The developer can customize by enabling or disabling some of the features, with no need for code manipulations. Its purpose is to enable the developer to create a WYSIWYG (What You See Is What You Get) type of UI. These settings are configured in the `OlvApplication.config.xml` file.

Behavior Customization

The application behavior is governed by its configuration. A set of preferences, defaults and other application issues may be configured. For example it is possible to configure how a viewer should be opened based on the type of document being opened, or how search results are presented, as text, thumbnails or snippets.

Customizations of Client Skin Settings

The following customizations are available:

- Virtual directory naming convention.
- Protocol used
- Configure the location of the data provider

Upgrade Procedure

It is commonly recommended to use the following application structure:

- **Application (template) folder** contains the most recent application release.
- **Skin folder** contains only changes and customizations per client.
- **Compiled Skin folder** a merged folder that contains the application with all relevant changes.

The building procedure is then performed as follows:

- Copy the content of the Application folder to the Compiled Skin folder.
- Copy the content of the Skin folder to the Compiled Skin folder. This operation overrides Application folder copy procedure, since the same file naming conventions are used for the files in the Application folder and in the Skin folder.

The three folders should exist on the client installation, making it easy to estimate what an upgrade procedure would cost, and it enables protecting client specific customizations.

The upgrade procedure is based on having a specific application HTML layout. The upgrade procedure may become difficult if the layout of the application is changed.

Building a Simple Web Application

This section demonstrates how to build a simple Web SDK application from start to finish. The sample application has two panels that are controlled by a tab strip with two tabs. One panel lists the documents, and the other panel displays a document using FlashViewer. Clicking a document in the document list (left panel) opens the flash-viewer panel, showing the selected document. You can click the tabs on the tab strips, at the top of the page, to go back and forth between the two panels.

This example is intended to show how easy it is to create a simple application using the Olive Web SDK. Although the full SDK offers a rich collection of controls, each with an extensive API, the basic principles are relatively simple:

1. Make a few adjustments to the HTML page to provide it with access to the Web SDK.
2. Add the appropriate controls as HTML elements, with the appropriate styling through CSS and images.
3. Create styling for dynamic data that comes from the server, through the file Layout.xml.
4. Allow the controls to communicate with each other by attaching JavaScript handler methods to SDK events.

Accessing the SDK and Placing Controls on a Page

Start with a small HTML page that uses the Web SDK, a page with only client-side controls on it: a **TabStrip** with two **TabItems**. To make this page work requires a virtual directory, in which the HTML file is placed, with a subdirectory **/Styles** with the required CSS file **OwcDemo.css** (**/Styles** has a subdirectory **/Images** for background images defined in the CSS file). Here is the complete HTML:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0
Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
xml:lang="en" lang="en"
5  xmlns:owc="http://www.olivesoftware.com/schema/owc.xsd">
  <head>
    <title>Olive Web Components - Demonstration</title>
    <meta http-equiv="Content-Type" content="text/html;
charset=utf-8" />
    <script language="javascript" type="text/javascript"
src="/Olive/WebComponents/OwcBind.js"></script>
10 <link href="Styles/OwcDemo.css" type="text/css"
rel="stylesheet" />
```

```

</head>
<body>
<div owc:Control="TabStrip"          owc:Decorate="true">
  <span owc:Control="TabItem">Document list</span>
15 <span owc:Control="TabItem">Flash viewer</span>
  </div>
</body>
</html>

```

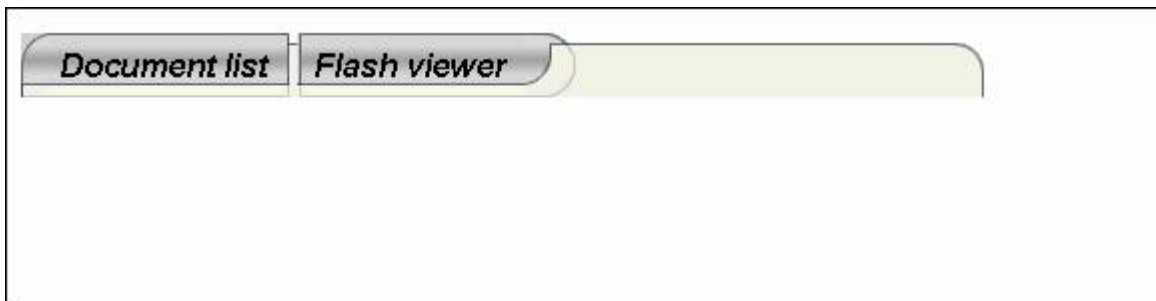


Figure 1: Simple Demo with a Tab Strip - Initial State



Figure 2: Tab Strip after Clicking the First Tab



Figure 3: Tab Strip after Clicking the Second Tab

Even some parts of this small file are unnecessary: the XML prologue (the three lines before the **<html>** root element), the attributes of the **<html>** element, and the **<title>** and **<meta>** elements. These lines should nevertheless be considered as an obligatory part of every SDK-enabled web page. They declare that the page is a valid XHTML page, encoded in Unicode, and using the Olive namespace **owc**. They will be required for forward compatibility with browsers that extend their XHTML support.

Note: The first line causes Internet Explorer 6 to render the page in "quirks mode."

If the Web page has a frameset instead of a body element, use the following DOCTYPE in lines 2-3:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"
```

```
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

The parts of the page that use the Web SDK are lines 9 and 13-16:

- * Line 9 imports the client-side JavaScript files.
- * Lines 13-16 attach SDK controls to the HTML **div** and **span** elements, with custom attributes beginning with the Olive namespace prefix **owc**.

- Note:**
1. The included script **OwcBind.js** includes many required scripts by writing **<script>** elements dynamically with **document.writeIn**.
 2. The custom **owc** attributes are parsed by a handler that is attached to the page "load" HTML event. The parser scans the HTML elements, looks for attributes and elements in the **owc** namespace, and dynamically adds JavaScript objects to the HTML elements, attaches handlers to HTML events, and creates further HTML elements.

Client-Side Controls; "Skinability"

The **TabStrip** control is a typical client-side control. It provides a convenient way to add a frequently-required client-side feature to a web page, the ability to display a horizontal or vertical strip of clickable tabs that change the display or execute a function. This control illustrates a basic design principle of the SDK, separation between style and functionality, so that different versions ("skins") of the same application may be provided by changing only the accessory files (CSS, images, string-tables, configuration files), with no change to the basic HTML and JavaScript files. The basic files can be modified in a single development branch, while the skins can be easily recompiled by recopying the accessory files.

Each tab in a tab strip is implemented as a sequence of three inline elements, representing the left, central, and right part of a tab. The SDK dynamically builds a group of these three elements for each tab, assigning predefined CSS class names to each element. The application developer must provide CSS definitions for these classes, with background images for the various possible positions: left and right edges for the first, middle and last tabs, and a 1-pixel-wide center image that is extended by *background-repeat* and superimposed on the text of the tab. A second version of all these classes displays the tab in a "selected" state, typically with a different color (see Figure 2 and Figure 3). This gives the appearance of fully-styled text without requiring you to create an image file containing text. The same images can then be used for different versions of the page having different texts. This section will not demonstrate how the text "Document list" and "Flash viewer" can be placed in an external file.

- Note:**
1. Creating the **TabStrip** is not as easy as it may appear; the file **OwcDemo.css** and its associated image files have been prepared in advance.
 2. The custom attribute **owc:Decorate="true"** tells the SDK to attach this system of CSS classes to the **TabStrip**. If **owc:Decorate="false"**, the SDK creates a tab strip with no styling.

SDK Events

Client-side controls are important not only because they provide convenient client-side features, but also because their behavior can be seamlessly combined with server-side controls using SDK events. To demonstrate the use of SDK events, make the first tab automatically selected when the page is loaded. This exercise illustrates two points; how to access the API of a control, and how to attach a handler to an appropriate SDK event, in order to execute an API method.

We must use an SDK event, since we cannot attach a handler to the HTML **load** event. When the page is loaded the SDK controls do not yet exist - the custom control elements parsing and creation is itself triggered by the **load** event.

To access the API of a control, we must extend the basic SDK page object. Any page that includes the file **OwcBind.js** automatically has an **OlivePage** object as a property of the window object. It oversees general SDK functionality at the page level. We extend the **OlivePage** object using the steps in the JavaScript below:

1. Declare a constructor class for our page (**DemoPageClass**, in this case).
2. Derive it from the **Olive.Page** class with the function **JScript_DeriveClass()**.
3. Register it as the page class for our page using the method **RegisterPageClass()**.

The implementation details are not important, but as a result of these steps, the built-in **OlivePage** object is extended with custom properties and methods of the **DemoPageClass** class.

We recommend placing JavaScript code in external files in the **/Script** subdirectory. Here we add another **<script>** element to include our file **OwcDemo.js**:

```
<script language="javascript" type="text/javascript"
src="Scripts/OwcDemo.js"></script>
```

After this, insert the following lines into the script file **OwcDemo.js**, extending the built-in page object with a custom method:

```
function DemoPageClass()
{
    this.activateTab = my_ActivateTab;
}
JScript_DeriveClass(DemoPageClass, Olive.Page);
Olive.WindowBinder.RegisterPageClass(window, DemoPageClass);
```

This is the implementation of our method:

```
function my_ActivateTab( nId ) {
    if( "undefined"== typeof nId || nId === null)
    { nId = 0; }
    var oTabStrip = OwcGetControl( "myTabStrip" );
```



```
oTabStrip.setActiveItem( nId );  
}
```

As shown, for a **TabStrip** to be useful, it must have an ID attribute, so that it can be conveniently accessed:

```
<div id="myTabStrip" owc:Control="TabStrip"  
owc:Decorate="true">
```

The method **activateTab()** uses two methods to activate the tab:

4. **OwcGetControl()** is a global utility function that finds the HTML element that has the given id, and returns the control object, a property **m_OwcControl** that is added to the HTML element at the time of SDK parsing. (The control and the HTML element have circular references: the control also has a reference to its HTML element, the property **HtmlElement**.)

5. **setActiveItem()** is a member method of the **TabStrip** control, taking as an argument the tab number (starting from index 0).

For the final step, attach a handler to the **PageLoaded** SDK event. The page fires a **PageLoaded** event when it finishes parsing the custom **owc** attributes and creating all the SDK controls on the page. There are several ways to attach a handler to an event, but the simplest is to define a method with the name of the event plus the prefix "on" (as with HTML event handlers). When an SDK object (page object, in this case) fires an event (such as the **PageLoaded** event), the SDK checks if the object has a method named "on" + {event-name}. It executes that method, if it finds it.

Add the method **onPageLoaded()**, which calls the **activateTab()** method, to the class definition of **DemoPageClass**. Our JavaScript file now looks like this:

```
function DemoPageClass()  
{  
  this.activateTab = my_ActivateTab;  
  this.onPageLoaded = my_PageLoadedHandler;  
}  
JScript_DeriveClass(DemoPageClass, Olive.Page);  
Olive.WindowBinder.RegisterPageClass(window, DemoPageClass);  
function my_ActivateTab( nId ) {  
  if( "undefined"== typeof nId || nId === null )  
  { nId = 0; }  
  var oTabStrip = OwcGetControl( "myTabStrip" );  
  oTabStrip.setActiveItem( nId );  
}  
function my_PageLoadedHandler() {  
  this.activateTab( 0 );  
}
```

Figure 4: Sample demo with PageLoadedhandler, initial state

Event Bubbling

All the SDK controls on a page are organized in a tree hierarchy, just like HTML elements, as illustrated in Figure 5. The root of the control-tree is the optional **Application** object, which can have one or more page child-objects. If a page has no application object, then the page object is the root of the control tree. Within a page, there can be one or more control child-objects, and some controls may contain other controls as nested sub-objects. Each control has references to its parent and child controls, in the property **Parent** and the array property **Controls**.

Figure 5: Typical Controls Tree

The sample page in Figure 6 has a page object as its root, with one child control on the page, a **TabStrip** control. The **TabStrip** has two sub-controls, the two **TabItem** controls.

Figure 6: Diagram of Sample Page

SDK events bubble up the SDK control tree, looking for a handler, just as HTML events bubble up the HTML tree. Once the handler is executed, either the event can continue bubbling up the tree, or the handler can stop the bubbling. So far, the handler was attached directly to the source object (the page object), and event bubbling has not yet been used. In addition, though, every time the **TabStrip** is clicked, it sends an event up the tree to the page. For the **TabStrip** to be useful, the page object must have a handler for the **ItemActivated** event of the **TabStrip**.

Communicating Between Controls with Event-Handlers

Now add another client-side control to the sample page, a control that is typically used with a **TabStrip** to create the tab-strip functionality of changing the display when a tab is clicked. Add a **PageSwitcher** control with two **PageItem** sub-controls below the **TabStrip** with its two **TabItem** sub-controls. A **PageSwitcher** uses only two predefined CSS classes, **Invisible** (usually defined as **display: none**) and **Visible** (usually **display: block**), unlike the more complicated CSS styling of a **TabStrip**. It has an API method of the same name as the **TabStrip**'s method, **setActiveItem()**, which simply makes the selected item **Visible** and any previously selected item **Invisible**.

After adding the **PageSwitcher**, add a handler to the page that receives click events from the **TabStrip** and tell the **PageSwitcher** to activate the appropriate **PageItem**.

The HTML **body** element now has the following sub-elements:

```
<div id="myTabStrip"          owc:Control="TabStrip"
owc:Decorate="true">

  <span owc:Control="TabItem">Document list</span>

  <span owc:Control="TabItem">Flash viewer</span>

</div>

<div id="myPageSwitcher"      owc:Control="PageSwitcher">
```

```

<div owc:Control="PageItem">
<span>Document list goes here</span>
</div>
<div owc:Control="PageItem">
<span>Document in the flash viewer goes here</span>
</div>
</div>

```

The JavaScript file looks like this:

```

function DemoPageClass()
{
this.activateTab = my_ActivateTab;
this.onPageLoaded = my_PageLoadedHandler;
this.onItemActivated = my_ItemActivatedHandler;
}
JScript_DeriveClass(DemoPageClass, Olive.Page);
Olive.WindowBinder.RegisterPageClass(window, DemoPageClass);
function my_ActivateTab( nId ) {
if( "undefined" == typeof nId || nId === null ) {nId = 0;}
var oTabStrip = OwcGetControl( "myTabStrip" );
oTabStrip.setActiveItem( nId );
var oPageSwitcher = OwcGetControl( "myPageSwitcher" );
oPageSwitcher.setActiveItem( nId );
}
function my_PageLoadedHandler() {
this.activateTab( 0 );
}
function my_ItemActivatedHandler( oEvent ) {
if( oEvent.srcObject.getControlType() !=
Olive.Controls.controlTypeNames.TabStrip ){return;}
var nId = oEvent.activeItemIndex;
var oPageSwitcher = OwcGetControl( "myPageSwitcher" );
oPageSwitcher.setActiveItem( nId );
}

```

Figure 7: First Tab Selected

Figure 8: Second Tab Selected

A handler receives data about an event through the argument that is passed (**oEvent**). Each event object has properties that are common to all events, such as **srcObject**, the object that originally fired the event before any bubbling. It also has properties that are specific to its event type, such as **activeItemIndex**, the index of the tab item that was selected. We use the **srcObject** property to check that the event came from a **TabStrip**, because a **PageSwitcher** sends the same kind of event (**ItemActivated**) when an item becomes activated. If the events sent from the **PageSwitcher** were not filtered out, we could conceivably have an infinite loop: A click on the **TabStrip** sends an event, which sets the active page, which sends an event, which sets the active page (...). In reality, the **ItemActivated** event is not sent if an already-selected item is selected. But it is good practice for a handler to check that the event came from the right control before it handles the event.

Server-side Controls

Client-side controls can add convenient functionality to a web page, but the real purpose of the SDK is to provide a simple and flexible way to access document data from the Olive data warehouse. We will now add a document list to the first **PageItem**. This requires small additions to our HTML and JavaScript files, as follows:

HTML - put a **DocList** control inside the first **PageItem**:

```
<div owc:Control="PageItem">
  <div id="myDocList"      owc:Control="DocList"
class="myDocList">
    <span>Document list goes here</span>
  </div>
</div>
```

JavaScript - display the contents of a document folder in the **PageLoaded** handler **my_PageLoadedHandler()**:

```
function my_PageLoadedHandler() {
  this.activateTab( 0 );
  var oDocList = OwcGetControl( "myDocList" );
  oDocList.displayFolderContent( null,
  "OliveUnitTests:||TestFiles" );
}
```

Note the **displayFolderContent()** method of the **DocList** class, which displays the data for the folder specified in the arguments. Here the first argument (**sTreeld**) is null and the second argument (**sFolderId**) gives a complete specification of the folder.

The **** message may be left inside the **DocList** control, since when server-side controls receive write the data from the server into their innerHTML, deleting the previous data. A wait message is first displayed ("**Downloading, please wait...**") and then overwritten by the actual data.

In addition to these familiar additions to HTML and JavaScript, several other steps must be taken to get the web page to request dynamic data from the server:

1. The virtual directory for the site must have an IIS application object, with a standard Olive **global.asp** file that defines Olive server objects.
2. All requests for dynamic data from the server go through one ASP file. It is recommended to take the standard SDK version of **getContent.asp** and place it in the **/Server** subdirectory. Most applications should thus have at least the three subdirectories **/Scripts**, **/Server**, and **/Styles**.
3. The Web page needs to receive the name and location of the ASP file **Server/getContent.asp**. This is done by including a separate script file that defines the string **OwcGlobals.DefaultDataProvider**:

```
HTML:

<script language="javascript" type="text/javascript"
src="Scripts/DemoGlobals.js"></script>

<script language="javascript" type="text/javascript"
src="/Olive/WebComponents/OwcBind.js"></script>

<script language="javascript" type="text/javascript"
src="Scripts/OwcDemo.js"></script>

DemoGlobals.js:

if ( "undefined"== typeof OwcGlobals )
    OwcGlobals = new Object();
if ( !OwcGlobals.DefaultDataProvider )
    OwcGlobals.DefaultDataProvider = "Server/GetContent.asp";
```

4. The server-side controls that take the form of lists or trees of data (document list, search results list, folder tree, and table-of-contents tree) receive multiple data items from the server, so the resulting HTML cannot be formatted directly. Instead, a layout template for a sample data item is provided in the configurable **Layout.xml** file, and the server applies this sample layout for each data item. A template for list headers and footers can also be provided, with placeholders for dynamic data such as pagination or the folder name. A basic version of **Layout.xml** is provided with the SDK, so the document list is viewable even without a custom layout. It is beyond the scope of this introduction to explain the structure of the **Layout.xml** file, but its basic components are as follows:

- A. Custom XML elements that define which (X)HTML section is used for formatting which control or part of a control
- B. Embedded (X)HTML code giving the layout for that part
- C. Embedded custom XML elements in the (X)HTML code, specifying where dynamic data is inserted

Figure 9: Document List Using the Default Layout.xml Template

Figure 10: Document List Using a Custom Layout.xml Template

Event Handlers for Dynamic Data (Content Items)

The last step in this sample application is adding a **FlashViewer** control to the second **PageItem**, so that we can view the selected documents. Adding the viewer control is relatively easy. Simply add a **div** sub-element to the **PageItem**, with the custom attribute **owc:Control="FlashViewer"**

```
<div owc:Control="PageItem">
  <div id="myFlashViewer"      owc:Control="FlashViewer"
    class="myFlashViewer"
    owc:FlashContentOptions="ReaderStyle=OwcDemo">
    <span>Document in the flash viewer goes here</span>
  </div>
</div>
```

The "ReaderStyle" in the FlashContentOptions attribute selects a viewer style that is defined in **FlashReaderSettings.xml** (see below). To use a **FlashViewer** control, add another **<script>** element to include the client-side code. The script **OwcBind.js** includes the general-purpose components that are likely to be used by most applications. Components with large script files that will not be used by some applications are not included by default and must be included separately. For example, since there are two document viewers, a Flash viewer and an HTML viewer, most applications will only use one of them, and will not require downloading both script files.

```
<script language="javascript"      type="text/javascript"
src="/Olive/WebComponents/OwcFlashViewer.js"></script>
```

A FlashViewer control also requires two configuration files. **FlashReaderSettings.xml**, the default name of the first file, provides the numerous fine points of flash viewer layout and behavior, such as the fields to display, colors, and other settings.

FlashLanguage.xml, the default name of the second file as specified in the **FlashReaderSettings.xml** file, supplies the string table for the viewer's menus, buttons, and messages.

Finally, when the page receives the **ContentItemClicked** event, it must notify the Flash viewer which document to open:

```
function DemoPageClass()
{
    //...
    this.onContentItemClicked = my_ContentItemClickedHandler;
    //...
}

function my_ContentItemClickedHandler( oEvent ) {
```

```

        if( oEvent.srcObject.getControlType() !=
            Olive.Controls.controlTypeNames.DocListItem &&

            oEvent.srcObject.getControlType() !=
            Olive.Controls.controlTypeNames.Thumbnail )
        {
            return;
        }

        var oContent = oEvent.OlvContentItem;
        var oFlashViewer = OwcGetControl( "myFlashViewer" );
        oFlashViewer.m_oContentItem = oContent;
        oFlashViewer.displayDocument( true );
        this.activateTab( 1 );
    }

```

Every data-bound control (a control that requests its data from the server) has a property that specifies the data that it requests, **m_oContentItem**. This property has a getter function **getContentItem()**, but no setter function. The content item has a data type, accessible by **getDataType()**, which specifies the type of its content, for example a tree of document folders, a document folder, a document, or a part of a document such as an entity, page, or table-of-contents entry. It also has ID properties specifying the content itself. For example:

- **Data Type** Identifying properties
- **folder tree** tree-id
- **document** tree-id, document-id
- **entity** tree-id, document-id, entity-id

Whenever a data-bound control is clicked, it fires a **contentItemClicked** event, which has a property **OlvContentItem**, in addition to properties of the event base class, such as **srcObject**. A handler in the Page or Application object can forward the content item to another control. In this case, the current content item (**m_oContentItem**) of the FlashViewer is set to the document specified in the **contentItemClicked** event, and then the Flash viewer displays the new current document. The argument **displayDocument()** specifies if to reload the viewer or just go to another page of the current document:

```

oFlashViewer.m_oContentItem = oContent;
oFlashViewer.displayDocument( true );

```

As usual, the handler first checks that the control that was clicked (**oEvent.srcObject**) was the correct control type, either a **DocListItem** or a **Thumbnail** image embedded within a **DocListItem**. If the header of the document list is clicked, however, the function must exit, since the content item of the **contentItemClicked** event specifies the entire document folder, which obviously cannot be displayed by FlashViewer.

What's Next?

The fundamental principles of using the Web SDK are demonstrated here:

- Accessing the SDK from a page
- Placing controls on the page
- Communicating between controls through events.

The Olive Web SDK is designed to allow access to the power of the Olive document warehouse (repository) with relatively simple controls, which can be dropped into any Web page and connected together with a minimal scripting and configuration. You can become more familiar with the SDK by reviewing the lists of controls and their public APIs, and by looking at a more sophisticated sample application, FileCabinet.

The following important topics have not been covered in this introduction:

- The application object, application configuration (**OlvApplication.config.xml**), data limitation and security
- The syntax of layout templates (**Layout.xml**)
- Managing popup windows, cross-window data exchange
- Managing and compiling skins of an application, using external string tables for HTML strings
- Menus for user actions on dynamic data sets, and some of the important action types (print, email)
- Controls for searching and displaying search results

ViewPoint Web SDK Reference Guide

This section includes a description of each module and component in the Olive Web SDK, as listed in the following sections:

[Application](#)

[Calendar](#)

[Command](#)

[Command Button](#)

[Content Item](#)

[Document](#)

[Entity](#)

[Folder](#)

[Folder Tree](#)

[Mail](#)

[Page](#)

[Print](#)

[Search Query](#)

[Search Result](#)

[TOC Entry](#)

[Control](#)

[Data Search Options](#)

[Search Query](#)

[Search Term](#)

[Data Type](#)

[Boolean](#)

[Date](#)

[Day](#)

[Email](#)

[Email List](#)

[Month](#)

[Number](#)

[Object](#)

[Sort By](#)

[String](#)

[Year](#)

[Date](#)

[Day](#)

[Document List](#)

[Document Viewer](#)

[Entity Viewer](#)

[Event Source](#)

[Flash Viewer](#)

[Folder Tree](#)

[Global Functions](#)

[I Active Item](#)

[I Command Item](#)

[I Command Source](#)

[I Complex Value](#)

[I Control Container](#)

[I Control Factory](#)

[I Data Bound](#)

[I Form](#)

[I Form Field](#)

[I Pane](#)

[I Pane List](#)

[I Pane List Item](#)

[I Pane Splitter](#)

[I Search Form Field](#)

[I Selection](#)

[I State](#)

[I UI Elements](#)

[I UI Element Types](#)

[I Value](#)

[Language List](#)

[List](#)

[List Item](#)

[Magnifier](#)

[Mail](#)

[Menu](#)

[Menu Item](#)

[Metadata](#)
[Metadata Field Value](#)
[Metadata Field](#)
[Month](#)
[Page](#)
[Page Item](#)
[Page Switcher](#)
[Pane](#)
[Pane List](#)
[Pane Splitter](#)
[Print List](#)
[Score Bar](#)
[Search Form](#)
[Search in Folder](#)
[Search in Folder Lookup](#)
[Search in Selected Folder](#)
[Search Option](#)
[Search Results](#)
[Search Term](#)
[Sort By](#)
[Tab Item](#)
[Tab Strip](#)
[Thumbnail](#)
[TOC](#)
[Tree](#)
[Tree Node](#)
[UI Element Type](#)
[Tab Item Position](#)
[Tree Box](#)
[Tree Icon](#)
[Tree Node Select](#)
[Tree Outline](#)
[Tree Sub-Nodes](#)
[Value](#)
[Viewer](#)

Web Application Base Class

Year

Application Overview

The Application object serves as a base class for user application class, meaning that all JavaScript code that contains the user application logic is encapsulated in the class derived from the Application class. This class is not created by the HTML element but rather by including the JavaScript file that contains this class.

WebAppBase Class Overview

The WebAppBase JavaScript class implements the Application class' functionality.

Inheritance Hierarchy

OwcEventsSource

OwcCmdTarget

WebAppBase

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[Application Overview](#)

Application Class Events

Application Events

Event	Description
applicationReady	Notify everybody about application ready state change

See Also

[Application Class](#)

Application applicationReady Event

Notify everybody about application ready state change.

Arguments

None

See Also

[WebAppBase Class](#)

Application Class Methods

Application Methods

Method	Description
initialize	???
loadSettings	???
parseSettings	???
parsePreferences	???
parseResources	???
parseMetadata	???
parseWindowOptions	???
isReady	???
onSettingsLoaded	???
getAppUrl	???
getPreference	Get preference value from the Preferences Array
getPreferenceBoolean	Get preference value from the Preferences Array as Boolean
getPreferenceNumber	Get preference value from the Preferences Array as decimal or hexadecimal number
getResString	???
getDefaultValue_MailFrom	???
getControllInitParams	Prepares initialization parameters for the specific control. Called during control's construction. ???
openWindow	Opening application pages
updateRequestParams	Called by control before submitting request to server to allow adding custom URL parameters ???
hookSendRequest	Called by control just before submitting request to server ???
onErrorOccured	???

reportError	???
registerAppCommands	???
addItemToFavorites	???
copyItemToClipboard	???
buildURL	???
parseURL	???
getItemTitle	???
getItemDocTitle	???
enableDebugMode	???

See Also

[Application Class](#)

getPreference Method

Returns the preference value from the array of preferences.

Arguments

sPrefName - This is the preference name string.

vDefault - If the preference array does not contain the requested preference, it returns this default value.

Syntax

```
getPreference(sPrefName, vDefault);
```

See Also

[Application Overview](#) | [WebAppBase Class Overview](#) | [WebAppBase Class Methods](#)

getPreferenceBoolean Method

This method returns the Boolean preference value from the array of preferences. If the stored value does not have the Boolean type, then this method converts it to this one.

Arguments

sPrefName - String that represents the name of the preference

vDefault - Default value is returned if the preference array does not contain the requested preference

Syntax

```
getPreferenceBoolean(sPrefName, vDefault);
```

See Also

[Application Overview](#) | [WebAppBase Class Overview](#) | [WebAppBase Class Methods](#)

getPreferenceNumber Method

Returns the numeric preference value from the array of preferences. If stored value does not have the number type, then method converts it to this one.

Arguments

sPrefName - This string represents the name of the preference.

vDefault - This default value will be returned if the preference array doesn't contain the requested preference.

Syntax

```
getPreferenceNumber(sPrefName, vDefault);
```

See Also

[Application Overview](#) | [WebAppBase Class Overview](#) | [WebAppBase Class Methods](#)

openWindow Method

Uses input parameters to open a window using the `window.open` method, using the last parameter that has the Boolean type.

Arguments

All arguments are described in the MSDN documentation for the *window.open* method.

Syntax

```
openWindow(sURL, sName, sFeatures, bReplace);
```

See Also

[Application Overview](#) | [WebAppBase Class Overview](#) | [WebAppBase Class Methods](#)

Application Class Properties

Inherited Properties

Property	Description
m_arrCommands	Registered commands array

OwcEvents	Array of supported Events
OwcEventHandlers	Array of supported Events Handlers

Application properties

Property	Description
m_oQueryString	Object that contains the Applications Query String ???
m_oCustomRequestParams	???
m_bSettingsLoaded	???
m_arrPreferences	Array of Applications preferences
m_arrStringTable	???
m_sAppUrl	???
DataProviderUrl	Applications Data Provider URL, by default GetContent.asp
m_oBaseWindow	???
m_oOpenedWindows	???
m_nBlankWndCounter	???

See Also

[Application Class](#)

DataProviderUrl Property

Applications Data Provider URL, by default it is the GetContent.asp file.

Syntax

Data Type	String
Default value	GetContent.asp

See Also

[Application Overview](#) | [WebAppBase Class Overview](#) | [WebAppBase Class Properties](#)

m_arrPreferences Property

This internal property represents the associative array that contains the Applications preferences.

Syntax

Data Type	Object (used as Array)
Default value	Empty array

Remarks

This array contains values for each named preference. The access to this array has to be done using the `getPreferenceXXX()` methods.

See Also

[Application Overview](#) | [WebAppBase Class Overview](#) | [WebAppBase Class Properties](#)

Command Control Overview

Command is a convenient way to execute the method of a parent control. The Command control is a nested subcontrol whose command name has been declared in a parent control (a control on an HTML element that is a parent of the command's element). When the Command control is activated (by clicking), the command bubbles up to the first parent control that has a handler defined for the command of that name. For example, a search form can have a nested command control under it with the command name "search" to execute the search; or a tree node can have a command control with the command name "expand" that expands or collapses the node.

A Command control launches the command when the HTML element is clicked. The Command control's HTML element is typically a clickable area or element with an image or text that looks like a button or link. A command can also be launched programmatically with the **doLaunchCommand** method of the Command control.

The command's handler is defined with the **Olive.CmdTarget.RegisterCommand()** function. You can either send commands for which some parent control already has an internally-defined handler, or use this function to define a handler on a page or application object.

A command resembles an event in the fact that it bubbles up, looking for a control that can handle the command. Unlike an event, however, a command's arguments are limited to a string, not a event object. Commands are also usually launched by a user click, whereas events are usually fired programmatically.

In addition to the command name, a Command control also has an attribute specifying the command's parameter(s). The parameters are used as the argument for the method that the command executes.

Syntax

```
<div id=idHtml/ owc:Control=Command owc:Command=/commandName/  
owc:CommandParams=/params/></div>
```

Remarks

This controls JavaScript class does not have events or methods.

See Also

[Command Attributes](#) | [Command Class](#) | [CommandButton Overview](#)

Command HTML Attributes

Command-specific Attributes

Attribute	Description
owc:Command	Name of the command
owc:CommandParams	Parameters of the command

See Also

[Command Overview](#) | [Command Class](#)

owc:Command HTML Attribute

Specifies the name of the Command.

Syntax

```
owc:Command=/sCommand/
```

Remarks

This attribute specifies the name used for the command. The string value of this parameter represents the name of a command that is declared by some control that is on an html parent element of the command control's element.

Example

```
owc:Command="ViewCalendar"
```

Usage

Use	Required
Default value	none

See Also

[Command Overview](#) | [Command Attributes](#) | [Command Class](#)

owc:CommandParams HTML Attribute

Parameters of the command.

Syntax

```
owc:CommandParams=/sParams/
```

Remarks

The string value of this attribute is sent as a parameter to the handler method that the Command executes. It is the handler method's responsibility to recognize the structure of the parameter. For example, the parameter can be one string, or several strings separated by commas.

Example

```
owc:CommandParams=from
```

Usage

Use	Optional
Default value	none

See Also

[Command Overview](#) | [Command Attributes](#) | [Command Class](#)

Command Class Overview

The Command component functionality is implemented by the Olive.Controls.Command JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Command

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[Command Overview](#)

OwcCommand Class Properties

Inherited Properties

Property	Description
----------	-------------

m_sId	Olive Web control unique ID (string)
WebApplication	Reference to the Olive Web application object
WebPage	Reference to the Olive Web page object
Parent	Reference to the parent (container) control
HtmlElement	Reference to the associated HTML element
m_arrControlTypes	Array of registered control types
Controls	Collection of controls (array)

OwcCommand properties

Property	Description
m_sCommand	This property contains the command name.
m_sCommandParams	This property contains the command parameters.

See Also

[Command Overview](#) | Command Class

Command Name

This property defines the name of the command.

Syntax

```
var sCommandName = commandControl.m_sCommand;
```

Sample Code

```
var sCommandName = commandControl.m_sCommand;
```

See Also

[Command Overview](#) | [Command Properties](#) | [Command Class](#)

Command Parameters

This property defines the parameters of the command.

Syntax

```
var sCommandParameters = commandControl.m_sCommandParams;
```

Remarks

This command is called with the `m_sCommandParams` string. The command handler determines the string format, which could contain several values separated by delimiters. It is the command handlers' responsibility to parse the string. For example, the string may contain the values 1 or 2,3 that can be used by the Print command for specifying the pages to be printed.

Sample Code

```
var sCommandParams = commandControl.m_sCommandParams;
```

See Also

[Command Overview](#) | [Command Properties](#) | [Command Class](#)

Command Control Example

The following example illustrates adding a command control to an HTML page. When the user clicks the Home link, the Home Page appears (file Home.htm). This is done by a function in the file **FileCabinetAppBind.js** that is included by this HTML page. This function implements the command.

EXAMPLE 1 - define a command on the page

JAVASCRIPT:

```
function DotPage() {
    this.appendDot = myPage_ appendDot;
    Olive.CmdTarget.RegisterCommand(this, "incrementValue",
    "incrementValue");
}
JScript_DeriveClass(DotPage, Olive.Page);
Olive.WindowBinder.RegisterPageClass(window, DotPage);
function myPage_ appendDot () {
    var sPrevValue = DHTML_getValue( this.oSomeHtmlElem );
    DHTML_setValue( this.oSomeHtmlElem, ( sPrevValue + "." ) );
}
```

HTML:

```
<span owc:Control="CommandButton" owc:Command="incrementValue"
>Increment</span>
```

EXAMPLE 2 - launch a command that is defined internally for a parent control

```
<div owc:Control="Calendar" id="divCalendar1" owc:Format="%A %d %B %Y"
owc:InitialDate="01/01/1999">
```

```
<td><input type="text" owc:Control="Year" size="4" /></td>
```

```

<td><select  owc:Control="Month"></select></td>
<td><select  owc:Control="Day"></select></td>
<td><input  type="button" value="Today"  owc:Control="Command"
owc:Command="Today"  /></td>
</div>

```

See Also

[Command Control](#) | [Command Class](#)

CommandButton Control Overview

The CommandButton control is a control representing a button, whose action is to send a command (see on the Command control).

Syntax

```

<html-element owc:Control = CommandButton owc:Command = /cmd/
owc:CommandParams=/params/>button-text-value
</html-element>

```

Remarks

This control inherits from the Command control and adds visual functionality. It is a command control with a specified visual interface. It allows you to create a button image with no text, and superimpose a text on the image. This makes it possible to change the text (for example, for internationalization) without changing the images.

The visual interface of the control is based on a set of CSS classes for the left, body and right parts of the button (see on the CommandButton CSS Classes).

The left and right classes can have background images that curve to form the ends of an oval button. Assign the URL of the image to the css property background-image for the classes .OwcButtonLeft / .OwcButtonRight.

The body should have a narrow 1-pixel-wide image that will be repeated horizontally to form the center body of the button, with straight sides on the top and bottom. Assign the URL of the narrow image to the property background-image for the classe .OwcButtonBody.

Put the text of the button in the innerHTML of the CommandButton element. This text will be superimposed on the repeated background image of the body.

You do not need to assign the css properties cursor:pointer and background-repeat:repeat-x (for the body image) / background-repeat:no-repeat (for left and right). These are assigned in the default css styles that are included when you include the script OwcBind.js.

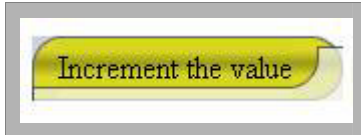
The reserved CSS classes (OwcButton, OwcButtonLeft, OwcButtonBody, OwcButtonRight) can be defined once for a page or application to give a uniform look to all buttons.

Alternatively, to give different looks to different buttons, use CSS specifiers with parent element ID's:

```
#button1 .OwcButtonLeft
{
    width: 10px;
    height: 22px;
    background-image: url(Images/btn_Left_Small_Green.gif);
    vertical-align: middle;
}
#button2 .OwcButtonLeft
{
    width: 13px;
    height: 32px;
    background-image: url(Images/ btn_Left_Big_Purple.gif);
    vertical-align: middle;
}
... (similarly for .OwcButton, .OwcButtonBody, .OwcButtonRight)
```

This control does not have its own attributes, and its JavaScript class does not have events, methods or properties.

Preview



See Also

[OwcButtonControl Class](#) | [CommandButton CSS Classes](#) | [Command Overview](#) | [Command Attributes](#)

CommandButton Class Overview

The CommandButton component functionality is implemented by the Olive.Controls.CommandButton JavaScript class. The Olive.Controls.CommandButton interface allows access to the CommandButton component properties and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Command

Olive.Controls.CommandButton

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[CommandButton Overview](#)

CommandButton Class Properties

This class contains only inherited properties.

See Also

[Command Control Overview](#) | [CommandButton Class](#)

CommandButton Control CSS Classes

CSS Class	Description
OwcButton	CSS class, general for the whole button
OwcButtonLeft	CSS class for the left part of the button
OwcButtonBody	CSS class for the body part of the button (for an image with 1px width)
OwcButtonRight	CSS class for the right part of the button

See Also

[CommandButton Overview](#) | [Command Overview](#)

CommandButton Control Example

The following example illustrates adding a CommandButton control to an HTML page. When the user clicks the Sendbutton, the SendMail command is launched.

```
HTML:
<span title="Send" owc:Control="CommandButton" owc:Command="SendMail">Send</span>
CSS CLASSES:
.OwcButton
{
    line-height: 22px;
    height: 23px;
    vertical-align: middle;
}
.OwcButtonLeft
```

```

{
    width: 10px;
    height: 22px;
    background-image: url(Images/btn_LeftSide.gif);
    vertical-align: middle;
}
.OwcButtonRight
{
    width: 10px;
    height: 22px;
    background-image: url(Images/btn_RightSide.gif);
    vertical-align: middle;
}
.OwcButtonBody
{
    height: 22px;
    background-image: url(Images/btn_Body.gif);
    text-decoration: none;
    vertical-align: middle;
}
.OwcButtonBody:hover
{
    color: blue;
    text-decoration: underline;
}

```

See Also

[CommandButton Overview](#) | [CommandButton Class](#) | [CommandButton CSS Classes](#)

ContentItem Overview

The ContentItem class is the base class for other ContentItem classes:

ContentItem Class	Description
Document	
Entity	

Folder	
FolderTree	
Mail	
Page	
Print	
SearchQuery	
SearchResult	
TocEntry	

See Also

[ContentItem Class](#)

ContentItem Class Overview

The full name for the class is `Olive.ContentItem`, which is the base class for other types of content items.

Usage

`Olive.ContentItem` class is an abstract class. It is the base functionality of its inheritors.

Following is general for content items, inheritors of the `Olive.ContentItem` class:

- Using of `owc:Data` HTML tag inside of HTML elements of SDK controls, that implement `Olive.Controls.IDataBound` interface, involves using of content item classes.
- Use `owc:Data` tag with corresponding attributes to use functionality of classes inherited from this `ContentItem` class. The main functionality of such classes is to send a request for some data to the server and keep information for following requests.

Requirements

Script Files	<code>OwcBind.js</code>
---------------------	-------------------------

See Also

[ContentItem Methods](#) | [ContentItem Properties](#) | [ContentItem Overview](#)

ContentItem Class Methods

ContentItem Methods

Method	Description
buildQueryString	Builds query string
contentCanLoad	Checks if the content can load

See Also

[ContentItem Class](#) | [ContentItem Properties](#)

buildQueryString Method

This method builds a query string with the necessary parameters to request from the server.

Parameters

No parameters

Syntax

```
oContentItem.buildQueryString();
```

Example

```
// get query string and use it for desired needs  
var sQueryStr = oContentItem.buildQueryString();
```

See Also

[ContentItem Class](#) | [ContentItem Methods](#)

contentCanLoad Method

The method checks whether the content can be loaded. Returns Boolean indicated the result of the checking.

Parameters

No parameters

Syntax

```
oContentItem.contentCanLoad();
```

Example

```
if (oContentItem.contentCanLoad() )
{
    oControl.contentLoad();
}
```

See Also

[ContentItem Class](#) | [ContentItem Methods](#)

ContentItem Class Properties

ContentItem properties

Property	Description
DataObjectType	Holds the type of content item

See Also

[ContentItem Class](#) | [ContentItem Methods](#)

DataObjectType Property

String value that holds the type of the content item

Syntax

Data Type	String
Value	null

Remarks

The property for every specific content item defines the type of the content item object. The property value is null for base class Olive.ContentItem.

Sample Code

```
var sDataType = oContentItem.DataObjectType;
```

See Also

[ContentItem Properties](#) | [ContentItem Class](#)

ContentItem Example

ContentItem class is a base class for its inheritors and cant be used as is. See examples of classes inherited from the ContentItem class.

See Also

[ContentItem Class](#)

ContentItem.Document Overview

Inheritance Hierarchy

ContentItem.Document Attributes	
ContentItem.Document Class	
ContentItem.Document Methods	
ContentItem.Document Properties	

See Also

[ContentItem Overview](#)

ContentItem.Document HTML Attributes

Attributes inherited from ContentItem class

Attribute	Description
type	Specifies a type of content item, equals to Document. Mandatory attribute

ContentItem.Document-specific Attributes

Attribute	Description
doc-href	DocHref of the document
doc-uid	The document's ID, as registered in Inventory
doc-cid	The document's ID, from its TOC.xml
doc-ver	The publication's Version number

Remarks

This attribute **type** is mandatory and set to Document.

It is mandatory to use at least one attribute **doc-href**, **doc-uid**, or **doc-cid**.

See Also

[ContentItem.Document Class](#)

doc-cid HTML Attribute

This attribute specifies a document ID in its TOC.xml file.

Syntax

```
<owc:Data type=Document doc-cid=/string/ />
```

Remarks

This is a required attribute, if doc-href and doc-uid are not defined. The value of the attribute must be a string with an the document's ID, as defined in its TOC.xml file.

```
<owc:Data type="Document" doc-cid =Tests:1 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.Document Attributes](#) | [ContentItem.Document Class](#)

doc-href HTML Attribute

This attribute specifies a BaseHref of the document.

Syntax

```
<owc:Data type=Document doc-href=/string/ />
```

Remarks

This is a required attribute, if doc-uid and doc-cid are not defined. The value of the attribute must be in the Olive format of BaseHref

Example

```
<owc:Data type="Document" doc-href=UnitTests/2004/04/05 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.Document Attributes](#) | [ContentItem.Document Class](#)

doc-uid HTML Attribute

This attribute specifies the document's ID in Inventory.

Syntax

```
<owc:Data type=Document doc-uid=/string/ />
```

Remarks

This attribute is required if doc-href and doc-cid are not defined. Its value attribute must be a string with a number in it.

```
<owc:Data type="Document" doc-uid =1 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.Document Attributes](#) | [ContentItem.Document Class](#)

doc-ver HTML Attribute

This attribute specifies the document version.

Syntax

```
<owc:Data type=Document doc-ver=/string/ />
```

Remarks

This is an optional attribute. Its value must be a string with the version number.

```
<owc:Data type="Document" doc-href =UnitTests/2004/04/05 doc-ver =2 />
```

Usage

HtmlElement	owc:Data
Use	Optional

Default value	no
----------------------	----

See Also

[ContentItem.Document Attributes](#) | [ContentItem.Document Class](#)

type HTML Attribute

This attribute specifies the content item type.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This is a required attribute. It must be set to **Document** for ContentItem.Document class.

Example

```
<owc:Data type="Document" />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.Document Attributes](#) | [ContentItem.Document Class](#)

ContentItem.Document Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.Document

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[ContentItem.Document Methods](#) | [ContentItem.Document Properties](#) | [ContentItem.Document Attributes](#)

ContentItem.Document Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds query string
contentCanLoad	Checks whether the content can load

See Also

[ContentItem.Document Class](#) | [ContentItem.Document Properties](#)

ContentItem.Document Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the type of the content item. It is equal to Document for ContentItem.Document class

See Also

[ContentItem.Document Class](#) | [ContentItem.Document Methods](#)

ContentItem.Document Example

The examples below describe how to use the ContentItem.Document class in the user application.

```
// example for Thumbnail control
<div owc:Control="Thumbnail">
  <owc:Data type="Document" doc-href="UnitTests/1600/01/01">
  </owc:Data>
</div>

// example for FlashViewer control
<div owc:Control="FlashViewer">
  <owc:Data type="Document" doc-uid="1">
  </owc:Data>
</div>
```

```
// example for Metadata control
<div owc:Control="Metadata">
  <owc:Data type="Document " doc-cid="Tests:1">
    </owc:Data>
    <table>
      <tr>
        <td>
          <table>
            <tr owc:Control="MetaField" owc:Field="dc:title"
owc:DisplayName="Title:" owc:ShowIf="exists">
              <td owc:Control="Title"></td>
              <td>
                <div owc:Control="Value" owc:Separator="<br/>">
                  </div>
                </td>
              </tr>
            </table>
          </td>
        </tr>
      </table>
    </div>
  </div>
```

See Also

[ContentItem.Document Class](#)

ContentItem.Entity Overview

Inheritance Hierarchy

[ContentItem.Entity Attributes](#)

[ContentItem.Entity Class](#)

[ContentItem.Entity Methods](#)

[ContentItem.Entity Properties](#)

See Also

[ContentItem Overview](#)

ContentItem.Entity HTML Attributes

Attributes inherited from [ContentItem](#) class

Attribute	Description
type	Specifies a type of content item, equals to Entity. Mandatory attribute

Attributes inherited from [ContentItem.Document](#) class

Attribute	Description
doc-href	DocHref of the document
doc-uid	The document's ID as registered in Inventory
doc-cid	The document's ID from its TOC.xml
doc-ver	Version number of the publication

ContentItem.Entity-specific Attributes

Attribute	Description
entityId	ID of the entity

Remarks

The attribute type must be used mandatory and set to Entity

It is mandatory to use at least one attribute from doc-href, doc-uid, doc-cid.

See Also

[ContentItem.Entity Class](#)

entityId HTML Attribute

This attribute specifies the entity's ID.

Syntax

```
<owc:Data type=Entity doc-uid =/string/ entityId=/string/ />
```

Remarks

This is a required attribute. One of the doc-href, doc-uid or doc-cid are also required .
The value of the attribute must be in the Olive format of EntityId

Example

```
<owc:Data type="Entity" doc-uid=5 entityId=Ar00100 />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.Entity Attributes](#) | [ContentItem.Entity Class](#) | [ContentItem.Document Attributes](#)

type HTML Attribute

Specifies a type of content item.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This is a required attribute. For ContentItem.Entity class the attribute must be set to Entity

Example

```
<owc:Data type="Entity" />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.Entity Attributes](#) | [ContentItem.Entity Class](#)

ContentItem.Entity Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.Entity>

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[ContentItem.Entity Methods](#) | [ContentItem.Entity Properties](#) | [ContentItem.Entity Attributes](#)

ContentItem.Entity Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds a query string
contentCanLoad	Checks whether the content can load

See Also

[ContentItem.Entity Class](#) | [ContentItem.Entity Properties](#)

ContentItem.Entity Class Properties

Properties inherited from [Olive.ContentItem](#) class

Property	Description
DataObjectType	Holds the type of the content item. It is equal to Entity for ContentItem.Entity class

See Also

[ContentItem.Entity Class](#) | ContentItem.Entity Methods

ContentItem.Entity Example

The examples below describe the using of the ContentItem.Entity class in the user application.

```
// example for EntViewer control
<div owc:Control="EntViewer">
  <owc:Data type="Entity" doc-href="UnitTests/1600/01/01"
entityId=Ar00200></owc:Data>
</div>
```

```
// example for Metadata control
<div owc:Control="Metadata">
  <owc:Data type="Entity" doc-uid="1" entityId=Ar00301>
  </owc:Data>
  <table>
  <tr>
  <td>
  <table>
  <tr owc:Control="MetaField" owc:Field="dc:title"
owc:DisplayName="Title:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
  <div owc:Control="Value" owc:Separator="<br/>">
  </div>
  </td>
  </tr>
  </table>
  </td>
  </tr>
  </table>
</div>
```

See Also

[ContentItem.Entity Class](#)

ContentItem.Folder Overview

Inheritance Hierarchy

ContentItem.Folder Attributes	
ContentItem.Folder Class	
ContentItem.Folder Methods	
ContentItem.Folder Properties	

See Also

[ContentItem Overview](#)

ContentItem.Folder HTML Attributes

Attributes inherited from [ContentItem](#) class

Attribute	Description
type	Specifies a type of content item, equals to Folder.

Attributes inherited from [ContentItem.FolderTree](#) class

Attribute	Description
tree-uid	The FolderTree's ID in Inventory

ContentItem.Folder-specific Attributes

Attribute	Description
folder-uid	The folder node's ID in the FolderTree
title	Title of the folder node

Remarks

The Folder content item is not used as other content items, such as Document, Entity, FolderTree, and others. The server usually sends **<owc:Data .. >** of the Folder type.

All folder-specific attributes are set at the server. The values of the attributes can only be received, but not changed.

See Also

[ContentItem.Folder Class](#)

folder-uid HTML Attribute

This attribute specifies the folder node's ID in the Folder Tree.

Syntax

```
<owc:Data type=Folder  folder-uid=/string/ />
```

Remarks

This attribute is set at the server.

```
<owc:Data type="Folder" folder-uid =5 />
```

Usage

HtmlElement	owc:Data
-------------	----------

Use	Optional
Default value	no

See Also

[ContentItem.Folder Attributes](#) | [ContentItem.Folder Class](#)

title HTML Attribute

This attribute specifies the folder node's title in the FolderTree.

Syntax

```
<owc:Data type=Folder title =/string/ />
```

Remarks

This attribute is set at the server.

```
<owc:Data type="Folder" title=Test Files />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.Folder Attributes](#) | [ContentItem.Folder Class](#)

type HTML Attribute

This attribute specifies the type of content item.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This is a required attribute for the ContentItem.Folder class the attribute, and must be set to Folder

Example

```
<owc:Data type="Folder" />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.Folder Attributes](#) | [ContentItem.Folder Class](#)

ContentItem.Folder Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.Folder

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[ContentItem.Folder Methods](#) | ContentItem.Folder Properties | [ContentItem.Folder Attributes](#)

ContentItem.Folder Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds query string
contentCanLoad	Checks whether the content can load

See Also

[ContentItem.Folder Class](#) | ContentItem.Folder Properties

ContentItem.Folder Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the type of the content item. It is equal to Folder for ContentItem.Folder class

See Also

[ContentItem.Folder Class](#) | [ContentItem.Folder Methods](#)

ContentItem.Folder Example

ContentItem.Folder is not used as other content items like Document, Entry, FolderTree, etc. You usually get <owc:Data .. > of the Folder type from the server.

```
// part of html received from the server for FolderTree
<div owc:Control="FolderTree">
  <owc:Data type="TreeLayout">
    <owc:NodeType name="Folder" cssClass="folderNode"
cssClassSelected="folderNodeSelected" />
  </owc:Data>
</div>

<owc:data type='FolderTreeData' displayName='Tree for unit
testing'></owc:data>

<table owc:Control="Folder">
<tr>
<td owc:ui="treeBox"></td>
<td owc:ui="treeNodeSelectableArea">
<span owc:ui="treeIcon"></span>
<span class="folderTitle">Test Newspapers</span>
<owc:Data type="Folder" tree-uid="2" folder-uid="2" title="Test
Newspapers"></owc:Data>
</td>
</tr>
</table>

<table owc:Control="Folder">
<tr>
<td owc:ui="treeBox"></td>
<td owc:ui="treeNodeSelectableArea">
<span owc:ui="treeIcon"></span>
<span class="folderTitle">Test Files</span>
<owc:Data type="Folder" tree-uid="2" folder-uid="3" title="Test
Files"></owc:Data>
</td>
</tr>
<tr owc:ui="treeSubNodes">
<td owc:ui="treeOutline"></td>
```

```
<td owc:ui="treeSubNodesTarget">
<table owc:Control="Folder">
<tr>
<td owc:ui="treeBox"></td>
<td owc:ui="treeNodeSelectableArea">
<span owc:ui="treeIcon"></span>
<span class="folderTitle">Test Books</span>
<owc:Data type="Folder" tree-uid="2" folder-uid="4" title="Test
Books"></owc:Data>
</td>
</tr>
</table>
</td>
</tr>
</table>
</div>
</div>
```

See Also

[ContentItem.Folder Class](#)

ContentItem.FolderTree Overview

Inheritance Hierarchy

ContentItem.FolderTree Attributes	
ContentItem.FolderTree Class	
ContentItem.FolderTree Methods	
ContentItem.FolderTree Properties	

See Also

[ContentItem Overview](#)

ContentItem.FolderTree HTML Attributes

Attributes inherited from [ContentItem](#) class

Attribute	Description
type	Specifies a type of content item, equals to the FolderTree. Mandatory attribute

ContentItem.FolderTree-specific Attributes

Attribute	Description
tree-uid	The FolderTree's ID in the Inventory
tree-name	Folder Tree name

Remarks

The attribute type is mandatory and must be set to the FolderTree.

It is mandatory to use at least one attribute from **tree-uid** or **tree-name**. The **tree-uid** attribute has higher priority.

See Also

[ContentItem.FolderTree Class](#)

tree-name HTML Attribute

This attribute specifies the name of the FolderTree.

Syntax

```
<owc:Data type=FolderTree tree-name =/string/ />
```

Remarks

This attribute is mandatory if **tree-uid** is not defined. Its value must be a string with the Folder Tree's name.

```
<owc:Data type="FolderTree" tree-name =OliveUnitTests />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.FolderTree Attributes](#) | [ContentItem.FolderTree Class](#)

tree-uid HTML Attribute

This attribute specifies an ID for the Folder tree in the Inventory.

Syntax

```
<owc:Data type=FolderTree tree-uid=/string/ />
```

Remarks

This attribute is mandatory if **tree-name** is not defined. Its value is a number string.

```
<owc:Data type="FolderTree" tree-uid =1 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.FolderTree Attributes](#) | [ContentItem.FolderTree Class](#)

type HTML Attribute

This attribute specifies a type of content item.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This attribute is mandatory and must be set to FolderTree for the ContentItem.FolderTree class

Example

```
<owc:Data type="FolderTree" />
```

Usage

HTML Element	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.FolderTree Attributes](#) | [ContentItem.FolderTree Class](#)

ContentItem.FolderTree Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.FolderTree

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ContentItem.FolderTree Methods](#) | [ContentItem.FolderTree Properties](#) | [ContentItem.FolderTree Attributes](#)

ContentItem.FolderTree Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds query string
contentCanLoad	Checks whether the content can load

See Also

[ContentItem.FolderTree Class](#) | [ContentItem.FolderTree Properties](#)

ContentItem.FolderTree Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the type of the content item. It is equal to FolderTree for ContentItem.FolderTree class

See Also

[ContentItem.FolderTree Class](#) | [ContentItem.FolderTree Methods](#)

ContentItem.FolderTree Example

The example below describe how to use the ContentItem.FolderTree class in the user application.

```
// example for FolderTree control
<div owc:Control="FolderTree">
  <owc:Data type="FolderTree" tree-uid="2">
  </owc:Data>
</div>
```

See Also

[ContentItem.FolderTree Class](#)

ContentItem.Mail Overview

Inheritance Hierarchy

[ContentItem.Mail Attributes](#)

[ContentItem.Mail Class](#)

[ContentItem.Mail Methods](#)

[ContentItem.Mail Properties](#)

See Also

[ContentItem Overview](#)

ContentItem.Mail HTML Attributes

Attributes inherited from ContentItem class

Attribute	Description
type	Specifies a type of content item, equals to Mail.

Remarks

The ContentItem.Mail class represents data that the server defines for a control that sends a mail request to the server.

See Also

[ContentItem.Mail Class](#)

type HTML Attribute

This attribute specifies the content item type.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This attribute is mandatory. It must be set to **Mail** for the **ContentItem.Mail** class.

Example

```
<owc:Data type="Mail" />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.Mail Attributes](#) | [ContentItem.Mail Class](#)

ContentItem.Mail Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.Mail

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[ContentItem.Mail Methods](#) | [ContentItem.Mail Properties](#) | [ContentItem.Mail Attributes](#)

ContentItem.Mail Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds a query string
contentCanLoad	Checks if the content can load

See Also

[ContentItem.Mail Class](#) | [ContentItem.Mail Properties](#)

ContentItem.Mail Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the content item type. It is equal to Mail for ContentItem.Mail class

See Also

[ContentItem.Mail Class](#) | [ContentItem.Mail Methods](#)

ContentItem.Mail Example

Mail is used differently to other content items like Document, Entry, and Page. It only forms a request to the server, without using following requests, as may be done with other content items. The mail control does not need to use a <owc:Data ..> HTML element, since everything is implemented by the Mail control and **IDataBound** interface.

To implement it, simply put the [Mail control](#) on the HTML page and use the **SendMail** command to send a request to the server. Refer to the [Mail control example](#) for more information.

See Also

[ContentItem.Mail Class](#) | [Mail Overview](#)

ContentItem.Page Overview

Inheritance Hierarchy

ContentItem.Page Attributes	
ContentItem.Page Class	
ContentItem.Page Methods	
ContentItem.Page Properties	

See Also

[ContentItem Overview](#)

ContentItem.Page HTML Attributes

Attributes inherited from [ContentItem](#) class

Attribute	Description
type	Specifies the content item type, equals to Page. Mandatory attribute

Attributes inherited from [ContentItem.Document](#) class

Attribute	Description
doc-href	The document's DocHRef
doc-uid	The document's ID as registered in Inventory
doc-cid	The document's ID from its TOC.xml
doc-ver	The publication's version number

ContentItem.Page-specific Attributes

Attribute	Description
page-no	The number of the page
label	Label of the page

Remarks

The attribute type is mandatory and set to Page.

It is mandatory to use at least one attribute from **doc-href**, **doc-uid**, and/or **doc-cid**.

See Also

[ContentItem.Page Class](#)

label HTML Attribute

This attribute specifies the page label.

Syntax

```
<owc:Data type=Page doc-uid =/string/ label=/string/ />
```

Remarks

This attribute is mandatory if the **page-no** attribute is not defined. At least one attribute of the **doc-href**, **doc-uid** or **doc-cid** are also required . Its value must be a string with a page label.

Example

```
<owc:Data type="Page" doc-uid=5 label=A1 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.Page Attributes](#) | [ContentItem.Page Class](#) | [ContentItem.Document Attributes](#)

page-no HTML Attribute

This attribute specifies the page number.

Syntax

```
<owc:Data type=Page doc-uid =/string/ page-no=/string/ />
```

Remarks

This attribute is mandatory if the label attribute is not defined. At least one of the attributes **doc-href**, **doc-uid** or **doc-cid** is also required. Its value must be a number string.

Example

```
<owc:Data type="Page" doc-uid=5 page-no=4 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.Page Attributes](#) | [ContentItem.Page Class](#) | [ContentItem.Document Attributes](#)

type HTML Attribute

This attribute specifies a type of content item.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This attribute is mandatory and must be set to **Page** for the ContentItem.Page class.

Example

```
<owc:Data type="Page" />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.Page Attributes](#) | ContentItem.Page Class

ContentItem.Page Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.Page

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ContentItem.Page Methods](#) | [ContentItem.Page Properties](#) | [ContentItem.Page Attributes](#)

ContentItem.Page Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds query string
contentCanLoad	Checks whether the content can load

See Also

[ContentItem.Page Class](#) | [ContentItem.Page Properties](#)

ContentItem.Page Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the content item type, which is equal to Page for ContentItem.Page class

See Also

[ContentItem.Page Class](#) | [ContentItem.Page Methods](#)

ContentItem.Print Overview

Inheritance Hierarchy

ContentItem.Print Attributes	
ContentItem.Print Class	
ContentItem.Print Methods	
ContentItem.Print Properties	

See Also

[ContentItem Overview](#)

ContentItem.Print HTML Attributes

Attributes inherited from [ContentItem](#) class

Attribute	Description
type	Specifies a type of content item, equals to PrintList.

Remarks

ContentItem.Print class defines a control's server-content, where it sends the server a print request.

See Also

[ContentItem.Print Class](#)

type HTML Attribute

This attribute Specifies the content item type.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This attribute is mandatory and must be set to **PrintList** for the ContentItem.Print class.

Example

```
<owc:Data type="PrintList" />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.Print Attributes](#) | ContentItem.Print Class

ContentItem.Print Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.Print

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ContentItem.Print Methods](#) | ContentItem.Print Properties | [ContentItem.Print Attributes](#)

ContentItem.Print Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds query string

contentCanLoad	Checks whether the content can load
-----------------------	-------------------------------------

See Also

[ContentItem.Print Class](#) | [ContentItem.Print Properties](#)

ContentItem.Print Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the type of the content item. It is equal to PrintList for ContentItem.Print class

See Also

ContentItem.Print Class | [ContentItem.Print Methods](#)

ContentItem.Print Example

ContentItem.Print is used differently to other content items like Document, Entity, Page etc. It only forms a request to the server, without following requests, as is possible for other content items.

```
<div owc:Control="PrintList">
  <owc:data type=printlist></owc:data>
</div>
```

See Also

ContentItem.Print Class | [PrintList Overview](#)

ContentItem.SearchQuery Overview

Inheritance Hierarchy

ContentItem.SearchQuery Class	
ContentItem.SearchQuery Methods	
ContentItem.SearchQuery Properties	

See Also

[ContentItem Overview](#)

ContentItem.SearchQuery Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.SearchQuery

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ContentItem.SearchQuery Methods](#) | [ContentItem.SearchQuery Properties](#)

ContentItem.SearchQuery Class Methods

Methods inherited from [Olive.ContentItem](#) class

Method	Description
buildQueryString	Builds a query string
contentCanLoad	Checks if the content can load

See Also

[ContentItem.SearchQuery Class](#) | [ContentItem.SearchQuery Properties](#)

ContentItem.SearchQuery Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the content item type, which is equal to SearchQuery for the ContentItem.SearchQuery class

See Also

ContentItem.SearchQuery Class | [ContentItem.SearchQuery Methods](#)

ContentItem.SearchQuery Example

This ContentItem.SearchQuery is not used by the <owc:Data > tag.



See Also

[ContentItem.SearchQuery Class](#)

ContentItem.SearchResults Overview

Inheritance Hierarchy

ContentItem.SearchResults Attributes	
ContentItem.SearchResults Class	
ContentItem.SearchResults Methods	
ContentItem.SearchResults Properties	

See Also

[ContentItem Overview](#)

ContentItem.SearchResult HTML Attributes

Attributes inherited from [ContentItem](#) class

Attribute	Description
type	Specifies a content item type, equals to SearchResult. Mandatory attribute

Attributes inherited from [ContentItem.Document](#) class

Attribute	Description
doc-href	The document's DocHRef
doc-uid	The document's ID as registered in Inventory
doc-cid	The document's ID from its TOC.xml
doc-ver	The publication's Version number

Attributes inherited from [ContentItem.Entity](#) class

Attribute	Description
entityId	The entity's ID

ContentItem.SearchResult-specific Attributes

Attribute	Description
page-no	Page number for the search result

primId	ID of the first primitive on the page with search results
score	Percentage score of the search result
collection	Collection that was used to index the document with search results

Remarks

SearchResult is used differently from other content items like Document, Entry, and Page. The server usually sends **<owc:Data .. >** of the SearchResult type.

All SearchResult-specific attributes are set at the server. It is possible only to get attribute values, but not to change them.

See Also

[ContentItem.SearchResult Class](#)

collection HTML Attribute

This attribute specifies the collection name that is used to index the document with search results.

Syntax

```
<owc:Data type=SearchResult collection=/string/ />
```

Remarks

This attribute is formed at the server.

Example

```
<owc:data type="SearchResult" collection=UnitTests />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.SearchResult Attributes](#) | [ContentItem.SearchResult Class](#)

page-no HTML Attribute

This attribute specifies the received search result's page number.

Syntax

```
<owc:Data type=SearchResult page-no=/number/ />
```

Remarks

This attribute is formed at the server.

Example

```
<owc:data type="SearchResult" page-no=5 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.SearchResult Attributes](#) | [ContentItem.SearchResult Class](#)

primId HTML Attribute

This attribute specifies the ID of the first primitive on the search results page.

Syntax

```
<owc:Data type=SearchResult primId=/string/ />
```

Remarks

This attribute is formed at the server.

Example

```
<owc:data type="SearchResult" primId=Ar0050000 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.SearchResult Attributes](#) | [ContentItem.SearchResult Class](#)

score HTML Attribute

This attribute specifies the search result score.

Syntax

```
<owc:Data type=SearchResult  score=/number/ />
```

Remarks

This attribute is formed at the server.

Example

```
<owc:data type="SearchResult" score=100 />
```

Usage

HtmlElement	owc:Data
Use	Optional
Default value	no

See Also

[ContentItem.SearchResult Attributes](#)

[ContentItem.SearchResult Class](#)

type HTML Attribute

This attribute specifies the content item type.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This attribute is mandatory and must be set to **SearchResult** for the ContentItem.SearchResult class.

Example

```
<owc:Data type="SearchResult" />
```

Usage

HtmlElement	owc:Data
Use	Required

Default value	no
---------------	----

See Also

[ContentItem.SearchResult Attributes](#) | [ContentItem.SearchResult Class](#)

ContentItem.SearchResult Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.Entity

`Olive.ContentItem.SearchResult`

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ContentItem.SearchResult Methods](#) | [ContentItem.SearchResult Properties](#) | [ContentItem.SearchResult Attributes](#)

ContentItem.SearchResult Class Methods

Methods inherited from [Olive.ContentItem](#) class

Method	Description
buildQueryString	Builds a query string
contentCanLoad	Checks whether the content can load

See Also

[ContentItem.SearchResult Class](#) | [ContentItem.SearchResult Properties](#)

ContentItem.SearchResult Class Properties

Properties inherited from **Olive.ContentItem** class

Property	Description
DataObjectType	Holds the type of the content item. It is equal to SearchResult for ContentItem.SearchResult class

See Also

[ContentItem.SearchResult Class](#) | [ContentItem.SearchResult Methods](#)

ContentItem.SearchResult Example

SearchResult is used differently from other content items like Document, Entry, and Page. The server usually sends the **<owc:Data .. >** of the SearchResult type.

```
// html for SearchResItem control, build at the server
<div owc:Control="SearchResItem" owc:ItemType="SearchResult">
  <owc:Data style="display:none" type="SearchResult" doc-
href="UnitTests/1600/01/02" doc-src-type="PDF"
  doc-name="Olive software" entityId="Ar00100" primId="Ar0010000" page-
no="1" Collection="UnitTests"
  Score="100" title="Olive Enterprise Publisher Product
Overview"></owc:Data>
</div>

// html for TocSearchRes control, build at the server
<table owc:Control="TocSearchRes" owc:NodeType="TocSearchResNode"
owc:ContentCmd="ViewSearchResult"
  owc:Download="1" owc:AutoLoad="true" owc:Expand="1">
  <tr>
    <td owc:ui="treeBox">
      <div class="tofill"></div>
    </td>
    <td owc:ui="treeNodeSelectableArea">
      <span owc:ui="treeIcon"></span>
      <span class="SearchResTitle">ActivePaper e-publishing</span>
      <span class="Score"><img owc:Control="scorebarentity" owc:MaxNumber="4"
owc:UrlPrefix="Layout/ArtScoreBar/"
owc:ImgSuffix="gif"></img>100%</span>
      <owc:Data style="display:none" type="SearchResult" doc-
href="Sample2/1600/01/01" doc-src-type="PDF"
      doc-name="ActivePaper e-publishing" entityId="Ar00100"
primId="Ar0010000" page-no="1"
      Collection="Sample_EP_New" Score="100" title="ActivePaper Multi-channel
E-publishing System"></owc:Data>
    </td>
  </tr>
</table>
```

See Also

[ContentItem.SearchResult Class](#)

ContentItem.TocEntry Overview

Inheritance Hierarchy

ContentItem.TocEntry Attributes	
ContentItem.TocEntry Class	
ContentItem.TocEntry Methods	
ContentItem.TocEntry Properties	

See Also

[ContentItem Overview](#)

ContentItem.TocEntry HTML Attributes

Attributes inherited from [ContentItem](#) class

Attribute	Description
type	Specifies a type of content item, equals to TocEntry.

ContentItem.TocEntry-specific Attributes

Attribute	Description
title	Title of the TocEntry
page_no	Page number of the TocEntry
prim_id	TocEntry's primitive ID
entityId	TocEntry's entity ID
hits	Number of search hits
IS_IC	Boolean value indicated if the TocEntry is an informational content
uid	ID of the TocEntry in Toc structure

FIRST_PAGE	Boolean value indicated if the TocEntry is on the first page
-------------------	--

Remarks

TocEntry is used differently from other content items like Document, Entry, and Page. The server usually sends <owc:Data .. > for the TocEntry type.

The server sets all TocEntry-specific attributes. It is only possible to get attribute values, but not to change them.

See Also

[ContentItem.TocEntry Class](#)

type HTML Attribute

This attribute specifies content item type.

Syntax

```
<owc:Data type=/string/ />
```

Remarks

This attribute is required, and must be set to TocEntry for ContentItem.TocEntry class.

Example

```
<owc:Data type="TocEntry" />
```

Usage

HtmlElement	owc:Data
Use	Required
Default value	no

See Also

[ContentItem.TocEntry Attributes](#) | [ContentItem.TocEntry Class](#)

ContentItem.TocEntry Class Overview

Inheritance Hierarchy

Olive.ContentItem

Olive.ContentItem.TocEntry

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[ContentItem.TocEntry Methods](#) | [ContentItem.TocEntry Properties](#) | [ContentItem.TocEntry Attributes](#)

ContentItem.TocEntry Class Methods

Methods inherited from Olive.ContentItem class

Method	Description
buildQueryString	Builds a query string
contentCanLoad	Checks if the content can load

See Also

[ContentItem.TocEntry Class](#) | [ContentItem.TocEntry Properties](#)

ContentItem.TocEntry Class Properties

Properties inherited from Olive.ContentItem class

Property	Description
DataObjectType	Holds the type of the content item. It is equal to TocEntry for ContentItem.TocEntry class

See Also

[ContentItem.TocEntry Class](#) | [ContentItem.TocEntry Methods](#)

ContentItem.TocEntry Example

TocEntry is used differently from other content items like Document, Entry, Page etc. The server usually sends **<owc:Data .. >** of the TocEntry type.

```
// part of html received from the server for TOC
<div owc:Control="Toc">
  <owc:Data type="TreeLayout"><owc:Data type="TreeLayout">
    <owc:NodeType name="TocDocNode" />
    <owc:NodeType name="TocEntryNode" />
  </owc:Data>
  <table owc:Control="TocDoc" owc:NodeType="TocDocNode"
owc:ExpandAll="1">
    <owc:Data type="Document" doc-cid="TEST:1" doc-ver="1" doc-
name="ActivePaper e-publishing"
    doc-href="UnitTests/1600/01/01" doc-page-count="30" doc-src-type="PDF"
doc-lang="English"
```

```

doc-format="1.0" doc-pv-ext="png"></owc:Data>
<tr>
<td owc:ui="treeBox"></td>
<td owc:ui="treeNodeSelectableArea">
<span owc:ui="treeIcon"></span>
<span class="TocDocTitle">ActivePaper e-publishing</span>
</td>
</tr>
<tr owc:ui="treeSubNodes">
<td owc:ui="treeOutline"></td>
<td owc:ui="treeSubNodesTarget">
<table owc:Control="TocEntry" owc:NodeType="TocEntryNode">
<tr>
<td owc:ui="treeBox"></td>
<td owc:ui="treeNodeSelectableArea">
<span owc:ui="treeIcon"></span>
<span class="TocEntryOuter">
<span class="TocEntryTitle"></span>
<span class="TocEntryPageNo">..... 2</span>
</span>
<owc:Data type="TocEntry" uid="100" TITLE="" PAGE_NO="2"
PRIM_ID="Ar0020000" />
</td>
</tr>
</tr>
<tr owc:ui="treeSubNodes">
<td owc:ui="treeOutline"></td>
<td owc:ui="treeSubNodesTarget">
<table owc:Control="TocEntry" owc:NodeType="TocEntryNode">
<tr>
<td owc:ui="treeBox"></td>
<td owc:ui="treeNodeSelectableArea">
<span owc:ui="treeIcon"></span>
<span class="TocEntryOuter">
<span class="TocEntryTitle">Important Notice</span>
<span class="TocEntryPageNo">..... 2</span>
</span>
<owc:Data type="TocEntry" uid="2" TITLE="Important Notice" PAGE_NO="2"
PRIM_ID="Ar0020000" />

```

```

</td>
</tr>
<tr  owc:ui="treeSubNodes">
<td  owc:ui="treeOutline"></td>
<td  owc:ui="treeSubNodesTarget"></td>
</tr>
</table>
<table  owc:Control="TocEntry"  owc:NodeType="TocEntryNode">
<tr>
<td  owc:ui="treeBox"></td>
<td  owc:ui="treeNodeSelectableArea">
<span  owc:ui="treeIcon"></span>
<span  class="TocEntryOuter">
  <span class="TocEntryTitle">ActivePaper  Product Overview</span>
  <span  class="TocEntryPageNo">..... 3</span>
</span>
  <owc:Data type="TocEntry"  uid="3"  TITLE="ActivePaper Product Overview"
PAGE_NO="3"  PRIM_ID="Ar0030000"  />
  </td>
</tr>
<tr  owc:ui="treeSubNodes">
<td  owc:ui="treeOutline"></td>
<td  owc:ui="treeSubNodesTarget"></td>
</tr>
</table>
<table  owc:Control="TocEntry"  owc:NodeType="TocEntryNode">
<tr>
<td  owc:ui="treeBox"></td>
<td  owc:ui="treeNodeSelectableArea">
<span  owc:ui="treeIcon"></span>
<span  class="TocEntryOuter">
  <span  class="TocEntryTitle">Production Workflow Concept</span>
  <span  class="TocEntryPageNo">..... 6</span>
</span>
  <owc:Data type="TocEntry"  uid="4"  TITLE="Production Workflow Concept"
PAGE_NO="6"  PRIM_ID="Ar0060000"  />
  </td>
</tr>

```

```

<tr owc:ui="treeSubNodes">
  <td owc:ui="treeOutline"></td>
  <td owc:ui="treeSubNodesTarget">
    <table owc:Control="TocEntry" owc:NodeType="TocEntryNode">
      <tr>
        <td owc:ui="treeBox"></td>
        <td owc:ui="treeNodeSelectableArea">
          <span owc:ui="treeIcon"></span>
          <span class="TocEntryOuter">
            <span class="TocEntryTitle">ActivePaper Publishing process</span>
            <span class="TocEntryPageNo">..... 7</span>
          </span>
          <owc:Data type="TocEntry" uid="5" TITLE="ActivePaper Publishing
process" PAGE_NO="7" PRIM_ID="Ar0070000" />
        </td>
      </tr>
      <tr owc:ui="treeSubNodes">
        <td owc:ui="treeOutline"></td>
        <td owc:ui="treeSubNodesTarget"></td>
      </tr>
    </table>
  </td>
</tr>
</table>
</td>
</tr>
</table>
</td>
</tr>
</table>
<table owc:Control="TocEntry" owc:NodeType="TocEntryNode"><tr><td
owc:ui="treeBox"></td>
  <td owc:ui="treeNodeSelectableArea">
    <span owc:ui="treeIcon"></span>
    <span class="TocEntryOuter">
      <span class="TocEntryTitle">ActivePaper Applications Catalog</span>
      <span class="TocEntryPageNo">..... 12</span>
    </span>
    <owc:Data type="TocEntry" uid="12" TITLE="ActivePaper Applications
Catalog" PAGE_NO="12" PRIM_ID="Ar0120000" />
  </td>
</tr>

```

```

</tr>
<tr  owc:ui="treeSubNodes">
<td  owc:ui="treeOutline"></td>
<td  owc:ui="treeSubNodesTarget">
<table  owc:Control="TocEntry"  owc:NodeType="TocEntryNode">
<tr>
<td  owc:ui="treeBox"></td>
<td  owc:ui="treeNodeSelectableArea">
<span  owc:ui="treeIcon"></span>
<span  class="TocEntryOuter">
    <span  class="TocEntryTitle">ActivePaper Daily</span>
    <span  class="TocEntryPageNo">..... 13</span>
</span>
    <owc:Data type="TocEntry"  uid="13" TITLE="ActivePaper Daily"
PAGE_NO="13" PRIM_ID="Ar0130000" />
</td>
</tr>
</table>
</td>
</tr>
</table>
</td>
</tr>
</table>
</div>

```

See Also

[ContentItem.TocEntry Class](#)

Control Overview

The Control class is a base class from which all SDK controls are inherited. It provides the basic internal functionality to parse, create, and initialize controls, and provides the following functionality to a public API:

1. References to HTML and SDK-JavaScript objects: Part of this functionality - access to child controls - provided by the IControlsContainer interface - with the following references:

- **HtmlElement** - The HTML element on which the control is defined
- **m_sId** - HTML element's ID attribute. This serves as the control's unique ID
- **getControlType()** - Get the control type name

- **getOwnerPage()** - Get the OlivePage property of the control's ownerDocument.
- **WebApplication** - The application object
- **WebPage** - The page object for the control's web page
- **Parent** - The parent object in the SDK control hierarchy
- **Controls** - The control's child objects

The [IControlContainer interface](#) provides methods to find child objects.

The JavaScript property **m_OwcControl** provides the HTML elements with access to the SDK control.

2. Events that report that various stages in the control creation process:

- **controlCreating** - Fires at the start of the creation process
- **controlCreated** - Fires at the end of the creation process
- **controlInitialized** - Fires when the control is initialized, either with data from the server or with other data

3. Functions that associate a context menu to the control:

- **setContextMenu** - Assign a context menu to the control
- **getContextMenu** - Retrieves the context menu that has been assigned to the control
- **enableContextMenu** - Enables or disables context menu response to right-clicking the control

See Also

[EventSource Overview](#)

Control HTML Attributes

Control Attributes

Attribute	Description
owc:Control	The name of the control type
id	A unique HTML id that can be used for identifying the control
owc:init-state	Specifies initial state for the control

See Also

Control Overview | [Control Class](#)

owc:control HTML Attribute

This attribute defines the name of the control type.

Syntax

```
owc:control=/sName/
```

Remarks

The value of this attribute must be the legal name of an SDK control, such as "DocList" or "Command". The name is not case-sensitive, since it is stored internally in lower-case form.

Example

```
owc:control=DocList
```

Usage

Use	Mandatory
Default value	none

See Also

Control Overview | [Control Attributes](#)

owc:init-state HTML Attribute

This attribute defines the control's initial state.

Syntax

```
owc:init-state=/sState/
```

Remarks

The value of this attribute can be a string with states defined in the SDK, separated by a comma.

Example: "Active, VisibleMask" or "Minimized".

Example

```
owc:init-state=Minimized
```

Usage

Use	Optional
Default value	none

See Also

Control Overview | [Control Attributes](#)

Control Class Overview

The Control base class functionality is implemented by the *Olive.Controls.ControlJavaScript* class.

The *Olive.Controls.Control* interface allows access to the Control properties, methods, and events.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Implemented Interfaces

Olive.IControlContainer

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[Control Methods](#) | [Control Properties](#) | [Control Events](#) | [Control Overview](#)

Control Class Events

Control Events

Event	Description
controlCreating	The control is about to be created: fired at the start of the creation process
controlCreated	The control has been created: fired at the end of the creation process
controlInitialized	The control has been initialized: fired when the control has been initialized with data from the server or from elsewhere (such as date data for the calendar control)

See Also

Control Overview

controlCreated Event

This event is activated once the control has been created, at the end of the creation process.

Properties

The event object has no additional properties. Its basic event properties already specify the control.

Remarks

This event can be handled in order to perform any task that must be done after the control is created.

Example

```
// Handler of the event
function Application_onControlCreated(oEvent)
{
    // desirable action
}
```

See Also

[Control Events](#) | [Control Overview](#)

controlCreating Event

This event is activated when the control is about to be created, and is fired at the start of the creation process.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following expando property:

1. ControlElement the HTML element on which the control is defined

Remarks

This event can be handled in order to perform any task that must be done before the control is created.

Example

```
// Handler of the event
function Application_onControlCreating(oEvent)
{
    // desirable action
}
```

See Also

[Control Events](#) | [Control Overview](#)

controlInitialized Event

This event is activated when the control is initialized, and is fired when the control has been initialized with data from the server or from elsewhere, such as date data for the calendar control.

Properties

This event object has no additional properties. The basic event properties already specify the control.

Remarks

This event can be handled in order to perform any task that must be done after the control has been initialized with data.

Example

```
// Handler of the event
function Application_onControlInitialized(oEvent)
{
    // desirable action
}
```

See Also

[Control Events](#) | [Control Overview](#)

Control Methods

Control Methods

Method	Description
getControlType()	Get this control's type (the string type name) Arguments: none
getOwnerPage()	Gets the OlivePage property of the ownerDocument of this control's element. Arguments: none
setContextMenu(rContextMenu, bDoNotAutoEnable)	Assign a context menu to this control. Arguments: rContextMenu (string or object) - the context menu to be assigned. bDoNotAutoEnable (boolean) - if true, assign the menu but do not enable it.

getContextMenu(bScanAncestors)	Get the context menu associated with this control. Arguments: bScanAncestors (boolean) - if this object does not have a context menu, look for a context menu in ancestor controls.
enableContextMenu(bEnable)	Enable or disable the context menu. Argument: bEnable (boolean) - enable (true) or disable (false)

See Also

[Control Overview](#) | [Control Events](#) | [Control Properties](#)

enableContextMenu Method

This method enables or disables the context menu. When a context menu is enabled, there is an HTML event handler for right-click that makes the menu visible and clickable.

Parameters

bEnable - Boolean - enable (true) or disable (false)

Return value

none

Syntax

```
oControl.enableContextMenu( bEnable )
```

See Also

[Control Overview](#) | [Control Methods](#) | [Control Class](#)

getContextMenu Method

This method gets the context menu associated with this control.

Parameters

bScanAncestors - Boolean - If this object does not have a context menu, it looks for a context menu in ancestor controls.

Return value

object - context menu control

Syntax

```
oMenu = oControl.getContextMenu( bScanAncestors )
```

Remarks

This method is intended for internal use. Since the application assigned the context menu with **setContextMenu()**, it normally knows the context menu for a control. It is not normal for different context menus to be assigned and unassigned dynamically, so that the application would need to find which one is the current menu.

See Also

[Control Overview](#) | [Control Methods](#) | [Control Class](#)

getControlType Method

This method gets the control type (the type name string)

Parameters

none

Return value

string

Syntax

```
sType = oControl.getControlType()
```

Remarks

The return value is in lower-case - internally, control types are lower-case only, although the HTML attribute Control is not case-sensitive.

See Also

[Control Overview](#) | [Control Methods](#) | [Control Class](#)

getOwnerPage Method

This method gets the OlivePage property for this control's element's ownerDocument.

Parameters

none

Return value

object

Syntax

```
oPage = oControl.getOwnerPage()
```

Example

```
var oPage = oSrc.getOwnerPage();
var oArguments = { contentItem : oContentItem };
oPage.openPopup("ShowMetadata.htm", "Metadata", oArguments);
```

See Also

[Control Overview](#) | [Control Methods](#) | [Control Class](#)

setContextMenu Method

This method assigns a context menu to this control.

Parameters

rContextMenu - (string or object) The context menu to be assigned.

bDoNotAutoEnable - (boolean)

true: assign the menu but do not enable it.

false: assign the menu and enable it - default

Return value

none

Syntax

```
oControl.setContextMenu( rContextMenu, bDoNotAutoEnable )
```

Example

```
var oDocList = oPage.FindControlById("ctrlDocList");
var oContextMenu = oPage.FindControlById("ctrlContextMenu");
oDocList.setContextMenu(oContextMenu, true);
```

Remarks

This control is usually a list or tree control (DocList, SearchRes, Toc) with sub-items. The purpose of a context menu is to allow the user to apply various functions (print, show metadata, etc.) to any of a list of items.

See Also

[Control Overview](#) | [Control Methods](#) | [Control Class](#)

OwcControl Class Properties

OwcControl properties

Property	Description
----------	-------------

WebApplication	Reference to the Olive Web application object
WebPage	Reference to the Olive Web page object
Parent	Reference to the parent (container) control
HtmlElement	Reference to the associated HTML element
m_sId	Web control's unique ID (string) - the ID attribute of the HTML element
m_bCtxMenuEnabled	DHTML event handlers providing context menu support are attached (boolean)
m_oCtxMenu	Reference to the associated context menu

See Also

[OwcControl Class](#)

HtmlElement

Type: object

Reference to the associated HTML element.

Remarks

This property can be used to read or write features of the HTML element.

The HTML element also has a reference to the SDK control: m_OwcControl.

See Also

[Control Overview](#) | [Control Class](#) | [Control Properties](#)

m_bCtxMenuEnabled

Type: boolean

This property is attached a DHTML event handler when it provides context menu support. (true/false).

Remarks

This property is for internal use.

To use externally from the Application, test if getContextMenu returns an object or null.

See Also

[Control Overview](#) | [Control Class](#) | [Control Properties](#)

m_oCtxMenu

Type: object

References the associated context menu.

Remarks

This property is for internal use.
For external use from the Application, use getMenu().

See Also

[Control Overview](#) | [Control Class](#) | [Control Properties](#)

m_sId

Type: string

Olive Web control's unique ID - the HTML element's ID attribute

Remarks

This property can be used to distinguish controls defined in the HTML, when more than one control of the same type is on the page.

If the ID attribute is undefined, the control will not have **m_sId**. The server-generated HTML will also not have ID attributes, so controls on the HTML elements do not have **m_sId** properties.

See Also

[Control Overview](#) | [Control Class](#) | [Control Properties](#)

Parent

Type: object

References the parent (container) control.

Remarks

The parent control is the control on the first HTML ancestor element that has an SDK control, but not necessarily on the HTML parent element. Every control, except for the highest-level control, has a Parent, either the Application (if one exists) or the Page (if there is no Application).

See Also

[Control Overview](#) | [Control Class](#) | [Control Properties](#)

WebApplication

Type: object

Reference to the Olive Web application object

Remarks

This property is a convenient way to obtain application-level data.

See Also

[Control Overview](#) | [Control Class](#) | [Control Properties](#)

WebPage

Type: object

Reference to the Olive Web page object for the control

Remarks

This property is a convenient way to access the page object for the HTML page on which the control is defined.

See Also

[Control Overview](#) | [Control Class](#) | [Control Properties](#)

Data.SearchOptions Class Overview

This class is Olive.Data.SearchOptions, which is the base class for two other classes for data search:

- [Olive.Data.SearchTerm](#) - an internal class that stores search query terms
- [Olive.Data.SearchQuery](#) - the main class that stores all search query parameters.

The API for this class includes methods that enable reading and writing the following search options:

- **stemming:** getStemmingMode() / setStemmingMode()
- **phonic:** getPhonicMode() / setPhonicMode()
- **synonyms:** getSynonymsMode() / setSynonymsMode()
- **fuzzy level:** getFuzzyLevel() / setFuzzyLevel()

An application can create or receive an [Olive.Data.SearchQuery](#) object and use this method to directly read or write option flags.

For an explanation of these options, see on the SearchOptions attribute, [owc:FieldName](#).

For reading and writing options, use constant variables:

Olive.Data.SearchOptions.Mode.Undefined = -1;

Olive.Data.SearchOptions.Mode.Enabled = 1;

Olive.Data.SearchOptions.Mode.Disabled = 0;

Olive.Data.SearchOptions.Operator.And = 0x00000000;

Olive.Data.SearchOptions.Operator.Or = 0x10000000;

Olive.Data.SearchOptions.Operator.Not = 0x20000000;

Olive.Data.SearchOptions.Operator.RangeStart = 0x00000000;

Olive.Data.SearchOptions.Operator.RangeEnd = 0x40000000;

```

Olive.Data.SearchOptions.Operator.AndOrMask = 0x10000000;
Olive.Data.SearchOptions.Operator.RangeMask = 0x40000000;
Olive.Data.SearchOptions.Operator.Mask = 0x70000000;
Olive.Data.SearchOptions.OperatorName.And = "and";
Olive.Data.SearchOptions.OperatorName.Or = "or";
Olive.Data.SearchOptions.OperatorName.Not = "not";
Olive.Data.SearchOptions.OperatorName.RangeStart = "range-start";
Olive.Data.SearchOptions.OperatorName.RangeEnd = "range-end";
Olive.Controls.SearchOption.OptionName.PageSize = "pagesize";
Olive.Controls.SearchOption.OptionName.SortBy = "sort-by";
Olive.Controls.SearchOption.OptionName.HitsOnly = "hits-only";
Olive.Controls.SearchOption.OptionName.PropagateHits = "propagate-hits";
Olive.Controls.SearchOption.OptionName.Stemming = "stemming";
Olive.Controls.SearchOption.OptionName.Phonic = "phonic";
Olive.Controls.SearchOption.OptionName.Synonyms = "synonyms";
Olive.Controls.SearchOption.OptionName.FuzzyLevel = "fuzzy-level";

```

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[Data.SearchOptions Methods](#) | [SearchForm Overview](#)

Data.SearchOptions Class Methods

Data.SearchOptions-specific Methods

Event	Description
getStemmingMode()	Returns the stemming mode
setStemmingMode(nMode)	Assigns the stemming mode
getPhonicMode()	Returns the phonics mode
setPhonicMode(nMode)	Assigns the phonics mode
getSynonymsMode()	Returns the synonyms mode
setSynonymsMode(nMode)	Assigns the synonyms mode

getFuzzyLevel()	Returns the fuzzy level
setFuzzyLevel(nLevel)	Assigns the fuzzy level
clear()	Erases the options flags
copyOptions(oOptions)	Assigns the option-values contained in oOptions to the internal value
getOptionsString()	Returns a comma-separated string of option-value pairs "option1:val1,option2:val2,option3:val3"
getOperatorMask()	Returns a number representing a bit sequence of operators (and, or, not, RangeStart/End) for which values are set
getOperatorFlags()	Returns a number representing a bit sequence of operators (and, or, not, RangeStart/End) for which values are set to true
parseOperators(sOperators, reSeparator)	Sets the internal search operators according to a delimiter-separated string of operators sOperators , separated by the delimiter reSeparator

See Also

[Data.SearchOptions Overview](#) | [SearchForm Overview](#)

clear Method

This method clears all search flags and search modes, and sets the fuzzy level to -1, meaning that the fuzzy level is also erased.

Parameters

No parameters

Syntax

```
oSearchOptions.clear();
```

Example

```
var oSearchQuery = oSearchForm.getValue();
oSearchQuery.clear();
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

copyOptions Method

This method assigns the option-values contained in **oOptions** to the internal value. It clears all search flags, search modes and fuzzy level at the beginning and sets new options to the search option object.

Parameters

oOptions - An object of options

Syntax

```
oSearchOptions.copyOptions(oOptions);
```

Example

```
oSearchOptions.copyOptions(oOptionsFromQuery);
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

getFuzzyLevel Method

This method returns the fuzzy level.

Parameters

No parameters

Return value

nFuzzyLevel - Number

Remarks

Fuzzy level - Number - include words that differ from the search word in number of letters (0-5), defining the fuzzy level, for example for fuzzy level 1 - the words "explain", "explein" and explain will be considered as the same search words.

Syntax

```
oSearchOptions.getFuzzyLevel();
```

Example

```
var oSearchQuery = oSearchForm.getValue();  
var nFuzzyLevel = oSearchQuery.getFuzzyLevel();
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

getOperatorFlags Method

This method returns operator flags, a number representing a bit sequence of the operators (and, or, not, RangeStart/End) that have values set to true.

Use constant variables for operator mask values:

```
Olive.Data.SearchOptions.Operator.And = 0x00000000;  
Olive.Data.SearchOptions.Operator.Or = 0x10000000;  
Olive.Data.SearchOptions.Operator.Not = 0x20000000;  
Olive.Data.SearchOptions.Operator.RangeStart = 0x00000000;  
Olive.Data.SearchOptions.Operator.RangeEnd = 0x40000000;  
Olive.Data.SearchOptions.Operator.AndOrMask = 0x10000000;  
Olive.Data.SearchOptions.Operator.RangeMask = 0x40000000;  
Olive.Data.SearchOptions.Operator.Mask = 0x70000000;
```

Parameters

No parameters

Syntax

```
oSearchOptions.getOperatorFlags();
```

Example

```
var nMask = oSearchOptions.getOperatorFlags();
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

getOperatorMask Method

This method returns the operator mask, a number representing a bit sequence of the operators (and, or, not, RangeStart/End) that have set values

Use constant variables for operator mask values:

```
Olive.Data.SearchOptions.Operator.And    = 0x00000000;  
Olive.Data.SearchOptions.Operator.Or     = 0x10000000;  
Olive.Data.SearchOptions.Operator.Not    = 0x20000000;  
Olive.Data.SearchOptions.Operator.RangeStart    = 0x00000000;  
Olive.Data.SearchOptions.Operator.RangeEnd     = 0x40000000;  
Olive.Data.SearchOptions.Operator.AndOrMask    = 0x10000000;  
Olive.Data.SearchOptions.Operator.RangeMask    = 0x40000000;
```

Olive.Data.SearchOptions.Operator.Mask = 0x70000000;

Parameters

No parameters

Syntax

```
oSearchOptions.getOperatorMask();
```

Example

```
var nMask = oSearchOptions.getOperatorMask();
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

getOptionsString Method

This method gathers all defined search options and forms a string with the search options separated by commas.

For example: "HitsOnly:1,PropagateHits:1" or "Recursive:1".

Parameters

No parameters

Syntax

```
oSearchOptions.getOptionsString();
```

Example

```
varsOptions = oSearchOptions.getOptionsString();
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

getPhonicMode Method

This method returns the Phonic mode.

Parameters

No parameters

Return value

nMode - Number

Remarks

Phonic mode - Boolean - include words that sound like the search word, for example "site" from "sight".

Use constant variables for reading and writing values:

Olive.Data.SearchOptions.Mode.Undefined = -1;

Olive.Data.SearchOptions.Mode.Enabled = 1;

Olive.Data.SearchOptions.Mode.Disabled = 0;

Syntax

```
oSearchOptions.getPhonicMode();
```

Example

```
var oSearchQuery = oSearchForm.getValue();
var nPhonicMode = oSearchQuery.getPhonicMode();
if( Olive.Data.SearchOptions.Mode.Enabled == nPhonicMode )
{
    //do something if the phonic mode is enabled
}
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

getStemmingMode Method

This method returns the stemming mode.

Parameters

No parameters

Return value

nMode - number

Remarks

Stemming mode - Boolean - include words from the same stem as the search word, such as "bound" and "binding" from "bind".

Use constant variables for reading and writing values:

Olive.Data.SearchOptions.Mode.Undefined = -1;

Olive.Data.SearchOptions.Mode.Enabled = 1;

Olive.Data.SearchOptions.Mode.Disabled = 0;

Syntax

```
oSearchOptions.getStemmingMode();
```

Example

```
var oSearchQuery = oSearchForm.getValue();
var nStemmingMode = oSearchQuery.getStemmingMode();
if( Olive.Data.SearchOptions.Mode.Enabled == nStemmingMode )
{
    //do something if the stemming mode is enabled
}
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

getSynonymsMode Method

This method returns the synonyms mode.

Parameters

No parameters

Return value

nMode - Number

Remarks

Synonyms mode - Boolean - include synonym words of the search word, for example "explain" and "clarify" from "interpret".

Use constant variables for reading and writing values:

Olive.Data.SearchOptions.Mode.Undefined = -1;

Olive.Data.SearchOptions.Mode.Enabled = 1;

Olive.Data.SearchOptions.Mode.Disabled = 0;

Syntax

```
oSearchOptions.getSynonymsMode();
```

Example

```
var oSearchQuery = oSearchForm.getValue();
var nSynonymsMode = oSearchQuery.getSynonymsMode();
if( Olive.Data.SearchOptions.Mode.Enabled == nSynonymsMode)
```



```
{  
    //do something if the synonyms mode is enabled  
}
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

parseOperators Method

This method sets the internal search operators according to a delimiter-separated string of operators **sOperators**, separated by the delimiter **reSeparator**.

Parameters

sOperators - A string with operators

reSeparator

A regular expression used for parsing the sOperators string

Syntax

```
oSearchOptions.parseOperators(sOperators, reSeparator);
```

Example

```
var sOperators = "not, or";  
if (sOperators)  
    oSearchOptions.parseOperators(sOperators, /[ , \t]/);
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

setFuzzyLevel Method

This method assigns the fuzzy level.

Parameters

nLevel - A number with a fuzzy level

Remarks

Fuzzy level - Number - include words that differ from the search word in number of letters (0-5), defined in the fuzzy level, for example for fuzzy level 1 - the words "explain", "explein" and explain will be considered as the same search words.

Syntax

```
oSearchOptions.setFuzzyLevel(nLevel);
```

Example

```
var oSearchQuery = oSearchForm.getValue();  
var nFuzzyLevel = 1;  
oSearchQuery.setFuzzyLevel(nFuzzyLevel);
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

setPhonicMode Method

This method assigns the Phonic mode.

Parameters

nMode - A number with a phonetic mode

Remarks

Phonic mode - Boolean - include words that sound like the search word, for example "site" from "sight".

Use constant variables for reading and writing values:

Olive.Data.SearchOptions.Mode.Undefined = -1;

Olive.Data.SearchOptions.Mode.Enabled = 1;

Olive.Data.SearchOptions.Mode.Disabled = 0;

Syntax

```
oSearchOptions.setPhonicMode(nMode);
```

Example

```
var oSearchQuery = oSearchForm.getValue();  
var nEnabledPhonicMode = Olive.Data.SearchOptions.Mode.Enabled;  
oSearchQuery.setPhonicMode(nEnabledPhonicMode);
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

setStemmingMode Method

This method assigns the stemming mode.

Parameters

nMode - A number with a stemming mode

Remarks

Stemming mode - Boolean - include words from the same stem as the search word, for example "bound" and "binding" from "bind".

Use constant variables for reading and writing values:

Olive.Data.SearchOptions.Mode.Undefined = -1;

Olive.Data.SearchOptions.Mode.Enabled = 1;

Olive.Data.SearchOptions.Mode.Disabled = 0;

Syntax

```
oSearchOptions.setStemmingMode(nMode);
```

Example

```
var oSearchQuery = oSearchForm.getValue();  
var nDisableStemmingMode = Olive.Data.SearchOptions.Mode.Disabled;  
oSearchQuery.setStemmingMode(nDisableStemmingMode);
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

setSynonymsMode Method

This method returns to synonyms mode.

Parameters

nMode - A number with a synonyms mode

Remarks

Synonyms mode - Boolean - include synonyms of the search word, for example "explain" and "clarify" from "interpret".

Use constant variables for reading and writing values:

* Olive.Data.SearchOptions.Mode.Undefined = -1;

* Olive.Data.SearchOptions.Mode.Enabled = 1;

* Olive.Data.SearchOptions.Mode.Disabled = 0;

Syntax

```
oSearchOptions.setSynonimsMode(nMode);
```

Example

```
var oSearchQuery = oSearchForm.getValue();
```

```
var nEnabledSynonymsMode = Olive.Data.SearchOptions.Mode.Enabled;
oSearchQuery.setSynonymsMode(nEnabledSynonymsMode);
```

See Also

[Data.SearchOptions Methods](#) | [Data.SearchOptions Overview](#)

Data.SearchQuery Class Overview

The full name for this class is Olive.Data.SearchQuery. It is the main class that stores all the parameters of a search query. The class, additionally to methods of the base class, has its own method for working with options like page size (how many search results to show per page), sort by, hits only mode (show TOC with only those entries that have hits), propagate hits mode (display TOC entries with the total number of search hits for the parent plus all search hits in the child entries). Please refer to additional explanation of the options to Olive.Controls.SearchOptions.FieldName attribute

The class has the array of constants of special fields option for indicating where to search - in folder or within document:

- Olive.Data.SearchQuery.SpecialFields.Folder = "olv:Folder"
- Olive.Data.SearchQuery.SpecialFields.Document = "olv:Document"

Inheritance Hierarchy

[Olive.Data.SearchOptions](#)

Olive.Data.SearchQuery

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[Data.SearchQuery Methods](#)

Data.SearchQuery Class Methods

Overridden Methods of [Olive.Data.SearchOptions](#) class

Event	Description
clear()	Erases the options flags of the base class and specific options (sort by, page size and other fields)

Data.SearchQuery-specific Methods

Event	Description
copyFrom(oSrc)	Copies options of the base class and specific options,

	like sort by, page size and other fields
getPageSize()	Returns the page size option
setPageSize(nPageSize)	Assigns the page size option
getSortBy()	Returns the sort by option
setSortBy(oSortBy)	Assigns the sort by option
getHitsOnlyMode()	Returns the hits only mode option
setHitsOnlyMode(nMode)	Assigns the hits only mode option
getPropagateHitsMode()	Returns the propagate hits mode option
setPropagateHitsMode(nMode)	Assigns the propagate hits mode option
contentBuildParams(oParams)	Builds parameters field, searchoptions, pagesize and sortby with data and push them into the oParams parameter
toString()	Converts an array with parameters, that were build by contentBuildParams method, to a query string
findSearchInTerm(sSearchIn, oOptions)	Looks up in an internal array for a search term, corresponding to the specified search term field name and options
addSearchInField(sSearchIn, oValue, oOptions)	Adds a filed with sSearchIn filed name, oValue value and oOptions options to an internal array
copySearchInField(sFieldName, oSrcSeQuery)	Copies search query data from oSrcSeQuery into an internal field
addSearchInContentField(sSearchIn, oContentItem, oOptions)	Builds a query string with data from the oContentItem and adds a filed to the SearchQuery object with sSearchIn field name, value of this built string and oOptions
addSearchInFolder(oFolder, oOptions)	Adds olv:Folder special field to the object
addSearchInDocument(oDocument, oOptions)	Adds olv:Document special field to the object
getSearchInField(sSearchIn, oOptions)	Gets the value of the search in field, specified in sSearchIn parameter

See Also

[SearchQuery Overview](#)

addSearchInContentField Method

This method builds a query string with data from the **oContentItem** and adds a field to the **SearchQuery** object with the **sSearchIn** field name, value of this built string and **oOptions**.

Parameters

sSearchTerm - A string of a search term field name

oContentItem - An instance of a content item object with data for the value for **sSearchTerm** field

oOptions - An instance of the [Olive.Data.SearchTerm](#) object with options to set for the search term

Syntax

```
oSearchQuery.addSearchInContentField(sSearchTerm, oContentItem, oOptions);
```

Example

```
// add field for olv:Document with data from oContentItem (a built string with data can be dohref=uid:1)
oSearchQuery.addSearchInContentField(olv:Document, oContentItem, oOptions);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

addSearchInDocument Method

This method adds a special field to the object with **olv:Document** field name, value from the parameter **oDocument** and options from **oOptions**.

Parameters

oDocument - An object with data for the field (search within Document); can be a content item object of the Olive.ContentItem.Documenttype or string with a value

oOptions - An instance of the Olive.Data.SearchTermobject with options to set for the search term

Syntax

```
oSearchQuery.addSearchInDocument(oDocument, oOptions);
```

Example

```
// add field for olv:Document with data from oDocumentContentItem (a
built string with data can be dohref=uid:2)
oSearchQuery.addSearchInDocument(oDocumentContentItem, oOptions);
// add field for olv:Document with data from sDocumentData
var sDocumentData = dohref=UnitTests/1600/01/01;
oSearchQuery.addSearchInDocument(sDocumentData, oOptions);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

addSearchInFolder Method

This method adds a special field to the object having the **olv:Folder** field name, value from the parameter **oFolder** and options from the **oOptions**.

Parameters

oFolder - An object with data for the field (search in folder); can be a content item object of the [Olive.ContentItem.Folder](#) type or string with a value

oOptions - An instance of the [Olive.Data.SearchTerm](#) object with options to set for the search term

Syntax

```
oSearchQuery.addSearchInFolder(oFolder, oOptions);
```

Example

```
// add field for olv:Folder with data from oFolderContentItem(a
built string with data can be tree=uid:1,folder=uid:2)
oSearchQuery.addSearchInFolder(oFolderContentItem, oOptions);
// add field for olv:Folder with data from sFolderData
varsFolderData = tree=Repository,folder=Files;
oSearchQuery.addSearchInFolder(sFolderData, oOptions);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

addSearchInTerm Method

This method adds a new field with the **sSearchIn** field name, **oValue** value and **oOptions** options to an internal array. If a field is already named **sSearchIn**, then this method adds **oValue** to it.

Parameters

sSearchTerm - A string of a search term field name

oValue - A value of a search term field

oOptions - An instance of the Olive.Data.SearchTerm object with options to set for the search term

Syntax

```
oSearchQuery.addSearchInTerm(sSearchTerm, oValue, oOptions);
```

Example

```
// sSearchTerm can be strings like Text, DocType, dc:title,
// dc:description, dc:date and others field names defined in a search form
var sSearchTerm = Text;
var oValue = olive;
oSearchQuery.addSearchInTerm(sSearchTerm, oValue, oOptions);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

clear Method

This method calls the base class method, where it erases search flags, search modes, and the fuzzy level. It then erases options specific to the search query class: page size, sort by, hits only, propagate hits and other fields defined for the object.

Parameters

No parameters

Syntax

```
oSearchQuery.clear();
```

Example

```
oSearchQuery.clear();
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

contentBuildParams Method

This method builds a parameters field; **searchoptions**, **pagesize** and **sortby** with data, and pushes them into the **oParams** parameter.

Parameters

oParams - An array of the parameters for building a request to the server

Syntax

```
oSearchQuery.contentBuildParams(oParams);
```

Example

```
oSearchQuery.contentBuildParams(oParams);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

copyFrom Method

This method copies options and internal fields (field name and values) from the **oSrc** parameter and writes the data into internal fields

Parameters

oSrc - An object of the search query data type

Syntax

```
oSearchQuery.copyFrom(oSrc);
```

Example

```
oSearchQuery.copyFrom(oSourceSearchQuery);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

copySearchInTerm Method

This method copies query data from **oSrcSeQuery** into an internal field.

Parameters

sSearchTerm - A string of a search term field name

oSrcSeQuery - An instance of the SearchQuery data type

Syntax

```
oSearchQuery.copySearchInTerm(sSearchTerm, oSrcSeQuery);
```

Example

```
// sSearchTerm can be strings like Text, DocType, dc:title,
dc:description, dc:date and others field names defined in a search form
var sSearchTerm = Text;
oSearchQuery.copySearchInTerm(sSearchTerm, oSrcSeQuery);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

findSearchInTerm Method

This method looks into an internal array for a search term that corresponds to a specified search term field name (**sSearchIn**) and options (**oOptions**), and returns a found search term object.

Parameters

sSearchIn - A string of a search term field name

oOptions - An instance of the Olive.Data.SearchTermobject with options to match to a found search term

Syntax

```
oSearchQuery.findSearchInTerm(sSearchIn, oOptions);
```

Example

```
// sSearchIn can be strings like Text, DocType, dc:title,
dc:description, dc:date and others field names defined in a search form
var sSearchIn = Text;
var oFoundSearchTerm = oSearchQuery.findSearchInTerm(sSearchIn,
oOptions);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

getHitsOnly Method

This method returns the value of the flag **Olive.Data.SearchOptions.Flags.HitsOnly**, defined in the object

Parameters

No parameters

Remarks

The value of the flag is Number nMode indicated whether to show TOC of the search results with only TOC entries with hits or all TOC entries. The method returns one of the following values:

- Olive.Data.SearchOptions.Mode.Undefined = -1;
(flag hits only was not defined, so the same like it is set to false)
- Olive.Data.SearchOptions.Mode.Enabled = 1;
(flag hits only set to true)
- Olive.Data.SearchOptions.Mode.Disabled = 0;
(flag hits only set to false)

Syntax

```
oSearchQuery.getHitsOnly();
```

Example

```
var nMode = oSearchQuery.getHitsOnly();
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

getPageSize Method

This method returns a number that specifies the page size, as defined in the object

Parameters

No parameters

Remarks

Page number is the maximum number of search results to show on each page of results. The default is one result per page

Syntax

```
oSearchQuery.getPageSize();
```

Example

```
var nPageSize = oSearchQuery.getPageSize();
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

getPropagateHits Method

This method returns the value of the flag

Olive.Data.SearchOptions.Flags.PropagateHits, defined in the object

Parameters

No parameters

Remarks

The value of the flag is the number **nMode** indicates when to show a TOC of the search results with total number of hits for TOC entries and their child entries. The method returns one of the following values:

- Olive.Data.SearchOptions.Mode.Undefined = -1; (flag propagate hits was not defined, so the same like it is set to false)
- Olive.Data.SearchOptions.Mode.Enabled = 1; (flag propagate hits set to true)
- Olive.Data.SearchOptions.Mode.Disabled = 0; (flag propagate hits set to false)

Syntax

```
oSearchQuery.getPropagateHits();
```

Example

```
var nMode = oSearchQuery.getPropagateHits();
```

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

getSearchInField Method

This method gets the internal field value specified in the **sSearchIn** parameter.

Parameters

sSearchIn - A string of a search term field name

oOptions - An instance of the **Olive.Data.SearchTerm** object with options to match to a found search term

Syntax

```
oSearchQuery.getSearchInField(sSearchIn, oOptions);
```

Example

```
// sSearchIn can be strings like Text, DocType, dc:title,  
// dc:description, dc:date and others field names defined in a search form  
var sSearchIn = Text;  
var oValue = oSearchQuery.getSearchInField(sSearchIn, oOptions);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

getSortBy Method

This method returns the sort by option, as defined in the object.

Parameters

No parameters

Remarks

Sort by option is an object with a specified option.

Its possible values may be **/field-name/ + ";" + /direction/** ("asc"/"desc" for ascending/descending). For example, "dc:title;asc"

Syntax

```
oSearchQuery.getSortBy();
```

Example

```
var oSortBy = oSearchQuery.getSortBy();
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

setHitsOnly Method

This method assigns the hits only option.

Parameters

nMode - A number with a mode to set for the flag
Olive.Data.SearchOptions.Flags.HitsOnly

Remarks

The value of **nMode** indicates if to show a TOC only with entries with hits or all TOC entries. The method returns one of the following values:

- Olive.Data.SearchOptions.Mode.Undefined= -1;
(flag hits only was not defined, so the same like it is set to false)
- Olive.Data.SearchOptions.Mode.Enabled= 1;
(flag hits only set to true)
- Olive.Data.SearchOptions.Mode.Disabled= 0;
(flag hits only set to false)

Syntax

```
oSearchQuery.setHitsOnly(nMode);
```

Example

```
// set hits only option to true  
oSearchQuery.setHitsOnly(1);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

setPageSize Method

This method assigns the page size.

Parameters

nPageSize - A number for the option

Remarks

Page number is the maximum number of search hits that appear on each page. One hit per page is the default.

Syntax

```
oSearchQuery.setPageSize(nPageSize);
```

Example

```
var nPageSize = 10;  
oSearchQuery.setPageSize(nPageSize);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

setPropagateHits Method

This method assigns the propagate hits option.

Parameters

nMode - A number sets the mode for the flag
Olive.Data.SearchOptions.Flags.PropagateHits

Remarks

The value of the nMode indicates whether to show TOC of the search results with total number of hits for TOC entries and their child entries. The method returns one of the following values:

- Olive.Data.SearchOptions.Mode.Undefined = -1;
(flag propagate hits was not defined, same as set to false)
- Olive.Data.SearchOptions.Mode.Enabled = 1;
(flag propagate hits set to true)
- Olive.Data.SearchOptions.Mode.Disabled = 0;
(flag propagate hits set to false)

Syntax

```
oSearchQuery.setPropagateHits(nMode);
```

Example

```
// set propagate hits option to true
oSearchQuery.setPropagateHits(1);
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

setSortBy Method

This method assigns the sort by option.

Parameters

oSortBy - An object with specified sort by options

Remarks

Sort by option is an object with a specified option. It may have the values **/field-name/ + ";" + /direction/** ("asc"/"desc" for ascending/descending). For example, "dc:title;asc"

Syntax

```
oSearchQuery.setSortBy(oSortBy);
```

Example

```
// here oSearchByControl is an instance of the Olive.Controls.SearchBy
class representing searchby control; its getValue method returns
selected option of the control
oSearchQuery.setSortBy(oSearchByControl.getValue());
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

toString Method

This method converts an array with parameters built by the **contentBuildParams** method into a query string.

Parameters

No parameters

Syntax

```
oSearchQuery.toString();
```

Example

```
var sQueryString = oSearchQuery.toString();
```

See Also

[Data.SearchQuery Methods](#) | [Data.SearchQuery Overview](#)

Data.SearchTerm Class Overview

The full name for this class is Olive.Data.SearchTerm. This class is for storing a search query term. In addition to methods of the base class, This class has its own method to match options, building content parameters, copying options and getting values.

Inheritance Hierarchy

Olive.Data.SearchOptions

Olive.Data.SearchTerm

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[Data.SearchTerm Methods](#)

Data.SearchTerm Class Methods

Data.SearchTerm-specific Methods

Event	Description
matchOptions(oOptions)	Checks if the options in the object and in the parameter match
contentBuildParams(oParams)	Builds a string of all parameters defined in the object and pushes it into the parameter oParams

copyFrom(oSrc)	Copies options, field name and values from the oSrc parameter
getValue()	Returns values that were set in the object

See Also

[SearchTerm Overview](#)

contentBuildParams Method

This method builds a string of all the object's parameters and pushes it into the parameter **oParams**. For example, the method can add the element **Field** with the value **Text;;olive** as an element of the **oParams** array.

Parameters

oParams - An array of the parameters for building a request to the server

Syntax

```
oSearchTerm.contentBuildParams(oParams);
```

Example

```
oSearchTerm.contentBuildParams(oParams);
```

See Also

[Data.SearchTerm Methods](#) | [Data.SearchTerm Overview](#)

copyFrom Method

This method copies options and internal fields (field name and values) in the **oSrc** parameter.

Parameters

oSrc - An object of the search term data type

Syntax

```
oSearchTerm.copyFrom(oSrc);
```

Example

```
oSearchTerm.copyFrom(oSourceTerm);
```

See Also

[Data.SearchTerm Methods](#) | [Data.SearchTerm Overview](#)

getValue Method

This method returns values that were set in the object.

Parameters

No parameters

Syntax

```
oSearchTerm.getValue();
```

Example

```
var oValues = oSearchTerm.getValue();
```

See Also

[Data.SearchTerm Methods](#) | [Data.SearchTerm Overview](#)

matchOptions Method

This method checks if the options in the object and in the parameter **oOptions** match to each other (the same flags and flag values were set)

Parameters

oOptions - An object with search options in it

Remarks

This method compares the options, comparing their masks and flags.

Syntax

```
oSearchTerm.matchOptions(oOptions);
```

Example

```
var bMatchedOptions = oSearchTerm.matchOptions(oOptions);  
if (bMatchedOptions)  
{  
    // do desirable actions  
}
```

See Also

[Data.SearchTerm Methods](#) | [Data.SearchTerm Overview](#)

DataType Overview

The class' full name is: Olive.Controls.DataType class.

It is a base class for other data type classes used in control classes implemented [Olive.Controls.IValue](#) interface. It has general functionality to format, parse, validate, and create new values, compare values, and get a default format.

There are 11 data type classes inherited from the Data Type base class:

[Olive.DataType.Object](#)

Olive.DataType.String

[Olive.DataType.Boolean](#)

[Olive.DataType.Number](#)

[Olive.DataType.Date](#)

[Olive.DataType.Year](#)

[Olive.DataType.Month](#)

[Olive.DataType.Day](#)

[Olive.DataType.EMail](#)

[Olive.DataType.EMailList](#)

[Olive.DataType.SortBy](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType Methods](#)

DataType Methods

DataTypespecific Methods

Event	Description
getDefaultFormat	Returns a default format.
formatValue	Formats the value converting it to a string presentation.
parseValue	Returns a value converted to a necessary type if it is needed.
validateValue	Validates the value, returns a Boolean value indicated whether the value is valid or not.

createNewValue	Creates a new value of a predefined type.
compareValue	Compares two values to its equality.

See Also

[DataType Overview](#)

compareValue Method

This method compares if two values are equal.

Parameters

oPrevValue - Previous value

oNewValue - New value

Syntax

```
oDataType.compareValue(oPrevValue, oNewValue);
```

Example

```
// compare 2 values
var bEqualValues = oDataType.compareValue(oPrevValue, oNewValue);
if (bEqualValues)
{
    // desirable actions
}
else
{
    // desirable actions
}
```

See Also

[DataType Methods](#) | [DataType Overview](#)

createNewValue Method

This method creates a new value of the corresponding data type.

Parameters

No parameters

Syntax

```
oDataType.createNewValue();
```

Example

```
// create a new value
var oNewValue = oDataType.createNewValue();
```

See Also

[DataType Methods](#) | [DataType Overview](#)

formatValue Method

This method formats the value into a string.

Parameters

oValue - A value for formatting

sFormat - A format to apply during formatting; optional parameter; If it is not defined, usually a default format is used.

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for some string; sFormattedValue will be the
// same as oValue
var oValue = some string;
var sFormattedValue = oDataType.formatValue(oValue);
// get a formatted value for some boolean; sFormattedValue will be
// false
var oValue = false;
var sFormattedValue = oDataType.formatValue(oValue);
// get a formatted value for some number; sFormattedValue will be 45
var oValue = 45;
var sFormattedValue = oDataType.formatValue(oValue);
```

See Also

[DataType Methods](#) | [DataType Overview](#)

getDefaultFormat Method

This method returns a default format for predefined type values. It can be used in the [formatValue\(\)](#) method if a format was not defined as a parameter of the method. For the

DataType base class the method returns null, so every inherited data type class that should have a default format for its data type should override this method.

Parameters

No parameters

Syntax

```
oDataType.getDefaultFormat();
```

Example

```
// get a default format for some data type
var sDefaultFormat = oDataType.getDefaultFormat();
```

See Also

[DataType Methods](#) | [DataType Overview](#)

parseValue Method

This method returns a value that is converted to a corresponding data type. In the base class (DataType), it returns the value from the parameter.

Parameters

oValue - A value to parse

sFormat - A format to apply during parsing; optional parameter.

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```
// get a formatted value for some string; sParsedValue will be the same string
var oValue = some string;
var sParsedValue = oDataType.parseValue(oValue);

// get a formatted value for some boolean; bParsedValue will be boolean false
var oValue = false;
var bParsedValue = oDataType.parseValue(oValue);

// get a formatted value for some number; nParsedValue will be a number 45
var oValue = 45;
var nParsedValue = oDataType.parseValue(oValue);
```

See Also

[DataType Methods](#) | [DataType Overview](#)

validateValue Method

This method checks if the value is valid. The version of the base class always returns true.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies when to throw error.

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is valid  
var bValidValue = oDataType.validateValue(oValue);
```

See Also

[DataType Methods](#) | [DataType Overview](#)

DataType.Boolean Overview

This class is for the Boolean data type, which inherits the base functionality of the `Olive.Controls.DataType` class and overrides the following methods according to the rules for boolean object types:

[parseValue](#)

[formatValue](#)

[validateValue](#)

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.Boolean

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Boolean Methods](#)

DataType.Boolean Methods

Methods inherited from the base class [Olive.Controls.DataType](#)

Event	Description
getDefaultFormat	Returns a default format
compareValue	Compares two values
createNewValue	Creates a new object

Overridden Methods of the base class [Olive.Controls.DataType](#)

Event	Description
parseValue	Parses the value and returns a corresponding Boolean value
validateValue	Checks if the value is Boolean
formatValue	Returns the value converted to a string

See Also

[DataType.Boolean Overview](#)

formatValue Method

This method formats the value to a string. If the second parameter **sFormat** is defined, then the method returns **true** for any value except for **null**, and returns false if undefined.

Parameters

oValue - A value for formatting

sFormat - An applied format; optional parameter. Expected format string is a combination of two strings that are separated with semicolon.
The first string is a "false" value, and the second string is a "true" value

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for the string true without sFormat;  
sFormattedValue will be the string true
```



```

var oValue = true;
var sFormattedValue = oDataBooleanType.formatValue(oValue);
// get a formatted value for the string yes without sFormat;
sFormattedValue will be the string yes
var oValue = yes;
var sFormattedValue = oDataBooleanType.formatValue(oValue);
// get a formatted value for the string yes with sFormat;
sFormattedValue will be the string true
var oValue = yes;
var sFormat = true;false;
var sFormattedValue = oDataBooleanType.formatValue(oValue, sFormat);

```

See Also

[DataType.Boolean Methods](#) | [DataType.Boolean Overview](#)

parseValue Method

This method parses a value and converts it to a Boolean value.

Parameters

oValue - A value to parse

sFormat - An applied format; optional parameter. The expected format string is combination of two strings that are separated by a semicolon. The first string is a "false" value, the second string is a "true" value

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```

// get a parsed value for the Boolean true; bParsedValue will be
Boolean true
var oValue = true;
var bParsedValue = oDataBooleanType.parseValue(oValue);
// get a parsed value for the Boolean false; bParsedValue will be
Boolean false
var oValue = false;
var bParsedValue = oDataBooleanType.parseValue(oValue);
// get a parsed value for the string true (the same for yes,on and 1);
bParsedValue will be Boolean true
var oValue = true;
var bParsedValue = oDataBooleanType.parseValue(oValue);

```

```
// get a parsed value for the string false (the same for any string
except for yes,on and 1); bParsedValue will be Boolean false
var oValue = false;
var bParsedValue = oDataBooleanType.parseValue(oValue);
// get a parsed value for the number 1 (or others except for 0);
bParsedValue will be Boolean true
var oValue = 1;
var bParsedValue = oDataBooleanType.parseValue(oValue);
// get a parsed value for the number 0; bParsedValue will be Boolean
false
var oValue = 0;
var bParsedValue = oDataBooleanType.parseValue(oValue);
// get a parsed value for null (the same for undefined); bParsedValue
will be Boolean false
var oValue = null;
var bParsedValue = oDataBooleanType.parseValue(oValue);
```

See Also

[DataType.Boolean Methods](#) | [DataType.Boolean Overview](#)

validateValue Method

This method checks if a value is Boolean.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter that specifies when to throw an error.

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a Boolean
// returns true
var oValue = true;
var bValidBooleanValue = oDataBooleanType.validateValue(oValue);
// returns false
var oValue = 55;
var bValidBooleanValue = oDataBooleanType.validateValue(oValue);
```

See Also

[DataType.Boolean Methods](#) | [DataType.Boolean Overview](#)

DataType.Date Overview

This class is for the Date data type and inherits a base functionality of the `Olive.Controls.DataType` class. It includes 4 methods:

[parseValue](#)

[formatValue](#)

[validateValue](#) according to the rules for Date type objects

getDefaultFormat for defining a default format for a Date value.

Inheritance Hierarchy

`Olive.Controls.DataType`

`Olive.Controls.DataType.Date`

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Date Methods](#)

DataType.Date Methods

Methods inherited from the base class [Olive.Controls.DataType](#)

Event	Description
compareValue	Compares two values
createNewValue	Creates a new object

Overridden Methods of the base class `Olive.Controls.DataType`

Event	Description
parseValue	Parses the value and returns a Date value corresponding the value
validateValue	Checks if the value is of the Date type
formatValue	Converts a value to a string

getDefaultFormat	Returns a default format
-------------------------	--------------------------

See Also

[DataType.Date Overview](#)

formatValue Method

This method formats the value, converting it to a string according to the **sFormat** or a default format, if the parameter **sFormat** is not defined.

Parameters

oValue - A value for formatting

sFormat - An applied format; Optional parameter; The default format is used if it is not defined.

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string 12/05/1972 according to a default
format %m/%d/%Y
var oValue = new Date(1972, 11, 5);
var sFormattedValue = oDataDateType.formatValue(oValue);
// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string 05 December 1972 according to the
specified format
var oValue = new Date(1972, 11, 5);
var sFormat = %d %B %Y;
var sFormattedValue = oDataDateType.formatValue(oValue, sFormat);
// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string 05 Dec 72 according to the specified
format
var oValue = new Date(1972, 11, 5);
var sFormat = %d %b %y;
var sFormattedValue = oDataDateType.formatValue(oValue, sFormat);
```

See Also

[DataType.Date Methods](#) | [DataType.Date Overview](#)

getDefaultFormat Method

This method returns a default format for the date values. A default format may be set in the `Locale_Default` array.

Parameters

No parameters

Syntax

```
oDataType.getDefaultFormat();
```

Example

```
// get a default format for values of the date type
var sDefaultFormat = oDataDateType.getDefaultFormat();
```

See Also

[DataType.Date Methods](#) | [DataType.Date Overview](#)

parseValue Method

This method parses a value and converts it to a Date value.

Parameters

oValue - The value being parsed

sFormat - The format applied during parsing; optional parameter

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```
// get a parsed value for the object of the Date type; oParsedValue
// will be the same Date object
var oValue = new Date();
var oParsedValue = oDataDateType.parseValue(oValue);

// get a parsed value for the string Jan 5, 1996 08:47:00;
// oParsedValue will be Date object corresponding to the specified date
var oValue = Jan 5, 1996 08:47:00;
var oParsedValue = oDataDateType.parseValue(oValue);

// get a parsed value for null (the same for undefined); oParsedValue
// will be null
var oValue = null;
var oParsedValue = oDataDateType.parseValue(oValue);
```

See Also

[DataType.Date Methods](#) | [DataType.Date Overview](#)

validateValue Method

This method checks for Date type value.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies when to throw error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a Date
// returns true
var oValue = new Date();
var bValidDateValue = oDataDateType.validateValue(oValue);
// returns false
var oValue = 55;
var bValidDateValue = oDataDateType.validateValue(oValue);
```

See Also

[DataType.Date Methods](#) | [DataType.Date Overview](#)

DataType.Day Overview

The Day objects class inherits a base functionality of the [Olive.Controls.DataType.Number](#) class and includes 4 methods:

[parseValue](#)

[formatValue](#)

[validateValue](#) according to the rules for Day objects

[getDefaultFormat](#) to define the Day value's a default format

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.Number

Olive.Controls.DataType.Day

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Day Methods](#)

DataType.Day Methods

Methods inherited from the base class [Olive.Controls.DataType.Number](#)

Event	Description
compareValue	Compares two values for its equality.
createNewValue	Creates a new object.

Overridden Methods of the base class [Olive.Controls.DataType.Number](#)

Event	Description
parseValue	Parses the value and returns a number corresponding to a Day value
validateValue	Checks if the value is number grater than 0 and less than 32
formatValue	Converted a value to a string
getDefaultFormat	Returns a default format

See Also

[DataType.Day Overview](#)

formatValue Method

This method formats the value, converting it to a string according to **sFormat** or the default format, if the parameter **sFormat** is not defined.

Parameters

oValue - A value for formatting

sFormat - An applied format; Optional parameter; The default format is used if it is not defined.

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string 05 according to a default format %d
var oValue = new Date(1972, 11, 5);
var sFormattedValue = oDataDayType.formatValue(oValue);

// get a formatted value for the date of 1 January 1972;
sFormattedValue will be the string 001 according to the specified
format %j (day of year)
var oValue = new Date(1972, 0, 1);
var sFormat = %j;
var sFormattedValue = oDataDayType.formatValue(oValue, sFormat);

// get a formatted value for the number 9 (the same with the string 9);
sFormattedValue will be the string 09 according to the default format %d
var oValue = 9;
var sFormattedValue = oDataDayType.formatValue(oValue);

// get a formatted value for the number 9 (the same with the string 9);
sFormattedValue will be the string 009 according to the specified format
%j
var oValue = 9;
var sFormat = %j;
var sFormattedValue = oDataDayType.formatValue(oValue, sFormat);
```

See Also

[DataType.Day Methods](#) | [DataType.Day Overview](#)

getDefaultFormat Method

This method returns a default format value for the Day of the month as a decimal number (01-31).

Parameters

No parameters

Syntax

```
oDataType.getDefaultFormat();
```

Example

```
// get a default format (%d Day of month)
var sDefaultFormat = oDataDayType.getDefaultFormat();
```


See Also

[DataType.Day Methods](#) | [DataType.Day Overview](#)

parseValue Method

This method parses a value and converts it to a Day value. If the **oValue** is a Date object, then the method gets a number corresponded to the **Day** in the object (a number between 1 and 31), otherwise, the **oValue** is parsed by the same rules as [Olive.DataType.Number](#)

Parameters

oValue - A value to parse

sFormat - A format to apply during parsing; optional parameter

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```
// get a parsed value for the object of the Date type; nParsedValue
// will be an actual number of the Day: 10
var oValue = new Date(2005, 1, 10);
var nParsedValue = oDataDayType.parseValue(oValue);
// get a parsed value for the number 15; nParsedValue will be the same
// number 15
var oValue = 15;
var nParsedValue = oDataDayType.parseValue(oValue);
// get a parsed value for 20; nParsedValue will be number 20
var oValue = 20;
var nParsedValue = oDataDayType.parseValue(oValue);
```

See Also

[DataType.Day Methods](#) | [DataType.Day Overview](#)

validateValue Method

This method checks if the value is greater than 0 and less than 32.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies to throw an error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a Day
// returns true
var oValue = 19;
var bValidDayValue = oDataDayType.validateValue(oValue);
// returns false
var oValue = -505;
var bValidDayValue = oDataDayType.validateValue(oValue);
// returns false
var oValue = 55;
var bValidDayValue = oDataDayType.validateValue(oValue);
```

See Also

[DataType.Day Methods](#) | [DataType.Day Overview](#)

DataType.Email Overview

This class is for objects that represent an e-mail address string. It inherits a base functionality of the [Olive.Controls.DataType.String](#) class and includes the [validateValue](#) method, according to the rules for an e-mail address string.

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.String

Olive.Controls.DataType.Email

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Email Methods](#)

DataType.Email Methods

Methods inherited from the base class [Olive.Controls.DataType.String](#)

Event	Description
-------	-------------

getDefaultFormat	Returns a default format
compareValue	Compares two values
createNewValue	Creates a new object
parseValue	Returns an empty string for null or undefined values, and value converted to string otherwise
formatValue	Returns an empty string for null or undefined values, and value converted to string otherwise

Overridden Methods of the base class **Olive.Controls.DataType.String**

Event	Description
validateValue	Checks whether the value is a valid e-mail address.

See Also

[DataType.Email Overview](#)

validateValue Method

This method checks if the e-mail address is valid.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies when to throw an error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a string with valid e-mail address
// returns true
var oValue = sample@olivesoftware.com;
var bValidEmailValue = oDataEmailType.validateValue(oValue);
// returns false
var oValue = notvalid$email.com;
var bValidEmailValue = oDataEmailType.validateValue(oValue);
```

See Also

[DataType.Email Methods](#) | [DataType.Email Overview](#)

DataType.EmailList Overview

This class is for objects that represent a list of e-mail addresses. It inherits a base functionality of the [Olive.Controls.DataType.String](#) class and includes the [validateValue](#) method according to the rules for list of e-mail addresses.

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.String

Olive.Controls.DataType.EmailList

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.EmailList Methods](#)

DataType.EmailList Methods

Methods inherited from the base class [Olive.Controls.DataType.String](#)

Event	Description
getDefaultFormat	Returns a default format
compareValue	Compares two values
createNewValue	Creates a new object
parseValue	Returns an empty string for null or undefined values, and value converted to string otherwise
formatValue	Returns an empty string for null or undefined values, and value converted to string otherwise

Methods of the base class Olive.Controls.DataType.String

Event	Description
validateValue	Checks whether the value is a valid list of e-mail addresses separated by semicolon.

See Also

DataType.EmailList Overview

validateValue Method

This method checks if the value is a list of valid e-mail addresses separated by semicolons.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies when to throw an error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a string with valid e-mail addresses
// returns true
var oValue = sample@olivesoftware.com;
var bValidEMailListValue = oDataEMailListType.validateValue(oValue);
// returns true
var oValue = sample@olivesoftware.com; filecabinet@olive.com;
var bValidEMailListValue = oDataEMailListType.validateValue(oValue);
// returns false
var oValue = sample@olivesoftware.com, filecabinet@olive.com;
var bValidEMailListValue = oDataEMailListType.validateValue(oValue);
```

See Also

[DataType.EMailList Methods](#) | [DataType.EMailList Overview](#)

DataType.Month Overview

This class is for Month objects, and inherits a base functionality of the [Olive.Controls.DataType.Number](#) class. It includes 4 methods:

[parseValue](#)

[formatValue](#)

[validateValue](#) according to the rules for objects represented a Month

[getDefaultFormat](#) to define a default format for the Month value

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.Number

Olive.Controls.DataType.Month

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Month Methods](#)

DataType.Month Methods

Methods inherited from the base class [Olive.Controls.DataType.Number](#)

Event	Description
compareValue	Compares two values
reateNewValue	Creates a new object

Overridden Methods of the base class [Olive.Controls.DataType.Number](#)

Event	Description
parseValue	Parses the value and returns a number corresponding to a Month value.
validateValue	Checks whether the value is number grater than 0 and less than 13.
formatValue	Returns a converted to string value.
getDefaultFormat	Returns a default format.

See Also

[DataType.Month Overview](#)

formatValue Method

This method formats the value, converting it to a string according to **sFormat** or the default format, if the parameter **sFormat** is not defined.

Parameters

oValue - A value for formatting

sFormat - An applied format ; Optional parameter; The default format is used if this is not defined.

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string December according to a default
format %B

var oValue = new Date(1972, 11, 5);
var sFormattedValue = oDataMonthType.formatValue(oValue);

// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string Dec according to the specified
format %b

var oValue = new Date(1972, 11, 5);
var sFormat = %b;
var sFormattedValue = oDataMonthType.formatValue(oValue, sFormat);

// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string 12 according to the specified format
%m

var oValue = new Date(1972, 11, 5);
var sFormat = %m;
var sFormattedValue = oDataMonthType.formatValue(oValue, sFormat);

// get a formatted value for the number 12 (the same with the string
12); sFormattedValue will be the string December according to the
default format %B

var oValue = 12;
var sFormattedValue = oDataMonthType.formatValue(oValue);

// get a formatted value for the number 12 (the same with the string
12); sFormattedValue will be the string Dec according to the specified
format %b

var oValue = 12;
var sFormat = %b;
var sFormattedValue = oDataMonthType.formatValue(oValue, sFormat);
```

See Also

[DataType.Month Methods](#) | [DataType.Month Overview](#)

getDefaultFormat Method

This method returns a default Month format. A default format is a full Month name.

Parameters

No parameters

Syntax

```
oDataType.getDefaultFormat();
```

Example

```
// get a default format (%B full Month name representation)
var sDefaultFormat = oDataMonthType.getDefaultFormat();
```

See Also

[DataType.Month Methods](#) | [DataType.Month Overview](#) [_Ref-116221952](#)

parseValue Method

This method parses a value and converts it to a value that represents a Month. If the **oValue** is a Date object, then the method gets a number corresponded to the month in the object (from 1 to 12). Otherwise, the **oValue** is parsed by the same rules as [Olive.DataType.Number](#).

Parameters

oValue - A value to parse

sFormat - A format to apply during parsing; optional parameter

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```
// get a parsed value for the object of the Date type; nParsedValue
// will be an actual number of the Month: 2
var oValue = new Date(2005, 1, 10);
var nParsedValue = oDataMonthType.parseValue(oValue);
// get a parsed value for the number 12; nParsedValue will be the same
// number 12
var oValue = 12;
var nParsedValue = oDataMonthType.parseValue(oValue);
// get a parsed value for 2; nParsedValue will be number 2
var oValue = 2;
var nParsedValue = oDataMonthType.parseValue(oValue);
```

See Also

[DataType.Month Methods](#) | [DataType.Month Overview](#)

validateValue Method

This method checks if the value is a number greater than 0 and less than 13.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specified when to throw an error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a Month
// returns true
var oValue = 9;
var bValidMonthValue = oDataMonthType.validateValue(oValue);
// returns false
var oValue = -5;
var bValidMonthValue = oDataMonthType.validateValue(oValue);
// returns false
var oValue = 55;
var bValidMonthValue = oDataMonthType.validateValue(oValue);
```

See Also

[DataType.Month Methods](#) | [DataType.Month Overview](#)

DataType.Number Overview

This class is the for Number data type, which inherits a base functionality of the [Olive.Controls.DataType](#) class and includes 3 methods:

[parseValue](#)

[formatValue](#)

[validateValue](#) according to the rules for Number type objects.

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.Number

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Number Methods](#)

DataType.Number Methods

Methods inherited from the base class [Olive.Controls.DataType](#)

Event	Description
getDefaultFormat	Returns a default format.
compareValue	Compares two values
createNewValue	Creates a new object.

Overridden Methods of the base class Olive.Controls.DataType

Event	Description
parseValue	Parses the value and returns a Number value corresponded to the value.
validateValue	Checks whether the value is of the Number type or not.
formatValue	Returns a converted to string value.

See Also

[DataType.Number Overview](#) [D:\Web SDK API\MadCap\Web SDK API Reference\Content\WebSDK\DataType.Number\DataType.Month Overview.htm](#)

formatValue Method

This method formats the value, converting it to a string.

Parameters

oValue - A value for formatting

sFormat - A format to apply during formatting; optional parameter

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for the string 44; sFormattedValue will be the string 44
var oValue = 44;
var sFormattedValue = oDataNumberType.formatValue(oValue);
```

See Also

[DataType.Number Methods](#) | [DataType.Number Overview](#)

parseValue Method

This method parses a value and converts it to a Number value.

Parameters

oValue - A value to parse

sFormat - A format to apply during parsing; optional parameter

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```
// get a parsed value for the Number 55; nParsedValue will be Number 55
var oValue = 55;
var nParsedValue = oDataNumberType.parseValue(oValue);
// get a parsed value for the string 404; nParsedValue will be Number 404
var oValue = 404;
var nParsedValue = oDataNumberType.parseValue(oValue);
// get a parsed value for the string 23 cows; nParsedValue will be Number 23
var oValue = 23 cows;
var nParsedValue = oDataNumberType.parseValue(oValue);
// get a parsed value for null (the same for undefined); nParsedValue will be Number 0
var oValue = null;
var nParsedValue = oDataNumberType.parseValue(oValue);
```

See Also

[DataType.Number Methods](#) | [DataType.Number Overview](#)

validateValue Method

This method checks if the value is of Number type.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specified when to throw an error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a Number
// returns true
var oValue = 44;
var bValidNumberValue = oDataNumberType.validateValue(oValue);
// returns false
var oValue = 55;
var bValidNumberValue = oDataNumberType.validateValue(oValue);
```

See Also

[DataType.Number Methods](#) | [DataType.Number Overview](#)

DataType.Object Overview

This class is for the object data type, which inherits a base functionality of the [Olive.Controls.DataType](#) class and includes the [createNewValue](#) method to create a new value of the object type with the possibility to use a custom constructor.

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.Object

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Object Methods](#)

DataType.Object Methods

Methods inherited from the base class [Olive.Controls.DataType](#)

Event	Description
getDefaultFormat	Returns a default format
formatValue	Converts the value to a string format
parseValue	Returns a value converted to a necessary type, if needed
validateValue	Validates the value, returns a Boolean value that indicate if the value is valid
compareValue	Compares two values to its equality

Overridden Methods of the base class Olive.Controls.DataType

Event	Description
createNewValue	Creates a new value of the type object

See Also

[DataType.Object Overview](#)

createNewValue Method

The method creates a new value of the object data type. If the class has its own constructor, then the method creates a new object with this constructor.

Parameters

No parameters

Syntax

```
oDataType.createNewValue();
```

Example

```
// create a new object value  
var oNewObjectValue = oObjectDataType.createNewValue();
```

See Also

[DataType.Object Methods](#) | [DataType.Object Overview](#)

DataType.SortBy Overview

This class represents objects of Olive.Data.SortOptions class. The class inherits a base functionality of the [Olive.Controls.DataType.Object](#) class and overrides 4 methods:

[parseValue](#)

[formatValue](#)

[validateValue](#) according to the rules for objects of SortOptions class

[createNewValue](#) to create a new value of the class

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.Object

Olive.Controls.DataType.SortBy

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.SortBy Methods](#)

DataType.SortBy Methods

Methods inherited from the base class [Olive.Controls.DataType.Object](#)

Event	Description
compareValue	Compares two values
getDefaultFormat	Returns a default format

Methods within the base class **Olive.Controls.DataType.Object**

Event	Description
parseValue	Parses the value and returns an object of the SortOptions type.
validateValue	Checks whether the value is a valid sort by object.
formatValue	Returns a converted to string value.
createNewValue	Creates a new object of the Olive.Data.SortOptions class.

See Also

[DataType.SortBy Overview](#)

createNewValue Method

This method creates a new instance of the Olive.Data.SortOptions type.

Parameters

No parameters

Syntax

```
oDataType.createNewValue();
```

Example

```
// create a new object of SortOptions type
var oNewSortOptions = oSortByDataType.createNewValue();
```

See Also

[DataType.SortBy Methods](#) | [DataType.SortBy Overview](#)

formatValue Method

This method formats the value, converting it to a string.

Parameters

oValue - A value for formatting

sFormat - An applied format; Optional parameter; Not used.

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for the search options of sorting by ascending
// date; the sFormattedValue will be string dc:date;asc
var sFormattedValue = oDataSortByType.formatValue(oValue);
```

See Also

[DataType.SortBy Methods](#) | [DataType.SortBy Overview](#)

parseValue Method

This method parses a value and converts it to an object of the Olive.Data.SortOptions class. If the oValue is a string with sort by options, then the method creates new object of the SortOptions type and puts the options in it.

Parameters

oValue - A parsed value

sFormat - A format applied during parsing; Optional parameter; Not used.

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```
// get a parsed value for the object of the SortOptions type;
// oParsedValue will be the same instance (oValue)
var oParsedValue = oDataSortByType.parseValue(oValue);
// get a parsed value for the string dc:date;asc; oParsedValue will be
// an instance of SortOptions class with option of sorting by ascending
// date
var oValue = dc:date;asc;
```

```
var oParsedValue = oDataSortByType.parseValue(oValue);
```

See Also

[DataType.SortBy Methods](#) | [DataType.SortBy Overview](#)

validateValue Method

This method checks if the value is an object of the Olive.Data.SortOptions type.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies when to throw an error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is an instance of SortOptions class
// returns true
var oValue = new Olive.Data.SortOptions();
var bValidSortByValue = oDataSortByType.validateValue(oValue);
```

See Also

[DataType.SortBy Methods](#) | [DataType.SortBy Overview](#)

DataType.String Overview

This class is for String data type, which inherits a base functionality of the Olive.Controls.DataTypeclass and includes 3 methods according to the rules for string type objects:

[parseValue](#)

[formatValue](#)

[validateValue](#)

Inheritance Hierarchy

Olive.Controls.DataType

Olive.Controls.DataType.String

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.String Methods](#)

DataType.String Methods

Methods inherited from the base class [Olive.Controls.DataType](#)

Event	Description
getDefaultFormat	Returns a default format
compareValue	Compares two values
createNewValue	Creates a new object

Methods of the base class **Olive.Controls.DataType**

Event	Description
parseValue	Returns an empty string for null or undefined values, and value converted to string otherwise
validateValue	Checks if the value is a string type
formatValue	Returns an empty string for null or undefined values, and value converted to string otherwise

See Also

[DataType.String Overview](#)

formatValue Method

This method formats the value, converting it to a string. If the value is null or undefined, the method returns an empty string.

Parameters

oValue - A value for formatting

sFormat - An applied format; Optional parameter; If it is not defined, the default format is usually used. Any format is not used in the method

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for some string; sFormattedValue will be the  
same as oValue
```

```

var oValue = some string;
var sFormattedValue = oDataStringType.formatValue(oValue);
// get a formatted value for null; sFormattedValue will be
var oValue = null;
var sFormattedValue = oDataStringType.formatValue(oValue);

```

See Also

[DataType.String Methods](#) | [DataType.String Overview](#)

parseValue Method

This method returns a value converted to a string.

Parameters

oValue - A value to parse

sFormat - A format to apply during parsing; Optional parameter. Not any format is used in this method

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```

// get a parsed value for some string; sParsedValue will be the same
string
var oValue = some string;
var sParsedValue = oDataStringType.parseValue(oValue);
// get a parsed value for some boolean; sParsedValue will be string
false
var oValue = false;
var sParsedValue = oDataStringType.parseValue(oValue);
// get a parsed value for some number; sParsedValue will be string 45
var oValue = 45;
var sParsedValue = oDataStringType.parseValue(oValue);
// get a parsed value for null; sParsedValue will be
var oValue = null;
var sParsedValue = oDataStringType.parseValue(oValue);

```

See Also

[DataType.String Methods](#) | [DataType.String Overview](#)

validateValue Method

This method checks if the value is a string type.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies when to throw an error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a string
// returns true
var oValue = a value;
var bValidStringValue = oDataStringType.validateValue(oValue);
// returns false
var oValue = 55;
var bValidStringValue = oDataStringType.validateValue(oValue);
```

See Also

[DataType.String Methods](#) | [DataType.String Overview](#)

DataType.Year Overview

This class is for objects that represent a Year, which is a base functionality of the [Olive.Controls.DataType.Number](#) class and includes 4 methods:

[parseValue](#)

[formatValue](#)

[validateValue](#) according to the rules for objects that represent a Year

[getDefaultFormat](#) to define a default format for a Year value

Inheritance Hierarchy

Olive.Controls.DataType

 Olive.Controls.DataType.Number

 Olive.Controls.DataType.Year

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DataType.Year Methods](#)

DataType.Year Methods

Methods inherited from the base class [Olive.Controls.DataType.Number](#)

Event	Description
compareValue	Compares two values
createNewValue	Creates a new object

Methods of the base class **Olive.Controls.DataType.Number**

Event	Description
parseValue	Parses the value and returns a number corresponding to a Year value
validateValue	Checks if the value is a positive number
formatValue	Returns a converted to string value
getDefaultFormat	Returns a default format

See Also

[DataType.Year Overview](#)

formatValue Method

This method formats the value, converting it to a string representation of the value according to **sFormat**, or a default format if the **sFormat** parameter is not defined.

Parameters

oValue - A value for formatting

sFormat - An applied format; Optional parameter; If not defined, a default format is used.

Syntax

```
oDataType.formatValue(oValue, sFormat);
```

Example

```
// get a formatted value for the date of 5 December 1972;  
sFormattedValue will be the string 1972 according to a default format  
%Y
```

```

var oValue = new Date(1972, 11, 5);
var sFormattedValue = oDataYearType.formatValue(oValue);
// get a formatted value for the date of 5 December 1972;
sFormattedValue will be the string 72 according to the specified format
%y
var oValue = new Date(1972, 11, 5);
var sFormat = %y;
var sFormattedValue = oDataYearType.formatValue(oValue, sFormat);
// get a formatted value for the number 2006 (the same with the string
2006); sFormattedValue will be the string 2006 according to the default
format %Y
var oValue = 2006;
var sFormattedValue = oDataYearType.formatValue(oValue);
// get a formatted value for the number 2006 (the same with the string
2006); sFormattedValue will be the string 06 according to the specified
format %y
var oValue = 2006;
var sFormat = %y;
var sFormattedValue = oDataYearType.formatValue(oValue, sFormat);

```

See Also

[DataType.Year Methods](#) | [DataType.Year Overview](#)

getDefaultFormat Method

This method returns a default format for values that represent a year. The default format is a full calendar year for the class.

Parameters

No parameters

Syntax

```
oDataType.getDefaultFormat();
```

Example

```

// get a default format (%Y full year representation)
var sDefaultFormat = oDataYearType.getDefaultFormat();

```

See Also

[DataType.Year Methods](#) | [DataType.Year Overview](#)

parseValue Method

This method parses a value, converting it to a value that represents a year. If the **oValue** is a Date object, then the method gets a full year from the object, otherwise the **oValue** is parsed by the same rules as [Olive.DataType.Number](#).

Parameters

oValue - A value to parse

sFormat - A format to apply during parsing; optional parameter

Syntax

```
oDataType.parseValue(oValue, sFormat);
```

Example

```
// get a parsed value for the object of the Date type; nParsedValue
// will be a number represented a Year in the date object - 2005
var oValue = new Date(2005, 1, 1);
var nParsedValue = oDataType.parseValue(oValue);

// get a parsed value for the number 1988; nParsedValue will be the
// same number 1988
var oValue = 1988;
var nParsedValue = oDataType.parseValue(oValue);

// get a parsed value for 2006; nParsedValue will be number 2006
var oValue = 2006;
var nParsedValue = oDataType.parseValue(oValue);
```

See Also

[DataType.Year Methods](#) | [DataType.Year Overview](#)

validateValue Method

This method checks if the value is a positive number.

Parameters

oValue - A value to validate

bDoNotThrowError - A Boolean parameter specifies if to throw error

Syntax

```
oDataType.validateValue(oValue, bDoNotThrowError);
```

Example

```
// check whether the value is a Year
// returns true
var oValue = 1999;
var bValidYearValue = oDataYearType.validateValue(oValue);
// returns false
var oValue = -2006;
var bValidYearValue = oDataYearType.validateValue(oValue);
// returns false
var oValue = 55;
var bValidYearValue = oDataYearType.validateValue(oValue);
```

See Also

[DataType.Year Methods](#) | [DataType.Year Overview](#)

Date Overview

The Date control reads and writes a formatted date/time value in an HTML element. The basic functionality of parsing, formatting, and validating Date strings comes from the data type [Olive.Controls.DataType.Date](#).

(All Value controls have a private DataType property, which provides the specific functionality to parse, format, and validate values for the data type.)

In addition to the date string functionality, the Date control has a method [setToday\(\)](#) and a command "**Today**" to assign the value to the current date.

The "**Today**" command is only useful as a subcontrol within a Calendar control, which inherits from Date. As a subcontrol of a Date control, the command control will erase itself, since changes to the Date control replace its inner HTML.

A Date control is therefore simply a Value control with DataType "Date", which has an additional method and command to assign the current date to the control. If this additional feature is not needed, it is equivalent to writing either

```
<div owc:Control="Value" owc:DataType="Date">
```

or

```
<div owc:Control="Date">
```

Syntax

```
<html-element owc:Control = "Date" owc:Format="/format string/">
</html-element>
```

Example

```
<input id="spanDate" type="text" size="10" owc:Control="Date"
owc:Format="%Y/%m/%d" owc:Command="setToday" value="2001/12/31"
maxlength="10" />

<input type="button" value="Set to today" onclick="setToTodaysDate()" />
function setToTodaysDate() {
    var oDate = OwcGetControl("spanDate");
    if( oDate ) {
        oDate.setToday();
    }
}
```

Preview



Inheritance

Two controls inherit from the Date control: Calendar and DayTable.

Calendar is a complex nested value control which maintains a consistent value for the wrapper Calendar control and the individual date-related subcontrols (Year, Month, Day, DayTable, and/or Date).

DayTable shows the days of the month in a calendar-like table of weeks with 7 columns for days, input by clicking, and highlighting one day.

PUBLIC API

Methods

* [setToday\(\)](#): assigns the value of the current date/time to the control

Commands

* [Today](#): executes the setToday() method

The functionality of the Value control comes from the [Olive.Controls.DataType.Date](#) control. For details, see the documentation on that control.

See Also

[Date Methods](#) | [Date Commands](#) | [Olive.Controls.DataType Overview](#) | [Value Overview](#)

Date Class Overview

The Date class functionality is implemented by the Olive.Controls.Date JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Value

Olive.Controls.Date

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[Date Methods](#) | [Date Overview](#)

Date Commands

Date Commands

Command	Description
Today	Assigns today's date to the control

See Also

[Date Methods](#) | [Date Overview](#)

Today Command

The Today command assigns today's date to the Date control.

Syntax

object.launchCommand(Today)

where *object* refers to a subcontrol of the Date control.

Parameters

none

Remarks

The "**Today**" command is only useful as a subcontrol within a **Calendar** control, which inherits from Date. As a subcontrol of a Date control, the command control will erase itself, since changes to the Date control replace its inner HTML.

Sample Code

The following example illustrates using the Command control for launching the Today command on a Calendar control

```

<div id="MyCalendar"
  owc:Control="Calendar"
  owc:InitialDate="12/04/2003" >
  <table>
  <tr>
  <td>Date:</td>
  <td><input type="text" owc:Control="Date"/></td>
  <td><input id="butToday" type="button" value="Today"
owc:Control="Command" owc:Command="Today"/></td>
  </tr>
  <tr>
  <td><input type="text" owc:Control="Year" size="4"/></td>
  <td><select owc:Control="Month"></select></td>
  <td><select owc:Control="Day"></select></td>
  </tr>
  </table>
  <div owc:Control="DayTable"></div>
</div>

```

The following example illustrates launching the Today command from a command control.

```

var oToday      = getObj("butToday");
oToday.doLaunchCommand();

```

The following example illustrates using the setToday() method as an alternative to the Today command.

```

var oMyDate      = getObj("MyDate");
oMyDate.setToday();

```

See Also

[Date Class](#) | [Date Commands](#) | [Date Overview](#)

Date Class Methods

Date Methods

Method	Description
setToday	Set the value of the control to the current date/time.

See Also

[Date Class](#) | [Date Overview](#)

setToday

This method sets the value of the control to the current date and time, updating the HTML display and firing the event "valueChanged" if the prior value was a different date or time.

Parameters

none

Syntax

```
oControl.setToday();
```

See Also

[Date Commands](#) | [Date Overview](#)

Day Overview

The Day control reads and writes the day portion of a date/time value in an HTML element. It is designed to be a subcontrol of a Calendar control, where the day part of the date value is shown and changed through a Day control, and the month and year are shown and changed through Month or Year controls.

The basic functionality of parsing and validating day strings comes from the data type [Olive.Controls.DataType.Day](#). The Day control also has an internal "**mask**" property that enables data exchange with a parent Calendar control. When the value of the Day control is changed, the "**valueChanged**" event updates the day part of the Calendar date/time value, and the Calendar control then updates the other date-related subcontrols that are affected by a change in day (DayTable and Date).

Possible values of the Format HTML attribute are:

%d - two-digit day ("02")

%#d - two-digit day without zero padding ("2")

For an example of a Day control in a Calendar control, see [Calendar Examples](#).

Syntax

```
<html-element owc:Control = "Day" owc:Format="/format string/">  
</html-element>
```

Example

```
<input id="iDay" type="text" size="2" owc:Control="Day"  
owc:Format="%#d" value="2" maxlength="2" />
```

See Also

[Calendar Overview](#) | [Value Overview](#) | [Olive.Controls.DataType Overview](#)

DocList Overview

The DocList component lists the documents in a repository folder. It typically displays metadata and/or a thumbnail image of each document, going from page to page of a long document list, and selecting a document (typically in viewing order in a viewer component).

The DocList component inherits from List component the following pagination-related features: the attributes owc:UsePagination and owc:PageSize, and the commands prevPage / nextPage / firstPage / lastPage.

The appearance and behaviour of the document list must be determined by the application's Layout.xml file (it is not possible to put this information directly into the html pages, since it must be replicated for each doc-list-item). The behavior can include clicking to open a document in a flash-viewer or html-viewer - this is done by adding a "ContentCommand" control to the list-item definition for the Document list in Layout.xml.

The DocList Control does not have its own CSS classes, and its JavaScript class does not have its own commands, events and properties.

Syntax

```
<div id="divDocList" class="DocListOuter Hidden" owc:Control="DocList"
owc:UsePagination="true" owc:PageSize="2" >
</div>
```

See Also

[DocList Class](#) | [List Overview](#)

DocList HTML Attributes

DocList-specific Attributes

The following attributes are derived from the [List](#) control.

For further details, see [List Attributes](#).

Attribute	Description
owc:UsePagination	Divides the results into pages (true) or displays all the results on one page (false). Type Boolean; optional; default value = false.
owc:PageSize	Number of items per page. This is only used if owc:UsePagination="true" Type number, must be greater than 0.

See Also

[List Overview](#) | [DocList Overview](#) | [SearchRes Overview](#)

DocList Class Overview

The **DocList** component functionality is implemented by the **Olive.Controls.DocList** JavaScript class.

The **Olive.Controls.DocList** interface enables access to DocList component properties and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.List

Olive.Controls.DocList

Implemented Interfaces

[Olive.IDataBound](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[DocList Methods](#) | [DocList Overview](#)

OwcDocList Class Methods

OwcDocList Methods

Method	Description
displayFolderContent (sTreeld, sFolderId, nPageNoToDisplay, nItemsPerPage, sSelectDocId)	Retrieves the data from the server and displays it.
navigateToPage()	Calls the displayFolderContent() method without parameters to load the Folders Doc List according to current Content Item. This method is called from the OwcList base class to navigate to the next/previous/first/last page.

See Also

[DocList Overview](#) | [OwcDocList Class](#)

displayFolderContent Method

This method retrieves data from the server. If the control already has a [ContentItem](#) (for example, provided by clicking a row in a folder-tree) and loaded data, these parameters are unnecessary.

Parameters

All parameters are optional.

sTreeId - String parameter that denotes the Tree ID.

sFolderId - String parameter that denotes the Folder ID.

nPageNoToDisplay - If the list extends over more than one page, then this number parameter defines which page of the list should be displayed.

nItemsPerPage - This number parameter defines how many documents per page should be displayed.

sSelectDocId - [Not yet implemented] This string parameter denotes a document Id that should be selected.

Syntax

```
OwcDocListControl.displayFolderContent(sTreeId, sFolderId,  
nPageNoToDisplay, nItemsPerPage, sSelectDocId);
```

See Also

[DocList Overview](#) | [OwcDocList Class](#) | [OwcDocListControl Class Methods](#)

navigateToPage Method

This method calls the **displayFolderContent()** method, without parameters, to load the Folders **DocList** according to the current Content Item. It is called from the [OwcList](#) base class to navigate to the **next**, **previous**, **first**, or **last** page.

Parameters

This method does not have parameters.

Syntax

```
OwcDocListControl.navigateToPage ();
```

See Also

[DocList Overview](#) | [OwcDocListControl Class](#) | [OwcDocListControl Class Methods](#)

<http://www.olivesoftware.com/>

DocList Example

The example below specifies a DocList control that uses pagination.

```
<div id="divDocList" owc:Control="DocList" owc:UsePagination="true"
owc:PageSize="10" >
</div>
```

See Also

[List Overview](#) | [DocList Overview](#) [_Ref267457857](#)

DocViewer Control Overview

The DocViewer component shows the pages of the document. Its includes additional features such as:

- * Search result entity highlighting in different styles
- * Mouse-over entity highlighting entities
- * Click-open an entity
- * Page corner flip image navigation
- * Highlighted search result hits with pointer arrows

Syntax

```
<div id=divDocViewer owc:Control=DocViewer
owc:HighlightSearchEntities=/flag/ owc:HighlightSearchType=/type/
owc:EntitySearchHighlightColor=/file or color/
owc:HighlightMouseEntities=/flag/ owc:HighlightMouseType=/type/
owc:EntityMouseHighlightColor=/file or color/ owc:ShowHits=/flag/
owc:ArrowUrl=/file/ owc:Primitives=/flag/ owc:Corners=/flag/
owc:UsePagesRange=/flag/ owc:DoublePageMode=/mode/></div>
```

Preview



See Also

DocViewer Attributes | DocViewer CSS Classes | DocViewer [Class](#)

Attributes DocViewer HTML

DocViewer Specific Attributes

Attribute	Description
owc:HighlightSearchEntities	boolean - whether entities found in searches should be highlighted
owc:HighlightSearchType	Type of highlighting for entities found: an image file or a colored border
owc:EntitySearchHighlightColor	The color for entities found (a color or an image file, depending on owc:HighlightSearchType)
owc:HighlightMouseEntities	boolean - whether entities should be highlighted when the mouse moves over
owc:HighlightMouseType	Type of highlighting for entities when the mouse moves over: an image file or a colored border

owc:EntityMouseHighlightColor	The color for entities when the mouse moves over (a color or an image file, depending on owc:HighlightMouseType)
owc:ShowHits	boolean - whether search hits should be shown on the page
owc:ArrowUrl	The URL of the file that contains an arrow image for highlighting entities found in searches.
owc:Primitives	boolean - whether the control fires an event when the user clicks on an entity
owc:Corners	boolean - whether to show page-corner images for navigation by clicking on the corners
owc:UsePagesRange	boolean - whether to limit the number of available pages to a fixed range before and after the current page
owc:DoublePageMode	When to show two pages at a time - always ("always"), or only for pages which are defined as double pages in the PrXML ("auto").

See Also

DocViewer CSS Classes | DocViewer Class

owc:HighlightSearchEntities HTML Attribute

This attribute defines when entities found in searches are highlighted.

Syntax

```
owc:HighlightSearchEntities=bFlag
```

Remarks

Highlighting entities found in searches is done by color or by an arrow image - see [owc:HighlightSearchType](#).

Example

```
owc:HighlightSearchEntities="true"
```

Usage

Type	Boolean
Use	Optional

Default value	false
----------------------	-------

See Also

DocViewer CSS Classes | DocViewer Class

owc:HighlightSearchType HTML Attribute

This attribute defines how found entries are highlighted: with an image file or a colored border.

Syntax

```
owc:HighlightSearchType=sType
```

Remarks

This optional string attribute can have one of two values:

1. **online:** Each primitive in the found entity is enclosed in a border frame, whose color is defined by the **owc:EntitySearchHighlightColor** attribute.
2. **imagefill:** Each primitive in the found entity will be covered by a semi-transparent image and this image file is defined by the [owc:EntitySearchHighlightColor](#) attribute.

Example

```
owc:HighlightSearchType="outline"
or
owc:HighlightSearchType="imagefill"
```

Usage

Use	Optional
Default value	none (no highlighting)

See Also

DocViewer CSS Classes | DocViewer Class

owc:EntitySearchHighlightColor HTML Attribute

This attribute is the entity color found (a color or an image file, depending on [owc:HighlightSearchType](#))

Syntax

```
owc:EntitySearchHighlightColor=color
```

Remarks

When the value of the *owc:HighlightSearchType* attribute is *outline* then *color* specifies a color in RGB hexadecimal values; if its value is *imagefill* then *color* specifies the background image file.

Example

```
owc:EntitySearchHighlightColor="Layout/FFFF00.png"
```

Usage

Use	Optional
Default value	"#FFFF00" (yellow)

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:HighlightMouseEntities HTML Attribute

This attribute defines when entities are highlighted on a mouse-over.

Syntax

```
owc:HighlightMouseEntities=bFlag
```

Example

```
owc:HighlightMouseEntities="true"
```

Usage

Type	Boolean
Use	Optional
Default value	false

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:HighlightMouseType HTML Attribute

This attribute defines the entity highlighting type when a mouse mover over an image file or a colored border.

Syntax

```
owc:HighlightMouseType=sType
```

Remarks

This optional string attribute can have one of these values:

1. **online:** Each primitive in the highlighted entity is enclosed in a colored border frame that is defined by the [owc:EntityMouseHighlightColor](#) attribute.
2. **imagefill:** Each primitive in the highlighted entity will be covered by a semi-transparent image as defined by the [owc:EntityMouseHighlightColor](#) attribute.

Example

```
owc:HighlightMouseType="outline"  
or  
owc:HighlightMouseType="imagefill"
```

Usage

Use	Optional
Default value	none (no highlighting)

See Also

DocViewer CSS Classes | DocViewer Class

owc:EntityMouseHighlightColor HTML Attribute

The color for entities when the mouse moves over (a color or an image file, depending on owc:HighlightMouseType).

Syntax

```
owc:EntityMouseHighlightColor=color
```

Remarks

When the value of the *owc:HighlightMouseType* attribute is *outline* then *color* specifies a color in RGB hexadecimal values; if its value is *imagefill* then *color* specifies the background image file.

Example

```
owc:EntityMouseHighlightColor="Layout/FFFF00.png"
```

Usage

Use	Optional
Default value	"#FFFF00" (yellow)

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:ShowHits HTML Attribute

This attribute defines when search hits appear on the page.

Syntax

```
owc:ShowHits=bFlag
```

Remarks

Hits appear with a highlighted color or with an arrow image.

Example

```
owc:ShowHits="true"
```

Usage

Type	Boolean
Use	Optional
Default value	true

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:ArrowUrl HTML Attribute

The URL of the file that contains an arrow image for highlighting entities found in searches.

Syntax

```
owc:ArrowUrl=sFile
```

Remarks

If an entity to be highlighted is small, this arrow makes it easier to find it on the page.

Example

```
owc:ArrowUrl="Layout/arrow.gif"
```

Usage

Use	Optional
-----	----------

Default value	null
----------------------	------

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:Primitives HTML Attribute

This attribute defines when the control fires an event when the user clicks on an entity.

Syntax

```
owc:Primitives=bFlag
```

Remarks

The application can handle this event by performing an action on the selected entity, for example opening it in the entity viewer.

Example

```
owc:Primitives="true"
```

Usage

Type	Boolean
Use	Optional
Default value	false

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:Corners HTML Attribute

This attribute defines when to show page-corner page-flip images to navigate between pages by clicking the corners.

Syntax

```
owc:Corners=bFlag
```

Remarks

A corner image looks like curved corner of a page that is about to be turned. If enabled, it appears when the mouse moves over the corner, and if you click on it, the viewer moves to the next or previous page.

Example

```
owc:Corners="true"
```

Usage

Type	Boolean
Use	Optional
Default value	false

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:UsePagesRange HTML Attribute

This attribute defines when to limit the number of available (downloaded) pages to a range before and after the current page.

Syntax

```
owc:UsePagesRange=bFlag
```

Remarks

The purpose of page range is to allow users to do a search and look at the surrounding context of several pages, but to prevent them from seeing the entire document.

Example

```
owc:UsePagesRange="false"
```

Usage

Type	Boolean
Use	Optional
Default value	true

See Also

[DocViewer CSS Classes](#) | [DocViewer Class](#)

owc:DoublePageMode HTML Attribute

This attribute defines when to display documents in two page spreads - always ("always"), or only pages that are defined as double pages in the PrXML ("auto").

Syntax

```
owc:DoublePageMode=auto  
or  
owc:DoublePageMode=always
```

Remarks

In *auto* mode, the control always shows pages that are defined as double pages in the PrXML in a double page spread. When mode is *always*, the Component always shows double pages. Default auto.

Example

```
owc:DoublePageMode=always
```

Usage

Use	Optional
Default value	auto

See Also

DocViewer CSS Classes | DocViewer Class

DocViewer Class Overview

Olive.Controls.DocViewer JavaScript class implements DocViewer component functionality. The Olive.Controls.DocViewer interface enables access to the DocViewer component events, methods, and properties.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Viewer

Olive.Controls.DocViewer

Requirements

Script Files	OwcDocViewer.js
---------------------	-----------------

See Also

[DocViewer Properties](#) | [DocViewer Methods](#) | [DocViewer Events](#)

DocViewer Events

DocViewer Events

Event	Description
viewerCurrentHitChanged	Fired when the user navigates to the next or previous hit

DocViewer Events from IDataBound

Event	Description
contentItemClicked	When permitted, fired when user clicks on an entity on the page

DocViewer Events from Viewer

Event	Description
viewerLoaded	Fired when all control data has been received and processed content loaded

See Also

[Viewer Class](#) | [DocViewer Class](#) | [DocViewer Properties](#) | [DocViewer Methods](#)

DocViewer viewerCurrentHitChanged Event

This event is fired when the user navigates to the next or previous hit.

Arguments

1. Array of hits: `arrHits`
2. Previous hit: `prevHit`
3. Current hit: `currentHit`

See Also

[Viewer Class](#) | [DocViewer Class](#) | [DocViewer Properties](#) | [DocViewer Methods](#)

OwcDocViewer Class Methods

OwcDocViewer methods

Method	Description
isNextPageAvail	Check if the next page is available - used in navigation
isPrevPageAvail	Check if the previous page is available - used in navigation
nextPage	Go to the next page

prevPage	Go to the previous page
setLegalPageRange	Specify the range of pages that are allowed to be retrieved from the server
getContentHTMLString	Returns a string representing the content of the component, transformed using one of the supported styles
viewerHighlightEntity	Set or clear the highlighted entity

See Also

[Viewer Class](#) | [DocViewer Class](#) | [DocViewer Properties](#) | [DocViewer Events](#)

isNextPageAvail Method

This method is used in navigation to check if the next page is available. If the next page is available, it returns true; otherwise it returns false.

Parameters

None

Syntax

```
var bIsNextAvail = docViewerControl.isNextPageAvail();
```

See Also

[Viewer Class](#) | [DocViewer Class](#) | [DocViewer Properties](#) | [DocViewer Events](#)

isPrevPageAvail Method

This method is used in navigation to check if the previous page is available. If the previous page is available, it returns true; otherwise it returns false.

Parameters

None

Syntax

```
var bIsPrevAvail= docViewerControl.isPrevPageAvail();
```

See Also

[Viewer Class](#) | [DocViewer Class](#) | [DocViewer Properties](#) | [DocViewer Events](#)

nextPage Method

Go to the next page.

Parameters

None

Syntax

```
var bNextPageResult = docViewerControl.nextPage();
```

See Also

Viewer Class | DocViewer Class | [DocViewer Properties](#) | [DocViewer Events](#)

prevPage Method

Go to the previous page.

Parameters

None

Syntax

```
var bPrevPageResult = docViewerControl.prevPage();
```

See Also

Viewer Class | DocViewer Class | [DocViewer Properties](#) | [DocViewer Events](#)

setLegalPageRange Method

Specify the range of pages that are allowed to be retrieved from the server. This method does not have a return value.

Parameters

sBasePageNo - String that denotes the base page number. Other parameters define the range in both directions.

sPagesAfter - String that denotes how many pages after the base page can be brought from the server.

sPagesBefore - String that denotes how many pages before the base page can be brought from the server.

Syntax

```
docViewerControl.setLegalPageRange(basePageNo, pagesAfter, pagesBefore);
```

Example

```
docViewerControl.setLegalPageRange(10, 1, 1);
```

See Also

Viewer Class | DocViewer Class | [DocViewer Properties](#) | [DocViewer Events](#)

getContentHTMLString Method

This method returns a string that represents the component's content, as it was transformed using one of the supported styles. If style is unspecified, the content is returned as is.

Parameters

sStyle - This string defines the style of transformation. The possible values are:

- **print** transform for use in the Print window.
- **static** transform for the position:relative style layout.

Syntax

```
var htmlContent = docViewerControl.getContentHTMLString(sStyle);
```

Example

```
var htmlContent = docViewerControl.getContentHTMLString(print);
```

See Also

Viewer Class | DocViewer Class | [DocViewer Properties](#) | [DocViewer Events](#)

viewerHighlightEntity Method

Set or clear the highlighted entity. This method does not have a return value.

Parameters

sEntityId - This string parameter denotes the entity. The highlighting is set or cleared for this entity.

bHighlight - This Boolean parameter defines if highlighting is set (true) or cleared (false).

Syntax

```
docViewerControl.viewerHighlightEntity(sEntityId, bHighlight);
```

Example

```
docViewerControl.viewerHighlightEntity(Ar00102, true); // Set highlighting  
docViewerControl.viewerHighlightEntity(Ar00102, false); // Clear highlighting
```

See Also

Viewer Class | DocViewer Class | [DocViewer Properties](#) | [DocViewer Events](#)

DocViewer Class Properties

DocViewer properties

Property	Description
layoutSet	The SDK layout folder
bHighlightSearchEntities	Flag that defines if to highlight entities found in searches
highlightSearchType	Type of search highlighting: "outline" or "imagefill"
highlightSearchColor	A color or file, depending on the search highlighting type
bHighlightMouseEntities	Flag that defines if to highlight an entity on a mouse-over
highlightMouseType	Type of mouse highlighting: "outline" or "imagefill"
highlightMouseColor	A color or file, depending on the mouse highlighting type
bDisplayHits	Flag that defines if search hits will be highlighted
hits	Array of hits objects
hitQuads	Array of hits when there is more than one hit per word
hitImgs	Array of hits images
selHitIndex	Selected hit index
selHitPageNo	Page number of the selected hit
arrowToHighlightUrl	URL of the Arrow image file
leftForArrow	Left coordinate of the arrow image
topForArrow	Top coordinate of the arrow image
divForArrow	Entity's primitive that the arrow applies to
imgArrow	Image arrow HTML object
bEntityPrims	Flag that defines if the control fires an event when some entity is clicked
entitiesOnPage	Array that contains all entities on a page
primitivesOnPage	Array that for each entity contains its primitives
embeddedPictures	Array that for each entity contains its embedded pictures

highlightedSearchDivs	Array that contains the primitives of an entity found in a search
primDivs	Array that contains highlighted HTML objects used for the mouse HTML events
imagesCache	Array that contains the cached images
dblPageMode	String that defines how server treats double pages
bCorners	Flag that defines if the control will create corner images for the page
bUsePagesRange	Flag that defines if the control will use a permitted-pages range
legalPagesAfter	Number of permitted pages after the base permitted page
legalPagesBefore	Number of permitted pages before the base permitted page

See Also

DocViewer Overview | DocViewer Class

layoutSet

The SDK layout folder.

Syntax

```
var layoutSet = docViewerControl.layoutSet;
```

Sample Code

```
var layoutSet = docViewerControl.layoutSet;
```

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

bHighlightSearchEntities

Boolean flag that defines when to highlight entities found in searches

Syntax

```
var bHighlightSearchEntities= docViewerControl.bHighlightSearchEntities;
```

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

highlightSearchType

Highlighting of search hits: "outline" or "imagefill"

Syntax

```
var highlightSearchType= docViewerControl.highlightSearchType;
```

Sample Code

```
var highlightSearchType= docViewerControl.highlightSearchType;
```

Remarks

By default this property's value is `outline`. Its value can be set by the `OWC:HighlightSearchType` attribute.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

highlightSearchColor

A color or file, depending on the search highlighting type

Syntax

```
var highlightSearchColor= docViewerControl.highlightSearchColor;
```

Sample Code

```
var highlightSearchColor= docViewerControl.highlightSearchColor;
```

Remarks

By default this property's value is `#FFFF00` (yellow color). Its value can be set by the `OWC:EntitySearchHighlightColor` attribute.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

bHighlightMouseEntities

Flag that defines when to highlight an entity if the mouse is moved over it

Syntax

```
var bHighlightMouseEntities= docViewerControl.bHighlightMouseEntities;
```

Sample Code

```
var bHighlightMouseEntities= docViewerControl.bHighlightMouseEntities;
```

Remarks

This property's default value is `false`; the found entity is not highlighted.

It is set to `true` (and the entity is highlighted) when the `owc:HighlightMouseEntities` attribute isn't `false`.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

highlightMouseType

Type of mouse highlighting: "outline" or "imagefill".

Syntax

```
var highlightMouseType= docViewerControl.highlightMouseType;
```

Sample Code

```
var highlightMouseType= docViewerControl.highlightMouseType;
```

Remarks

This property's default value is `outline`.

Its value can be set by the `owc:HighlightMouseType` attribute.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

highlightMouseColor

A color or file, depending on the mouse highlighting type.

Syntax

```
var highlightMouseColor= docViewerControl.highlightMouseColor;
```

Sample Code

```
var highlightMouseColor= docViewerControl.highlightMouseColor;
```

Remarks

This property's default value is `#FFFF00` (yellow color).

Its value can be set by the `owc:EntityMouseHighlightColor` attribute.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

bDisplayHits

Flag that defines if search hits are highlighted

Syntax

```
var bDisplayHits = docViewerControl.bDisplayHits;
```


Sample Code

```
var bDisplayHits = docViewerControl.bDisplayHits;
```

Remarks

This property's default value is true; the hits are highlighted.

Its value can be changed to false by the `owc:ShowHits` attribute.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

hits

Array of hits objects. This is an internal property.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

hitQuads

Array of hits when there is more than one hit per word. This is an internal property.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

hitImgs

Array of hits images. This is an internal property.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

selHitIndex

Selected hit index.

Syntax

```
var selHitIndex = docViewerControl.selHitIndex;
```

Sample Code

```
var selHitIndex = docViewerControl.selHitIndex;
```

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

selHitPageNo

Page number of the selected hit.

Syntax

```
var selHitPageNo = docViewerControl.selHitPageNo;
```

Sample Code

```
var selHitPageNo = docViewerControl.selHitPageNo;
```

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

arrowToHighlightUrl

URL of the Arrow image file. This file represents the image of an arrow and can be used when the found (highlighted) entity is too small and difficult to view on the page.

Syntax

```
var arrowToHighlightUrl= docViewerControl.arrowToHighlightUrl;
```

Sample Code

```
var arrowToHighlightUrl= docViewerControl.arrowToHighlightUrl;
```

Remarks

This property's default value is null (no arrow file).

Its value can be set by the `owc:ArrowUrl` attribute.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

leftForArrow

Left coordinate of the arrow image. This is an internal property.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

topForArrow

Top coordinate of the Arrow HTML image element. This is an internal property.

See Also

DocViewer Overview | DocViewer Class | [DocViewer Properties](#)

divForArrow

Entity's primitive that the arrow applies to. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

imgArrow

Image Arrow HTML object. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

bEntityPrims

Flag that defines if the control fires an event when some entity is clicked

Syntax

```
var bEntityPrims = docViewerControl.bEntityPrims;
```

Sample Code

```
var bEntityPrims = docViewerControl.bEntityPrims;
```

Remarks

This property's default value is `false`.

Its value can be set by the `owc:Primitives` Boolean attribute.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

entitiesOnPage

Array that contains all entities on a page. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

primitivesOnPage

Array that for each entity contains its primitives. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

embeddedPictures

Array that for each entity, containing its embedded pictures. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

highlightedSearchDivs

Array that contains the primitives of an entity found in a search. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

primDivs

Array that contains highlighted HTML objects used for the mouse HTML events. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

imagesCache

Array that contains the cached images. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

dblPageMode

String that defines how server treats double pages: "auto" or "always".

"**auto**" value shows pages according to their type (single or double pages).

"**always**" value shows each page as a double spread.

Syntax

```
var dblPageMode = docViewerControl.dblPageMode;
```

Sample Code

```
var dblPageMode = docViewerControl.dblPageMode;
```

Remarks

This property's default value is `auto`.

Its value can be set by the `owc:DoublePageMode` attribute.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

bCorners

Flag that defines if the control will create corner images for the page.

Syntax

```
var bCorners = docViewerControl.bCorners;
```

Sample Code

```
var bCorners = docViewerControl.bCorners;
```

Remarks

This property's default value is `false`.

Its value can be set by the `owc:Corners` attribute.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

bUsePagesRange

Flag that defines if the control will use a permitted-pages range

Syntax

```
var bUsePagesRange = docViewerControl.bUsePagesRange;
```

Sample Code

```
var bUsePagesRange = docViewerControl.bUsePagesRange;
```

Remarks

This property's default value is `true`.

Its value can be set by the `owc:UsePagesRange` attribute.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

legalPagesAfter

Number of permitted pages after the base permitted page. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

legalPagesBefore

Number of permitted pages before the base permitted page. This is an internal property.

See Also

[DocViewer Overview](#) | [DocViewer Class](#) | [DocViewer Properties](#)

DocViewer Control CSS Classes

CSS Class	Description
owcCorner	CSS class for corners
owcArrow	CSS class for arrow

See Also

[DocViewer Overview](#) | [DocViewer Class](#)

DocViewer Example

The example below describes the DocViewer control.

```
<div id="divDocViewer"
  owc:Control="DocViewer"
  owc:HighlightSearchEntities="true"
  owc:HighlightSearchType="imagefill"
  owc:EntitySearchHighlightColor="Layout/FFFF00.png"
  owc:HighlightMouseEntities="true"
  owc:HighlightMouseType="imagefill"
  owc:EntityMouseHighlightColor="Layout/FF0000.png"
  owc:ShowHits="true"
  owc:ArrowUrl="Layout/arrow.gif"
  owc:Primitives="true"
  owc:Corners="true"
  owc:UsePagesRange="false"
  owc:DoublePageMode="always"
>
</div>
```

See Also

[DocViewer Overview](#) | [DocViewer Class](#)

EntViewer Overview

The EntViewer component shows the entity in the document. Its includes additional features, such as addedcolor highlighting or an arrow, to words were in searches.

Syntax

```
<html-element owc:Control = EntViewer
```

Preview

All Furniture Reduced!

SAVE UP TO 40%

SAVE 10% Windows Treats!
 10% discount on all window treatments. Reg. \$19.99, now \$17.99. Call 1-800-888-8888. Reg. \$19.99, now \$17.99. Call 1-800-888-8888.

\$1,499 ~~\$2,478~~
 Executive Leather Sofa, Chair
 6-Piece leather sofa with matching chairs. Reg. \$2,478, now \$1,499. Call 1-800-888-8888. Reg. \$2,478, now \$1,499. Call 1-800-888-8888.

\$559 ~~\$700~~
 6-Piece Western Lodge Black Table Set
 6-Piece western lodge black table set. Reg. \$700, now \$559. Call 1-800-888-8888. Reg. \$700, now \$559. Call 1-800-888-8888.

Only \$1,499!
 Leather Sofa, Chair & Ottomans

99.99 **99.99** **99.99** **115.99** **112.99** **119.99**
 Red Wine, White Wine, Sparkling Wine, etc.

\$16.99
 4-Piece Candle Set
 4-Piece candle set. Reg. \$16.99, now \$16.99. Call 1-800-888-8888. Reg. \$16.99, now \$16.99. Call 1-800-888-8888.

\$26.99
 6-Piece Picture Set
 6-Piece picture set. Reg. \$26.99, now \$26.99. Call 1-800-888-8888. Reg. \$26.99, now \$26.99. Call 1-800-888-8888.

\$20.00
 All World Market Coffee
 All World Market coffee. Reg. \$20.00, now \$20.00. Call 1-800-888-8888. Reg. \$20.00, now \$20.00. Call 1-800-888-8888.

\$10.00
 3-Piece Storage Boxes
 3-Piece storage boxes. Reg. \$10.00, now \$10.00. Call 1-800-888-8888. Reg. \$10.00, now \$10.00. Call 1-800-888-8888.

WHITE WINES			RED WINES		
Brand	Reg.	Now	Brand	Reg.	Now
Chateau d'Aud	\$19.99	\$17.99	Chateau d'Aud	\$19.99	\$17.99
Chateau d'Aud	\$19.99	\$17.99	Chateau d'Aud	\$19.99	\$17.99
Chateau d'Aud	\$19.99	\$17.99	Chateau d'Aud	\$19.99	\$17.99
Chateau d'Aud	\$19.99	\$17.99	Chateau d'Aud	\$19.99	\$17.99
Chateau d'Aud	\$19.99	\$17.99	Chateau d'Aud	\$19.99	\$17.99

\$20.00
 All World Market Coffee
 All World Market coffee. Reg. \$20.00, now \$20.00. Call 1-800-888-8888. Reg. \$20.00, now \$20.00. Call 1-800-888-8888.

WORLD MARKET

WORLD MARKET Furniture, Home Decor, Kitchenware, Bedding, Bath, and more. Call 1-800-888-8888. Reg. \$19.99, now \$17.99. Call 1-800-888-8888. Reg. \$19.99, now \$17.99. Call 1-800-888-8888.

Pay no thing until March 2001!
 0% financing available on all purchases. Call 1-800-888-8888. Reg. \$19.99, now \$17.99. Call 1-800-888-8888.

[EntViewer Attributes](#) | [EntViewer CSS Classes](#) | [EntViewer Class](#)

EntViewer HTML Attributes

EntViewer Specific Attributes

Attribute	Description
owc:SelectedHitStyle	CSS class for selected hits
owc:UnselectedHitStyle	CSS class for unselected hits
owc:SelectedHitColor	Color for selected hits
owc:UnselectedHitColor	Color for unselected hits
owc:ArrowUrl	URL of the file that contains the arrow image
owc:LayoutSet	Path where all styles and images are defined
owc:IsForward	Boolean attribute defines when to show hits from the first to the next or in reverse
owc:HighlightArrow	Boolean attribute defines when to show an arrow on the selected hit

See Also

EntViewer CSS Classes | [EntViewer Class](#)

owc:SelectedHitStyle HTML Attribute

Specifies a style for selected hits

Syntax

```
<html-element owc:SelectedHitStyle = /style/ >  
...  
</html-element>
```

Remarks

This attribute can have only 2 values "outline" or "imagefill".

With outline, selected hits are displayed by a box with border as defined in the owc:SelectedHitColor attribute color.

In the second case, imagefill, selected hits have a box filled as defined in the owc:SelectedHitColor attribute color.

If only one of the two attributes owc:SelectedHitStyle or owc:UnselectedHitStyle is defined then the value of the defined attribute is set for the both attributes.

Example

```
<div owc:Control="EntViewer" owc:SelectedHitStyle="imagefill">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	no

See Also

EntViewer Control | EntViewer Attributes | [EntViewer Class](#)

owc:UnselectedHitStyle HTML Attribute

Specifies a style for unselected hits

Syntax

```
<html-element owc:UnselectedHitStyle = /style/ >  
...  
</html-element>
```

Remarks

This attribute can have one of only 2 values "outline" or "imagefill".

With outline, unselected hits are displayed by a box with a border as defined in owc:UnselectedHitColor attribute "color".

With imagefill, unselected hits have a box filled as defined in owc:UnselectedHitColor attribute "color".

If only one of the two attributes owc:SelectedHitStyle or owc:UnselectedHitStyle is defined, then the value of the defined attribute is set for the both attributes.

Example

```
<div owc:Control="EntViewer" owc:UnselectedHitStyle="imagefill">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	no

See Also

EntViewer Control | EntViewer Attributes | [EntViewer Class](#)

owc:SelectedHitColor HTML Attribute

Specifies a color for selected hits

Syntax

```
<html-element owc:SelectedHitColor = /color / >  
...  
</html-element>
```

Remarks

The value of this attribute defines a color used to display selected hits.

If only one of the attributes owc:SelectedHitColor or owc:UnselectedHitColor is defined, then the value of the defined attribute is set for the both attributes.

Example

```
<div owc:Control="EntViewer" owc:SelectedHitColor="#FFFF00">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	null

See Also

EntViewer Control | EntViewer Attributes | [EntViewer Class](#)

owc:UnselectedHitColor HTML Attribute

Specifies a color for unselected hits

Syntax

```
<html-element owc:UnselectedHitColor = /color / >  
...  
</html-element>
```

Remarks

The value of this attribute defines a color used for displaying unselected hits.

If only one attribute owc:SelectedHitColor or owc:UnselectedHitColor is defined, then the value of the defined attribute is set for the both attributes.

Example

```
<div owc:Control="EntViewer" owc:UnselectedHitColor="#0000FF">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	null

See Also

EntViewer Control | EntViewer Attributes | [EntViewer Class](#)

owc:ArrowUrl HTML Attribute

URL of the file that contains the arrow image

Syntax

```
<html-element owc:ArrowUrl = /file/ >  
...  
</html-element>
```

Remarks

This attribute is used when the found entity is too small, and the arrow makes it easier to find it on the page.

Example

```
<div owc:Control="EntViewer" owc:ArrowUrl="Layout/arrow.gif">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	no

See Also

EntViewer Control | EntViewer Attributes | [EntViewer Class](#)

owc:LayoutSet HTML Attribute

Specifies a path to the CSS and image files

Syntax

```
<html-element owc:LayoutSet = /path/ >  
...  
</html-element>
```

Remarks

Example

```
<div owc:Control="EntViewer" owc:LayoutSet="Layout">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	no

See Also

EntViewer Control | EntViewer Attributes | [EntViewer Class](#)

owc:IsForward HTML Attribute

Boolean attribute defines to show hits from the first to the next or in reverse.

Syntax

```
<html-element owc:IsForward = /boolean/ >  
...  
</html-element>
```

Remarks

Example

```
<div owc:Control="EntViewer" owc:IsForward="true">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	true

See Also

[EntViewer Control](#) | [EntViewer Attributes](#) | [EntViewer Class](#)

owc:HighlightArrow HTML Attribute

Shows an arrow on the selected hit

Syntax

```
<html-element owc:HighlightArrow = /boolean/ >
...
</html-element>
```

Remarks

Example

```
<div owc:Control="EntViewer" owc:HighlightArrow="true">
```

Usage

Parent element	EntViewer
Use	Optional
Default value	false

See Also

[EntViewer Control](#) | [EntViewer Attributes](#) | [EntViewer Class](#)

EntViewer Class Overview

The EntViewer component functionality is implemented by the Olive.Controls.EntViewer JavaScript class. The Olive.Controls.EntViewer interface allows access to the EntViewer component properties and methods.

Inheritance Hierarchy

- Olive.EventsSource
- Olive.Object
- Olive.CmdTarget
- Olive.Controls.Control
 - Olive.Controls.EntViewer

Requirements

Script Files	OwcEntViewer.js
--------------	-----------------

See Also

[EntViewer Methods](#) | [EntViewer Events](#) | [EntViewer Commands](#)

EntViewer Class Events

EntViewer Events

Event	Description
entviewerCalled	The SDK does not fire this event, but if it is fired from the application, the EntViewer control has a handler that reads the content data from the event object and loads the entity viewer. This event is deprecated.
entviewerLoaded	Fired when the entity viewer has received server data and processed it.

See Also

[EntViewer Class](#) | [EntViewer Methods](#) | [EntViewer Commands](#)

entviewerCalled Event

The event is activated when the entity viewer is called.

Arguments

The event object that passed to the event handler is an instance of the OwcEvent class. It uses content data from the srcObject property of the event object to initialise the control. It assumes that the event originated with a document viewer (clicking on an entity).

Remarks

This event is an alternative way to call an entity viewer, to send a request to the server, with data from the parameter of the event. Another components or application can fire this event to call for an entviewer. The class OwcEntViewer has its own handler of this event.

NOTE: This event is deprecated. It is not recommended to fire an event on an object itself.

Example

```
function my_onContentItemClicked( oEvent ) {  
    var oEntViewer = OwcGetControl( "divEntViewer" );  
    oEntViewer.fireOwcEvent( oEvent );  
}
```

See Also

[EntViewer Events](#) | [EntViewer Class](#)

entviewerLoaded Event

The event is activated when the entity viewer is loaded.

Arguments

The event object that passes to the event handler is an instance of the OwcEvent class

Remarks

This event is fired to inform other controls that the entity viewer is ready, loaded, and all initial actions are finished.

Example

```
function my_onEntviewerLoaded( oEvent ) {  
    // do some action  
}
```

See Also

[EntViewer Events](#) | [EntViewer Class](#)

EntViewer Class Methods

EntViewer methods

Method	Description
isNextHitAvail	Check if the next hit is available - used in navigation
isPrevHitAvail	Check if the previous hit is available - used in navigation
nextHit	Go to the next entity's hit
prevHit	Go to the previous entity's hit
gotoNextChunk	Go to the next entity's chunk
gotoPrevChunk	Go to the previous entity's chunk
getLoadingStatus	Return the loading status

See Also

[EntViewer Class](#) | [EntViewer Overview](#)

isNextHitAvailable Method

This method checks if the next hit is available. It can be called to set enabled or disabled styles on the hits navigation buttons (next / prior hit).

Parameters

No parameters

Syntax

```
objEntViewer.isNextHitAvailable();
```

Example

```
if (objEntViewer.isNextHitAvailable())
{
    butNextHit.setEnabled(true);
}
else
{
    butNextHit.setEnabled(false);
}
```

See Also

[EntViewer Methods](#) | [EntViewer Class](#) | [EntViewer Overview](#)

isPrevHitAvailable Method

The method checks if a previous hit is available. It can be called to set enabled or disabled styles on the hits navigation buttons (next / previous hit).

Parameters

No parameters

Syntax

```
objEntViewer.isPrevHitAvailable();
```

Example

```
if (objEntViewer.isNextHitAvailable())
{
    butPrevHit.setEnabled(true);
}
else
{
    butPrevHit.setEnabled(false);
}
```


See Also

[EntViewer Methods](#) | [EntViewer Class](#) | [EntViewer Overview](#)

nextHit Method

This method returns the next hit from the array, sets the selected highlighting on the current hit, and removes the selected highlighting from the previously selected hit. This method also looks for instances of the next hit in other chunks of the article.

There is no need to use the method [isNextHitAvailable](#) separately, since it is already checked in this method.

Parameters

No parameters

Syntax

```
objEntViewer.nextHit();
```

See Also

[EntViewer Methods](#) | [EntViewer Class](#) | [EntViewer Overview](#)

prevHit Method

This method returns the previous hit from the array, sets the selected highlighting on the current hit, and removes the selected highlighting from the previously selected hit. This method also looks for instances of the previous hit in other chunks of the article.

There is no need to use the method [isPrevHitAvailable](#) separately, since it is already checked in this method.

Parameters

No parameters

Syntax

```
objEntViewer.prevHit();
```

See Also

[EntViewer Methods](#) | [EntViewer Class](#) | [EntViewer Overview](#)

gotoNextChunk Method

This method checks if the next chunk is available, and builds a request to the server for the next chunk in the entity. It is used in requests for next hits.

Parameters

isForward - Boolean parameter defines going forward or backward to the next chunk

Syntax

```
objEntViewer.gotoNextChunk(/boolean/);
```

Example

```
objEntViewer.gotoNextChunk(true);
```

See Also

[EntViewer Methods](#) | [EntViewer Class](#) | [EntViewer Overview](#)

gotoPrevChunk Method

This method checks if a previous chunk is available and builds a request to the server for the previous chunk in the entity. It is used in requests for previous hits.

Parameters

isForward - Boolean parameter defines going forward or backward to the previous chunk

Syntax

```
objEntViewer.gotoPrevChunk(/boolean/);
```

Example

```
objEntViewer.gotoPrevChunk(true);
```

See Also

[EntViewer Methods](#) | [EntViewer Class](#) | [EntViewer Overview](#)

getLoadingStatus Method

This method returns the component's loading status (boolean value)

Parameters

No parameters

Syntax

```
objEntViewer.getLoadingStatus();
```

See Also

[EntViewer Methods](#) | [EntViewer Class](#) | [EntViewer Overview](#)

EntViewer Class Properties

EntViewer properties

Property	Description
curHitBox	Index of the current hit in the hit objects array
hits	Array of hits
searchStr	Requested hits from server according to search pattern
documentID	Document ID value from attribute or from Event when the Entity Viewer is called
chunk	Chunk number value from attribute or from Event when the Entity Viewer is called
entityWidth	Entity width
entityHeight	Entity height
printWidth	Page width for printing. Depends on page size: A4, Letter
printHeight	Page height for printing. Depends on page size: A4, Letter
highlightDivs	Array of highlighted Divs hits
selectedHitStyle	Selected hit style: outline or image
unselectedHitStyle	Unselected hit style: outline or image
selectedHitColor	Selected hit color
unselectedHitColor	Unselected hit color
isHighlighting	Whether to use highlighting for hits
bAddArrowToHighlight	Whether to use an arrow for highlighting
bArrowLoaded	Whether the arrow is loaded
arrowToHighlightUrl	The URL to the arrow file
imgArrow	HTML element for the arrow

See Also

[EntViewer Class](#) | [EntViewer Overview](#)

curHitBox Property

Index of current hit in the hit objects .

Syntax

Data Type	Number
Default value	0

Remarks

Initial value of this property is zero and then it is changed when user navigates among hits.

Sample Code

```
var nIndex= entViewerControl.curHitBox;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

hits Property

Array of hits objects.

Syntax

Data Type	Array
Default value	empty

Sample Code

```
var oHitObject = entViewerControl.hits[0];
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

searchStr Property

This property is used to request hits from server, according to this search pattern.

Syntax

Data Type	String
------------------	--------

Default value	null
---------------	------

Sample Code

```
var sCollName = entViewerControl.collName;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

documentID Property

This property defines the document to which the entity belongs

Syntax

Data Type	String
Default value	(empty string)

Sample Code

```
var sDocumentId = entViewerControl.documentID;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

chunk Property

Chunk of the entity (which part of an entity which is divided among pages).

Syntax

Data Type	Number
Default value	-2

Sample Code

```
var nChunk = entViewerControl.chunk;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

entityWidth Property

This property recalculates the entity's width, based on the widths of its primitives.

Syntax

Data Type	Number
Default value	680

Remarks

This value represents the entity's maximum width, calculated on the basis of its primitives. This value is then stored inside the HTML element and can be used by the application to resize the window when its content is loaded.

Sample Code

```
var nEntityWidth = entViewerControl.entityWidth;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

entityHeight Property

This property recalculates the entity height based on the heights of its primitives.

Syntax

Data Type	Number
Default value	700

Remarks

This value represents the entity's maximum height, calculated on the basis of its primitives. This value is then stored inside the HTML element and can be used by the application to resize the window when its content is loaded.

Sample Code

```
var nEntityHeight = entViewerControl.entityHeight;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

printWidth Property

This property defines printing page width, depending on page type: A4, Letter, A3, Legal.

Syntax

Data Type	Number
-----------	--------

Default value	640
---------------	-----

Remarks

This property defines the page width according to page type. The real width depends also on the print mode: Portrait or Landscape.

Sample Code

```
var nPrintWidth = entViewerControl.printWidth;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

printHeight Property

This property defines printing page height, depending on page type: A4, Letter, A3, Legal.

Syntax

Data Type	Number
Default value	910

Remarks

This property defines the page height according to page type. The real height depends also on the print mode: Portrait or Landscape.

Sample Code

```
var nPrintHeight = entViewerControl.printHeight;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

highlightDivs Property

Array of highlighted HTML <div> elements, each representing a search hit.

Syntax

Data Type	Array
Default value	empty

Sample Code

```
var oHit = entViewerControl.highlightDivs[0];
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

selectedHitStyle Property

Property of the selected hit style: outline or imagefill.

Syntax

Data Type	String
Default value	SelHitBox

Remarks

This property defines the style of the selected hit. It may be **outline** (contour) or **imagefill** (semi-transparent image). This value is defined by the [owc:SelectedHitStyle](#) attribute. The default class name is equal to SelHitBox for the selected hit HTML element.

Sample Code

```
var sSelectedHitStyle = entViewerControl.selectedHitStyle;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

unselectedHitStyle Property

This property defines the style of unselected hits: outline or imagefill.

Syntax

Data Type	String
Default value	NormHitBox

Remarks

This property defines the style of the unselected hits. It may be **outline** (contour) or **imagefill** (semi-transparent image). This value is defined by the [owc:UnselectedHitStyle](#) attribute. The default class name is equal to NormHitBox for the unselected hit HTML element.

Sample Code

```
var sUnselectedHitStyle = entViewerControl.unselectedHitStyle;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

selectedHitColor Property

This property defines the color of a selected hit.

Syntax

Data Type	String
Default value	SelHitBox

Remarks

This property defines the color of a selected hit. Its value depends on the hit style. If the hit style is **outline**, then the color is defined as the HTML color (e.g., #FF00FF). If its style is **imagefill** then the color is defined as the name (without the .png extension) of the image file. This value is defined by the [owc:SelectedHitColor](#) attribute. By default, it sets the class name equal to SelHitBox for the selected hit HTML element.

Sample Code

```
var sSelectedHitColor = entViewerControl.selectedHitColor;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

unselectedHitColor Property

This property defines the color of unselected hits.

Syntax

Data Type	String
Default value	NormHitBox

Remarks

This property defines the color of unselected hits. Its value depends on the hits style. If hits style is **outline**, then color is defined as the HTML color (e.g., #FF00FF). If its style is **imagefill**, then the color is defined as the name (without the .png extension) of the image file. This value is defined by the [owc:UnselectedHitColor](#) attribute. The default class name is equal to NormHitBox for the unselected hit HTML element.

Sample Code

```
var sUnselectedHitColor = entViewerControl.unselectedHitColor;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

isHighlighting Property

This Boolean value defines how to use hit highlighting: as highlighted HTML <div> elements or by defining the appropriate class names.

Syntax

Data Type	Boolean
Default value	false

Sample Code

```
var bIsHighlighting = entViewerControl.isHighlighting;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

bAddArrowToHighlight Property

This property defines if an arrow is used for highlighting

Syntax

Data Type	Boolean
Default value	false

Remarks

This property shows a semi-transparent arrow that points to the selected hit. It is used when the hit is small and difficult to see. This property is defined by the owc:HighlightArrow attribute.

Sample Code

```
var bAddArrowToHighlight = entViewerControl.bAddArrowToHighlight;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

bArrowLoaded Property

This internal property is a flag that is set when the arrow image is loaded.

Syntax

Data Type	Boolean
Default value	false

Sample Code

```
var bArrowLoaded = entViewerControl.bArrowLoaded;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

arrowToHighlightUrl Property

This property defines the URL of the arrow image file

Syntax

Data Type	String
Default value	null

Remarks

This property is used to define the arrow image file URL. The value of this property is defined by the owc:ArrowUrl attribute.

Sample Code

```
var sArrowToHighlightUrl = entViewerControl.arrowToHighlightUrl;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

imgArrow Property

HTML element for arrow.

Syntax

Data Type	Number
Default value	-2

Remarks

This is an internal property.

Sample Code

```
var oImgArrow = entViewerControl.imgArrow;
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#) | EntViewer Properties

EntViewer Control CSS Classes

CSS Class	Description
NormHitBox	CSS class for normal (unselected) hit boxes
SelHitBox	CSS class for selected hit boxes
owcArrow	CSS class for the arrow

See Also

[EntViewer Overview](#) | [EntViewer Class](#)

EntViewer Example

The example below describes how to use the EntViewer control in the user application.

```
<div id="divEntViewer"
  owc:Control="EntViewer"
  owc:LayoutSet="Layout"
  owc:SelectedHitStyle="imagefill"
  owc:UnselectedHitStyle="imagefill"
  owc:SelectedHitColor="#FFFF00"
  owc:UnselectedHitColor="#0000FF"
  owc:HighlightArrow="true"
  owc:ArrowUrl="Layout/arrow.gif"
>
</div>
```

See Also

[EntViewer Overview](#) | [EntViewer Class](#)

EventSource Overview

The Web SDK has a mechanism for SDK objects (JavaScript) to send and handle events, in addition to the native HTML mechanism that handles browser events (such as onclick, onload, etc.). This mechanism is provided by the methods and properties of the `Olive.EventSource` class, from which all controls are derived by inheritance.

An SDK object typically fires an event of a certain name when the state of the control changes, for example when the control has been initialized or when data has been processed or received from the server. Before it is fired, the event object can have data properties assigned that describe the state of the object that fires the event.

The event bubbles up from the control to parent controls, in the hierarchy of nested controls, looking for a control that handles an event of that name. The bubbling starts with the control itself and bubbles up to the page and then to the application, if one exists.

When it finds a control that has defined a handler for the event, the handler is executed, using the event object to obtain data about the event. If more than one handler has been attached to the event, all the handlers are executed.

The bubbling stops after the handler is executed. If no object has a handler defined for the event, it bubbles up to the top of the hierarchy chain, and no action is taken.

An application can use the SDK events mechanism in several ways:

1. A page or application can define a static handler for an event that some control can fire. To do this, add a custom attribute to the HTML for the control, with the name of the default handler ("on" + the event name).

For example, to do something on a page when e-mail has been sent successfully, write the following:

```
owc:onMailSentOk="showMessageOnMailSent"
```

The function `showMessageOnMailSent()` should be defined on the HTML page.

2. A handler can be attached or detached dynamically, with the methods [attachOwcEventHandler\(\)](#) and [detachOwcEventHandler\(\)](#).

3. In theory, since the pages and the application are SDK controls, they can define and fire events with the methods [registerOwcEventsClass2\(\)](#), [createOwcEventObject\(\)](#), and [fireOwcEvent\(\)](#). In practice, this is not usually necessary, since the pages and the application can call methods directly, without using the event-handling mechanism.

4. In theory, the application can find an SDK object on a page which has a default event handler defined, and fire one of its events. In practice, it is better to execute the handler directly. The purpose of the event-handling mechanism is to make it convenient to respond to an event that occurs in a separate and possibly unknown source object.

See Also

[Control Overview](#) | [EventSource Class](#)

EventSource Class Overview

The EventSource base class functionality is implemented by the Olive.EventSource JavaScript class.

The Olive.EventSource interface allows access to the EventSource methods.

Inheritance Hierarchy

[Olive.EventSource is the root of the inheritance hierarchy. All controls inherit from Olive.EventSource.]

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[EventSource Overview](#)

EventSource Class Methods

EventSource Methods

Method	Description
registerOwcEventsClass2	Register an event name as the class of an event that this control can fire, specifying features of the event such as whether it bubbles and what its default handler method is.
attachOwcEventHandler	Attach a function to be a handler for an event.
detachOwcEventHandler	Detach a handler from an event.
detachAllOwcEventHandlers	Detach all handlers for an event.
createOwcEventObject	Create an event object that can be sent as the data object when the event is fired.
fireOwcEvent	Fire an event.

See Also

[EventSource Class](#)

registerOwcEventsClass2

This method registers an event name as the class of an event that this control can fire. It specifies features of the event, such as when it bubbles, and its default handler method.

Parameters

sName – string, name of the event

sHandlerProperty – string, name of the event's handler property

bSupportsBubbling - boolean

Does the event bubble up to ancestor controls, if this control does not have a handler?

rDefaultAction - function or string

A function or executable string specifying the action to be executed when the event is handled

oDefaultActionObject - object

Default object on which the default action is to be performed

cDefaultActionType - constant

Type of default action - One of these two constants: **Delegate.Type.Generic** or **Delegate.Type.Method**

Return Value

oEventClass - object

The object representing the event of this type.

Syntax

```
oEventClass = oControl.registerOwcEventsClass2( sName, sHandlerProperty,  
bSupportsBubbling, rDefaultAction, oDefaultActionObject, cDefaultActionType );
```

Example

```
oEventClass = oControl.registerOwcEventsClass2( "myEventOccurred", "onMyEventOccurred", true,  
myAppl_handlerFunction, null, Delegate.Type.Method );
```

See Also

[EventSource Overview](#) | [EventSource Class](#)

attachOwcEventHandler

This method attaches a function to be a handler for an event.

Parameters

rEventClass - string or object

Name of the event, or the EventClass object for the event.

rHandler - function or string

A function or executable string specifying the action to be executed when the event is handled

rObject - object

The object on to which the action is to be performed

cActionType - constant

Type of the action. One of these two constants: `Delegate.Type.Generic` or `Delegate.Type.Method`

Return Value

bSucceeded - boolean

Was the method executed successfully?

Syntax

```
bSucceeded = oControl.attachOwcEventHandler( rEventClass, rHandler, rObject, cType );
```

Example

```
bSucceeded = oControl.attachOwcEventHandler( "myEventOccurred", myAppl_handlerFunction, null, Delegate.Type.Method );
```

See Also

[EventSource Overview](#) | [EventSource Class](#)

detachOwcEventHandler

This method detaches a handler from an event, identifying it by the handler, object, and actionType used in the method [attachOwcEventHandler\(\)](#).

Parameters

rEventClass - string or object

Name of the event, or the EventClass object for the event.

rHandler - function or string

A function or executable string specifying the action that is executed when the event is handled

rObject - object

The object on to which the action is to be performed

cActionType - constant

Type of the action. One of these two constants: `Delegate.Type.Generic` or `Delegate.Type.Method`

Return Value

bSucceeded - boolean

Was the method executed successfully?

Syntax

```
bSucceeded = oControl.detachOwcEventHandler( rEventClass, rHandler, rObject, cType );
```


Example

```
bSucceeded = oControl.detachOwcEventHandler( "myEventOccurred",  
myAppl_handlerFunction, null, Delegate.Type.Method );
```

See Also

[EventSource Overview](#) | [EventSource Class](#)

detachAllOwcEventHandlers

This method detaches all handlers for an event.

Parameters

rEventClass - string or object

Name of the event, or the EventClass object for the event.

Return Value

bSucceeded - boolean

Was the method executed successfully?

Syntax

```
bSucceeded = oControl.attachOwcEventHandler( rEventClass );
```

Example

```
bSucceeded = oControl.attachOwcEventHandler( "myEventOccurred" );
```

See Also

[EventSource Overview](#) | [EventSource Class](#)

createOwcEventObject

This method creates an event object that can be sent as the data object when the event is fired.

Parameters

rEventClass - string or object

Name of the event, or an EventClass object representing an event of this type

Return Value

oEvent - object

The event object created. This should be used for firing the event, as the object of [fireOwcEvent\(\)](#).

Syntax

```
oEvent = oControl.createOwcEventObject( rEventClass );
```

Example

```
oEvent = oControl.createOwcEventObject( "myEventOccurred" );
```

See Also

[EventSource Overview](#) | [EventSource Class](#)

fireOwcEvent

This method fires an event.

Parameters

oEvent - object

The event object created by **createOwcEventObject()**

Syntax

```
oControl.fireOwcEvent( oEvent );
```

See Also

[EventSource Overview](#) | [EventSource Class](#)

FlashViewer Control Overview

The FlashViewer component shows a document in an interactive and rich interface, resembling reading and paging through a book. It includes additional features:

- Search results hits in a configurable highlight box, on the page
- Highlighted TOC entities while navigating
- Text mode view shows the digital text representation of an entity by clicking on it
- Smart zoom positions the enlarged box to best fit the clicked text area.
- Regular zoom enlarges the clicked area by repeated clicking
- Navigation by clicking the interactive page corners.
- Navigation and zoom keyboard shortcuts
- Interactive links on pages: web links, mail to links, go to page links, see box links (go to page reversible links)
- View rich media on pages
- Rotated the book view
- Page flipping animation effect that can be turned on and off

- Toggle between single-page and double-page spreads
- Document printing options

Syntax

```
<html-element owc:Control = FlashViewer
  owc:FlashComponentOptions = /options/
  owc:bgColor = /color/
  owc:onCommand = /handler/
  owc:onControlCreated = /handler/
  owc:onControlInitialized = /handler/
  owc:onErrorOccured = /handler/
  owc:onValueChanged = /handler/ >
</html-element>
```

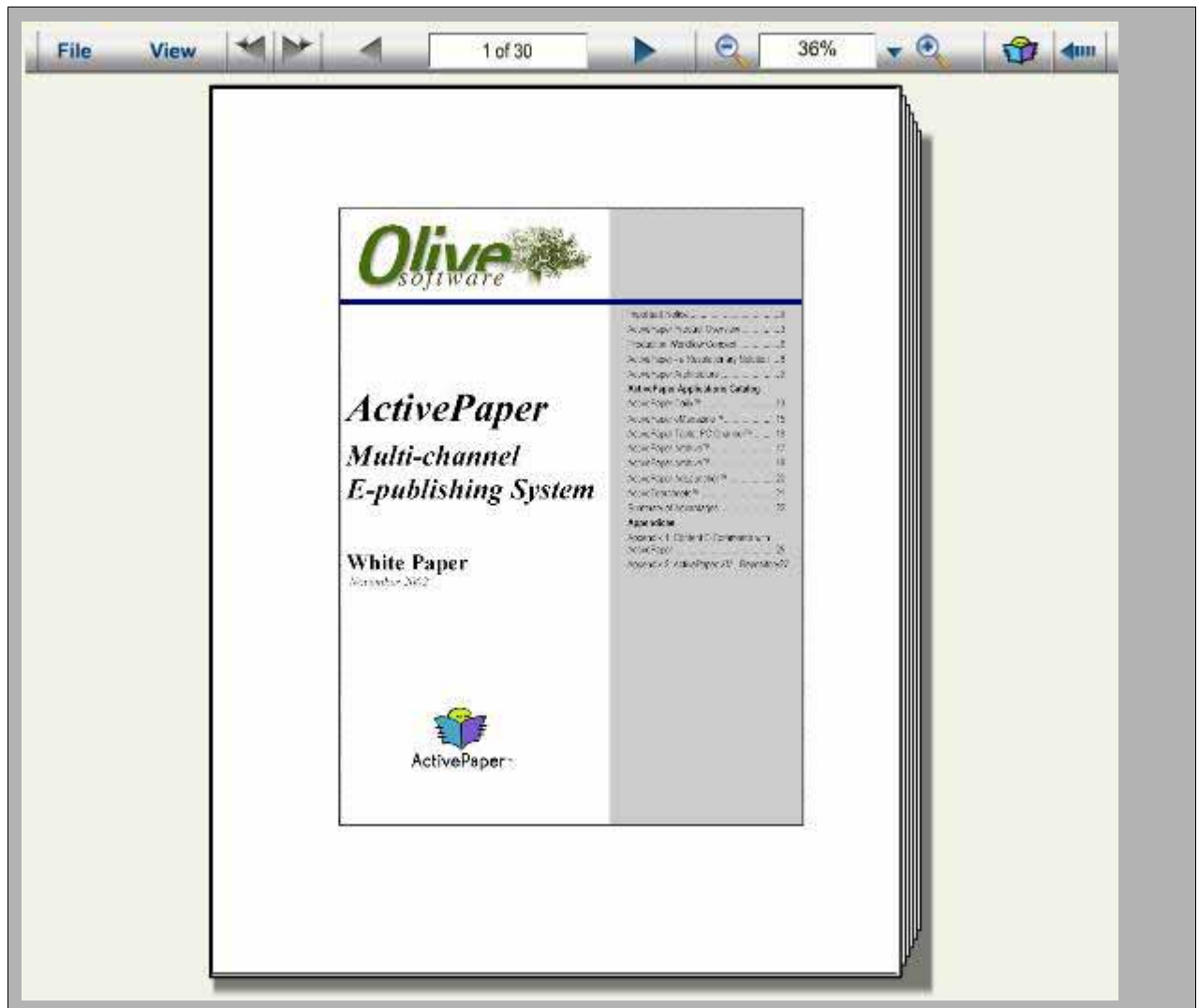
Remarks

FlashViewer component has its own configuration file for its many configurable features.

It is recommended to define a reader style in FlashReaderSettings.xml and use the style parameter as one of the options (in the [owc:FlashContentOptions](#) attribute).

The text for the toolbars and messages in FlashViewer is configured in the Language.xml file. The name of this file is specified in the FlashReaderSettings.xml file, <Language> node.

Preview



See Also

[FlashViewer Class](#) | [FlashViewer Attributes](#) | [FlashViewer Options](#)

Flash Viewer Options

The following parameters control the FlashViewer's look and feel.

These parameters are defined in three places:

FlashReaderSettings.xml file, as xml node names and values

[owc:FlashContentOptions](#) custom HTML attribute, separated by two vertical lines ("par1=val1||par2=val2||...")

[displayDocument\(\)](#) in the second argument, as an associative array of name-value pairs.

Remarks

displayDocument overrides **owc:FlashContentOptions**, which overrides **FlashReaderSettings.xml**

FlashReaderSettings.xml also has two places to define options, under the root node /BookReaderSettings and under /BookReaderSettings/ReaderStyle/{name-of-style}; the option for a style overrides the general option for /BookReaderSettings. The name of the reader style should be given in #2 owc:FlashContentOptions: "...||ReaderStyle={name-of-style}||..."

Parameter	Values	Description
Path	Path to the folder in Olives Warehouse	The Path is made up of the publication name together with the subdirectory name. This parameter displays the relevant document. For Example: PUB/2005/10/20 Note: This parameter is mandatory.
BookCollection	Collection name defined in Content Administrator. E.g. Sample_EP	This is the name entered in the Olive Content Administrator application, under Index Bindings (found under Settings in the Publication Details window). This parameter enables users to search a document. Note: This parameter is mandatory for searching.
Language	English, Hebrew, French	The issue language, i.e. if a documents text is French then the value should be French. The default is English, i.e. if no parameter is entered in the URL, the software assumes that a user would prefer English.
OnePageMode	0 or 1	This parameter displays a document in either double-page or single-page mode. 0 = Double Page Mode (false) - default 1 = One Page Mode (true)
MouseMode	0, 1, 2	The initial effect of clicking on the page. The user can click on View in the menu to change the mouse mode. 0 = Magazine Reading Style (smart zoom) - default 1 = Document Reading Style (standard zoom) 2 = Text Mode (pop-up text windows)

ReaderStyle	Name of a subnode of the <ReaderStyle> node in the FlashReaderSettings.xml file	One of the reading styles defined in the FlashReaderSettings.xml file. Users can define as many as needed. See parameter example below.
ViewFullCover	true, false	Defines if a document is opened showing the front and back pages or only the front page. true = Front and back pages false = Only the front page
Page	(integer)	Open the document on a specific page. For example: Page=5 will open the document on the fifth page.

Example

Below is an example of what a ReaderStyle node looks like:

```
<ReaderStyle>
  <Normal>
    <Loader>swf/Loader_new.swf</Loader>
    <ReaderInitMode>Normal</ReaderInitMode>
  </Normal>
  <Modem>
    <Loader>swf/Loader.swf</Loader>
    <ReaderInitMode>TextView</ReaderInitMode>
  </Modem>
</ReaderStyle>
```

Manipulating Documents on the same Server

The parameters listed in the table below are per server (i.e. they contain the parameters defined for all publications per server) and are edited or changed in the FlashReaderSettings.xml file.

Under ReaderStyle, define a new node for [ABCReaderStyle].

An existing ReaderStyle subnode can be copied and renamed to a new ReaderStyle node (For Example: copy FileCabinet and rename it to ABCReaderStyle). Add or change any parameters for the new ReaderStyle node.

Parameter	Values	Description
<Verification enabled="false">{a codeword}</Verification>	A string	Verification based on encrypted strings using code word

<LogFile logging = "on">BookReader.log</LogFile>	A file name	Name of log file
UsePNGFastConversion	true, false	Fast conversion from PNG to SWF files
<OfflineBookFolder auto-cache="on">	A physical directory path	Save files to this directory for the work offline feature
RepUrl	Virtual directory name	The virtual directory of the Warehouse
AppName	EP	The application name for the Web SDK is EP
ReverseRTLStrings	true, false	Reverse string direction in right-to-left languages, such as Arabic or Hebrew.
ShowPrimitiveBox	true, false	Show highlight box for primitives
ConnectChunks	true, false	Connect the chunks of an entity that is divided into more than one page.
TOCMode	0, 1, 2	0 - The table of Contents appears within the Flash viewer 1 - TOC is an external HTML element 2 - Disable TOC
PlayCorner	true, false	Play animation of moving corners until a page is turned.
Preload	A number	How many pages to preload adjacent to the current page, for faster paging.
ToolbarMinimized	true, false	Start with the toolbar minimized
EnableFlippingEffect	true, false	Enable or disable the flipping effect (the animated appearance of a page turning, when you change the page). If true, a check mark () appears alongside the flip effect option in the viewer. true = Enables flipping effect false = Disables flipping effect
EnablePagesPrint	true, false	Enable or disable the printing feature. true = Enables Print feature false = Disables Print feature

DefaultZoomScale	100, 50 (%)	Sets the smart zoom level. 100% is the default setting.
CloseContentOnArt	true, false	TOC in FlashViewer may stay open or not when the user selects an article. true = Close the TOC when an article has been selected false = Do not close the TOC automatically
ViewTOCSections	true, false	Readers see different articles (within the TOC) under different sections. true = Users will see the different sections within the TOC false = Users will see only the articles without the sections
ViewAdText	true, false	Allow users to view ads in text mode. true = Enable viewing ads in text mode false = Disable viewing ads in text mode
MouseMode	0, 1, 2	Initial mouse effects in the viewer. Click on the View menu to change the mouse mode (Smart Zoom, Regular Zoom or Text Mode). 0 = Smart Zoom 1 = Regular Zoom 2 = Text Mode
Language	Language.xml	The name of the XML file containing the strings for FlashViewer's toolbar texts and messages.
ReaderBackgroundColor	Enter RGB color e.g. #808080	FlashViewer's background color.
<HighlightBoxes> <LinesWidth>1</LinesWidth> <LinesColor>#FEF232</LinesColor> <BackgroundColor>#FEF232</BackgroundColor> </HighlightBoxes>		The width, line color and box colors of the search hit boxes.

<pre> <HighlightCaptionBoxes> <LinesWidth>1</LinesWidth> <LinesColor>#006DF2</LinesColor> <BackgroundColor>#006DF2</BackgroundColor> </HighlightCaptionBoxes> </pre>		The width, line color, and box colors of the article headlines (captions).
<pre> <HighlightArticles switch="off"> <HighlightOnMouseOver>no </HighlightOnMouseOver> <Color>#FF0000</Color> </HighlightArticles> </pre>		The color highlighting of entire articles.
<pre> <HighlightLinks switch="on"> <Color>#C096EA</Color> </HighlightLinks> </pre>	on, off	Highlight links, and in what color. on = will be highlighted (recommended) off = will not be highlighted (not recommended)
<pre> <ZoomMenuValues>75 100 150 200</ZoomMenuValues> </pre>	For example: 75, 100, 150	The FlashViewer's zoom settings (in the pre-selected zoom drop down menu). Predefined zoom levels can be changed to any percentage.
<pre> <Zoom> <ZoomOutOnFlip>>false </ZoomOutOnFlip> <ZoomOutRegularOnFlip>>false </ZoomOutRegularOnFlip> <ZoomOutOnLink>true </ZoomOutOnLink> <ZoomOutOnTocEntry>true </ZoomOutOnTocEntry> </Zoom> </pre>		Go back to normal zoom size when a page is turned, or a link is clicked, or a TOC entry is selected.

<pre> <Margins> <BookPos_LeftMargin>5 </BookPos_LeftMargin> <BookPos_RightMargin>5 </BookPos_RightMargin> <BookPos_TopMargin>4 </BookPos_TopMargin> <BookPos_BottomMargin>4 </BookPos_BottomMargin> </Margins> </pre>	For example: 5, 5, 4, 4	The left, right, top and bottom margins in pixels.
UseCustomCursor	yes, no	Enable the following regular mouse cursors: hand/finger, magnify glass and arrow with a hand writing. yes = Enables the regular mouse cursor no = Disables the regular mouse cursor (recommended)
ShowLinkCursor	yes, no	Enable the link, e-mail and info cursors. yes = Enables the above cursors (recommended) no = Disables the above cursors
ShowScrollBars	yes, no	Display scroll bars.
<pre> <StatisticsSettings ENABLED="true" DEFAULT="false"> - <Publications> < Publication NAME="Six" ENABLED="true"/ </Publications> </StatisticsSettings> </pre>	ENABLED = true/false	This is the application's main statistics server parameter. It controls the Statistics Server feature for all publications, or a single publication. If ENABLED=false, then no document residing on this server will have the statistics server feature enabled. If ENABLED=true and DEFAULT=true then the Urchin Statistics Server feature will be enabled for all publications except those that have a sub-node defined If ENABLED=true and DEFAULT=false then the Urchin Statistics Server feature will be disabled for all publications except those that have a sub-node defined See the example on the left - here, the Urchin Statistics Server feature is disabled for all publications except "Six".

Enabling PDF downloads - (The style below is to be added to the FlashReaderSettings.xml file):

<pre><Skins> <skin id="1" resolution="1" language="english"> swf/SkinAM_X.swf</skin> </Skins></pre>	swf/SkinAM_X.swf	The file SkinAM_X.swf must be in the swf directory. To enable PDF download in FlashViewer, create a mapping (virtual directory under VD Repository) with the publications warehouse name, pointing to the publications warehouse folder. This can be done in the Content Administrator application. Click Settings then double-click on the publication, then click "Map on server". Or, perform this manually using the IIS MMC (Note - Map on server fails more than it works in the distributed warehouse environment). In a distributed warehouse environment, if the mapping was done automatically, it has to be edited (properties), click Connect and enter the distributed warehouse user credentials.
---	------------------	--

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#) | [FlashViewer Attributes](#)

FlashViewer HTML Attributes

Flash Viewer Specific Attributes

Attribute	Description
owc:FlashContentOptions	Specifies FlashViewer parameters, as separated by two vertical lines: "par1=val1 par2=val2 ..." One of the parameters should be "ReaderStyle", with most of the options for that style given in FlashReaderSettings.xml. See "FlashViewer Options" for details.
owc:bgColor	Specifies the background color of the viewer component The value is given as RGB hexadecimal numbers: "808080"

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#) | [FlashViewer Options](#)

owc:bgColor

This parameter specifies the background color for the Flash reader.

Syntax

```
<html-element owc:bgColor = /color/ >
...
</html-element>
```

Remarks

The color is an RGB color value represented as a hexadecimal number.

Example

```
<div owc:Control="FlashViewer" owc:bgColor="#F2F2E6">
```

Usage

Control	FlashViewer
Use	Optional
Default value	808080 light gray.

See Also

FlashViewer Control | [FlashViewer Attributes](#) | [FlashViewer Options](#)

owc:FlashContentOptions

This attribute specifies FlashViewer parameters.

Syntax

```
<html-element owc:FlashContentOptions = /string/ >
...
</html-element>
```

Remarks

The string contains pairs of parameter=value separated by ||, i.e. it has the following format parameter1=value1||parameter2=value2

For most parameters, it is recommended to use ReaderStyle, with a declaration of the appropriate style in FlashReaderSettings.xml. See on FlashReader Options.

Example

```
<div owc:Control="FlashViewer"
owc:FlashContentOptions="ReaderStyle=FileCabinet||TOCMode=1||OnePageMode=0||OpenTOCA
tStart=0||ShowPrimitiveBox=true">
```

Usage

Control	FlashViewer
Use	Optional
Default value	Default values are defined in <code>FlashReaderSettings.xml</code>

See Also

FlashViewer Control | [FlashViewer Attributes](#) | [FlashViewer Options](#)

FlashViewer Class Overview

The FlashViewer component functionality is implemented by the `Olive.Controls.FlashViewer` JavaScript class.

The `Olive.Controls.FlashViewer` interface allows access to the FlashViewer component methods and events.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Viewer

Olive.Controls.FlashViewer

Requirements

Script Files	OwcFlashViewer.js
--------------	-------------------

See Also

[FlashViewer Overview](#) | [FlashViewer Methods](#) | [Viewer Overview](#)

FlashViewer Class Events

FlashViewer Events

Event	Description
flashEventReceived	An event was received by the FlashViewer control from the Flash browser object. The event can be any of the following specific events (print, pageTurned, pageModeChanged, toggleFrame).
print	The user clicked on the print button.
pageTurned	A page was turned in the document.

pageModeChanged	The page mode was changed from 2-page to 1-page mode or vice versa.
toggleFrame	The TOC pane was opened or closed.

Remarks

The Flash browser object can initiate any of the four specific listed events. When the event is received by the FlashViewer control, it first fires a general event, "flashEventReceived", and then the specific event.

If the Flash browser object initiates an event that is not one of the four registered events, the FlashViewer control fires the "flashEventReceived" event, and then it calls the virtual member function processCustomFlashEvent(), which is defined to be empty {} in the SDK.

For all these cases (the general event, the specific event, and the virtual function) it passes the parameters received from the Flash object for the event (see on each specific event).

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#)

flashEventReceived Event

An event was received by the FlashViewer control from the Flash browser object. The event can be any of the following specific events:

- * print
- * pageTurned
- * pageModeChanged
- * toggleFrame

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following expando property:

1. oFlashEventData (object) - a name-value associative array. For individual values, see the specific events (print, pageTurned, pageModeChanged, toggleFrame)

Supports bubbling: true

Example

```
// Handler of the event
function Application_onFlashEventReceived(oEvent)
{
    // desirable action
}
```

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#)

print Event

The user clicked the print button.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following expando properties:

1. sWndName (string) - browser name of the print window
2. sPath (string) - URL of the print window
3. sDataProto (string) - data protocol (can be "file")
4. RightPage (number) - right page currently displayed
5. LeftPage (number) - left page currently displayed (0 if there is no left page)
6. BookLength (number) - total number of pages
7. Resolution (number) - image resolution
8. graphicImgExt (string) - file name extension, e.g. ".png"
9. textImgExt (string) - file name extension, e.g. ".png"
10. defaultImgExt (string) - file name extension, e.g. ".png"
11. PageWidth (number) - width in pixels
12. PageHeight (number) - height in pixels
13. bMultilayer (boolean) - whether this is a multi-layer document
14. sBaseHref (string) - BaseHref path of the document

Supports bubbling: true

Example

```
// Handler of the event
function Application_onPrint(oEvent)
{
    // desirable action
}
```

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#)

pageTurned Event

A page was turned in the document.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following expando properties:

1. OnePageMode (boolean) - whether the viewer is in one page mode (true) or two page mode (false)
2. currentPageId (number) - the current page

Supports bubbling: true

Example

```
// Handler of the event
function Application_onPageTurned(oEvent)
{
    // desirable action
}
```

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#)

pageModeChanged Event

The page mode was changed between double-page spread to single-page or the opposite

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following expando properties:

1. OnePageMode (boolean) - whether the viewer is in one page mode (true) or two page mode (false)

Supports bubbling: true

Example

```
// Handler of the event
function Application_onPageModeChanged(oEvent)
```



```
{  
    // desirable action  
}
```

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#)

toggleFrame Event

The TOC pane was opened or closed.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with no expando properties.

Supports bubbling: true

Example

```
// Handler of the event  
function Application_onToggleFrame(oEvent)  
{  
    // desirable action  
}
```

See Also

[FlashViewer Overview](#) | [FlashViewer Class](#)

FlashViewer Class Methods

FlashViewer Methods

Method	Description
displayDocument (bNewReader, oReaderSettingss)	Opens a Flash reader and displays the document in it, or go to a different page of an open document.
closeTOC ()	Sends a message to the Flash object to close the TOC.
gotoPage (nPage, sPrimId)	Go to a specified page in the currently open document.

Virtual functions that can be implemented in the application

Method	Description
--------	-------------

[processCustomFlashEvent\(\)](#)

The default handler for any custom event sent by the flash browser object that is not one of the registered FlashViewer events. It is empty {} unless it is overridden.

See Also

[FlashViewer Overview](#)

displayDocument Method

This method opens a Flash reader and displays a document in it, or goes to a different page of an open document.

Parameters

boolean - **bNewReader**

true: open a Flash reader; false: go to a different page of an open document

object - **oReaderSettings**

An associative array of parameter names and values for the Flash Reader, that will be passed to the Flash object in its query string (for example, OpenTOCAtStart, OnePageMode, ...).

Syntax

```
oFlashViewer.displayDocument(bNewReader, oReaderSettings);
```

Example

```
oFlashViewer.displayDocument( true,  {"OnePageMode" : true} );
```

See Also

[FlashViewer Overview](#) | [FlashViewer Methods](#) | [FlashViewer Events](#)

closeTOC Method

This method sends a message to the Flash object to close the TOC.

Parameters

No parameters

Syntax

```
oFlashViewer.closeTOC();
```

See Also

[FlashViewer Overview](#) | [FlashViewer Methods](#) | [FlashViewer Events](#)

gotoPage Method

Go to a specified page in the currently open document.

Parameters

number - **nPage**

Go to the page with this page number

string - **sPrimId**

Go to the page with this primitive ID.

Syntax

```
oFlashViewer.gotoPage(nPage, sPrimId);
```

Example

```
oFlashViewer.gotoPage( 3, "Ar0010000" );
```

See Also

[FlashViewer Overview](#) | [FlashViewer Methods](#) | [FlashViewer Events](#)

processCustomFlashEvent Method

This is the default handler for any custom event sent by the Flash browser object that is not one of the registered FlashViewer events. It is empty {} unless it is overridden.

Parameters

object - **oCustomData**

A name-value associative array

Syntax

```
oFlashViewer.processCustomFlashEvent = function myHandler(oCustomData) {  
    //further processing  
}
```

See Also

[FlashViewer Overview](#) | [FlashViewer Methods](#) | [FlashViewer Events](#)

FolderTree Overview

The FolderTree component shows a tree of document folders in a repository. The typical functionality of a folder tree includes navigating through the folder tree by means of user-input nodes (clicking on "+" opens the subtree, clicking on "-" closes the subtree), and selecting a row of the tree (typically to display a document list of documents in the folder, or to limit a search to the folder).

The FolderTree component inherits from Owctree the attribute owc:MultiSelect (default value = false). If this is set to "true", it is possible to select several nodes by control-click (for example, to perform some repository-related action in an application like "Director").

The appearance and behaviour of the folder tree must be determined by the application's Layout.xml file (it is not possible to put this information directly into the HTML pages, since it must be replicated for each folder-tree-item). The behaviour can include clicking to open a document list for the folder - this is done by adding a "content-cmd" attribute to the tree-node-type definition for Folder in Layout.xml. The appearance can be set by adding a cssClass attribute to the owc:NodeType node in the HTML definition (see below).

Syntax

```
<div id="divFolderTree" owc:Control="FolderTree"
owc:onSelectionChanged="onFolderSelectionChanged">
  <owc:Data type="TreeLayout">
    <owc:NodeType name="Folder" iconUrl="Layout/folder.gif"
cssClass="folderNode" cssClassSelected="folderNodeSelected" />
    <owc:NodeType name="Inbox" iconUrl="Layout/inbox.gif"
cssClass="folderNode" cssClassSelected="folderNodeSelected" />
  </owc:Data>
</div>
in Layout.xml:
<owct:tree-template for="FolderTree">
  <owct:tree-node-type name="Folder" content-cmd="ViewFolderContent">
    <owct:tree-node-layout>
      <cssClass>folderNode</cssClass>
      <iconUrl>Layout/folder.gif</iconUrl>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>folderNodeSelected</cssClass>
    </owct:tree-node-layout>
  </owct:tree-node-type>
  ...
```

See Also

[FolderTree Class](#) | [Tree Overview](#)

FolderTree HTML Attributes

FolderTree Specific Attributes

Attribute	Description
-----------	-------------

owc:SpecialFolders	This attributes determines when to include folders that are designated as "special folders" in the Inventory, such as the folder of unassociated documents. Type Boolean; optional; default value = false.
---------------------------	---

See Also

[FolderTree Overview](#) | [FolderTree Class](#) | [Tree Overview](#)

OwcFolderTree Class Overview

The FolderTree component is implemented by the Olive.Controls.FolderTree JavaScript class.

The Olive.Controls.FolderTree interface provides access to the FolderTree component methods and events.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Tree

Olive.Controls.FolderTree

Implemented Interfaces

Olive.IDataBound

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[FolderTree Overview](#) | [FolderTree Methods](#) | FolderTree Events | [Tree Overview](#)

OwcFolderTree Class Methods

OwcFolderTree Methods

Method	Description
displayTree(Treeld)	Download the data for the folder tree and display it on the page. Arguments: Treeld - tree uid in inventory (number), or tree name (string)
getDisplayName()	Returns the display name of the folder tree.

See Also

[FolderTree Overview](#) | [FolderTree Class](#)

displayTree Method

This method downloads and displays the folder tree data.

An application should refer to a folder tree by its name, not by number, since the number is not constant.

Parameters

vTree - Number or string, tree-uid in inventory (number), or tree name (string)

Syntax

```
oFolderTree.displayTree( sTree );
```

Example

```
var sFolderTree = this.WebApplication.getPreference("Tree");
if (sFolderTree)
{
    oFolderTree.displayTree(sFolderTree);
}
```

See Also

[FolderTree Overview](#) | [FolderTree Class](#)

getDisplayName Method

This method returns the display name of the folder tree. It is used to display the name somewhere else on the page in addition to on the tree itself.

Parameters

No parameters

Syntax

```
oFolderTree.getDisplayName();
```

Example

```
var sName = oFolderTree.getDisplayName();
oHtmlCaptionText.innerHTML = sName;
```

See Also

[FolderTree Overview](#) | [FolderTree Class](#)

OwcFolderTree Class Properties

OwcFolderTree properties

Property	Description
displayName	Display name of the Folder Tree

See Also

[FolderTree Overview](#) | [FolderTree Class Overview](#)

displayName Property

This property specifies the display name of the Folder Tree.

This property should be accessed by the member method `getDisplayName()`, and not directly.

Syntax

Data Type	String
Default value	empty string

Remarks

The data for this property comes with the folder-tree data from the server.

Sample Code

```
var sDisplayName= folderTreeControl.displayName;
```

See Also

[FolderTree Overview](#) | [FolderTree Class Overview](#)

FolderTree Example

The example below shows the FolderTree control used in the application.

```
<div id="divFolderTree" owc:Control="FolderTree"
owc:onSelectionChanged="onFolderSelectionChanged">
  <owc:Data type="TreeLayout">
    <owc:NodeType name="Folder" iconUrl="Layout/folder.gif"
cssClass="folderNode"
cssClassSelected="folderNodeSelected" />
    <owc:NodeType name="Inbox" iconUrl="Layout/inbox.gif"
cssClass="folderNode"
cssClassSelected="folderNodeSelected" />
  </owc:Data>
</div>
```

```

</div>
in Layout.xml:
<owct:tree-template for="FolderTree">
  <owct:tree-node-type name="Folder" content-
cmd="ViewFolderContent">
    <owct:tree-node-layout>
      <cssClass>folderNode</cssClass>
      <iconUrl>Layout/folder.gif</iconUrl>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>folderNodeSelected</cssClass>
    </owct:tree-node-layout>
  </owct:tree-node-type>
  ...

```

See Also

[FolderTree Overview](#) | [FolderTree Class Overview](#)

Global Functions

Cross browser functions

Functions	Description
Olive.CmdTarget.RegisterCommand	Registers command on object; is usually applied on class prototypes.
Olive.Page.GetPageForWindow	Retrieves Olive page object for the window object.
Olive.Controls.IDataBound.getContentItem	Retrieves content item object, seeking it also in ancestors.
Olive.WindowBinder.RegisterPageClass	Registers page class for the window.
Olive.WindowBinder.RegisterPageCustomPrototype	Registers page prototype for the window.
OwcGetControlFromHtmlElem	Retrieves a control object by HTML element.
JScript_DeriveClass	Associates one class as a derived class from another one.

OwcGetControl	Retrieves a control object by ID attribute of the HTML element.
Object.Clone	Clones an object.
String.parseBoolean	Parses a string and returns Boolean value: true for strings 1, on, true, yes and false otherwise.
DOM.CreateFragmentFromContent	Creates a document fragment of DOM model objects from content. ??
DOM.MoveContentToFragment	Moves content of the source HTML element to the document fragment. ??
DOM.MoveContentToParent	Moves the content to the parent element. ??
DOM.ClearContent	Clears the content. ??
DOM.PasteContent	Pastes the content to the source element. ??
OwcGlobals_standardInit	?? more like a member function of the OwcGlobals class
Olive.Resources.defineResString	Defines a resource string.
Olive.Resources.getResString	Retrieves a resource string for the specified resource name.
Olive.Resources.defineGlobalResString	Defines a global resource string.
Olive.Errors.getErrorInfo	Retrieves error info from an internal array of errors by specified error ID.
Olive.Errors.createError	Creates an error object.
Olive.Errors.defineKnownError	Defines a global error.
Olive.Errors.getLocalizedFormatString	Retrieves a localized format string.
Olive.Errors.getLocalizedErrorMsg	Retrieves a localized error message.
String.parseInt	Returns an integer converted from a string.
String.format	Calls String_format_arg() function and

	returns the result formatted string.
String_format_arg	Returns formatted string.
Number.prototype.toPrecision	Overridden by the base functionality of the Number Javascript object. Returns a string with the specified number of digits.
Function.prototype.apply	Overridden by the base functionality of the Function Javascript object. Applies an object method, substituting another object for the current object.
JScript_isDate	
Object_CloneWithoutConstructor	
Object_Destroy	
Array_contains	
Array_indexOf	
Array_appendItem	
Array_removeItem	
Array_append	
Array_remove	
Array_difference	
EMail_validateAddress	
EMail_validateAddressList	
Function_override	
generateUniqueID	
Olive.WebAppBase.ApplyPrototype	
Olive.EventSource.hookEvent	
Olive.Object.FindParentImplementing	
Olive.Controls.getContextMenu	

Olive.Controls.RegisterControlType	
Olive.ContentItem.buildRefParams	
Olive.Controls.IDataBound.getHtmlContentTargetNull	Retrieves null as an HTML content target, meaning that you don't want to clear the original content of HTML element after receiving data from server.
Olive.Controls.IDataBound.Request_onDataReceived	??
Olive.Controls.IDataBound.Request_onDataError	??
Olive.Controls.DataType.getDefaultFormat	
Olive.Controls.DataType.toString	
Olive.Controls.DataType.parseValue	
Olive.Controls.DataType.validateValue	
Olive.Controls.DataType.createNewValue	
Olive.Controls.DataType.compareValue	
Olive.Controls.DataType.RegisterDataType	
Olive.Controls.DataType.RegisterClassDataType	
Olive.Controls.DataType.LookupDataType	
Olive.Controls.DataType.String.formatValue	
Olive.Controls.DataType.String.parseValue	
Olive.Controls.DataType.String.validateValue	
Olive.Controls.DataType.Boolean.formatValue	
Olive.Controls.DataType.Boolean.parseValue	
Olive.Controls.DataType.Boolean.validateValue	
Olive.Controls.DataType.Number.formatValue	
Olive.Controls.DataType.Number.parseValue	

Olive.Controls.DataType.Number.validateValue	
Olive.Controls.DataType.Date.getDefaultFormat	
Olive.Controls.DataType.Date.formatValue	
Olive.Controls.DataType.Date.parseValue	
Olive.Controls.DataType.Date.validateValue	
Olive.Controls.DataType.Year.getDefaultFormat	
Olive.Controls.DataType.Year.formatValue	
Olive.Controls.DataType.Year.parseValue	
Olive.Controls.DataType.Year.validateValue	
Olive.Controls.DataType.Month.getDefaultFormat	
Olive.Controls.DataType.Month.formatValue	
Olive.Controls.DataType.Month.parseValue	
Olive.Controls.DataType.Month.validateValue	
Olive.Controls.DataType.Day.getDefaultFormat	
Olive.Controls.DataType.Day.formatValue	
Olive.Controls.DataType.Day.parseValue	
Olive.Controls.DataType.Day.validateValue	
Olive.Controls.DataType.Email.validateValue	
Olive.Controls.DataType.EmailList.validateValue	
OwcListRegisterItemType	
Olive.Page.GetPageForDocument	Retrives Olive page object for the document object.
OwcTreeRegisterNodeType	
Olive.WindowOptions.getDefaultOptions	
Olive.WindowOptions.parsePosValue	

Olive.WindowBinder.GetBinderForWindow	Retrieves OliveBinder object of the window.
Olive.WindowBinder.HTML_onPageLoaded	??
Olive.XHTTP.Initialize	
Olive.XHTTP.CreateRequest	
Olive.XHTTP.Request.ApplyPrototype_XMLHTTP	
Olive.XHTTP.Request.ApplyPrototype_XmlIsland	
XMLHTTP_CreateRaw	Internal?
XMLHTTP_CreateMsXml2	Internal?

Olive.CmdTarget.RegisterCommand Method

This function registers a command on an object. It is usually applied on class prototypes.

Parameters

rObject - Object on which to register the command

sCommandName - String that specifies a name of the command

sHandlerProp - String that specifies a name of the command handler

sUpdateUiStateProp - String that specifies a name of the handler for updating UI state

Syntax

```
Olive.CmdTarget.RegisterCommand(rObject, sCommandName, sHandlerProp,
sUpdateUiStateProp);
```

Example

```
Olive.CmdTarget.RegisterCommand(this, showmetadata, ShowItemMetadata,
updateCommandUiState);
Olive.CmdTarget.RegisterCommand(rObject, "ShowSearchRes", "ShowSearchRes");
```

See Also

[Global Functions](#)

Olive.Page.GetPageForWindow Method

This function retrieves an Olive page object for the window object.

Parameters

oWindow - Window JavaScript object

Syntax

```
Olive.Page.GetPageForWindow(oWindow);
```

Example

```
var oPage = Olive.Page.GetPageForWindow(oWindow);
```

See Also

[Global Functions](#)

Olive.Page.GetPageForDocument Method

This function retrieves an Olive page object for the Document object.

Parameters

oDocument - Document JavaScript object

Syntax

```
Olive.Page.GetPageForDocument(oDocument);
```

Example

```
var oPage = Olive.Page.GetPageForDocument(oDocument);
```

See Also

[Global Functions](#)

Olive.Controls.IDataBound.getContentItem Method

This function retrieves content item object, seeking it also in ancestors.

Parameters

oControl - WebSDK control object

Syntax

```
Olive.Controls.IDataBound.getContentItem(oControl);
```

Example

```
var oContentItem = Olive.Controls.IDataBound.getContentItem(oTreeNode);
```

See Also

[Global Functions](#)

Olive.WindowBinder.RegisterPageClass Method

This function registers the window's page class.

Parameters

oWindow - Window JavaScript object

rPageClass - Page class object, derived from the [Olive.Page](#) class

Syntax

```
Olive.WindowBinder.RegisterPageClass(oWindow, rPageClass);
```

Example

```
Olive.WindowBinder.RegisterPageClass(window, Application_CalendarPage);
```

See Also

[Global Functions](#)

Olive.WindowBinder.RegisterPageCustomPrototype Method

This function registers the window's page prototype.

Parameters

oWindow - Window JavaScript object

pfbApplyCustomProto - Pointer to the page prototype's function

Syntax

```
Olive.WindowBinder.RegisterPageCustomPrototype(oWindow, pfbApplyCustomProto);
```

Example

```
Olive.WindowBinder.RegisterPageCustomPrototype(window,  
Application_MetadataPage_ApplyProto);
```

See Also

[Global Functions](#)

OwcGetControlFromHtmlElem Method

This function retrieves a control object according to the HTML element.

Parameters

oHtmlElem - HTML element

bScanAncestors - Boolean that specifies when to scan ancestor HTML elements, searching for a Control object within

Syntax

```
OwcGetControlFromHtmlElem(sControlId, bScanAncestors);
```

Example

```
var oPaneSplitter = OwcGetControlFromHtmlElem(this);
```

See Also

[Global Functions](#)

JScript_DeriveClass Method

This function associates one class as derived from another one.

Parameters

rDerivedClass - Reference to the derived class

rBaseClass - Reference to the base class

Syntax

```
JScript_DeriveClass(rDerivedClass, rBaseClass);
```

Example

```
JScript_DeriveClass(Application_CalendarPage, Olive.Page);
```

See Also

[Global Functions](#)

OwcGetControl Method

This function retrieves a control object according to the HTML element's ID attribute.

Parameters

sControlId - String that specifies the HTML element's ID attribute

bScanAncestors - Boolean that specifies when to scan ancestor HTML elements, searching for a Control object within

Syntax

```
OwcGetControl(sControlId, bScanAncestors);
```

Example

```
var oFlashViewerControl = OwcGetControl(divFlashViewer);
```


See Also

[Global Functions](#)

Object_Clone Method

This function clones an object.

Parameters

objSrc - A source object for cloning

Syntax

```
Object_Clone(objSrc);
```

Example

```
// Clones command parameters for further changes of the cloned object  
var oContentItem = Object_Clone(oCmdParams);
```

See Also

[Global Functions](#)

String_parseBoolean Method

This function parses a string and returns Boolean value: true for strings 1, on, true, and yes; and false otherwise.

Parameters

sValue - String for parsing

bDefault - Default Boolean value

Syntax

```
String_parseBoolean(sValue, bDefault);
```

Example

```
var bDebugMode = String_parseBoolean(sApplicationDebugParam, false);
```

See Also

[Global Functions](#)

DOM.CreateFragmentFromContent Method

This function creates a document fragment of HTML elements from the content.

Parameters

oSrcElem - Source HTML element from which to create the fragment

Syntax

```
DOM.CreateFragmentFromContent(oSrcElem);
```

Example

```
var oFragment = DOM.CreateFragmentFromContent(oHtmlElement);
```

See Also

[Global Functions](#)

DOM.MoveContentToFragment Method

This function moves content of the source HTML element to the document fragment.

Parameters

oSrcElem - Source HTML element

Syntax

```
DOM.MoveContentToFragment(oSrcElem);
```

Example

```
var oFragment = DOM.MoveContentToFragment(oHtmlElement);
```

See Also

[Global Functions](#)

DOM.MoveContentToParent Method

This function moves the source HTML content to the parent element.

Parameters

oSrcElem - Source HTML element

Syntax

```
DOM.MoveContentToParent(oSrcElem);
```

Example

```
DOM.MoveContentToParent(oHtmlElement);
```

See Also

[Global Functions](#)

DOM.ClearContent Method

This function clears the content.

Parameters

oSrcElem - Source HTML element

Syntax

```
DOM.ClearContent(oSrcElem);
```

Example

```
DOM.ClearContent(oHtmlElement);
```

See Also

[Cross Browser Functions](#)

DOM.PasteContent Method

This function pastes the content to the source element.

Parameters

oSrcElem - Source HTML element

oContent - HTML element with a content to paste

bAppend - Boolean specifies whether append the content to the source element or clear the content before pasting the new content

Syntax

```
DOM.PasteContent(oSrcElem, oContent, bAppend);
```

Example

```
DOM.PasteContent(oHtmlElement, oContent, true);
```

See Also

[Global Functions](#)

Olive.Resources.defineResString Method

This function defines a resource string and places it in the internal array of resource strings.

Parameters

sResName - String that specifies a resource string with an identical name

sResString - String that specifies a resource string value

bDoNotReplace - Boolean that specifies when to replace the resource string; if that string is already in the internal array of resource strings, or replace with the new value

Syntax

```
Olive.Resources.defineResString(sResName, sResString, bDoNotReplace);
```

Example

```
Olive.Resources.defineResString(Downloading, Downloading data, please wait);
```

See Also

[Global Functions](#)

Olive.Resources.getResString Method

This function retrieves a resource string for the specified resource name.

Parameters

sResName - String that specifies a resource string with an identical name

Syntax

```
Olive.Resources.getResString(sResName);
```

Example

```
var sResString = Olive.Resources.getResString(Downloading);
```

See Also

[Global Functions](#)

Olive.Resources.defineGlobalResString Method

This function defines a global resource string and places it in the global array of resource strings.

Parameters

sResConstName - String that specifies a constant global resource string with an identical name

sResName - String that specifies a resource string with an identical name

sResString - String that specifies a resource string value

bDoNotReplace - Boolean that specifies when to replace the resource string; if that string is already in the internal array of resource strings, or replace it with the new value

Syntax

```
Olive.Resources.defineGlobalResString(sResConstName, sResName, sResString, bDoNotReplace);
```

Example

```
Olive.Resources.defineGlobalResString(g_sDownloadResName, Downloading, Downloading data, please wait);
```

See Also

[Global Functions](#)

Olive.Errors.getErrorInfo Method

This function retrieves error information from an internal errors array according to a specified error ID.

Parameters

nErrId - Number that specifies an error ID

Syntax

```
Olive.Errors.getErrorInfo(nErrId);
```

Example

```
Olive.Errors.getErrorInfo(Olive.Errors.FirstSdkErrorId);
```

See Also

[Global Functions](#)

Olive.Errors.createError Method

This function creates an error object by an error ID. It can get an additional parameter (string that specifies an error message) through the parameter arguments.

Parameters

nErrId - Number that specifies the error ID

Syntax

```
Olive.Errors.createError(nErrId, /*, ... */);
```

Example

```
var oError = Olive.Errors.createError(Olive.Errors.Code.LoadDataFailed, sErrMsg);
```

See Also

[Global Functions](#)

Olive.Errors.defineKnownError Method

This function defines a global error.

Parameters

sConstName - String that specifies a constant global error identical name

nErrId - Number that specifies the error ID

sErrMsgFormatResId - String that specifies a resource formatted error message string

nErrCode - Optional; number that specifies an error code

Syntax

```
Olive.Errors.defineKnownError(sConstName, nErrId, sErrMsgFormatResId, nErrCode);
```

Example

```
Olive.Errors.defineKnownError("CannotLoadSettings", Olive.Errors.FirstSdkErrorId,  
Olive.Resources.ResName.Err_LoadSettings);
```

See Also

[Global Functions](#)

Olive.Errors.getLocalizedFormatString Method

This function retrieves a localized format string.

Parameters

sErrorKey - String that specifies an error key

Syntax

```
Olive.Errors.getLocalizedFormatString(sErrorKey);
```

Example

```
Olive.Errors.getLocalizedFormatString(oErrorInfo.errorMsgFormatResId);
```

See Also

[Global Functions](#)

Olive.Errors.getLocalizedMessage Method

This function retrieves a localized error message.

Parameters

sErrorKey - String that specifies an error key

oValue - Object of parameters that is used for formatting the message

Syntax

```
Olive.Errors.getLocalizedMessage(sErrorKey, oValue);
```

Example

```
// get localized error message for "BAD_STRING" error
var sErrorMessage = Olive.Errors.getLocalizedMessage("BAD_STRING");

// get localized error message for "EMailBadAddress" error; here oValue is a
string with invalid e-mail address
var sErrorMessage = Olive.Errors.getLocalizedMessage("EMailBadAddress", oValue);
```

See Also

[Global Functions](#)

String_parseInt Method

This function returns an integer that is converted from a string. The string is parsed with the base 10. If the string does not start from a number, the function returns a default value, or null if default value is undefined.

Parameters

sValue - String for parsing

defaultVal - Default integer value

Syntax

```
String_parseInt(sValue, defaultVal);
```

Example

```
var nFirstLegalPage = String_parseInt( DHTML_getAttr(oDataElement,
"firstLegalPage"), 1 );
```

See Also

[Global Functions](#)

String_format Method

This function returns a formatted string. It can get arguments to format the string.

Parameters

sFormat - String for formatting

Syntax

```
String_format(sFormat /*, ... */);
```

Example

```
var sFormattedMessage = String_format(Error loading settings: \'%1\',  
sServerMsg);
```

See Also

[Global Functions](#)

String_format_arg Method

This function returns a formatted string.

Parameters

sFormat - String for formatting

oArguments - Array of arguments to substitute parameters in a formatted string

nStartFromArg - Index from which to start reading arguments

Syntax

```
String_format_arg(sFormat, oArguments, nStartFromArg);
```

Example

```
// here in arguments there are 2 values for substituting in the formatted string  
var sFormattedMessage = String_format_arg(The following value for the attribute  
\'%1\' is invalid: \'%2\', arguments, 1);
```

See Also

[Global Functions](#)

Number.prototype.toPrecision Method

This function returns a string with a specified number of digits. It is an overridden functionality of the Number JavaScript object.

Parameters

prec - Precision number specifies the desired precision of the returned number

Syntax

```
Number.prototype.toPrecision( prec );
```

Example

```
// sPrecisedNumber will be equal to 55.12
var nSomeNumber = new Number(55.12345);
var sPrecisedNumber = nSomeNumber.toPrecision(3);
```

See Also

[Global Functions](#)

Function.prototype.apply Method

This function applies an object method, substituting another object for the current object. It overrides the base functionality of the JavaScript object Function.

Parameters

oScope - The object to be used as the current object

args - Array of arguments to be passed to the function

Syntax

```
Function.prototype.apply(oScope, args);
```

Example

```
return oCallback.apply(oThis, arrArguments);
```

See Also

[Global Functions](#)

Olive.Controls.IDataBound.getHtmlContentTargetNull Method

This function retrieves null as an HTML content target, meaning that you don't want to clear the original content of HTML element after receiving data from server. It is used for to override the [getHtmlContentTarget\(\)](#) method of a class that implements Olive.Controls.IDataBound interface.

Parameters

No parameters

Syntax

```
Olive.Controls.IDataBound.getHtmlContentTargetNull();
```

Example

```
// in a constructor of some control
rObject.getHtmlContentTarget =
Olive.Controls.IDataBound.getHtmlContentTargetNull();
```

See Also

[Global Functions](#)

Olive.WindowBinder.GetBinderForWindow Method

This function retrieves the window's OliveBinder object. The **OliveBinder** class is responsible to bind the browser window to the Olive Web SDK, creating an Olive page object and initiating HTML parsing.

Parameters

oWindow - Window JavaScript object

Syntax

```
Olive.WindowBinder.GetBinderForWindow(oWindow);
```

Example

```
var oWindowBinder = Olive.WindowBinder.GetBinderForWindow(window);
```

See Also

[Global Functions](#)

IActiveItem Overview

The IActiveItem interface provides functionality for objects that support the activation of a single sub-item, for example the *PageSwitcher* control or the *Tab* control. It contains methods to read and write which sub-item is active, and to get information about the sub-items. It also can fire events when a new sub-item is activated, both before and after the activation (itemActivating, itemActivated).

All methods and events that use index numbers to identity sub-items use numbers starting from 0 (for example, the second item has index 1).

See Also

[IActiveItem Methods](#) | [IActiveItem Events](#)

IActiveItem Events

Event	Description
itemActivating	Fired by setActiveItem() if the new sub-item is different from the old value and argument bDoNotNotify is not true. The event is fired before the active/inactive state of the old and new active items has been changed.
itemActivated	Fired by setActiveItem() if the new sub-item is different from the old value and argument bDoNotNotify is not true. The event is fired after the active/inactive state of the old and new active items has been changed.

See Also

[IActiveItem Overview](#)

itemActivated Event

This event is fired by setActiveItem() if the new sub-item is different from the old value and argument bDoNotNotify is not true. The event is fired after the active/inactive state of the old and new active items has been changed.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following expando property:

1. **prevActiveItemIndex** - the index of the previous active item.
2. **prevActiveItem** - a reference to the previous active item.
3. **activeItemIndex** - the index of the new active item.
4. **activeItem** - a reference to the new active item.

Remarks

If the event handler for this event sets the property cancelDefaultAction to true, the active item will not be changed to the new item.

Example

```
// Handler of the event
function Application_onItemActivated(oEvent)
{
    // desirable action
}
```

See Also

[IActiveItem Events](#) | [IActiveItem Overview](#)

itemActivating Event

This event is fired by `setActiveItem()` if the new sub-item is different from the old value and argument `bDoNotNotify` is not true. The event is fired before the active/inactive state of the old and new active items has been changed.

Properties

The event object passed to the event handler is an instance of the `Olive.Event` class, with the following expando property:

1. **prevActiveItemIndex** - the index of the previous active item.
2. **prevActiveItem** - a reference to the previous active item.
3. **activeItemIndex** - the index of the new active item.
4. **activeItem** - a reference to the new active item.

Remarks

If the event handler for this event sets the property `cancelDefaultAction` to true, the active item will not be changed to the new item.

Example

```
// Handler of the event
function Application_onItemActivating(oEvent)
{
    // desirable action
    // oEvent.cancelDefaultAction = true; // will cause setActiveItem() to do nothing
}
```

See Also

[IActiveItem Events](#) | [IActiveItem Overview](#)

IActiveItem Methods

Method	Description
getItemsCount()	Returns the number of sub-items.
getItem(itemIndex)	Returns the sub-item specified by index number <code>itemIndex</code> (starting from 0).

indexOfItem(oltem)	Returns the index number of sub-item oltem (starting from 0).
getActiveItemIndex()	Returns the index number of the active sub-item (starting from 0).
getActiveItem()	Returns the active sub-item.
setActiveItem(vItem, bDoNotNotify)	Sets sub-item vItem (index-number or object) to be the active sub-item. Fires events itemActivating and itemActivated unless bDoNotNotify is true.

See Also

[IActiveItem Overview](#)

getActiveItem Method

This method returns the active sub-item.

Parameters

none

Return value

object

Syntax

```
var oItem = oControl.getActiveItem();
```

Example

```
var oPage = oPageSwitcher.getActiveItem();
```

See Also

[IActiveItem Overview](#) | [IActiveItem Methods](#) | [IActiveItem Events](#)

getActiveItemIndex Method

This method returns the index number of the active sub-item (starting from 0).

Parameters

none

Return value

number

Syntax

```
var nIndex = oControl.getActiveItemIndex()
```

Example

```
var nPage = oPageSwitcher.getActiveItemIndex();
```

See Also

[IActiveItem Overview](#) | [IActiveItem Methods](#) | [IActiveItem Events](#)

getItem Method

This method returns the sub-item specified by index number `itemIndex` (starting from 0).

Parameters

itemIndex - number

The index number of the requested item.

Return value

object

Syntax

```
var oItem = oControl.getItem( nIndex );
```

Example

```
var oPage = oPageSwitcher.getItem( nIndexOfPreviousActiveItem );
```

See Also

[IActiveItem Overview](#) | [IActiveItem Methods](#) | [IActiveItem Events](#)

getItemCount Method

This method returns the number of sub-items.

Parameters

none

Return value

number

Syntax

```
var nCount = oControl.getItemCount();
```

Example

```
var nCount = oPageSwitcher.getItemCount();  
if( nActiveIndex == ( nCount - 1 ) ) {
```

```
oPageSwitcher.setActiveItem( 0 );//go back to the first item
}
```

See Also

[ActiveItem Overview](#) | [ActiveItem Methods](#) | [ActiveItem Events](#)

indexOfItem Method

This method returns the index number of sub-item oltem (starting from 0).

Parameters

oltem – object, the item whose index number is requested.

Return value

number

Syntax

```
var nIndex = oControl.indexOfItem( oItem );
```

Example

```
var nIndex = oPageSwitcher.indexOfItem(oPage);
```

See Also

[ActiveItem Overview](#) | [ActiveItem Methods](#) | [ActiveItem Events](#)

setActiveItem Method

Sets sub-item vltem (index-number or object) to be the active sub-item.

Parameters

vltem - variant (number or object)

Index number or reference to the item that is to be set to the active item.

bDoNotNotify - boolean

If true - Do not fire events itemActivating and itemActivated

Return value

none

Syntax

```
oControl.setActiveItem( nIndex );
oControl.setActiveItem( oItem );
```

Example

```
oPageSwitcher.setActiveItem(0); //activate the first page-switcher item
```

See Also

[ActiveItem Overview](#) | [ActiveItem Methods](#) | [ActiveItem Events](#)

ICmdItem Overview

The interface represents by Olive.Controls.ICmdItem class.

This interface should be implemented by classes that support launching commands upon being click.

An example of such class is [Olive.Controls.MenuItem](#) that represents the MenuItem control.

The interface has general functionality for updating state of a command item.

Implemented Interfaces

[Olive.IUiElements](#)

[Olive.ICmdSource](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ICmdItem Methods](#)

ICmdItem Methods

Methods inherited from the [Olive.ICmdSource](#) interface

Event	Description
updateCmdUiState	Updates some kind of states of the UI element of the controls implemented the interface

See Also

[ICmdItem Overview](#)

updateCmdUiState Method

This method updates a state of the UI element represented the command item. It can update following states:

- * enable/disable
- * show/hide

* check/uncheck

It can also change text or icon of the command item.

Parameters

oCmdUiState

An instance of Olive.CmdUiStateclass

Syntax

```
oCmdItem.updateCmdUiState(oCmdUiState);
```

Example

```
var oCmdUiState = new Olive.CmdUiState(rSrcControl, sCmdName, oCmdParams);  
//here rSrcControl source control with data  
//sCmdName name of the command  
//oCmdParams parameters of the command  
oMenuItem.updateCmdUiState();
```

See Also

[ICmdItem Methods](#) | [ICmdItem Overview](#)

ICmdSource Overview

This interface represents by Olive.ICmdSource class.

It should be implemented by classes that support launching commands and reflecting UI state of commands. For example, by buttons that launch a command when clicked.

Examples of such classes are Olive.Controls.Menuclass that represents Menu control and classes that implement Olive.Controls.ICmdItem interface.

The interface has the general functionality of sending and updating commands, updating the UI state of the command and getting/setting a command target object.

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ICmdSource Methods](#)

ICmdSource Methods

ICmdSource-specific Methods

Event	Description
-------	-------------

getCmdTarget	Returns a command target
setCmdTarget	Assigns a command target
sendCommand	Launches a command
updateCommand	Updates target, source control of the command and calls <code>updateCmdUiState()</code> method
updateCmdUiState	Updates some kind of states of the UI element of the controls implemented the interface

See Also

[ICmdSource Overview](#)

getCmdTarget Method

This method returns a command target object.

Parameters

No parameters

Syntax

```
oCmdSourceControl.getCmdTarget();
```

Example

```
var oCmdTarget = oMenu.getCmdTarget();
```

See Also

[ICmdSource Methods](#) | [ICmdSource Overview](#)

setCmdTarget Method

The method updates a state of the UI element represented the command item. It can update following states:

- enable/disable
- show/hide
- check/uncheck

It can also change text or icon of the command item.

Parameters

oCmdTarget - An instance of an SDK control

Syntax

```
oCmdSource.setCmdTarget(oCmdTarget);
```

Example

```
oMenu.setCmdTarget(oSomeControl);
```

See Also

[ICmdSource Methods](#) | [ICmdSource Overview](#)

sendCommand Method

This method launches the command.

Parameters

sCmdName - A name of the command

oCmdParams - Parameters for the command

Syntax

```
oCmdSource.sendCommand(sCmdName, oCmdParams);
```

Example

```
oMenu.sendCommand(sCmdName, oCmdParams);
```

See Also

[ICmdSource Methods](#) | [ICmdSource Overview](#)

updateCommand Method

This method updates the command target object, source control object of the command and calls updateCmdUiState() method

Parameters

sCmdName - A name of the command

oCmdParams - Parameters for the command

rSrcControl - A source control, optional parameter

Syntax

```
oCmdSource.updateCommand(sCmdName, oCmdParams, rSrcControl);
```

Example

```
oMenu.updateCommand(sCmdName, oCmdParams);
```

See Also

[ICmdSource Methods](#) | [ICmdSource Overview](#)

updateCmdUiState Method

This method updates the UI state of the command. It should inherit classes/interfaces interested in some special UI updates

Parameters

oCmdUiState - An instance of Olive.CmdUiStateclass

Syntax

```
oCmdSource.updateCmdUiState(oCmdUiState);
```

Example

```
var oCmdUiState = new Olive.CmdUiState(rSrcControl, sCmdName, oCmdParams);  
//here rSrcControl source control with data  
//sCmdName name of the command  
//oCmdParams parameters of the command  
oCmdSource.updateCmdUiState(oCmdUiState);
```

See Also

[ICmdSource Methods](#) | [ICmdSource Overview](#)

ComplexValue Overview

This interface represents the Olive.Controls.IComplexValue class.

It is for classes of compound controls that support data exchange between a JavaScript object and nested controls. Examples of such classes are Olive.Controls.Calendar that represents a Calendar control and classes that implement the Olive.Controls.IForm interface.

The interface has general functionality to update controls from values of HTML elements and vice versa.

Implemented Interfaces

[Olive.Controls.IValue](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IComplexValue Methods](#)

IComplexValue Methods

Overridden methods inherited from IValue interface

Event	Description
updateControl	Assigns the internal values to the HTML element and to the HTML elements of its nested controls
updateData	Assigns the HTML value to the internal value of the control and its nested controls.

IComplexValuespecific Methods

Event	Description
createNewValue	Creates a new instance of the type defined with the owc:DataType attribute

See Also

[IComplexValue Overview](#)

createNewValue Method

This method creates and returns a new instance of the type defined with the owc:DataType attribute

Parameters

No parameters

Syntax

```
oComplexValueControl.createNewValue();
```

Example

```
// create new value for calendar class that implements IComplexValue interface  
var oNewDate = oCalendar.createNewValue();
```

See Also

[IComplexValue Methods](#) | [IComplexValue Overview](#)

updateControl Method

This method assigns the internal values to the HTML element and to the HTML elements of its nested controls. The method returns a boolean value indicated if scanning was canceled.

Parameters

oDataObject - It is an object with data to save to the internal values of the control and its nested controls. Optional parameter. If not defined, the values are updated with the current values.

Syntax

```
oComplexValueControl.updateControl(oDataObject);
```

Example

```
// save current values of calendar control and its nested controls in an internal data
var bScanningCancelled = oCalendar.updateControl();
```

See Also

[IComplexValue Methods](#) | [IComplexValue Overview](#)

updateData Method

This method assigns the HTML values that were saved during updateControl() to the control and its nested controls. The method returns false if scanning was canceled or the value is not valid, or returns true if scanning was not canceled and value is valid.

Parameters

oDataObject - An object with data to update the control. Optional parameter. If not defined, the control is updated with previously saved data.

Syntax

```
oComplexValueControl.updateData(oDataObject);
```

Example

```
// update calendar control and its nested controls with previously saved data
var bUpdated = oCalendar.updateData();
```

See Also

[IComplexValue Methods](#) | [IComplexValue Overview](#)

IControlContainer Overview

The interface represents the Olive.Controls.IControlContainer class.

This interface should be implemented by classes that support controls containment. It is currently implemented by:

- **Olive.Controls.Control** that is base class for all controls

- **Olive.Page** that is base class for page control

The IControlContainer interface provides the basic internal functionality to bind controls, parse HTML pages, create, initialize and clear controls, and it exposes a public API for the following functionality types:

- Add and remove a child control
- Find control/controls by id/type
- Get a control
- Get a count of child controls
- Get an index of a child control
- Check whether the child control is first/last
- Set active mast for child controls

The IControlContainer includes parsing of <owc:data..> elements in HTML pages.

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IControlContainer Methods](#) | [IControlContainer Properties](#) | [IControlContainer Attributes](#)

IControlContainer HTML Attributes

IControlContainer-specific Attributes

Attribute	Description
owc:control	Specifies the name of the control type
owc:ui	Specifies a kind of UI elements

See Also

[IControlContainer Overview](#) | [IControlContainer Methods](#) | [IControlContainer Properties](#)

owc:control HTML Attribute

Specifies the control's name

Syntax

```
<html-element owc:control = /name of the control/ >
...
</html-element>
```

Remarks

Example

```
<div owc:control="pane"> ... </div>
```

Usage

Control	Any control
Use	Required
Default value	no

See Also

[IControlContainer Overview](#) | [IControlContainer Attributes](#)

wc:ui HTML Attribute

Specifies a type of UI element

Syntax

```
<html-element owc:ui = /type of the UI element/ >  
...  
</html-element>
```

Remarks

Example

```
<div owc:ui="SearchInAllFolders"> ... </div>
```

Usage

Control	Any control
Use	Required
Default value	no

See Also

[IControlContainer Overview](#) | [IControlContainer Attributes](#)

IControlContainer Methods

IControlContainerspecific Methods

Event	Description
bindControls	Binds HTML elements to controls recursively, calling <code>parseHtmlElem</code>
parseHtmlElem	Parses HTML elements like <code>owc:data</code> , binds UI elements, and creates a control if there is an HTML attribute <code>owc:control</code>
initializeControls	Initializes child controls
clearControls	Clears array with child controls
clearSelections	Clears selection if defined
addControl	Adds a child control to the property <code>Controls</code>
removeControl	Removes a child controls from the property <code>Controls</code>
findControlById	Finds a child control by its ID
findControlByType	Finds a first child control by its control type
findControlsByType	Finds all child controls by their control types
getControlsCount	Returns the <code>Controls</code> (child controls) array size or a number of child controls of a specified type, if a control type has a parameter
getControl	Returns a child control by its index in the array <code>Controls</code> , or by its control type
getControlRev	Returns a child control by its index in the array <code>Controls</code> , or by its control type, looking for the control in reverse direction (from the last item of the array)
indexOfControl	Returns the index of the control in the array <code>Controls</code>
isFirstControl	Checks whether the control is a first control in the <code>Controls</code> array
isLastControl	Checks whether the control is a last control in the <code>Controls</code> array
setActiveMask	Sets some active mask defined for the control

See Also

[IControlContainer Overview](#) | [IControlContainer Properties](#) | [IControlContainer Attributes](#)

bindControls Method

This method returns bound HTML elements to controls, recursively calling `parseHtmlElem`. You can call this method during initialization of a container object.

Parameters

oDOMNode - An instance of a Dom node, an enter point for parsing and biding.

oPage - An instance of a page object represented a web page.

Syntax

```
oContainer.bindControls(oDOMNode, oPage);
```

Example

```
// bind controls on a page, starting from a root element  
oPage.bindControls(oRootElement, oPage);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

parseHtmlElement Method

This method parses HTML elements like `owc:data`, binds UI elements, and creates a control if there is the HTML attribute `owc:control`. It is used in `bindControls` method for recursive parsing of the HTML page.

Parameters

oCurHtmlElem - A current Dom node element, undergone the parsing.

oPage - An instance of a page object represented a web page.

Syntax

```
oContainer.parseHtmlElement(oCurHtmlElem, oPage);
```

Example

```
// parses a current HTML element, returns a newly created control, if there was  
// defined HTML attribute owc:control for the current HTML element  
var oCreatedControl = oPage.parseHtmlElem(oCurHtmlElem, oPage);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

initializeControls Method

This method initializes child controls: goes along the `Controls` property, calls `initControl()` method of every child control and fires event control initialized

Parameters

No parameters

Syntax

```
oContainer.initializeControls();
```

Example

```
// initializes child controls  
oPage.initializeControls();
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

clearControls Method

This method destroys every child control in the Controls property and sets Controls to null. You can call this method during termination of a container object.

Parameters

oHtmlParentElement - An instance of a DOM node, an entry point to parse and bind. Optional parameter.

Syntax

```
oContainer.clearControls(oHtmlParentElement);
```

Example

```
// delete child controls  
oPage.clearControls();
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

clearSelections Method

This method clears selections defined for child controls or for the parent control.

Parameters

No parameters

Syntax

```
oContainer.clearSelections();
```

Example

```
// clear selections of child controls
oPage.clearSelections();
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

addControl Method

This method adds a child control to property Controls

Parameters

oControl - A current Dom node element, undergone the parsing.

Syntax

```
oContainer.addControl(oControl);
```

Example

```
// clear selections of child controls
oPage.addControl(oControl);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

removeControl Method

This method adds a child control to the property Controls. It returns the removed control.

Parameters

iControl - An index of the control in Controls array for removing.

bTerminate - A Boolean value indicated whether to terminate the control or only to remove from the Controls array.

Syntax

```
oContainer.removeControl(iControl, bTerminate);
```

Example

```
// remove the 3d child control from the Controls array and terminate the control
(call termControl() method)
oPage.removeControl(2, true);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

findControlById Method

This method finds a child control by its ID

Parameters

sId - A string with id of the HTML element

Syntax

```
oContainer.FindControlById(sId);
```

Example

```
// find Toc control on the page by its id
var oToc = oPage.FindControlById(divToc);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

findControlByType Method

This method finds the first child control by its control type

Parameters

sControlTypeName - A string with the name of the control type

Syntax

```
oContainer.FindControlByType(sControlTypeName);
```

Example

```
// find Toc control on the page by its type name
var oToc = oPage.FindControlByType(toc);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

findControlsByType Method

This method finds all child controls of the specified control type

Parameters

oControlType - An instance of a control

Syntax

```
oContainer.findControlsByType(oControlType);
```

Example

```
// find Toc controls (an array) on the page by its type
var arrToc = oPage.findControlsByType(Olive.Controls.Toc)
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

getControlsCount Method

This method returns a length of the array Controls (child controls) or a number of child controls of a specified type, if a control type is received as a parameter

Parameters

oControlType - An instance of a control. Optional parameter

Syntax

```
oContainer.getControlsCount(oControlType);
```

Example

```
// get the count of child controls in the container oPage
var nChildControls = oPage.getControlsCount()
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

getControl Method

This method returns a child control by its index in the Controls array or its control type

Parameters

iIndex - An index of a desirable child control in the Controls array

oControlType - A string with a control type name. Optional parameter

Syntax

```
oContainer.getControl(iIndex, oControlType);
```

Example

```
// get the fifth child control from the Controls array
```

```
var oFifthChildControl = oPage.getControl(4)
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

getControlRev Method

This method returns a child control by its index in the Controls array or control type, looking for the control in reverse direction (from the last item of the array)

Parameters

iIndex - An index of a desirable child control in the Controls array

oControlType - A string with a control type name. Optional parameter

Syntax

```
oContainer.getControlRev(iIndex, oControlType);
```

Example

```
// get the last child control from the Controls array  
var oLastChildControl = oPage.getControlRev(0)
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

indexOfControl Method

This method returns the index of the control in the Controls array.

Parameters

oControl - An instance of a child control

oControlType - A string with a control type name. Optional parameter

Syntax

```
oContainer.indexOfControl(oControl, oControlType);
```

Example

```
// get the index of the specified Toc node object  
var nTocNodeIndex = oPage.indexOfControl(oTocNode)
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

isFirstControl Method

This method checks if the control is the first control in the Controls array.

Parameters

oControlParam - An instance of a child control undergone the checking

oControlType - A string with a control type name. Optional parameter

Syntax

```
oContainer.isFirstControl(oControlParam, oControlType);
```

Example

```
// check whether the specified toc node object is a first child node in the page object  
var bFirstControl = oPage.isFirstControl(oTocNode);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

isLastControl Method

This method checks if the control is the last control in the Controls array.

Parameters

oControlParam - An instance of a child control undergone the checking

oControlType - A string with a control type name. Optional parameter

Syntax

```
oContainer.isLastControl(oControlParam, oControlType);
```

Example

```
// check whether the specified toc node object is a last child node in the page object  
var bLastControl = oPage.isLastControl(oTocNode);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

setActiveMask Method

This method sets an active mask, defined for the control. It is called when the analog method is defined for a child control.

Parameters

sMaskType - A type of mask

dwBitmask - An active value mask

nItemCount - A length of the value mask

sControlType - A string with a control type name. Optional parameter

Syntax

```
oContainer.SetActiveMask(sMaskType, dwBitmask, nItemCount, sControlType);
```

Example

```
// set active mask for months in calendar  
oCalendar.SetActiveMask(OwcActiveMask_Month, this.m_dwActiveMonthsMask, 12,  
Olive.Controls.controlTypeNames.Month);
```

See Also

[IControlContainer Methods](#) | [IControlContainer Overview](#)

IControlContainer Properties

IControlContainer-specified properties

Property	Description
Controls	Collection of controls (array of child controls)

See Also

[IControlContainer Overview](#) | [IControlContainer Methods](#) | [IControlContainer Attributes](#)

Controls Property

Holds an array of child controls

Syntax

Data Type	Array
Default value	null

Remarks

This property is filled when child controls are added to a control

Sample Code

```
// get collection of the child controls
```

```
var arrControls = oControl.Controls;
```

See Also

[IControlContainer Overview](#) | [IControlContainer Properties](#)

IControlFactory Overview

The interface represents by Olive.Controls.**IControlFactory** class.

This interface **should be implemented by classes** that support registration of control types and creation of controls.

The interface is currently implemented by [Olive.Controls.IControlContainer](#) interface and Olive.Controls.ControlFactory class - general controls factory class.

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IControlFactory Methods](#)

IControlFactory Methods

IControlFactoryspecific Methods

Event	Description
registerControlType	Registers the control type in an internal array
getRegisteredControlType	Returns a control type object registered in the interface
createControlInstance	Creates an instance of a specified control type
createControl	Calls the createControlInstance method and attaches the created control to a specified Page, HTML element and Parent object

See Also

[IControlFactory Overview](#)

registerControlType Method

This method registers the control type in an internal array.

Parameters

sControlType - String that specifies control type name

sControlClass - String that specifies control class name

Syntax

```
oControlClass.registerControlType(sControlType, sControlClass);
```

Example

```
Olive.Controls.registerControlType(Olive.Controls.controlTypeNames.TOC,  
"Olive.Controls.Toc");
```

See Also

[IControlFactory Methods](#) | [IControlFactory Overview](#)

getRegisteredControlType Method

This method returns a control type object registered in the interface.

Parameters

sControlType - String that specifies control type name

Syntax

```
oControlClass.getRegisteredControlType(sControlType);
```

Example

```
var oControlType =  
oControl.getRegisteredControlType(Olive.Controls.controlTypeNames.TOC);
```

See Also

[IControlFactory Methods](#) | [IControlFactory Overview](#)

createControlInstance Method

This method creates an instance of a specified control type, and returns the created control instance.

Parameters

sControlType - String that specifies control type name

Syntax

```
oControlClass.createControlInstance(sControlType);
```

Example

```
var oTocControl =  
oControl.createControlInstance(Olive.Controls.controlTypeNames.TOC);
```

See Also

[IControlFactory Methods](#) | [IControlFactory Overview](#)

createControl Method

This method calls [createControlInstance](#) method, which creates an instance of a specified control type and attaches it to a specified Page, HTML element and Parent object. The method returns the created control instance.

Parameters

sControlType - String that specifies control type name

oPage - Page object where the control is created

oCurHtmlElem - Current HTML object control is created

oControlParent - Parent control of created control

Syntax

```
oControlClass.createControl(sControlType, oPage, oCurHtmlElem,  
oControlParent);
```

Example

```
var oTreeNodeControl =  
oTreeControl.createControl(Olive.Controls.controlTypeNames.  TreeNode,  
oPage, oCurHtmlElem, oTreeControl);
```

See Also

[IControlFactory Methods](#) | [IControlFactory Overview](#)

IDataBound Overview

This interface is represented by Olive.Controls.**IDataBound class**.

It should be implemented by classes that get data from the server, such as [DocList](#), [FlashViewer](#), Toc, and others.

The interface has general API to load content, send requests to the server and process server responses. It also has events that are fired when a request is sent, when response is received, when clicking/double-clicking/right-clicking HTML elements with content item in it for calling corresponding content commands; events of state and selection changed and others.

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[IDataBound Methods](#) | IDataBound Events

IDataBound Events

IDataBound-specific events

Event	Description
sendingRequest	Fired by sendRequestRaw() just before sending a request to server.
dataReceived	Fired by processResponse() when a response was received from server and before any parsing and processing the data.
dataProcessed	Fired by processResponse() when a response was received from server and after all parsing and processing the data.
contentItemClicked	Fired by the handler of the click DHTML event, attached to an HTML element with content item within. Fired before launching a content command.
contentItemDbClicked	Not in use. Can be fired by a dblclick DHTML event handler, attached to an HTML element with content item within.
contentItemRightClicked	Not in use. Can be fired by a contextmenu DHTML event handler, attached to an HTML element with content item within.
contentItemActiveStateChanged	Not in use. Can be fired when active state of HTML element with content item within has changed.
contentSelectionChanged	Not in use. Can be fired when selection state of HTML element with content item within has changed.
folderTreeNameReceived	Fired when data has received and parsed for FolderTree control.

See Also

[IDataBound Overview](#) | [IDataBound Methods](#)

sendingRequest Event

This event is fired by `sendRequestRaw()` just before sending a request to the server.

Properties

The event object that is passed to the event handler is an instance of the `Olive.Event` class, with the following property:

`Request` an instance of `Olive.XHTTP.Request` class created for the request.

Remarks

If the event handler for this event sets the property `cancelDefaultAction` to true, the request is not be sent to server.

Example

```
// Handler of the event
function Application_onSendingRequest(oEvent)
{
    // desirable action
    // oEvent.cancelDefaultAction = true; //will cause sendRequestRaw() not
    // to send the request
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

dataRecieved Event

This event is fired by the `processResponse()` method when a response was received from the server and before the data is parsed and processed.

Properties

The event object passed to the event handler is an instance of the `Olive.Event` class, with the following property:

Request an instance of `Olive.XHTTP.Request` class.

Remarks

Example

```
// Handler of the event
function Application_onDataReceived(oEvent)
{
    // desirable action
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

dataProcessed Event

This event is fired by the `processResponse()` method when a response was received from the server and after the data was parsed and processed.

Properties

The event object that is passed to the event handler is an instance of the Olive.Event class, with the following properties:

1. Request an instance of Olive.XHTTP.Request class.
2. curObjectType control type of a current control, if it is defined

Remarks

Example

```
// Handler of the event
function Application_onDataProcessed(oEvent)
{
    // desirable action
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

contentItemClicked Event

This event is fired by handler function of click DHTML event, attached to every control with a content item in it. Using this event is not recommended. Use the attribute [owc:ContentCmd](#) for controls with data or attribute [content-cmd](#) for tree nodes is a better way to react on content item clicks.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following property:

1. OlvContentItem an instance of a class inherited from Olive.ContentItem class; it is a content item of a control on which click was done

Remarks

Example

```
// Handler of the event
function PanelPage_ContentItemClicked(oEvent)
{
    // desirable action
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

contentItemDbClicked Event

This event is currently not in use. It is fired by handler function of dblclick DHTML event, attached to every control with a content item within.

Properties

The event object that is passed to the event handler is an instance of the Olive.Event class, with the following property:

OlvContentItem an instance of a class inherited from Olive.ContentItem class; it is a content item of a control on which click was done

Remarks

Example

```
// Handler of the event
function PanelPage_ContentItemDbClicked(oEvent)
{
    // desirable action
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

contentItemRightClicked Event

This event is fired by handler function of the context menu DHTML event, which can be attached to a control with a content item within.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following property:

OlvContentItem an instance of a class inherited from [Olive.ContentItem](#) class; it is a content item of a control on which right click was done

Remarks

Example

```
// Handler of the event
function PanelPage_ContentItemRightClicked(oEvent)
{
    // desirable action
}
```



```
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

contentItemActivateStateChanged Event

This event is currently not in use. It is fired when the activate state of content item is changed.

Properties

The event object that is passed to the event handler is an instance of the Olive.Event class.

Remarks

Example

```
// Handler of the event
function PanelPage_ContentItemActivateStateChanged(oEvent)
{
    // desirable action
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

contentSelectionChanged Event

This event is currently not in use. It is fired when selection of content items is changed.

Properties

The event object passed to the event handler is an instance of the Olive.Event class.

Remarks

Example

```
// Handler of the event
function PanelPage_ContentSelectionChanged(oEvent)
{
    // desirable action
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

folderTreeNameReceived Event

This event is fired when data for the [FolderTree](#) control was received and parsed.

Properties

The event object that is passed to the event handler is an instance of the Olive.Event class.

Remarks

Example

```
// Handler of the event
function Application_onFolderTreeNameReceived(oEvent)
{
    // desirable action
}
```

See Also

[IDataBound Events](#) | IDataBound Overview

IDataBound Methods

IDataBoundspecific Methods

Event	Description
Group of methods to load content	
contentBuildParams	Builds the parameters for a query string.
contentCanLoad	Checks for content that can load. This method calls the contentCanLoad method of an instance of a contentItem class or class, inherited from the ContentItem class, created for an instance of a class that implements the interface. This method can be overridden.
contentLoad	Loads the content: forms request to server and sends it. Can be overridden.
getContentItem	Returns a content item instance, created for an instance of a class that implements the interface. Can be overridden.

	There is a global version of the method, Olive.Controls.IDataBound.getContentItem , that can be called by the application.
Group of methods to send requests to server	
buildRequestUrl	Builds the requested URL string.
getDataProviderUrl	Returns the property DataProviderUrl.
sendRequestRaw	Sends a raw request, making some preliminary actions.
setLoadingDataMessage	Sets loading data message into HTML object where the content is loaded.
requestData	Gets a full URL and redirects it to sendRequestRaw method.
Group of methods tor process server response	
processResponse	Calls some methods types that parse data from the response, and fires events before and after parsing.
processResponseText	Parses the text part of the response.
processResponseData	Parses part of the response with special data.
processResponseHtml	Pastes HTML content into the target content HTML object.
getHtmlContentTarget	Returns an HTML element, target element for downloaded data.
postHtmlContentPaste	Performs desirable actions after parsing HTML content and before displaying requested data.

See Also

[IDataBound Overview](#) | [IDataBound Events](#) | [ContentItem Class](#)

contentBuildParams Method

This method builds query string parameters. Every class that implements the IDataBound interface can override this method. By default, this method tries to call the method with the same name as the internal content item object.

Parameters

oParams - Object of the query string

Syntax

```
oMetadata.contentBuildParams(oParams);
```

Example

```
oToc.contentBuildParams(oParams);
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

contentCanLoad Method

This method checks if the content data can be loaded. Every class that implements the `IDataBound` interface can override this method. By default, it calls the method with the same name as the internal content item object.

Parameters

No parameters

Syntax

```
oFolderTree.contentCanLoad();
```

Example

```
var bCanLoad = oFolderTree.contentCanLoad();  
if (bCanLoad)  
oFolderTree.contentLoad();
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

contentLoad Method

This method loads the content data. Every class that implements the `IDataBound` interface can override this method. By default, it checks if the content can be loaded. If yes, then it builds a request for the server and sends it.

Parameters

No parameters

Syntax

```
oMetadata.contentLoad();
```

Example

```
oToc.contentLoad();
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

getContentItem Method

This method returns an instance of the content item object. Every class that implements the `IDataBound` interface can override this method. By default, it returns an internal reference to a content item object, also looking for this object through parent objects.

There is a global copy of this method; [Olive.Controls.IDataBound.getContentItem](#)

Parameters

No parameters

Syntax

```
oMetadata.getContentItem();
```

Example

```
var oContentItem = oToc.getContentItem();
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

buildRequestUrl Method

This method builds requested URL string. It adds some standard parameters for requests like *type* and *for to* query strings, and returns a URL starting with the data provider.

Parameters

oParams - Object of the query string

Syntax

```
oMetadata.buildRequestUrl(oParams);
```

Example

```
var sUrl = oToc.buildRequestUrl(oParams);
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

getDataProviderUrl Method

This method returns the internal property `DataProviderUrl`, which holds the data provider URL string.

Parameters

No parameters

Syntax

```
oMetadata.GetDataProviderUrl();
```

Example

```
var sDataProviderUrl = oToc.GetDataProviderUrl();
```

See Also

[IDataBound Methods](#) | IDataBound Overview

sendRequestRaw Method

This method sends a raw request and performs some preliminary actions.

Parameters

oHtmlContentTarget - HTML object, target object for the content to download

sUrl - String that specifies URL request

oPostData - String that specifies data that should be posted

sMethod - String that specifies method for loading: GET (by default) or POST

bAsync - Boolean that specifies whether the request would be asynchronous (by default) or not

Syntax

```
oMetadata.SendRequestRaw(oHtmlContentTarget, sUrl, oPostData, sMethod, bAsync);
```

Example

```
// there is no oPostData;  
// therefore default values are used for sMethod and bAsync  
oToc.SendRequestRaw(oTocTarget, sUrl);
```

See Also

[IDataBound Methods](#) | IDataBound Overview

setLoadingDataMessage Method

The method sets loading data message into HTML object where the content will be loaded. Every class, that implements IDataBound interface, can override the method. By default, the method sets Downloading resource string as inner HTML of the content HTML object.

Parameters

oHtmlContentTarget - HTML object, target object for the content to download

Syntax

```
oFolderTree.setLoadingDataMessage(oHtmlContentTarget);
```

Example

```
oToc.setLoadingDataMessage(oTocTarget);
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

requestData Method

This method receives a full URL and redirects it to the [sendRequestRaw](#) method.

Parameters

oParams - Object of the query string

oHtmlContentTarget - HTML object, target object for the content to download; optional parameter, the value can be get from function [getHtmlContentTarget](#)

Syntax

```
oFolderTree.requestData(oParams, oHtmlContentTarget);
```

Example

```
// for LangList request data without parameters
oLangList.requestData();

// for Metadata request data with oParams parameters
oMetadata.requestData(oParams);
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

processResponse Method

Every class that implements the [IDataBound](#) interface can override the method.

By default, this method calls some method types that parse data from the response. It fires events before and after parsing; [dataReceived](#) and [dataProcessed](#).

Parameters

oRequest - Object of the request

Syntax

```
oFolderTree.processResponse(oRequest);
```

Example

```
oFlashViewer.processResponse(oRequest);
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

processResponseText Method

Every class that implements the `IDataBound` interface can override the method.

By default, this method parses the text part of the response. It returns a Boolean value that indicates if the response returns an error. If the method returns true, the requested data is ready to be pasted in the HTML content target. Otherwise, an error message appears.

Parameters

oRequest - Object of the request

Syntax

```
oFolderTree.processResponseText(oRequest);
```

Example

```
var bParsedOk = oFlashViewer.processResponseText(oRequest);
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

processResponseData Method

Every class that implements the `IDataBound` interface and wants to have special parsing of its data should override the method.

This method does nothing by default.

Parameters

oRequest - Object of the request

Syntax

```
oFolderTree.processResponseData(oRequest);
```


Example

```
oMetadata.processResponseData (oRequest) ;
```

See Also

[IDataBound Methods](#) | IDataBound Overview

processResponseHtml Method

Every class that implements the IDataBound interface can override the method.

By default, this method pastes HTML content into the target content HTML object. It calls methods to bind and initialize controls, and calls [postHtmlContentPaste](#) if it is defined.

Parameters

oRequest - Object of the request

Syntax

```
oFolderTree.processResponseHtml (oRequest) ;
```

Example

```
oMetadata.processResponseHtml (oRequest) ;
```

See Also

[IDataBound Methods](#) | IDataBound Overview

getHtmlContentTarget Method

Every class that implements IDataBound can override the method.

By default, this method returns an HTML element, the target element for downloaded data.

Parameters

No parameters

Syntax

```
oFolderTree.getHtmlContentTarget () ;
```

Example

```
var oHtmlContentTarget = oMetadata.getHtmlContentTarget () ;
```

See Also

[IDataBound Methods](#) | IDataBound Overview

postHtmlContentPaste Method

Every class that implements the `IDataBound` interface and requires a special actions after pasting the HTML content and before displaying it should override the method.

This method is undefined by default.

Parameters

oRequest - Object of the request

Syntax

```
oFolderTree.postHtmlContentPaste(oRequest);
```

Example

```
oMetadata.postHtmlContentPaste(oRequest);
```

See Also

[IDataBound Methods](#) | [IDataBound Overview](#)

IDataBound Properties

IDataBound-specified properties

Property	Description
DataProviderUrl	String with URL of a data provider.

See Also

[IDataBound Overview](#) | [IDataBound Methods](#) | [IDataBound Events](#)

DataBoundUrl Property

This property holds a string with the URL of a data provider

Syntax

Data Type	String
Default value	

Remarks

The property can be filled by the application object or by using the attribute [owc:data-provider](#) for controls that support working with data. There is `OwcGlobals.DefaultDataProvider` that can be set as a value of the property.

Sample Code

```
// read data provider URL from the preferences
oAppObj.DataProviderUrl = oAppObj.getPreference(OwcPref_AppDataProvider,
oAppObj.DataProviderUrl);

// set data provider URL to a default value from OwcGlobals
rObject.DataProviderUrl = OwcGlobals.DefaultDataProvider;
```

See Also

[IDataBound Overview](#) | [IDataBound Properties](#)

IForm Overview

The interface represents by Olive.Controls.IForm class.

This interface should be implemented by classes for form controls.

Examples of such classes are Olive.Controls.SearchForm of SearchForm control and Olive.Controls.MailForm of Mail control.

The interface has general API for working with its fields. It has a default handlers for controlCreating and validationError events.

Implemented Interfaces

Olive.Controls.IComplexValue

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IForm Methods](#) | IForm Events

IForm Events

Hooked events of Olive.Controls.IValue interface

Event	Description
validationError	The interface has its own handler where it cancels bubbling, displays an error message, and sets focus on a field with an invalid value

Hooked events of Olive.Controls.Control class

Event	Description
controlCreating	The interface has its own default action handler for a "control creating" event. It is used to automatically apply the Olive.Controls.IFormField interface to valued

controls that have `owc:FieldName` attribute (if they do not implement it)

See Also

[IForm Methods](#) | [IForm Overview](#)

IForm Methods

IFormspecific Methods

Event	Description
findFieldControls	Scans controls (recursively) and find all field controls matching field name
getFieldValue	Returns value of the filed
setFieldValue	Sets a new value to the field

See Also

[IForm Overview](#) | [IForm Events](#)

findFieldControls Method

This method scans the controls recursively and finds all field controls matching the field name.

Parameters

sFieldName - A field name for request

bFindFirstControl - A Boolean value indicated whether find only first control matching field name or all such controls

bForRead - A Boolean parameter indicated whether look for controls with only read flag access or not

Syntax

```
oFormControl.findFieldControls(sFieldName, bFindFirstControl, bForRead);
```

Example

```
// update calendar control and its nested controls with previously saved data  
var bUpdated = oFormControl.findFieldControls(sFieldName, bFindFirstControl, bForRead);
```

See Also

[IForm Methods](#) | [IForm Overview](#)

getFieldValue Method

This method returns the value for a requested field name.

Parameters

sFieldName - A field name for request

oDefaultValue - A default value for the field

Syntax

```
oFormControl.getFieldValue(sFieldName, oDefaultValue);
```

Example

```
// get value for the To field in mail form  
var sToAddress = oFormControl.getFieldValue(To);
```

See Also

[IForm Methods](#) | [IForm Overview](#)

setFieldValue Method

This method returns the value for a requested field name.

Parameters

sFieldName - A field name for request

oValue - A value to set

Syntax

```
oFormControl.setFieldValue(sFieldName, oValue);
```

Example

```
// set filecabinet@olivesoftware.comvalue for the To field in mail form  
var sToAddress = oFormControl.setFieldValue(To, filecabinet@olivesoftware.com);
```

See Also

[IForm Methods](#) | [IForm Overview](#)

IFormField Overview

The interface represents by Olive.Controls.IFormField class.

This interface should be implemented by classes for form field controls.

Examples of such classes are Olive.Controls.Field that represent Field control and classes that implement Olive.Controls.ISearchFormField interface.

The interface has general methods for receiving the field name and checking whether the field is required.

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[IFormField Methods](#) | [IFormField Attributes](#)

IFormField HTML Attributes

IFormField-specific Attributes

Attribute	Description
owc:field-name	Specifies a field name
owc:required	Specifies whether the field is required for the form or not

See Also

[IFormField Methods](#) | [IFormField Overview](#)

owc:FieldName HTML Attribute

Specifies a field name

Syntax

```
<html-element owc:FieldName = /string/ >  
...  
</html-element>
```

Remarks

The attribute's value can be a string that specifies the field's name

Example

```
<div owc:control="field " owc:FieldName="To"> ... </div>
```

Usage

Control	Controls implemented the interface
----------------	------------------------------------

Use	Required
Default value	no

See Also

[IFormField Overview](#) | [IFormField Attributes](#)

owc:required HTML Attribute

Specifies if a field is required

Syntax

```
<html-element owc:required= /string/ >
...
</html-element>
```

Remarks

The attribute's value can be 1, yes or true for required fields, and 0, no or false. It can be undefined for non-required fields.

Example

```
<div owc:control="field " owc:required="true"> ... </div>
```

Usage

Control	Controls implemented the interface
Use	Optional
Default value	false

See Also

[IFormField Overview](#) | [IFormField Attributes](#)

IFormField Methods

IFormFieldspecific Methods

Event	Description
getFieldName	Returns a field name
isFieldRequired	Returns a boolean value indicated if the field is required

See Also

[IFormField Overview](#)

getFieldName Method

The method returns a field name.

Parameters

No parameters

Syntax

```
oFormFieldControl.getFieldName();
```

Example

```
// get the field name  
var sFieldName = oFormFieldControl.getFieldName();
```

See Also

[IFormField Methods](#) | [IFormField Overview](#)

isFieldRequired Method

This method returns a Boolean value that indicates if the field is required in the form.

Parameters

No parameters

Syntax

```
oFormFieldControl.isFieldRequired();
```

Example

```
// get the boolean value
var bRequired = oFormFieldControl.isFieldRequired();
if (bRequired)
{
    // do desirable actions
}
```

See Also

[IFormField Methods](#) | [IFormField Overview](#)

IPane Overview

This interface represents by Olive.Controls.IPane class.

It should be implemented by classes for pane controls.

An example of such class is the Olive.Controls.Pane class, that represents the Pane control.

The IPane interface implements the Olive.Controls.IPaneListItem interface, with general functionality for any kind of pane item, and adds its own specific functionality (commands, ui elements)

Implemented Interfaces

Olive.Controls.IPaneListItem

Requirements

Script Files	OwcPanels.js
--------------	--------------

See Also

[IPane Methods](#) | [IPane Commands](#) | [IPane UiElements](#) | [IPane Attributes](#)

IPane HTML Attributes

Attributes inherited from Olive.Controls.IPanelItem interface

Property	Description
owc:pane-size	Specifies a size of the pane list item

IPane-specific Attributes

Attribute	Description
owc:resizable	Specifies whether the pane is resizable or not
owc:PaneMaximizeToolTip	Specifies a string for tool tips of a HTML element of the toggle command, when the pane is minimized
owc:PaneMinimizeToolTip	Specifies a string for tool tips of an HTML element of the toggle command, when the pane has a normal size

See Also

[IPane Overview](#) | [IPane Methods](#) | [IPane UiElements](#) | [IPane Commands](#) | [IPane CSS Classes](#)

owc:resizable HTML Attribute

This attribute specifies if the pane is resizable.

Syntax

```
<html-element owc:resizable = /boolean/ >  
...  
</html-element>
```

Remarks

Value of the attribute can be true or false

Example

```
<div owc:control="pane " owc:resizable="false"> ... </div>
```

Usage

Control	Controls implemented the interface
Use	Optional
Default value	true

See Also

[IPane Overview](#) | [IPane Attributes](#) | [Pane Overview](#)

owc:PaneMaximizeToolTip HTML Attribute

This attribute specifies a tool tips string of the toggle command's HTML element, when the pane is minimized.

Syntax

```
<html-element owc:PaneMaximizeToolTip = /string/ >
...
</html-element>
```

Remarks

If the attribute is not set, the application can set it in its locale resources.

Example

```
<div owc:control="pane " owc:PaneMaximizeToolTip="maximize the pane"> ... </div>
```

Usage

Control	Controls implemented the interface
Use	Optional
Default value	no

See Also

[IPane Overview](#) | [IPane Attributes](#) | [Pane Overview](#)

owc:PaneMinimizeToolTip HTML Attribute

This attribute specifies a tool tips string of the toggle command's HTML element, when the pane is at normal size.

Syntax

```
<html-element owc:PaneMinimizeToolTip = /string/ >
...
</html-element>
```

Remarks

If the attribute is not set, the application can set it in its locale resources

Example

```
<div owc:control="pane " owc:PaneMinimizeToolTip="minimize the pane"> ...
</div>
```

Usage

Control	Controls implemented the interface
Use	Optional
Default value	no

See Also

[IPane Overview](#) | [IPane Attributes](#) | [Pane Overview](#)

IPane Commands

IPane-specific Commands

Command	Description
minimize	Minimizes the pane
restore	Restores the size of the pane
toggle	Toggles between minimized state and normal size state
show	Shows the pane
hide	Hides the pane

See Also

[IPane Overview](#) | [IPane Methods](#) | [IPane UIElements](#) | [IPane Attributes](#) | [IPane CSS Classes](#)

minimize Command

The minimize command minimizes the pane, more exactly - the command minimizes only the content UI element of the pane.

Syntax

oPane. **launch Command (minimize)**

where *oPane* refers to a control that implements IPane interface

Parameters

none

Sample Code

The following example illustrates the Command control used to launch the minimize command on a Pane control.

```
<div id=MyPane owc:Control="pane">
  <input ui:header id="buttonMinimize" type="button" value="Minimize"
owc:Control="Command" owc:Command="minimize"/>
  <span ui:content>It is a ui:content part of the pane control that will
be minimized when button Minimize will be clicked</span>
  <span>It is a simple span element in the pane control that will not be
minimized when the button Minimize will be clicked. This part will be
always visible</span>
</div>
```

The following example illustrates launching the minimize command from a command control.

```
var oMinimizeCommand = getObj("buttonMinimize");
oMinimizeCommand.doLaunchCommand();
```

The following example illustrates the minimize() method (inherited from Olive.Controls.IPaneListItem interface) used as an alternative to the minimize command.

```
var oPane = OwcGetControl("MyPane");
oPane.minimize();
```

See Also

[IPane Overview](#) | [IPane Commands](#)

restore Command

The restore command restores the pane to its normal size, more exactly - the command restores only the content UI element of the pane.

Syntax

oPane. **launch Command(restore)**

where *oPane* refers to a control that implements IPane interface

Parameters

none

Sample Code

The following example illustrates the Command control being used to launch the restore command on a Pane control.

```
<div id=MyPane owc:Control="pane">
  <input ui:header id="buttonRestore" type="button" value="Restore"
owc:Control="Command" owc:Command="restore"/>
  <span ui:content>It is a ui:content part of the pane control that will
```

```

be restored when button      Restore will be clicked</span>

<span>It is a simple span element in the pane  control that will not be
restored when the  button      Restore will be clicked</span>

</div>

```

The following example illustrates launching the restore command from a command control.

```

var oRestoreCommand      = getObj("butttonRestore");
oRestoreCommand.doLaunchCommand();

```

The following example illustrates the restore() method (inherited from Olive.Controls.IPaneListItem interface) used as an alternative to the restore command.

```

var oPane      = OwcGetControl("MyPane");
oPane.restore();

```

See Also

[IPane Overview](#) | [IPane Commands](#)

toggle Command

The toggle command toggles the pane between states: minimized and normal size. The command will minimize/restore the content ui element of the pane.

Syntax

oPane. - launch Command (toggle)

where oPane refers to a control that implements IPane interface

Parameters

none

Sample Code

The following example illustrates using the Command control to launch the toggle command on a Pane control.

```

<div id=MyPaneowc:Control="pane">

  <input ui:header id="butttonToggle" type="button" value="Toggle"
owc:Control="Command" owc:Command="toggle"/>

  <span ui:content>It  is a      ui:content part of the pane control that
will  be minimized/restored when button Toggle will be clicked</span>

  <span>It is a simple  span element in the pane control that will not be
minimized/restored when the  button Toggle will be clicked. This part
will be always visible</span>

</div>

```

The following example illustrates launching the toggle command from a command control.

```
var oToggleCommand = getObj("buttonToggle");  
oToggleCommand.doLaunchCommand();
```

The following example illustrates using the toggle() method (inherited from Olive.Controls.IPaneListItem interface) as an alternative to the toggle command.

```
var oPane = OwcGetControl("MyPane");  
oPane.toggle();
```

See Also

[IPane Overview](#) | [IPane Commands](#)

show Command

The show command shows the pane

Syntax

oPane. launch Command(show)

where oPane refers to a control that implements IPane interface

Parameters

none

Sample Code

The following example illustrates the Command control being used to launch the show command on a Pane control.

```
<div id=MyPane owc:Control="pane">  
  <span>It is a pane control with show button for it.</span>  
  <input id="buttonShow" type="button" value="Show" owc:Control="Command"  
owc:Command="show"/>  
</div>
```

Because the command belongs to IPane interface and classes that implement the interface, the user can put an HTML element with a command control in it only inside of the HTML element with pane control (or other control that supports the IPane interface). But it is not relevant for the show command: if the pane is shown, you see the command control and you don't want to click on the button to show the pane (pane is shown), but if the pane is hidden, you don't see the command and you can't click on it to show the pane.

So use the launching command or method show() for the pane control to show the pane.

The following example illustrates launching the show command from a command control.

```
var oShowCommand = getObj("buttonShow");
oShowCommand.doLaunchCommand();
```

The following example illustrates using the show() method (inherited from **Olive.Controls.IPaneListItem** interface) as an alternative to the show command.

```
var oPane = OwcGetControl("MyPane");
oPane.show();
```

See Also

[IPane Overview](#) | [IPane Commands](#)

hide Command

The hide command hides the pane.

Syntax

oPane. **launch Command (hide)**

where oPane refers to a control that implements IPane interface

Parameters

none

Sample Code

The following example illustrates the Command control used to launch the hide command on a Pane control.

```
<div id=MyPane owc:Control="pane">
  <span>It is a pane control with hide button in it.</span>
  <input id="buttonHide" type="button" value="Hide" owc:Control="Command"
owc:Command="hide"/>
</div>
```

The following example illustrates launching the hide command from a command control.

```
var oHideCommand = getObj("buttonHide");
oHideCommand.doLaunchCommand();
```

The following example illustrates the hide() method (inherited from **Olive.Controls.IPaneListItem** interface) used as an alternative to the hide command.

```
var oPane = OwcGetControl("MyPane");
oPane.hide();
```


See Also

[IPane Overview](#) | [IPane Commands](#)

IPane CSS Classes

CSS Class	Description
PaneMinimizeCommand	CSS class for the image of the toggle command, when the pane has a normal size
PaneMaximizeCommand	CSS class for the image of the toggle command, when the pane is minimized

See Also

[IPane Overview](#) | [IPane Methods](#) | [IPane UiElements](#) | [IPane Commands](#) | [IPane Attributes](#)

IPane Methods

IPanespecific Methods

Event	Description
setPanelItemSize(nWidth, nHeight)	Sets width and height for the pane
getToolTip(bMinimized)	Returns a tool tip string that is set for the toggle command's HTML element

See Also

[IPane Overview](#) | [IPane Commands](#) | [IPane UiElements](#) | [IPane Attributes](#) | [IPane CSS Classes](#)

setPanelItemSize Method

This method sets width and height for the pane, getting into account the type of the pane item (header or content; see UiElements for the IPane interface). Layout is recalculated for the parent container, implemented the Olive.Controls.IPaneList interface

Parameters

nWidth - The type of the parameter is number

nHeight - The type of the parameter is number

Syntax

```
oPane.setPanelItemSize(nWidth, nHeight);
```

Example

```
// sets the predefined size for the pane item
var nWidth = 100px;
var nHeight = 80px;
oPane.setPaneItemSize(nWidth, nHeight);
```

See Also

[IPane Methods](#) | [IPane Overview](#)

getToolTip Method

This method returns a tool tip string that is set for the toggle command's HTML element. If the tool tips are defined for the HTML element, then they can be changed according to the size of the pane (minimized or has normal size). The tool tips for the HTML element is updates every time when toggle-command UI element is updated.

Parameters

bMinimized - A Boolean value indicated whether the pane is minimized or not

Syntax

```
oPane.getToolTip(bMinimized);
```

Example

```
// get tool tip string for HTML element represented toggle command,
when the pane is minimized
var sToolTip = oPane.getToolTip(true);

// get tool tip string for HTML element represented toggle command,
when the pane has its normal size
var sToolTip = oPane.getToolTip(false);
```

See Also

[IPane Methods](#) | [IPane Overview](#)

IPane UiElements

IPane-specific UiElements

Command	Description
header	UI element for panes for header
content	UI element for panes for content

toggle-command	UI element for HTML element represented toggle command
--------------------------------	--

See Also

[IPane Overview](#) | [IPane Methods](#) | [IPane Commands](#) | [IPane Attributes](#) | [IPane CSS Classes](#)

header UiElement

Defines an HTML element as the header of a pane item

Syntax

```
<html-element owc:ui=header >
...
</html-element>
```

Remarks

In every HTML element with pane control you can define 2 types of UI elements: header or content, which can behave differently to minimize/restore/toggle commands of the pane

Example

```
<div owc:control="pane"> <div owc:ui="header" >... </div> </div>
```

Usage

Control	A control, implemented IPane interface
Use	Optional
Default value	no

See Also

[IPane Overview](#) | [IPane UiElements](#)

content UiElement

Defines an HTML element as the content part of a pane item

Syntax

```
<html-element owc:ui=content >
...
</html-element>
```

Remarks

In every HTML element with a pane control you can define 3 types of UI elements: header or content, which can behave differently to minimize/restore/toggle commands of the pane

Example

```
<div owc:control="pane"> <div owc:ui="content" >... </div> </div>
```

Usage

Control	A control, implemented IPane interface
Use	Optional
Default value	no

See Also

[IPane Overview](#) | [IPane UiElements](#)

toggle-command UiElement

Defines an HTML element with a toggle command

Syntax

```
<html-element owc:ui=toggle-command >  
...  
</html-element>
```

Remarks

The HTML element of the UI element will contain interchangeable images of the button that represent the toggle command: one image for minimizing the pane and another controversial image for restoring the pane. The images can be defined with CSS classes defined for the interface.

Example

```
<div owc:control="pane"> <div owc:ui="toggle-command" > ... </div>  
</div>
```

Usage

Control	A control, implemented IPane interface
Use	Optional
Default value	no

See Also

[IPane Overview](#) | [IPane UiElements](#)

IPanelist Overview

This interface represents the Olive.Controls.IPanelist class, which is implemented by classes of pane list controls, controls that contain lists (row or column) of adjacent panes and/or splitters.

The IPanelist interface provides functionality for Olive.Controls.PaneList class that represents the PaneList control to create HTML pages with complicated layout.

Requirements

Script Files	OwcPanels.js
--------------	--------------

See Also

[IPanelist Methods](#) | [IPanelist Properties](#) | [IPanelist Attributes](#)

IPanelist HTML Attributes

Attributes inherited from Olive.Controls.IPanelistItem interface

Property	Description
owc:pane-size	Specifies a size of the pane list item

IPanelist-specific Attributes

Attribute	Description
owc:kind	Specifies a kind of a pane list, grouped in rows or columns

See Also

IPanelist Overview | [IPanelist Methods](#) | [IPanelist Properties](#)

owc:kind HTML Attribute

Specifies a kind of the pane-list rows or columns

Syntax

```
<html-element owc:kind = /string/ >  
...  
</html-element>
```

Remarks

Value of the attribute can be **rows** for forming horizontally grouped panes or **columns** for vertically-grouped panes. The default value is **rows**.

Example

```
<div owc:control="pane-list" owc:kind="columns">... </div>
```

Usage

Control	Controls implemented the interface
Use	Optional
Default value	rows

See Also

IPanelList Overview | [IPanelList Attributes](#) | [PanelList Overview](#)

IPanelList Methods

IPanelList specific Methods

Event	Description
isHorizontal	Defines whether the pane list is horizontal or not

See Also

IPanelList Overview | IPanelList Properties | [IPanelList Attributes](#)

isHorizontal Method

This method returns a Boolean value indicated if the panes in a pane list are grouped horizontally.

Parameters

No parameters

Syntax

```
oPanelList.isHorizontal();
```

Example

```
// check whether oPanelList (object of a class implemented IPanelList  
interface) horizontal or not  
var bHorizontal = oPanelList.isHorizontal();
```

See Also

IPanelList Overview | [IPanelList Methods](#)

IPanelList Properties

IPanelList-specified properties

Property	Description
kind	Holds a kind of the pane list rows or columns

See Also

IPanelList Overview | [IPanelList Methods](#) | [IPanelList Attributes](#)

kind Property

Holds a kind of a pane list

Syntax

Data Type	String
Default value	rows

Remarks

Value of the property can be rows for forming horizontally grouped panes or columns for vertically-grouped panes. The default value is rows.

Sample Code

```
// get the kind of oPanelList (object of a class implemented IPanelList interface)
var sKind = oPanelList.kind;
```

See Also

[IPanelList Overview](#) | [IPanelList Properties](#)

IPanelListItem Overview

This interface represents the Olive.Controls.IPanelListItem class, which includes general functionality for any pane type.

It is implemented by controls whose sizes are managed by the pane list control.

Implemented Interfaces

[Olive.UIElements](#)

Requirements

Script Files	OwcPanes.js
--------------	-------------

See Also

[IPanelListItem Methods](#) | [IPanelListItem Properties](#) | [IPanelListItem Attributes](#)

IPanelListItem HTML Attributes

IPanelListItem-specific Attributes

Property	Description
owc:pane-size	Specifies a size of the pane list item. The value of the attribute is held in resizablePane property of the Interface

See Also

[IPanelListItem Overview](#) | [IPanelListItem Methods](#) | [IPanelListItem Properties](#)

owc:pane-size HTML Attribute

This attribute specifies the size of the pane-list-item that is a part of another pane-list.

Syntax

```
<html-element owc:pane-size = /percent/ >
...
</html-element>
```

Remarks

The attribute's value can be a number of the percent of the pane-list-item

Example

```
<div owc:control="pane-list" owc:pane-size="40%"> ... </div>
```

Usage

Control	Controls implemented the interface
Use	Optional
Default value	no

See Also

[IPanelListItem Overview](#) | [IPanelListItem Attributes](#) | [PaneList Overview](#)

IPanelListItem Methods

IPanelListItemspecific Methods

Event	Description
isVisible	Defines if the pane item is visible
show	Sets the visible state for the pane item, as implemented in the interface
hide	Removes the visible state
isMinimized	Checks if the pane item is minimized
minimize	Minimizes a frame of the pane item
restore	Restores the size of the pane item
toggle	Toggles between states: Olive.IState.State.Minimized and Olive.IState.State.NormalSize
toggleVisible	Toggles between states: Olive.IState.State.Visible and Olive.IState.State.Invisible
isColumnItem	Checks if the pane item is in the column-grouped list
getPanelItemSizeFactor	Returns a factor (0-1 or -1 if undefined) of the pane item

See Also

[IPanelListItem Overview](#) | [IPanelListItem Properties](#) | [IPanelListItem Attributes](#)

isVisible Method

This method returns Boolean value that indicates if the pane is visible.

Parameters

No parameters

Syntax

```
oItem.isVisible();
```

Example

```
// check whether oPanelListItem (object of a class implemented  
IPanelListItem interface) visible or not  
var bVisible = oPanelListItem.isVisible();
```

See Also

[IPanelItem Methods](#) | [IPanelItem Overview](#)

show Method

This method shows the pane by setting the **Olive.IState.State.Visible** state for the pane. Layout is then recalculated for the pane container

Parameters

No parameters

Syntax

```
oItem.show();
```

Example

```
// show the pane item  
oPanelItem.show();
```

See Also

[IPanelItem Methods](#) | [IPanelItem Overview](#)

hide Method

This method hides the pane by removing Olive.IState.State.Visible state. Layout is recalculated for the pane container.

Parameters

No parameters

Syntax

```
oItem.hide();
```

Example

```
// hide the pane item  
oPanelItem.hide();
```

See Also

[IPanelItem Methods](#) | [IPanelItem Overview](#)

isMinimized Method

This method returns Boolean value that indicates if the pane is minimized.

Parameters

No parameters

Syntax

```
oItem.isMinimized();
```

Example

```
// check whether oPaneListItem (object of a class implemented  
IPaneListItem interface) minimized or not  
var bMinimized = oPaneListItem.isMinimized();
```

See Also

[IPaneListItem Methods](#) | [IPaneListItem Overview](#)

minimize Method

This method minimizes the pane by setting the **Olive.IState.State.Minimized** state. Layout is then recalculated for the pane container.

Parameters

No parameters

Syntax

```
oItem.minimize();
```

Example

```
// minimize the pane item  
oPaneListItem.minimize();
```

See Also

[IPaneListItem Methods](#) | [IPaneListItem Overview](#)

restore Method

This method restores the pane size by removing the **Olive.IState.State.Minimized** state. Layout is then recalculated for the pane container

Parameters

No parameters

Syntax

```
oItem.restore();
```

Example

```
// restore the pane item  
oPaneListItem.restore();
```

See Also

[IPaneListItem Methods](#) | [IPaneListItem Overview](#)

toggle Method

This method toggles between the states **Olive.IState.State.Minimized** and **Olive.IState.State.NormalSize**.

Parameters

No parameters

Syntax

```
oItem.toggle();
```

Example

```
// toggle the pane item: minimize the pane item if it was not minimized  
and restore if it was minimized  
oPaneListItem.toggle();
```

See Also

[IPaneListItem Methods](#) | [IPaneListItem Overview](#)

toggleVisible Method

This method toggles between the states **Olive.IState.State.Visible** and **Olive.IState.State.Invisible**.

Parameters

No parameters

Syntax

```
oItem.toggleVisible();
```

Example

```
// toggle the visibility: set visible state for the pane item if it was  
not visible and set invisible state if it was visible  
oPaneListItem.toggleVisible();
```

See Also

[IPanelListItem Methods](#) | [IPanelListItem Overview](#)

isColumnItem Method

This method returns Boolean value that indicates if the pane is in a column-grouped list.

Parameters

No parameters

Syntax

```
oItem.isColumnItem();
```

Example

```
// check whether oPanelListItem (object of a class implemented
// IPanelListItem interface) column item or not
var bColumnItem = oPanelListItem.isColumnItem();
```

See Also

[IPanelListItem Methods](#) | [IPanelListItem Overview](#)

getPanelItemSizeFactor Method

This method returns a factor (number between 0 and 1) for the size of the pane item, or - 1 if the factor is undefined. For example if the attribute pane-size was defined as 40%, then the method returns 0.4.

Parameters

No parameters

Syntax

```
oItem.getPanelItemSizeFactor();
```

Example

```
// restore the pane item
var nSizeFactor = oPanelListItem.getPanelItemSizeFactor();
```

See Also

[IPanelListItem Methods](#) | [IPanelListItem Overview](#)

IPanelListItem Properties

IPanelListItem-specified properties

Property	Description
paneContainer	Holds a reference to a parent pane list
resizablePane	Specifies if the pane is resizable. The default value is true.

See Also

[IPanelListItem Overview](#) | [IPanelListItem Methods.htm](#) | [IPanelListItem Attributes](#)

paneContainer Property

Holds a reference to a parent pane list

Syntax

Data Type	object
Default value	null

Remarks

Sample Code

```
// get a reference of the parent pane list  
var oParentPaneList = oPaneListItem.paneContainer;
```

See Also

[IPanelListItem Overview](#) | [IPanelListItem Properties](#)

resizablePane Property

Holds a Boolean value specified whether the pane is resizable or not.

Syntax

Data Type	boolean
Default value	true

Remarks

Sample Code

```
// get a Boolean value
```

```
var bResizable = oPaneListItem.resizablePane;
```

See Also

[IPaneListItem Overview](#) | [IPaneListItem Properties](#)

IPaneSplitter Overview

This interface represents the Olive.Controls.IPaneSplitter class.

It is implemented by pane splitter controls.

The IPaneSplitter interface implements the [IPaneListItem](#) interface with general functionality for any pane item, and also adds specific dragging functionality.

Implemented Interfaces

Olive.Controls.IPaneListItem

Requirements

Script Files	OwcPanes.js
--------------	-------------

See Also

[IPaneSplitter](#) Methods | [ISearchFormField Methods](#) | [ISearchFormField Overview](#)

IPaneSplitter Methods

IPaneSplitter specific Methods

Event	Description
onDragStart	Handler of ondragstart DHTML event. This method calculates possible minimum and maximum dragging coordinates, creates a tracker element and attaches it to the control that implements the interface.
onDragMove	Handler of ondragmove DHTML event
onDragEnd	Handler of ondragend DHTML event

See Also

[IPaneSplitter](#) Overview

onDragEnd Method

This method is a handler of the ondragend DHTML event.

It is called at the end of the dragging.

Parameters

oDragger - An instance of the Olive.MoveDragger type.

pointMouse - An object with coordinates of the mouse.

oEvent - An instance of DHTML event, usually mouseup or mouseout event.

bCancel - A Boolean value indicates if to cancel all dragging.

Syntax

```
oPaneSplitter.onDragUp(oDragger, pointMouse, oEvent, bCancel);
```

Example

```
var pointMouse = {x : getEventClientX(oEvent), y :  
getEventClientY(oEvent)};  
oPaneSplitter.onDragEnd(oDragger, pointMouse, oEvent, false);
```

See Also

[IPaneSplitter Methods](#) | [IPaneSplitter](#) Overview

onDragMove Method

This method is a handler of the ondragmove DHTML event.

It is called during dragging.

Parameters

oDragger - An instance of the Olive.MoveDragger type.

pointMouse - An object of the mouse coordinates.

oEvent - An instance of DHTML event, usually mousemove event.

Syntax

```
oPaneSplitter.onDragMove(oDragger, pointMouse, oEvent);
```

Example

```
var pointMouse = {x : getEventClientX(oEvent), y :  
getEventClientY(oEvent)};  
oPaneSplitter.onDragMove(oDragger, pointMouse, oEvent);
```

See Also

[IPaneSplitter Methods](#) | [IPaneSplitter](#) Overview

onDragStart Method

This method is a handler of the ondragstart DHTML event.

It is called when dragging is started. The method calculates possible minimum and maximum dragging coordinates, creates a tracker element, and attaches it to the control

that implements the interface. It traces the possible minimum and maximum dragging coordinates into the debug tracer.

Parameters

oDragger - An instance of the Olive.MoveDragger type.

Syntax

```
oPaneSplitter.onDragStart(oDragger);
```

Example

```
oPaneSplitter.onDragStart(oDragger);
```

See Also

[IPaneSplitter Methods](#) | [IPaneSplitter](#) Overview

ISearchFormField Overview

This interface represents the **Olive.Controls.ISearchFormField** class.

It is implemented by any search form field control in order to support data exchange with search query data object.

There are two methods in the interface: `readFromSearchData` and `writeToSearchData`, which are implemented differently according to the type of search option.

Implemented Interfaces

Olive.Controls.IFormField

Requirements

Script Files	OwcSearchForm.js
--------------	------------------

See Also

[ISearchFormField Methods](#) | [SearchOption Overview](#)

ISearchFormField Methods

Olive.ISearchFormField-specific Methods

Event	Description
readFromSearchData	Reads a value from the search data and puts it into the HTML element of a search option control
writeToSerchData	Writes a value into the search data from HTML element represented in a search option control

See Also

[ISearchFormField Overview](#)

readFromSearchData Method

This method reads a value from search data and puts it into the HTML element of a search option control. The method returns a value that was set to HTML element.

It performs the opposite operation to [writeToSearchData](#) method.

Parameters

oDataObject - An instance of Data type (Olive.Data.SearchOptions or Olive.Data.SearchQuery)

Syntax

```
oSearchFormField.readFromSearchData(oDataObject);
```

Example

```
// 1. create search query object and set sort by option for it
// 2. read data from the created search query object and set the value
into the HTML element
var oSearchQueryObject = new Olive.Data.SearchQuery();
oSearchQueryObject.setSortBy(relevance);
var oValue = oSearchOption.readFromSearchData(oSearchQueryObject);
```

See Also

[ISearchFormField Methods](#) | [ISearchFormField Overview](#)

writeToSearchData Method

This method writes a value into the search data from the HTML element represented a search option control.

It performs the reverse operation to [readFromSearchData](#) method.

Parameters

oDataObject - An instance of Data type (Olive.Data.SearchOptions or Olive.Data.SearchQuery)

Syntax

```
oSearchFormField.writeToSearchData(oDataObject);
```

Example

```
// read value of the HTML element and write it into the data of search
object
```

```
var oSearchQueryObject = new Olive.Data.SearchQuery();  
oSearchOption.writeToSearchData(oSearchQueryObject);
```

See Also

[ISearchFormField Methods](#) | [ISearchFormField Overview](#)

ISelection Overview

The interface represents the Olive.ISelection class.

It is implemented by classes that support item selection.

The interface has a general API to change or clear a selection, and get selected items and their content items. Controls that support selection can specify HTML attribute owc:MultiSelect to allow multiple selection.

Following are main modes of the interface:

- **Olive.ISelection.Mode.Select**
- **Olive.ISelection.Mode.Unselect**

Examples of such classes are Olive.Controls.List and Olive.Controls.Tree.

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ISelection Methods](#) | [ISelection Events](#)

ISelection Events

ISelection-specific Events

Event	Description
selectionChanging	Fired by selectionChange() just before changing the selection.
selectionChanged	Fired by selectionChange() after the selection was changed.

See Also

[ISelection Overview](#) | [ISelection Methods](#)

selectionChanged Event

This event is fired by selectionChange() after the selection was changed.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following properties:

- **prevSelection** - An array of previous selected items
- **newSelection** - An array of items to apply a new selection
- **arrSelectedItems** - An array of items for which selected mode was applied
- **arrUnselectedItems** - An array of items for which unselected mode was applied

Supports bubbling: true

Example

```
// Handler of the event
function Tree_onSelectionChanged(oEvent)
{
    if(oEvent.newSelection)
    {
        // desirable action
    }
}
```

See Also

[ISelection Overview](#) | ISelection Events

selectionChanging Event

This event is fired by selectionChange() just before changing the selection.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following properties:

- **prevSelection** - An array of previously selected items
- **newSelection** - An array of items to be set in the new selection
- **arrSelectedItems** - An array of items that need to be applied selected mode
- **arrUnselectedItems** - An array of items that need to be applied unselected mode

Supports bubbling: true

Example

```
// Handler of the event
function Tree_onSelectionChanging(oEvent)
{
    if(oEvent.newSelection)
    {
        // desirable action
    }
}
```

```
}  
  
}
```

See Also

[ISelection Overview](#) | ISelection Events

IState Overview

This interface represents the Olive.IState class.

The interface is implemented by classes that support states. The interface has general functionality to make a bitmask of state flags and state change notifications.

The interface has a general API to access the state (get/set methods), change, toggle and modify the state, update state behavior, and initializing the state from the HTML element.

Following are some main states of the interface:

- Olive.IState.State.Visible
- Olive.IState.State.Invisible
- Olive.IState.State.Enable
- Olive.IState.State.Disable
- Olive.IState.State.Inactive
- Olive.IState.State.Active
- Olive.IState.State.Cold
- Olive.IState.State.Hot
- Olive.IState.State.Collapsed
- Olive.IState.State.Expanded
- Olive.IState.State.Selected
- Olive.IState.State.NotSelected
- Olive.IState.State.DataLoaded
- Olive.IState.State.DataNotLoaded
- Olive.IState.State.NormalSize
- Olive.IState.State.Minimize

Examples of such classes are Olive.Controls.Menuclass, which represents a Menu control, and classes that implement Olive.IUIElements interface.

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IState Methods](#) | IState Events

IState HTML Attributes

IState-specific Attributes

Property	Description
owc:init-state	Specifies an initial state of the control

See Also

[IState Overview](#) | [IState Methods](#) | IState Events [_Ref-54374521](#)

owc:init-state HTML Attribute

This attribute specifies an initial state of a control implemented in the interface

Syntax

```
<html-element owc:init-state = /string/ >
...
</html-element>
```

Remarks

Value of the attribute can be a string that specifies states, separated by commas. For example, Active,Visible or Enable,Selected. You can use any state that is supported in the interface and in a control implemented by the interface.

Example

```
<div owc:control="tabitem" owc:init-state="Active">... </div>
```

Usage

Control	Controls implemented the interface
Use	Optional
Default value	no

See Also

[IState Overview](#) | IState Attributes

IState Events

IState-specific Events

Event	Description
stateChanged	Event fired by setState() after the state was changed

See Also

[IState Overview](#) | [IState Methods](#) | IState Attributes

stateChanged Event

This event is fired by stateChange() after the state was changed.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following properties:

1. **prevState** - A previous state
2. **newState** - A new state

Supports bubbling: true

Example

```
// Handler of the event
function Pane_onStateChanged(oEvent)
{
    if(oEvent.newState)
    {
        // desirable action
    }
}
```

See Also

[IState Overview](#) | IState Events [_Ref-54374521](#)

IState Methods

IStatespecific Methods

Event	Description
getState	Returns the state of the object

setState	Sets a specified state
changeState	Changes state: set or remove specified state
toggleState	Toggles between a specified state and its inverse state
modifyState	Removes one state and adds another one
isStateSet	Checks if the specified state is set for the object
updateStateBehaviors	Attaches or detaches DHTML events like mouseover and mouseout
initStateFromHtml	Initiates state (hidden, disable) from HTML element

See Also

[IState Overview](#) | IState Events | IState Attributes

IUIElements Overview

The interface represents by Olive.IUIElements class.

It is implemented by classes that support UI elements. The interface has general functionality to bind, initialize state, and update UI elements.

Examples of such classes are:

Olive.Controls.TabItemclass

Olive.Controls.PageSwitcher.Item class

Olive.Controls.ListItem class

Olive.Controls.List class

Olive.Controls.SearchInFolderLookup class

Olive.Controls.TreeNode class

and classes that implement

Implemented Interfaces

Olive.IState

Olive.IUIElemTypes

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IUIElements Methods](#)

IUIElements Methods

Inheriting Methods of Olive.Controls.Control class

Event	Description
termControl	Terminates the control

IUIElementsspecific Methods

Event	Description
bindUiElement	Attaches custom style and behavior to the UI element, saves a reference to the UI element
getUiElement	Returns an UI element by its UI element type name
initUiState	Initiates the UI element state
updateUiState	Updates the UI element state
updateUiStateHtml	Updates UI element with an HTML specific state (enabled/disables, visible/hidden, highlighting)

See Also

IUIElements Overview

IUIElemTypes Overview

The interface represents by Olive.IUIElemTypes class.

The interface should be implemented by classes that support storing UI element types. The interface has general functionality for registering and reciving UI types.

Examples of such classes are Olive.ListItemypeclass, Olive.Controls.TreeNodeTypeclass and classes that implement Olive.IUIElementsinterface.

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IUIElemTypes Methods](#)

IUiElemTypes Methods

IUiElemTypespecific Methods

Event	Description
registerUiElemType	Registers an UI element type
getUiElemType	Returns an UI element type

See Also

[IUiElemTypes Overview](#)

getUiElemType Method

This method returns a UI element type object by its element type name.

Parameters

sUiElemType - A string with UI element type name

Syntax

```
oControl.getUiElemType(sUiElemType);
```

Example

```
// get a tab position UI element type object by its type name
OwcUiElem_TabPosition, where constant OwcUiElem_TabPosition is equal to
tabpos
var oTabPositionUiElemType =
oTabItem.getUiElemType(OwcUiElem_TabPosition);
// get a tree icon UI element type object by its type name
OwcUiElem_TreeIcon, where constant OwcUiElem_TreeIcon is equal to
treeicon
var oTreeIconUiElemType = oTreeNode.getUiElemType(OwcUiElem_TreeIcon);
```

See Also

[IUiElemTypes Methods](#) | [IUiElemTypes Overview](#)

registerUiElemType Method

This method registers the UI element type.

Parameters

sUiElemType - A string with the UI element type name

oUiElemType - An object that represents a corresponding UI element type

Syntax

```
oControl.registerUiElemType(sUiElemType, oUiElemType);
```

Example

```
// create tab position UI element type and register it as
OwcUiElem_TabPosition type for a tab item object oTabItem, where
constant OwcUiElem_TabPosition is equal to tabpos
var oTabPositionUiElemType = new Olive.UiElemType.TabItemPos();
oTabItem.registerUiElemType(OwcUiElem_TabPosition,
oTabPositionUiElemType);

// create tree icon UI element type and register it as
OwcUiElem_TreeIcon type for a tree node object oTreeNode, where
constant OwcUiElem_TreeIcon is equal to treeicon
var oTreeIconUiElemType = new Olive.UiElemType.TreeIcon();
oTreeNode.registerUiElemType(OwcUiElem_TreeIcon, oTreeIconUiElemType);
```

See Also

[IUiElemTypes Methods](#) | [IUiElemTypes Overview](#)

WebAppBase Example

The following example illustrates how to create a unique instance of the custom application class and define a function OwcGetApplication that returns the instance definition of a custom application class with some inheriting methods

```
//----- The one and only instance of the application -----
-----
var g_appFileCabinet = new FileCabinetApp();
g_appFileCabinet.initialize();
function OwcGetApplication()
{
    return g_appFileCabinet;
}

//----- FileCabinetApp class implementation -----
-----
function FileCabinetApp_ApplyPrototype(rObject)
{
    rObject.m_iActivePage = FileCabinetPage_Home;
    rObject.onSettingsLoaded = FileCabinetApp_onSettingsLoaded;
    rObject.parseURL = FileCabinetApp_parseURL;    //replace function from
base class
    rObject.buildURL = FileCabinetApp_buildURL;    //replace function of base
```

```

class
}

function FileCabinetApp()
{
}

function FileCabinetApp_onSettingsLoaded()
{
    // Enable debugging features if Debug=true URL parameter is passed
    if (this.m_oQueryString)
    {
        var bDebugMode =
String_parseBoolean(this.m_oQueryString.getParam("Debug", 0), false);
        if (bDebugMode)
        {
            this.enableDebugMode();
            alert("Debug mode is enabled");
        }
    }

    // Update window title
    document.title = this.getResString("WindowCaption");

    // Update initial frame index (from application preferences)
    this.m_iActivePage = this.getPreferenceNumber("InitialTab",
FileCabinetPage_Home);

    // Parse URL parameters
    this.parseURL();
}

// replace base class function.
function FileCabinetApp_parseURL()
{
    if (!this.m_oQueryString)
        return;

    // Read query string parameters
    var nDocUid = parseInt(this.m_oQueryString.getParam("DocUid", 0), 10);
    if (!isNaN(nDocUid))
        this.m_sDocUid = nDocUid.toString();

    var sDocHRef = this.m_oQueryString.getParam("DocHRef", 0);

```

```

if (sDocHRef)
this.m_sDocHRef = sDocHRef;
if (this.m_sDocHRef != null || this.m_sDocUid != null)
{
var oCmdParams = new Olive.ContentItem.Document();
oCmdParams.m_sDocHRef = this.m_sDocHRef;
oCmdParams.m_sDocUid = this.m_sDocUid;

var sEntityId = this.m_oQueryString.getParam("EntityId", 0);
if (sEntityId && sEntityId != "") oCmdParams.m_sEntityId = sEntityId;

var nPageNo = parseInt(this.m_oQueryString.getParam("PageNo", 0), 10);
if(!isNaN(nPageNo)) oCmdParams.m_nPageNo = nPageNo;
var sPageLabel = this.m_oQueryString.getParam("PageLabel", 0);
if (sPageLabel && sPageLabel!= "") oCmdParams.m_sPageLabel =
sPageLabel;
var sPrimId = this.m_oQueryString.getParam("PrimId", 0);
if (sPrimId && sPrimId != "") oCmdParams.m_sPrimId = sPrimId;

this.oCmdParams = oCmdParams;
this.m_iActivePage = FileCabinetPage_Viewer;
}

// Read initial tab
var sInitialTab = this.m_oQueryString.getParam("Tab", 0);
if (sInitialTab)
{
switch(sInitialTab.toLowerCase())
{
case"home":
this.m_iActivePage = FileCabinetPage_Home; break;
case"library":
this.m_iActivePage = FileCabinetPage_Library; break;
case"viewer":
this.m_iActivePage = FileCabinetPage_Viewer; break;
case"searchres":
this.m_iActivePage = FileCabinetPage_SearchRes; break;

```

```

default:
    Debug.assert(false, "Invalid URL parameter: '" + sInitialTab + "' is
not a valid tab name");
}
}
}

// function building URL to open application at specified content item
// replace base class function.
function FileCabinetApp_buildURL(oContentItem)
{
    var sItemLink = this.getAppUrl();
    if (oContentItem && oContentItem.buildQueryString)
        sItemLink += oContentItem.buildQueryString();
    return sItemLink;
}

JScript_DeriveClass(FileCabinetApp, Olive.WebAppBase);
FileCabinetApp_ApplyPrototype(FileCabinetApp.prototype);

```

See Also

[WebAppBase Class](#) | [WebAppBase Events](#) | [WebAppBase Methods](#) | [WebAppBase Properties](#)

WebAppBase Class Methods

WebAppBase-specific Methods

Event	Description
initialize	Initializes the application object
loadSettings	Loads the application settings from the server. This method is called when initializing the method of the WebAppBase class
parseSettings	Parses the application settings received from the server
parsePreferences	Parses the application preferences. This method is called in the <code>parseSetting()</code> method.
parseResources	Parses the application resources. This method is called in the <code>parseSetting()</code> method.
parseWindowOptions	Parses the application window options. This method is called in the <code>parseSetting()</code> method.

<u>isReady</u>	Returns a Boolean value that indicates if all settings are loaded and parsed.
<u>onSettingsLoaded</u>	Overridable function for derived classes for desired actions when settings are loaded.
<u>getAppUrl</u>	Returns URL of the application.
<u>getPreference</u>	Returns a preference defined for the application.
<u>getPreferenceBoolean</u>	Returns a Boolean preference defined for the application.
<u>getPreferenceNumber</u>	Returns a Number preference defined for the application.
<u>getResString</u>	Returns a string from resources of the application.
<u>formatResString</u>	Returns a formatted resource string.
<u>getWindowOptions</u>	Returns window options defined in settings of the application.
<u>openWindow</u>	Opens a popup window. Deprecated-Replaced by openPopup of Olive.PopupInfo class.
<u>updateRequestParams</u>	Updates parameters from the request before submitting request to server to add custom URL parameters.
<u>hookSendRequest</u>	Called by the application control just before submitting request to server
<u>onErrorOccurred</u>	Central error handler
<u>reportError</u>	Reports an error
<u>prepareItemMailParams</u>	Prepares parameters for mail form (fill in from, subject, body fields)
<u>registerAppCommands</u>	Registers two default application commands
<u>addItemToFavorites</u>	Default application command add2favorites
<u>copyItemToClipboard</u>	Default application command copylink
<u>buildUrl</u>	Builds application specific URL
<u>parseUrl</u>	Parses URL of the application
<u>getItemTitle</u>	Returns a title of a content item

getItemDocTitle	Returns a document title of a content item
cloneObject	Creates a copy of an object
enableDebugMode	Enables debug mode for the application

See Also

[WebAppBase Class](#) | [WebAppBase Events](#) | [WebAppBase Properties](#)

WebAppBase Class Properties

WebAppBase-specific properties

Property	Description
DataProviderUrl	String value that holds URL to data provider
m_oQueryString	QueryString object holding parameters of query string

See Also

[WebAppBase Class](#) | [WebAppBase Events](#) | [WebAppBase Methods](#)

IValue Overview

This interface is for controls that support data exchange between an HTML element for inputting and/or displaying data, and a Javascript object.

The main functionality is to read the value of the HTML element and store in the object, or to format the object's value and write it to the HTML element. Values can also be parsed, validated, and compared, and an event is fired when the internal value of the object changes.

Note that in the naming conventions for value methods the HTML element is called the "control": updateControl() means write the internal value of the Javascript object to the HTML element according to some format.

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[IValue Methods](#) | [IValue Events](#) | [IValue Attributes](#)

IValue HTML Attributes

IValue-specific Attributes

	Description
--	-------------

Attribute	
owc:DataType	Data type of the control. Possible values: object, string, boolean, number, date, year, month, day, email [email address], email-list [addresses separated by ;], sort-by
owc:Format	The format used for reading and writing values.
owc:HtmlValue	boolean - whether the string value of the Html-element is its innerHTML

See Also

[IValue Overview](#) | [IValue Methods](#) | [IValue Events](#)

owc:DataType HTML Attribute

This attribute specifies the control's data type. If this attribute is not specified, the default data type is string for all HTML elements except for Checkbox, for which the default data type is Boolean.

Syntax

```
<html-element owc:Format = /string/ >
...
</html-element>
```

Remarks

Possible values are:

boolean - as in JavaScript. The read/write format for "true" and "false" can be configured with owc:Format

number - as in JavaScript. The number is converted to and from a string for reading/writing to HTML.

string - as in JavaScript. No formatting is needed to read/write to HTML.

object - an object

date - This and the following three date-related types can convert to and from a JavaScript Date object.

year

month

day

email - an email address "person@domain.name"

email-list - a list of addresses separated by ";":
"person1@domain.name1;person2@domain.name2"

sort-by - a field by which a list is sorted

Example

```
<input owc:control="Field" owc:DataType="Boolean"
owc:Format="bad:good"...> ... </input>
```

Usage

Control	Controls implementing the interface
Use	Optional
Default value	none

See Also

[IValue Overview](#) | [IValue Methods](#) | [IValue Attributes](#)

owc:Format HTML Attribute

This attribute is the format used to read and write values.

Syntax

```
<html-element owc:Format = /string/ >
...
</html-element>
```

Remarks

The owc:Format attribute is currently implemented for boolean and date value types.

The format string for Boolean values gives true and false strings separated by a semicolon (;), for example "invalid;valid".

The format string for all date-related value types has fprintf-like variable placeholders within a string of constants. The placeholders are "%" followed by a character. For example, "%m/%d/%Y" for Date (12/31/2006), "%B %d, %Y" spells out the month name in the current language locale (December 31, 2006), "%A, %b %d, %Y" adds the day name (Sunday, December 31, 2006).

NOTE: the date format string is not yet implemented for parsing values; it currently works for writing values.

Below is a full list of date placeholders that may be used after "%":

- a - Abbreviated weekday name
- A - Full weekday name
- o - one letter weekday name
- b - Abbreviated month name
- B - Full month name
- c - Date and time representation appropriate for locale

d - Day of month as decimal number (01 31)
 g - GMT representation of date and time
 H - Hour in 24-hour format (00 23)
 I - Hour in 12-hour format (01 12)
 j - Day of year as decimal number (001 366)
 m - Month as decimal number (01 12)
 M - Minute as decimal number (00 59)
 p - Current locale's A.M./P.M. indicator for 12-hour clock
 S - Second as decimal number (00 59)
 U - Week of year as decimal number, with Sunday as first day of week (00 53)
 w - Weekday as decimal number (0 6; Sunday is 0)
 W - Week of year as decimal number, with Monday as first day of week (00 53)
 x - Date representation for current locale
 X - Time representation for current locale
 y - Year without century, as decimal number (00 99)
 Y - Year with century, as decimal number
 z - Timezone offset
 % - Percent sign
 #{letter} for letters dHjmMSy - add zeros for left padding of small values ("01" instead of "1")

Example

```

<input owc:control="Field" owc:DataType="Boolean"
owc:Format="bad:good"...> ... </input>
  
```

Usage

Control	Controls implementing the interface
Use	Optional
Default value	none

See Also

[IValue Overview](#) | [IValue Methods](#) | [IValue Attributes](#)

owc:HtmlValue HTML Attribute

This attribute specifies if the string value of the HTML element is its innerHTML.

Syntax

```
<html-element owc:HtmlValue = /boolean/ >
...
</html-element>
```

Remarks

Usually an HTML element that is used as a field as some kind of input or select. The SDK has an algorithm that extracts the value of a Field element by looking for a selected option, checked property, value property, or innerText.

If `owc:HtmlValue="true"`, however, data exchange bypasses this algorithm and simply reads or writes the innerHTML of the element. This is useful if you want to use a read-only element (such as `div` or `span`) as a value field.

Example

```
<div owc:control="Field" owc:DataType="String" owc:HtmlValue="true"...>
... </input>
```

Usage

Control	Controls interface implementation
Use	Optional
Default value	false

See Also

[IValue Overview](#) | [IValue Methods](#) | [IValue Attributes](#)

IValue Class Events

IValue-specific Events

Event	Description
valueChanged	Fired by <code>setValue()</code> when the new value is different from the old value.
validationError	Fired by <code>validateValue()</code> when the new value is invalid.

See Also

[IValue Overview](#) | [IValue Methods](#) | [IValue Attributes](#)

validationError Event

This event is fired by `validateValue()` when the new value is invalid.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following properties:

1. **value** - The value that is validated
2. **error** - An exception object, created by DHTML_newError(nErrNum, sDesc)

Supports bubbling: true

Example

```
// Handler of the event
function Date_onValidationError(oEvent)
{
    // desirable action
}
```

See Also

[IValue Overview](#) | IValue Events

valueChanged Event

This event is fired by setValue() when the new value is different from the old value.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with the following properties:

1. **prevValue** - previous value of the control
2. **newValue** - new value of the control
3. **DoNotUpdateControl** - Boolean value if the control was updated with the new value

Supports bubbling: true

Example

```
// Handler of the event
function Calendar_onValueChanged(oEvent)
{
    if( !oEvent.DoNotUpdateControl )
    {
        // desirable action
    }
}
```

```
}  
  
}
```

See Also

[IValue Overview](#) | IValue Events

IValue Class Methods

IValue-specific Methods

Event	Description
getValue()	Returns the internal value of the object
setValue(oValue, bDoNotUpdateHTML)	Assigns a new value to the object
getFormat()	Returns the format of the object
setFormat(sFormat, bUpdateControl)	Assigns a format to the object, and updates the control's display
formatValue(oValue)	Returns the value that is formatted by the object's format setting
parseValue(oValue)	Returns the given value, parsed according to the data type and formatting
validateValue(oValue, bDoNotReportErrors)	Returns a boolean, true=valid, false=invalid
compareValue(oPrevValue, oNewValue)	Tests if the given values are equal
updateControl()	Assigns the internal value of the object to the HTML element (the "control")
updateData()	Assigns the HTML value to the internal value of the object
getHtmlValue(bParseValue)	Reads the string value of the HTML element (the "control")
setHtmlValue(oValue)	Writes the given value to the HTML element
reportValidationError(oValue, oError)	Fires an event with the event properties value and error (called by validateValue)

See Also

[Value Events](#) | IValue Attributes

LangList Control Overview

The LangList component loads the list of languages from the server into this Control.

Syntax

```
<span owc:Control=LangList></span>
```

OwcLangList Class Overview

The LangList class functionality is implemented by the Olive.Controls.LangList JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.LangList

Implemented Interfaces

Olive.IDataBound

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[LangList Overview](#)

LangList Example

The example below describes how to use the LangList control in the user application.

```
<div id="divLangList" owc:Control="LangList" class="Hidden">
</div>
```

See Also

[LangList Overview](#) | [OwcLangList Class Overview](#)

List Overview

The List control is an abstract base class that gives list functionality to its two derived classes, [DocList](#) (a list of documents in a folder) and [SearchRes](#) (a list of entities that contain search hits).

Paging

It is possible to divide up a list into page-sized subgroups, so that the user can page forwards and backwards. Going to a new page is done by sending a request to the server for the data for the new page, and replacing the current data with the new data. (This is different from trees, in which data that is already on the client-side is hidden and displayed when the user clicks to expand or close a node, without any new request to the server.) You can also make the "nextPage" command-button change appearance on the last page (when paging forwards is impossible) or the "prevPage" button on the first page. The normal and disabled button styles should be defined in a CSS file, using designated class names (OwcPrevPageButton, OwcPrevPageButton_d, OwcNextPageButton, OwcNextPageButton_d, OwcFirstPageButton, OwcFirstPageButton_d, OwcLastPageButton, and OwcLastPageButton_d). All pagination buttons are hidden when the data fits onto one page.

Sorting

You can sort the list with a "SortBy" sub-control within a List control. When the user changes the value of the "SortBy" element, a request is automatically sent to the server to update the list in the new sort order.

Selecting a List Item

The common reason to display a list of documents, or a list of entities of search results, is to allow the user to select an item and display it in a reader. For information on how to configure the behavior of lists, see the documentation for the [DocList](#) and [SearchRes](#) controls.

See Also

[DocList Overview](#) | [SearchRes Overview](#) | [List Class](#)

List HTML Attributes

List-specific Attributes

Attribute	Description
owc:UsePagination	Divide up the results into pages (true) or display all the results on one page (false). Type boolean; optional; default value = false.
owc:PageSize	The number of items per page. This is only used if UsePagination="true" Type number, must be greater than 0; default value = 1.

See Also

[List Overview](#) | [DocList Overview](#) | [SearchRes Overview](#)

owc:PageSize HTML Attribute

This attribute defines the maximum number of items on each page if list pagination is enabled.

Syntax

```
owc:PageSize=/nSize/
```

Remarks

This attribute defines the maximum number of items on each page. It can only be used if the owc:UsePagination attribute is "true". Its value must be greater than 0.

Example

```
owc:PageSize = "5"
```

Usage

Use	Optional
Default value	1

See Also

[List Overview](#) | [List Attributes](#) | [List Attribute owc:UsePagination](#)

owc:UsePagination HTML Attribute

This attribute specifies if a list is divided into pages.

Syntax

```
owc:UsePagination=/bFlag/
```

Remarks

This attribute determines if a list is divided into pages. If its boolean value is true, then the list will be divided into pages. If it is absent or false, then the list will not be divided into pages and all results will be shown on one page.

Example

```
owc:UsePagination = "true"
```

Usage

Use	Optional
Default value	false

See Also

[List Overview](#) | [List Attributes](#) | [List Attribute owc:PageSize](#)

List Class Overview

The List control functionality is implemented by the Olive.Controls.List JavaScript class. The Olive.Controls.List class allows access to the List commands and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.List

Implemented Interfaces

Olive.IUiElements

Olive.ISelection

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[List Overview](#) | [List Commands](#) | [List Methods](#)

List Class Methods

List Methods

Method	Description
PrevPage	Go to the previous page.
NextPage	Go to the next page.
FirstPage	Go to the first page.
LastPage	Go to the last page.

See Also

[List Overview](#) | [List Class](#)

List Class Methods

List Methods

Method	Description
sortBy(sField, bAscending, bDoNotReloadContent)	Sort the list, possibly reloading the content to display the data in the new sort order. Arguments: sField (string) - name of the field by which the list is to be sorted. It must be the name of a server-side metadata field for the list item (a document field or a search-result field). bAscending (boolean) - Sort in ascending order (true) or descending order (false). bDoNotReloadContent (boolean) - Change the sort order without reloading the content (true), or change the sort order and reload the content by retrieving new data from the server (false).
getItemType(itemType)	Get the ItemType object from the lists array of item types.
getSortOptions()	Return sort options defined for the list
setSortOptions(oSortOptions, bDoNotReloadContent)	Set sort options for the list.

See Also

[List Overview](#) | [List Class](#)

List CSS Classes

CSS Class	Description
OwcPrevPageButton	CSS class for Previous Page Button
OwcNextPageButton	CSS class for Next Page Button
OwcFirstPageButton	CSS class for First Page Button
OwcLastPageButton	CSS class for Last Page Button
OwcPrevPageButton_d	CSS class for disabled Previous Page Button

OwcNextPageButton_d	CSS class for disabled Next Page Button
OwcFirstPageButton_d	CSS class for disabled First Page Button
OwcLastPageButton_d	CSS class for disabled Last Page Button
Hidden	CSS class for Hidden Button, used internally to hide buttons (for example, the First Page or Previous Page buttons if the List is on the first page).

See Also

[List Overview](#) | [List Class](#)

List Control Example

The List component is a base class that is not used directly, but gives list functionality to its two derived classes, [DocList](#) (a list of documents in folders in the inventory) and [SearchRes](#) (a list of entities containing hits for a search).

For this reason, examples of List Control use are described in examples for these two classes.

See Also

[List Overview](#) | [ListControl Class](#)

ListItem Overview

The ListItem control is represented by an abstract base class for [Olive.Controls.DocList](#) item (items in a list of documents in a folder) and [Olive.Controls.SearchRes](#) item (items in a list of entities that contain search hits).

The list item's control type is registered in the [Olive.Controls.List](#) constructor class, so the control can not be used separately, but only inside a list control.

See Also

[ListItem Class](#) | [ListItem Attributes](#)

ListItem HTML Attributes

ListItem-specific Attributes

Attribute	Description
owc:ItemType	Specifies a type of the list item

See Also

[ListItem Overview](#)

owc:ItemType HTML Attribute

This attribute specifies the list item control type.

Syntax

```
<html-element owc:ItemType=/string/>
...
</html-element>
```

Remarks

This attribute can be set for control classes derived from the ListItem class: Olive.Controls.DocListItem (set to Document) or Olive.Controls.SearchResItem (set to SearchResult).

Although it usually specifies the type of list item the public method class OwcListRegisterItemType() used, the method of the Olive.Controls.List. In the constructor of the List class the method registers a default type of the list item: ListItem. The method has following signature: OwcListRegisterItemType(rObject, sItemTypeName, sControlType, sItemClass) where

- **rObject** is an instance of the Olive.Controls.List class
- **sItemTypeName** is a name of a registered list item type
- **sControlType** is a control type name (ListItem, DocListItem, SearchResItem or other inherited from them)
- **sItemClass** is a string that specifies a class of the list item control

Example

```
<div owc:Control="ListItem" owc:ItemType="document">... </div>
```

Usage

Control	ListItem
Use	Optional
Default value	no

See Also

[ListItem Overview](#) | [ListItem Attributes](#)

ListItem Class Overview

The ListItem control functionality is implemented by the Olive.Controls.ListItem JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.ListItem

Implemented Interfaces

[Olive.IUiElements](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[ListItem Methods](#) | [ListItem Overview](#)

ListItem Class Methods

ListItem-specific Methods

Method	Description
itemSelect	Clears selection of subitems in the list item if the <code>bSelect</code> parameter is false

See Also

[ListItem Overview](#) | [ListItem Class](#)

itemSelect Method

This method clears the selection of sub controls in the list item when the parameter `bSelect` is set to false

Parameters

bSelect - A Boolean value indicates when to clear selection of subcontrols in the list item.

Syntax

```
oListItem.itemSelect(bSelect);
```

See Also

[ListItem Class](#) | [ListItem Methods](#)

ListItem Control Example

The ListItem control is a base class that is not used directly, but gives ListItem functionality to its two derived classes, Olive.Controls.DocListitem (list item for a list of documents in folders in the inventory) and Olive.Controls.SearchResItem(list item for a list of entities containing hits for a search). Due to this reason examples of using the ListItem Control are described in examples for these two classes.

See Also

[ListItem Overview](#) | [ListItemControl Class](#)

Magnifier Control Overview

The Magnifier control shows a magnified image.

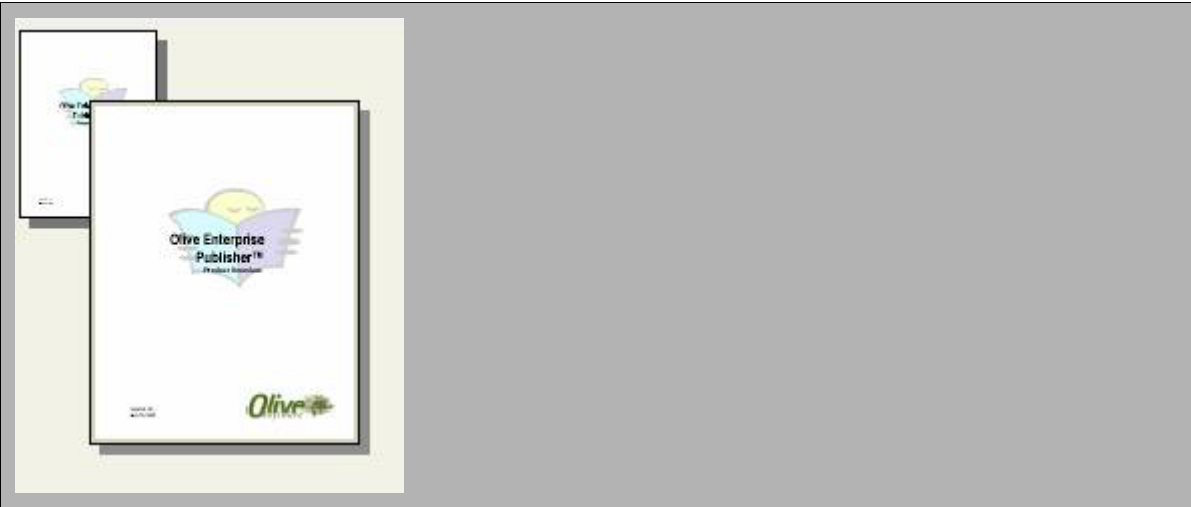
Syntax

```
<html-element owc:Control=Magnifier
  owc:MaxWidth=/number/ owc:MaxHeight=/number/
  owc:MagPercent=/number/ owc:SelfHiding = /boolean/ />
```

Remarks

A magnified image appears over the original image with an outline.

Preview



See Also

[Magnifier Class](#) | [Magnifier Attributes](#)

Magnifier HTML Attributes

Magnifier-specific Attributes

Attribute	Description
-----------	-------------

owc:MaxWidth	Specifies the maximal width for the Magnifier image
owc:MaxHeight	Specifies the maximal height for the Magnifier image
owc:MagPercent	Specifies the magnification percent
owc:SelfHiding	A Boolean attribute that specifies if the magnifier is hidden or appears automatically in response of events

See Also

[Magnifier Control](#) | [Magnifier Class](#)

Magnifier Class Overview

The Magnifier control functionality is implemented by Olive.Controls.Magnifier JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Magnifier

Requirements

Script Files	OwcMagnifier.js
---------------------	-----------------

See Also

[Magnifier Overview](#) | [Magnifier Methods](#)

--

Magnifier Class Methods

Magnifier Methods

Method	Description
hide	Hides the magnifier image

onMouseMove	DHTML event handler <i>mousemove</i> that is attached to document
onShow	Handler of the desired event, shows the magnifier on the source image object
setMagPercent	Sets the member variable m_nMagPercent
setMaxHeight	Sets the member variable m_nMaxHeight
setMaxWidth	Sets the member variable m_nMaxWidth

See Also

[Magnifier Class](#)

--

Magnifier Example

The example below describes the Magnifier control used in the user application. Place this code with other controls, consisting Thumbnail.

```
<div id="divMagnifier" owc:Control="Magnifier" owc:SelfHiding="true"
    owc:MaxHeight="200" owc:MaxWidth="200" owc:MagPercent="150">

</div>
```

See Also

[Magnifier Overview](#) | [Magnifier Class](#) | [Thumbnail Overview](#)

--

Mail Control Overview

The Mail control is a form control used to form a data request to sending an e-mail.

Syntax

```
<html-element owc:Control = Mail>
  <html-element owc:Control="Field" owc:FieldName="From" owc:DataType="email" />
  <html-element owc:Control="Field" owc:FieldName="To" owc:DataType="email" />
  <html-element owc:Control="Field" owc:FieldName="Cc" owc:DataType="email-list" />
```

```
<html-element owc:Control="Field" owc:FieldName="Subject" />
<html-element owc:Control="Field" owc:FieldName="MessageBodyAuto" />
<html-element owc:Control="Field" owc:FieldName="MessageBodyCustom" />
</html-element>
```

Remarks

The control is a container for field controls where users enter necessary data to send an e-mail, such as To, From, and Cc addresses, and Subject.

Preview



The preview shows a web form with the following elements:

- From***: A text input field.
- To***: A text input field.
- Cc**: A text input field.
- Subject**: A text input field.
- Message Body:**: A large text area containing the text "This is inserted text."
- Enter the text to be appended to the message:**: A text input field.
- Send**: A button at the bottom right.

See Also

[MailForm Class](#)

© Olive Software Inc. All rights reserved

MailForm Class Overview

The MailForm control functionality is implemented by Olive.Controls.MailForm JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

`Olive.Controls.MailForm`

Implemented Interfaces

[Olive.Controls.IForm](#)

Olive.Controls.IDataBound

Requirements

Script Files	OwcMail.js
---------------------	------------

See Also

[MailForm Methods](#) | [MailForm Overview](#)

© Olive Software Inc. All rights reserved

MailForm Class Methods

MailForm Methods

Method	Description
sendMail	Collects data from the form and sends request to the server

See Also

[MailForm Class](#)

© Olive Software Inc. All rights reserved

sendMail Method

This method collects data from all its fields, validates their formats, forms a request to the server, and sends the request.

It returns a Boolean value that indicates if the form fields have been successfully updated.

The e-mail will be sent then from the server.

Parameters

No parameters

Syntax

```
oMailForm.sendMail();
```

Example

```
var bFieldsUpdated = oMail.sendMail();  
if (bFieldsUpdated)  
{  
    // do desired actions  
}
```

See Also

[MailForm Methods](#) | [MailForm Class](#)

© Olive Software Inc. All rights reserved

Mail Example

The example below describes how to use the Mail control in the user application.

```

<div id="MailForm" owc:Control="Mail" owc:onInitialized="onMailInitialized" class="Content">
  <table class="MaxTable" border="0">
    <col class="LabelText" />
    <col width="100%" />
    <tr>
      <td class="Required">From*</td>
      <td><input owc:Control="Field" owc:FieldName="From" owc:Required="true" type="text"
id="From" name="From" value=" " owc:DataType="email" class="InputText" /></td>
    </tr>
    <tr>
      <td class="Required">To*</td>
      <td><input owc:Control="Field" owc:FieldName="To" owc:Required="true" type="text"
id="To" name="To" value=" " owc:DataType="email-list" class="InputText" /></td>
    </tr>
    <tr>
      <td>Cc</td>
      <td><input owc:Control="Field" owc:FieldName="Cc" type="text" id="Cc" name="Cc"
value="" owc:DataType="email-list" class="InputText" /></td>
    </tr>
    <tr>
      <td>Subject</td>
      <td><input owc:Control="Field" owc:FieldName="Subject" type="text" id="Subject"
name="Subject" value="" class="InputText" /></td>
    </tr>
    <tr>
      <td colspan="2" height="100%">
        <table cellpadding="0" cellspacing="0" width="100%">
          <tr>
            <td>
              <div class="LabelText" id="messageBody">Message Body:</div>
              <div owc:Control="Field" owc:FieldName="MessageBodyAuto" owc:HtmlValue="yes"
class="MailBody" id="divHTMLBody">This is hard-coded text in the e-mail.</div>
              <div class="LabelText">Enter the text to be appended to the message:</div>
              <textarea owc:Control="Field" owc:FieldName="MessageBodyCustom" id="BodyAppended"
class="MailBody Editable" NAME="BodyAppended"></textarea>
            </td>
          </tr>
        </table>
      </td>
    </tr>
  </table>

```

```
</tr>
</table>
</td>
</tr>
</table>
<div class="PopupButtonsRow">
  <span id="SendButton" title="Send e-mail" owc:Control="CommandButton"
owc:Command="SendMail">Send</span>
</div>
</div>
```

See Also

[Mail Overview](#) | [MailForm Class](#)

Menu Control Overview

The Menu control is used for both static menus (drop-down menus) and context menus.

Syntax

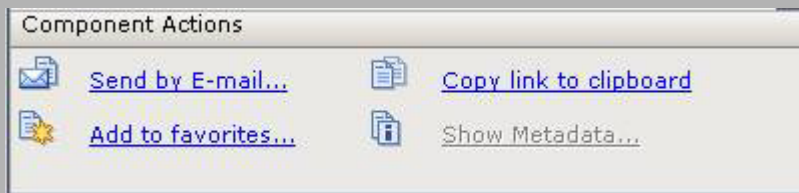
```
<html-element owc:Control=Menu  
  owc:MenuType=/string - type of the menu/>  
<!-- Nested controls -->  
</html-element>
```

Remarks

Menu control is a container for menu item controls.

Preview

Static menu:



Popup menu:



See Also

[Menu Class](#) | [Menu Attributes](#) | [MenuItem Control](#)

Menu HTML Attributes

Menu-specific Attributes

Attribute	Description
owc:MenuType	Specifies menu type

See Also

[Menu Overview](#) | [Menu Class](#)

owc:MenuType HTML Attribute

Specifies type of the menu: static or context

Syntax

```
<html-element owc:MenuType = /string/ >  
...  
</html-element>
```

Remarks

The attribute value can be **menu** for static menu and **contextmenu** for context menu. There is no default value.

Example

```
<div owc:Control="Menu" owc:MenuType="contextmenu"> ... </div>
```

Usage

Control	Menu
Use	Required
Default value	no

See Also

Menu Control | Menu Attributes | [Menu Class](#)

Menu Class Overview

The Menu control functionality is implemented by Olive.Controls.Menu JavaScript class

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Menu

Implemented Interfaces

[Olive.ICmdSource](#)

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[Menu Events](#) | Menu Methods | [Menu Overview](#)

Menu Class Events

Menu Events

Event	Description
menuCancelled	Fired when menu is canceled, to inform other controls

See Also

[Menu Class](#) | Menu Methods | Menu Properties

menuCanceledEvent

This event is activated when the menu is canceled.

Arguments

The event object that is passed to the event handler is an instance of the Olive.Event class. This object does not have additional properties.

Remarks

This event is fired to inform other controls that the menu is canceled, so that they can update status or perform desired actions

Example

```
// define handler for the event in html-page, for example for Toc control
// onMenuCancelledForToc function defined in an included js file
<div owc:Control=Toc onMenuCancelled=onMenuCancelledForToc>
...
</div>

// or define handler for the event in js-code
// oControl is an SDK control;
// onMenuCancelledForControl a function, handler for the event
oControl.onMenuCancelled = onMenuCancelledForControl;
// in the handler of the event do desired actions
function onMenuCancelledForControl(oEvent)
{
    var oControl = oEvent.srcObject;
    writeToLog(oControl, menu cancelled);
}
```

See Also

[Menu Class](#) | [Menu Events](#)

Menu Class Methods

Menu Methods

Event	Description
getCommandSource	Returns m_oCommandSource field
setCommandSource	Sets m_oCommandSource field
trackPopupMenu	Shows a popup menu in the correct position, capture and react

	to events
stopTracking	Hides the popup menu, detaches events, sends command when needed
updateMenuState	Scans for user-defined procedures for command source object

See Also

[Menu Class](#) | [Menu Events](#) | Menu Properties

Menu Example

The following example illustrates how to:

- Add a Menu to an HTML page using the MenuItem controls
- Use custom HTML attributes of the Menu control to create different menu types
- Use CSS classes to format menu layout

```
// In HTML
// 1. context menu
<div class="ContextMenuControl" owc:Control="Menu"
owc:MenuType="contextmenu">
  <table>
    <tr><td owc:Control="MenuItem" owc:Command="sendbyemail">Send by E-
mail...</td></tr>
    <tr><td owc:Control="MenuItem" owc:Command="copylink">Copy link to
clipboard</td></tr>
    <tr><td owc:Control="MenuItem" owc:Command="add2favorites">Add to
favorites</td></tr>
    <tr><td owc:Control="MenuItem" owc:Command="createshortcut">Create
Shortcut</td></tr>
    <tr><td owc:Control="MenuItem" owc:Command="showmetadata">Show
Metadata... </td></tr>
    <tr><td owc:Control="MenuItem" owc:Command="selectall">Select
All</td></tr>
  </table>
</div>
// 2. static menu
<div class="MenuTasks" owc:Control="Menu" owc:MenuType="menu">
  <table>
    <tr>
      <td owc:Control="MenuItem" owc:Command="sendbyemail"
class="SendMailImg">
```

```

Send by E-mail...</td>
<td owc:Control="MenuItem" owc:Command="copylink"
class="CopyToClipboardImg">
Copy link to clipboard</td>
</tr>
<tr>
<td owc:Control="MenuItem" owc:Command="add2favorites"
class="AddToFavoritesImg">
Add to favorites</td></tr>
<td owc:Control="MenuItem"
owc:Command="showmetadata"class="ShowMetadataImg">
Show Metadata... </td>
</tr>
</table>
</div>
// In CSS:
/* Styles for context menu: */
.ContextMenuControl
{
border-right: 2px outset;
border-top: 2px outset;
border-bottom: 2px outset;
border-left: 2px outset;
background-color: #D2D2D2;
position: absolute;
display: none;
visibility: hidden;
z-index: 25;
}
.ContextMenuControl table
{
font: 8pt Verdana;
}
/* Style for context menu item (not highlighted) */
.ContextMenuControl td
{
padding: 2px 18px 3px 18px;
white-space: nowrap;
}

```

```
color:    Black;
cursor:   pointer;
}

/* Style for highlighted context menu item */
.ContextMenuControl.Highlighted
{
color:    White;
background-color: #191970;
}

.ContextMenuControl.Disabled
{
color:    #808080;
}

/* Styles for static menu: */
.MenuTasks table
{
font:     8pt Verdana;
border-right: solid 1px #A5A8AE;
border-left:  solid 1px #A5A8AE;
border-bottom: solid 1px #A5A8AE;
-moz-box-sizing: border-box;
background-color: #EAE9E5;
width:    100%;
}

.MenuTasks td
{
text-decoration: underline;
color:    Blue;
cursor:   pointer;
white-space: nowrap;
}

.MenuTasks.MenuItemDisabled
{
font:     8pt Verdana;
text-decoration: underline;
color:    #808080;
cursor:   default;
}
```

```

white-space:    nowrap;
}
/* Images */
.SendMailImg
{
background-image:    url(Images/icon_SendMail.gif);
background-repeat:    no-repeat;
padding-left:    35px;
}
.CopyToClipboardImg
{
background-image:    url(Images/icon_CopyToClipboard.gif);
background-repeat:    no-repeat;
padding-left:    35px;
}
.AddToFavoritesImg
{
background-image:    url(Images/icon_AddToFavorites.gif);
background-repeat:    no-repeat;
padding-left:    35px;
}
.ShowMetadataImg
{
background-image:    url(Images/icon_ShowMetadata.gif);
background-repeat:    no-repeat;
padding-left:    35px;
}

```

See Also

[Menu Overview](#) | [Menu Class](#)

MenuItem Control Overview

The **MenuItem** control is a control for a menu item in the [menu](#) control.

Syntax

```

<html-element owc:Control=Menu /menu attributes /
  <html-element owc:Control=MenuItem owc:Command=/action/ >
    /public name of the menu item/

```

```
</html-element>

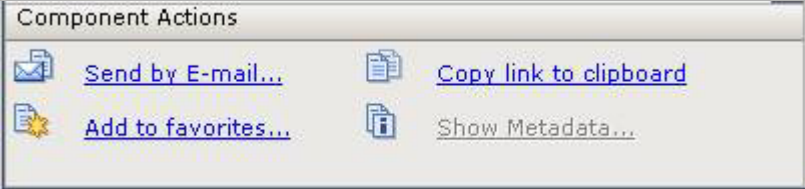
/other menu items/
</html-element>
```

Remarks

The MenuItem control must be inside of a [menu](#) control.
To add functionality to the menu item, use owc:command attribute.

Preview

Static menu:



Popup menu:



See Also

[MenuItem Class](#) | [MenuItem Attributes](#) | [Menu Control](#)

MenuItem HTML Attributes

MenuItem-specific Attributes

Attribute	Description
owc:command	Specifies an action for click on the menu item

See Also

[MenuItem Overview](#) | [MenuItem Class](#) | [Menu Overview](#) | [Menu Class](#)

owc:command HTML Attribute

This attribute specifies an action that is performed when a menu item is clicked.

Syntax

```
<html-element owc:command = /string/ >
...
</html-element>
```

Remarks

This is a required attribute. There is no default value. To make the defined command work, an application (or any other code) must implement this command.

Example

```
<div owc:Control="MenuItem" owc:command="sendemail">Send By E-mail</div>
```

Usage

Control	MenuItem
Use	Required
Default value	no

See Also

[MenuItem Control](#) | [MenuItem Attributes](#) | [MenuItem Class](#)

MenuItem Class Overview

The MenuItem control functionality is implemented by Olive.Controls.MenuItem JavaScript class

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.MenuItem

Implemented Interfaces

Olive.Controls.ICmdItem

Olive.IState

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[MenuItem Overview](#) | [MenuItem Methods](#)

MenuItem Class Methods

Inheriting from [ICmdSource](#) Interface

Method	Description
sendCommand	Sets the command source object from the menu as cmdParams for command, and calls the base class method

MenuItem Methods

Method	Description
getParentMenu	Returns Menu object

See Also

[MenuItem Class](#)

getParentMenu Method

This method returns that parent menu object that implements **ICmdSource** interface

Parameters

No parameters

Syntax

```
oMenuItem.getParentMenu();
```

Example

```
var oMenu = oMenuItem.getParentMenu();
```

See Also

[MenuItem Class](#) | [MenuItem Methods](#)

MenuItem Example

The following example illustrates:

- Add a Menu to an HTML page using MenuItem controls
- Create different command types using custom HTML attributes for the MenuItem control


```
padding: 2px 18px 3px 18px;
white-space: nowrap;
color: Black;
cursor: pointer;
}
/* Style for highlighted context menu item */
.ContextMenuControl.Highlighted
{
color: White;
background-color: #191970;
}
.ContextMenuControl.Disabled
{
color: #808080;
}
```

See Also

MenuItem Overview | MenuItem Class

Metadata Control Overview

This control shows metadata.

Syntax

```
<html-element owc:Control = Metadata >
  <owc:data type=/type-of-data/></owc:data>

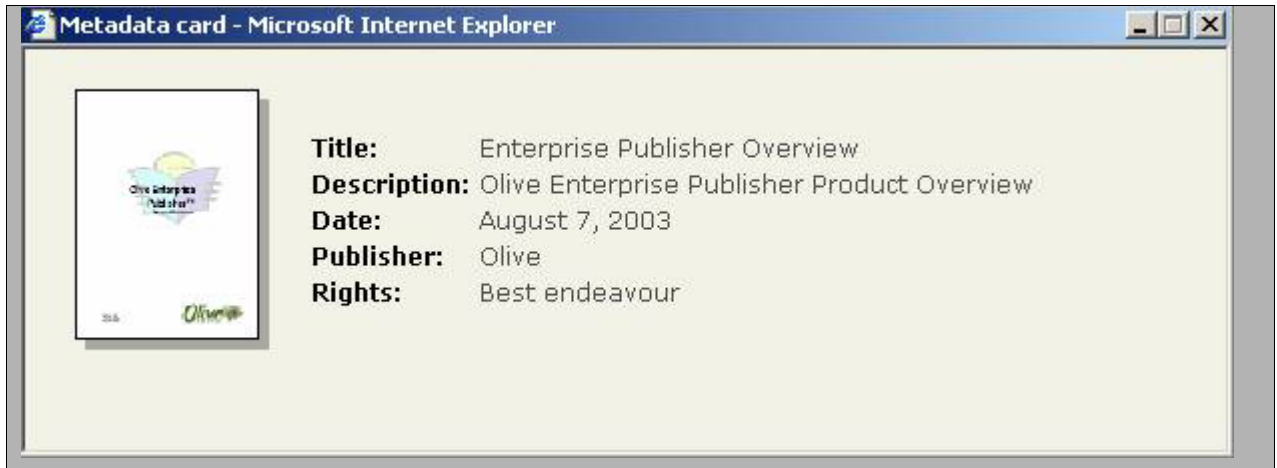
  <!-- Nested controls: MetadataField, Thumbnail, etc -->
</html-element>
```

Remarks

This control is a container for [metafield](#) and others controls, such as Thumbnail, that bring document, entity, or other information.

Preview

For Example:



See Also

Metadata Class

Metadata Class Overview

The Metadata control functionality is implemented by Olive.Controls.Metadata JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Metadata

Implemented Interfaces

Olive.Controls.IDataBound

Requirements

Script Files	OwcMetadata.js
--------------	----------------

See Also

[Metadata Overview](#)

Metadata Example

The examples below describe the using of the Metadata control in the user application.

```
// example for metadata for document
<div id="MetadataDoc" owc:Control="Metadata">
  <owc:Data type="Document" doc-href="UnitTests/1600/01/01">
  </owc:Data>
  <table>
  <tr>
  <td align="center" valign="top" class="Card">
  <div>
  <img owc:Control="Thumbnail" class="Thumbnail" />
  </div>
  </td>
  <td class="Card">
  <table id="tblCardDoc">
  <tr owc:Control="MetaField" owc:Field="dc:title"
owc:DisplayName="Title:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
  <div owc:Control="Value" owc:Separator="<br/>">
  </div>
  </td>
  </tr>
  <tr owc:Control="MetaField" owc:Field="dc:description"
owc:DisplayName="Description:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
  <div owc:Control="Value" owc:Separator="<br/>">
  </div>
  </td>
  </tr>
  <tr owc:Control="MetaField" owc:Field="dc:date"
owc:DisplayName="Date:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
  <div owc:Control="Value" owc:Separator="<br/>">
  </div>
  </td>
  </tr>
  <tr owc:Control="MetaField" owc:Field="dc:rights">
```

```

owc:DisplayName="Rights:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>
</table>
</td>
</tr>
</table>
</div>

```

// example for metadata for entity

```

<div id="MetadataEntity" owc:Control="Metadata">
  <owc:Data type="Entity" doc-href="UnitTests/1600/01/02"
entityId="Ar00100">
    </owc:Data>
    <table>
      <tr>
        <td align="center" valign="top" class="Card">
          <div>
            <img owc:Control="Thumbnail" class="Thumbnail" />
          </div>
        </td>
        <td class="Card">
          <table>
            <tr owc:Control="MetaField" owc:Field="dc:title"
owc:DisplayName="Title:" owc:ShowIf="exists">
              <td owc:Control="Title"></td>
              <td>
                <div owc:Control="Value" owc:Separator="<br/>">
                </div>
              </td>
            </tr>
          </table>
        </td>
      </tr>
    </table>
  </owc:Data>

```

```
</tr>

<tr owc:Control="MetaField" owc:Field="dc:description"
owc:DisplayName="Description:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr owc:Control="MetaField" owc:Field="dc:date"
owc:DisplayName="Date:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr owc:Control="MetaField" owc:Field="dc:rights"
owc:DisplayName="Rights:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr owc:Control="MetaField" owc:Field="dc:publisher"
owc:DisplayName="Publisher:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr id="PAGE_NO_ent" owc:Control="MetaField" owc:Field="PAGE_NO"
owc:DisplayName="Page Number:">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br>">
    </div>
```

```

</td>
</tr>
<tr id="ENTITY_TYPE_ent" owc:Control="MetaField"
owc:Field="ENTITY_TYPE" owc:DisplayName="Entity Type:">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br>">
    </div>
  </td>
</tr>
</table>
</td>
</tr>
</table>
</div>

```

See Also

Metadata Overview | Metadata Class

Value Control Overview

The Value control is used to display the value in a metadata field.

Syntax

```

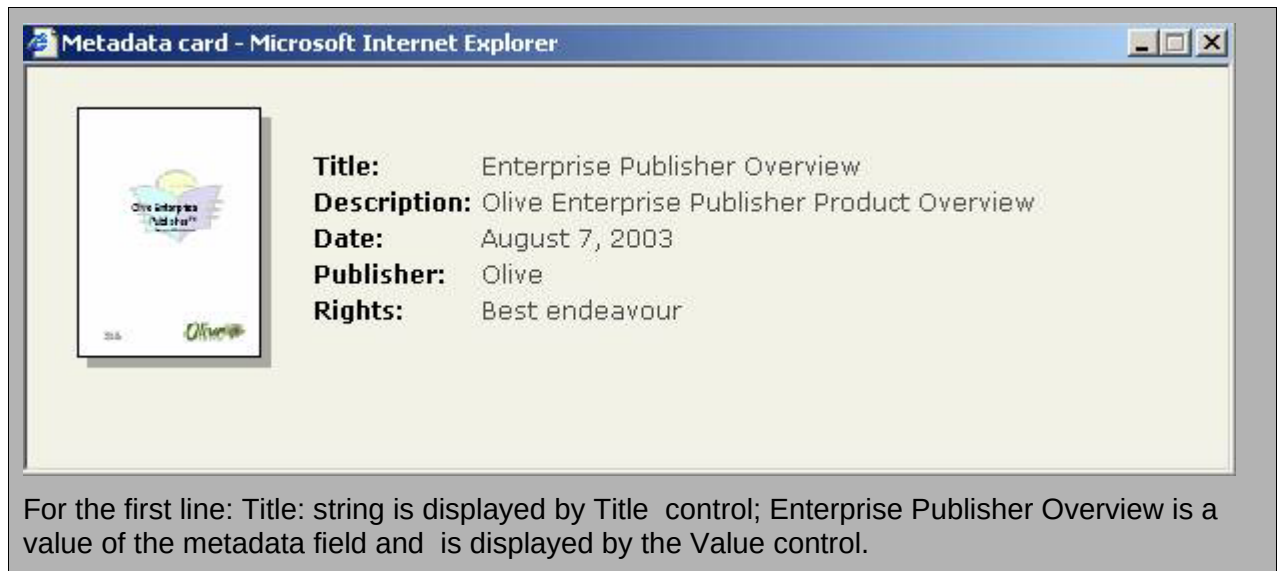
<html-element owc:control=Value
  owc:separator=/string/>
<!-- Nested controls -->
</html-element>

```

Remarks

The [MetaField](#) control is this control's container. The Value control can be used together with Title control, which is used to display the title of a metadata field.

Preview



For the first line: Title: string is displayed by Title control; Enterprise Publisher Overview is a value of the metadata field and is displayed by the Value control.

See Also

[MetadataFieldValue Class](#) | [Value Attributes](#) | [MetaField Control](#) | [Title Control](#)

Value HTML Attributes

Value-specific Attributes

Attribute	Description
owc:separator	Specifies a separator between metadata fields values when there are more than one value

See Also

[Value Overview](#) | [MetadataFieldValue Class](#)

owc:separator HTML Attribute

This attribute specifies the separator character between multiple values

Syntax

```
<html-element owc:separator = /string/ >  
...  
</html-element>
```

Remarks

Value of the attribute can be a desirable symbol or string with an HTML tag `<br />`.

Example

```
<div owc:control="Value" owc:separator="&lt;br/>"> ... </div>
```

Usage

Control	Value
Use	Optional
Default value	no

See Also

[Value Control](#) | [Value Attributes](#) | [MetadataFieldValue Class](#)

MetadataFieldValue Class Overview

The Value control functionality is implemented by the Olive.Controls.MetadataFieldValue JavaScript class

Inheritance Hierarchy

- Olive.EventsSource
- Olive.Object
- Olive.CmdTarget
- Olive.Controls.Control
- Olive.Controls.Value

Olive.Controls.MetadataFieldValue

Requirements

Script Files	OwcMetadata.js
--------------	----------------

See Also

Value Overview | MetadataFieldValue Methods

MetadataFieldValue Class Methods

Inheriting Methods from Olive.Controls.IValue interface

Event	Description
setValue	Sets value for the metadata field. Uses separator for multiple values if it is defined.

See Also

[MetadataFieldValue Class](#) | [MetadataFieldValue Methods](#)

setValue Method

This method sets a value into the metadata field value control

Parameters

oValue - An object of an SDK control that will react on clicks on MetadataFieldValue items

bDoNotUpdateControl - A Boolean value that indicates when to update the MetadataFieldValue state

Syntax

```
oMetadataFieldValue.setValue(oValue, bDoNotUpdateControl);
```

Example

```
// set oFieldValue into the metadata field value control and update the control  
oMetadataFieldValue.setValue(oFieldValue, false);
```

See Also

[MetadataFieldValue Class](#) | [MetadataFieldValue Methods](#)

Value Example

The examples below describe the using of the Value control in the user application.

```

// example for metadata for document
<div id="MetadataDoc" owc:Control="Metadata">
  <owc:Data type="Document" doc-href="UnitTests/1600/01/01">
  </owc:Data>
  <table>
  <tr>
  <td align="center" valign="top" class="Card">
  <div>
  <img owc:Control="Thumbnail" class="Thumbnail" />
  </div>
  </td>
  <td class="Card">
  <table id="tblCardDoc">
  <tr owc:Control="MetaField" owc:Field="dc:title"
owc:DisplayName="Title:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
  <div owc:Control="Value" owc:Separator="<br/>">
  </div>
  </td>
  </tr>
  <tr owc:Control="MetaField" owc:Field="dc:description"
owc:DisplayName="Description:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
  <div owc:Control="Value" owc:Separator="<br/>">
  </div>
  </td>
  </tr>
  <tr owc:Control="MetaField" owc:Field="dc:date"
owc:DisplayName="Date:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
  <div owc:Control="Value" owc:Separator="<br/>">
  </div>
  </td>
  </tr>
  <tr owc:Control="MetaField" owc:Field="dc:rights"

```

```

owc:DisplayName="Rights:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>
</table>
</td>
</tr>
</table>
</div>

```

See Also

[Value Overview](#) | [MetadataFieldValue Class](#)

MetaField Control Overview

This control shows metafield data.

Syntax

```

<html-element owc:Control = Metadata >

  <!-- Nested meta field controls -->
  <html-element owc:Control = MetaField
    owc:Field = /name of meta field/
    owc:DisplayName = /display name/>
    <html-element owc:Control = Title />
    <html-element owc:Control = Value />
  </html-element>
  <!-- other MetaField controls -->
</html-element>

```

Remarks

MetaField control is an element of its container Metadata control. On the other side it is a container of two other controls Title and Value.

Preview



See Also

MetaField Class | MetaField Attributes

MetaField HTML Attributes

MetaField-specific Attributes

Attribute	Description
owc:Field	Specifies the metadata field name
owc:DisplayName	Specifies the metadata field display name
owc:ShowIf	Specifies a criterion to show the metadata field

See Also

MetaField Overview | MetaField Class

owc:DisplayName HTML Attribute

This attribute specifies the display name of the metadata field

Syntax

```
<html-element owc:DisplayName = /string/ >  
...  
</html-element>
```

Remarks

This attribute is the desired name for the displayed metadata field

Example

```
<tr owc:Control="MetaField" owc:DisplayName="Title: ">
```

Usage

Control	MetaField
Use	Required
Default value	none

See Also

[MetaField Control](#) | [MetaField Attributes](#) | [MetaField Class](#)

owc:Field HTML Attribute

This attribute specifies the metadata field name

Syntax

```
<html-element owc:Field = /string/ >  
...  
</html-element>
```

Remarks

This attribute is a metadata field name

Example

```
<tr owc:Control="MetaField" owc:Field="dc:title">
```

Usage

Control	MetaField
Use	Required
Default value	none

See Also

[MetaField Control](#) | [MetaField Attributes](#) | [MetaField Class](#)

owc:ShowIf HTML Attribute

This attribute specifies a criterion to show the meta field

Syntax

```
<html-element owc:ShowIf = /string/ >
...
</html-element>
```

Remarks

If the attribute has value exists then metafield will not be showed if it doesnt exist

Example

```
<tr owc:Control="MetaField" owc:ShowIf="exists">
```

Usage

Control	MetaField
Use	Optional
Default value	none

See Also

[MetaField Control](#) | [MetaField Attributes](#) | MetaField Class

MetaField Class Overview

The MetaField control functionality is implemented by the Olive.Controls.MetaField JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.MetaField

Requirements

Script Files	OwcMetadata.js
--------------	----------------

See Also

MetaField Overview

MetaField Example

The example below describes shows the MetaField control in the user application.

```
// example for metadata for document
<div id="MetadataDoc" owc:Control="Metadata">
  <owc:Data type="Document" doc-href="UnitTests/1600/01/01">
  </owc:Data>
  <table>
    <tr>
      <td align="center" valign="top" class="Card">
        <div>
          <img owc:Control="Thumbnail" class="Thumbnail" />
        </div>
      </td>
      <td class="Card">
        <table id="tblCardDoc">
          <tr owc:Control="MetaField" owc:Field="dc:title"
owc:DisplayName="Title:" owc:ShowIf="exists">
            <td owc:Control="Title"></td>
            <td>
              <div owc:Control="Value" owc:Separator="<br/>">
            </div>
            </td>
          </tr>
          <tr owc:Control="MetaField" owc:Field="dc:description"
owc:DisplayName="Description:" owc:ShowIf="exists">
            <td owc:Control="Title"></td>
            <td>
              <div owc:Control="Value" owc:Separator="<br/>">
            </div>
            </td>
          </tr>
          <tr owc:Control="MetaField" owc:Field="dc:date"
owc:DisplayName="Date:" owc:ShowIf="exists">
            <td owc:Control="Title"></td>
            <td>
              <div owc:Control="Value" owc:Separator="<br/>">
            </div>
            </td>
          </tr>
        </table>
      </td>
    </tr>
  </table>
</div>
```

```

</td>
</tr>
<tr owc:Control="MetaField" owc:Field="dc:rights"
owc:DisplayName="Rights:" owc:ShowIf="exists">
<td owc:Control="Title"></td>
<td>
<div owc:Control="Value" owc:Separator="<br/>">
</div>
</td>
</tr>
</table>
</td>
</tr>
</table>
</div>

```

```

// example for metadata for entity

```

```

<div id="MetadataEntity" owc:Control="Metadata">
<owc:Data type="Entity" doc-href="UnitTests/1600/01/02"
entityId="Ar00100">
</owc:Data>
<table>
<tr>
<td align="center" valign="top" class="Card">
<div>
<img owc:Control="Thumbnail" class="Thumbnail" />
</div>
</td>
<td class="Card">
<table>
<tr owc:Control="MetaField" owc:Field="dc:title"
owc:DisplayName="Title:" owc:ShowIf="exists">
<td owc:Control="Title"></td>
<td>
<div owc:Control="Value" owc:Separator="<br/>">
</div>
</td>

```

```
</tr>

<tr owc:Control="MetaField" owc:Field="dc:description"
owc:DisplayName="Description:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr owc:Control="MetaField" owc:Field="dc:date"
owc:DisplayName="Date:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr owc:Control="MetaField" owc:Field="dc:rights"
owc:DisplayName="Rights:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr owc:Control="MetaField" owc:Field="dc:publisher"
owc:DisplayName="Publisher:" owc:ShowIf="exists">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br/>">
    </div>
  </td>
</tr>

<tr id="PAGE_NO_ent" owc:Control="MetaField" owc:Field="PAGE_NO"
owc:DisplayName="Page Number:">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br>">
    </div>
```

```

</td>
</tr>
<tr id="ENTITY_TYPE_ent" owc:Control="MetaField"
owc:Field="ENTITY_TYPE" owc:DisplayName="Entity Type:">
  <td owc:Control="Title"></td>
  <td>
    <div owc:Control="Value" owc:Separator="<br>">
    </div>
  </td>
</tr>
</table>
</td>
</tr>
</table>
</div>

```

See Also

MetaField Overview | MetaField Class

Month Overview

The Month control reads and writes the month portion of a date/time value in an HTML element. It is a subcontrol of a Calendar control, where the month part of the date value is shown and changed through a Month control, and the day and year are shown and changed by Day or DayTable and Year controls.

The basic functionality of parsing and validating month strings comes from the data type [Olive.Controls.DataType.Month](#). The Month control also has an internal "mask" property that enables it exchange data with a parent [Calendar](#) control. When the Month control's value changes, the "valueChanged" event updates the month part of the Calendar date/time value, and the Calendar control then updates the other date-related subcontrols that are affected by a change in month (DayTable and Date).

Possible values of the Format HTML attribute are:

%m - two-digit month ("02")

%#m - two-digit year without zero padding ("2")

%B - month name in the current locale ("Februario")

%b - abbreviated month name in the current locale ("Feb")

For an example of a Month control in a Calendar control, see [Calendar Examples](#).

Syntax

```
<html-element owc:Control = "Month" owc:Format="/format string/">
</html-element>
```

Example

```
<input id="iMonth" type="text" size="2" owc:Control="Month"
owc:Format="%#d" value="2" maxlength="2" />
```

See Also

[Calendar Overview](#) | [Value Overview](#) | [Olive.Controls.DataType Overview](#)

Page Overview

The internal Page control loads and unloads document page(s) and fires corresponded events, and includes a set of methods for opening, closing, and keeping track of pop-up windows.

This control and the browser page each reference to each other. The page control initializes after the Application object is created, and it processes the SDK controls on the page. It is the parent control of all first-level controls on the page (the control under which they are nested). If no Application control is above it, the page is the parent element in the SDK control hierarchy.

See Also

[Page Class](#) | [Application Overview](#)

OwcPage Class Overview

The Page class functionality is implemented by the Olive.Page JavaScript class.

The Olive.Page interface provides access to Page control events and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Page

Implemented Interfaces

Olive.IControlContainer

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[Page Overview](#) | [Page Events](#) | [Page Methods](#)

Page Events

Page Events

Event	Description
pageLoaded	Event fires after the page is loaded and its controls are bound and initialized
pageUnloaded	Event fires when the page starts the unloading process, after closing its popup windows

See Also

[Page Overview](#) | [Page Methods](#) | [Page Properties](#)

pageLoaded Event

This event fires after the page is loaded and its controls are bound and initialized

Arguments

None

Remarks

The Application can handle this event to set up the initial state of windows and their controls.

See Also

[Page Overview](#) | [Page Events](#) | [Page Methods](#)

pageUnloaded Event

This event fires when the page starts the unloading process, after closing its popup windows

Arguments

None

Remarks

It is preferable to use the oCallbackOnClose argument to the openPopup() method when implemented.

See Also

[Page Overview](#) | [Page Events](#) | [Page Methods](#)

Page Class Methods

Page Methods

Method	Description
<u>getBrowserWindow()</u>	Returns a reference to the page's associated browser window
<u>getPagePopupArguments()</u>	Returns the popup arguments with which this page was opened
<u>getWindowOptions(sName)</u>	Returns the window options that the application defines to open a window named sName or the default name
<u>openPopup(sUrl, oWindowOptionsNameOrRef, oArguments, oCallbackOnClose)</u>	A high-level function that opens a new popup window using the named window options set
<u>preparePopupWindow(oArguments, oCallbackOnClose)</u>	Creates a popup window marshaller object, which may later be configured and passed to the openPopupWindow()
<u>openPopupWindow(oPopupInfo, sUrl, oWindowOptions, sTarget)</u>	Opens a new popup window with the specified properties and features
<u>getPopupCount()</u>	Returns the number of popup windows opened by this page
<u>findPopupInfo(oWindow)</u>	Finds the SDK popup object associated with a browser window oWindow that was opened by this page
<u>closeAllPopupWindows(bDoNotWait)</u>	Closes all popup windows opened by this page and removes them from this page's collection of child windows. It also stops tracking the state of popup windows (if it is working)
<u>detachPage()</u>	Called when the opener window decides to close the popup window (this page) without waiting

Static functions that are used without a page object

Method	Description
<u>Olive.Page.GetPageForWindow(oWindow)</u>	Returns the page object for a window

Olive.Page.GetPageForWindow(oDocument)	Returns the page object for a document
---	--

See Also

[Page Overview](#) | [Page Events](#) | [Page Class](#)

Page Class Properties

Page properties

Property	Description
isFrameset	boolean - defines if the main element a <body> or <frameset>
isPopup	boolean - defines if this a popup window
popupInfo	Olive.PopupInfo object
Parent	The Parent control (WebApplication if it exists)
WebApplication	The Olive Web application object

See Also

[Page Overview](#) | [Page Events](#) | [Page Methods](#)

PageItem Control Overview

The PageItem control is a control representing an iframe in a page switcher. For more information, see on the [PageSwitcher](#) control.

Syntax JavaScript

```
<div owc:Control = PageSwitcher>JavaScript
  <iframe owc:Control="PageItem" id=iframe_1 class=Page Invisible src=file_1
></iframe>JavaScript
  <iframe owc:Control="PageItem" id=iframe_2 class=Page Invisible src=file_2
></iframe>JavaScript
  ..JavaScript
</div>
JavaScript
```

Remarks

The visual interface of each [TabItem](#) control is based on two CSS classes, *Visible* and *Invisible*. The PageItems will usually all be the same place and size when they are active, and the inactive pages will be invisible.

See Also

[PageItem Class](#) | [PageItem Css Classes](#) | [PageSwitcher Overview](#)

PageItem Class Overview

The PageItem component functionality is implemented by the Olive.Controls.PageSwitcher.Item JavaScript class. The Olive.Controls.PageSwitcher.Item interface allows access to the TabItem component properties and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.PageSwitcher.Item

Implemented Interfaces

Olive.IUiElements

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[PageItem Control Overview](#)

PageItem Class Events

PageItem Events inherited from IState

Event	Description
stateChanged	This event is fired when the state of a page item changes between active and inactive. It is usually more useful to handle the itemActivated event of PageSwitcher in order to perform an action when the user selects a tab.

See Also

[PageItem Class](#) | PageItem Methods | [PageItem Overview](#)

PageItem Control CSS Classes

The SDK does not have default CSS definitions for these classes. Any page that uses a PageSwitcher must define them.

CSS Class	Description
Visible	CSS class that defines the active page. A typical value is "width:100%; height:100%; z-index: 1"
Invisible	CSS class defines the inactive pages. A typical value is "width:1%; height:1%; z-index: 100".

See Also

[PageItem Overview](#) | [PageSwitcher Overview](#)

PageSwitcher Control Overview

The PageSwitcher control is a container for <iframe> HTML elements, each containing a [PageItem](#) Control. Its purpose is to get and set the active page (switch between pages).

Syntax

```
<div owc:Control = PageSwitcher>
  <iframe owc:Control="PageItem" id=iframe_1 class=Page Invisible src=file_1
  ></iframe>
  <iframe owc:Control="PageItem" id=iframe_2 class=Page Invisible src=file_2
  ></iframe>
  ..
</div>
```

Remarks

The control's visual interface is based on the pre-defined CSS classes *Visible* and *Invisible* for its HTML child <iframe> elements ([PageItem](#) sub-controls).

See Also

[PageSwitcher Class](#)

PageSwitcher Class Overview

The PageSwitcher component functionality is implemented by the Olive.Controls.PageSwitcher JavaScript class. The Olive.Controls.PageSwitcher interface provides access to PageSwitcher methods.

Class Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.PageSwitcher

Implemented Interfaces

Olive.IActiveItem

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[PageSwitcher Events](#) | [PageSwitcher Methods](#) | [PageSwitcher Overview](#)

PageSwitcher Class Events

PageSwitcher Events inherited from IActiveItem

Event	Description
itemActivating	Fired by setActiveItem() if the new sub-item is different from the old value and argument bDoNotNotify is not true. The event is fired before the active/inactive state of the old and new active items has been changed.
itemActivated	Fired by setActiveItem() if the new sub-item is different from the old value and argument bDoNotNotify is not true. The event is fired after the active/inactive state of the old and new active items has been changed.

See Also

[PageSwitcher Class](#) | [PageSwitcher Methods](#) | [PageSwitcher Overview](#)

PageSwitcher Class Methods

PageSwitcher Methods inherited from IActiveItem

Method	Description
getItemsCount()	Returns the number of sub-items.
getItem(itemIndex)	Returns the sub-item specified by index number itemIndex (starting from 0).
indexOfItem(oltem)	Returns the index number of sub-item oltem (starting from 0).
getActiveItemIndex()	Returns the index number of the active sub-item (starting from 0).
getActiveItem()	Returns the active sub-item.

setActiveItem(vItem, bDoNotNotify)	Sets sub-item vItem (index-number or object) to be the active sub-item. Fires events itemActivating and itemActivated unless bDoNotNotify is true.
------------------------------------	--

See Also

[PageSwitcher Class](#) | [PageSwitcher Events](#) | [PageSwitcher Overview](#)

PageSwitcher Example

The example below describes a PageSwitcher control used in an application.

```
HTML:
<div id="divPageSwitcher"  owc:Control="PageSwitcher" >
  <iframe  owc:Control="pageItem"  id="Fr_First"  src="First.htm"></iframe>
  <iframe  owc:Control="pageItem"  id="Fr_Second"
src="Second.htm"></iframe>
  <iframe  owc:Control="pageItem"  id="Fr_Third"  src="Third.htm"></iframe>
</div>
CSS Classes:
.Visible
{
  display:  block;
}
.Invisible
{
  display:  none;
}
```

See Also

[PageSwitcher Overview](#) | [PageSwitcher Class](#)

Pane Control Overview

The pane control is used for forming a page with a complicated layout.

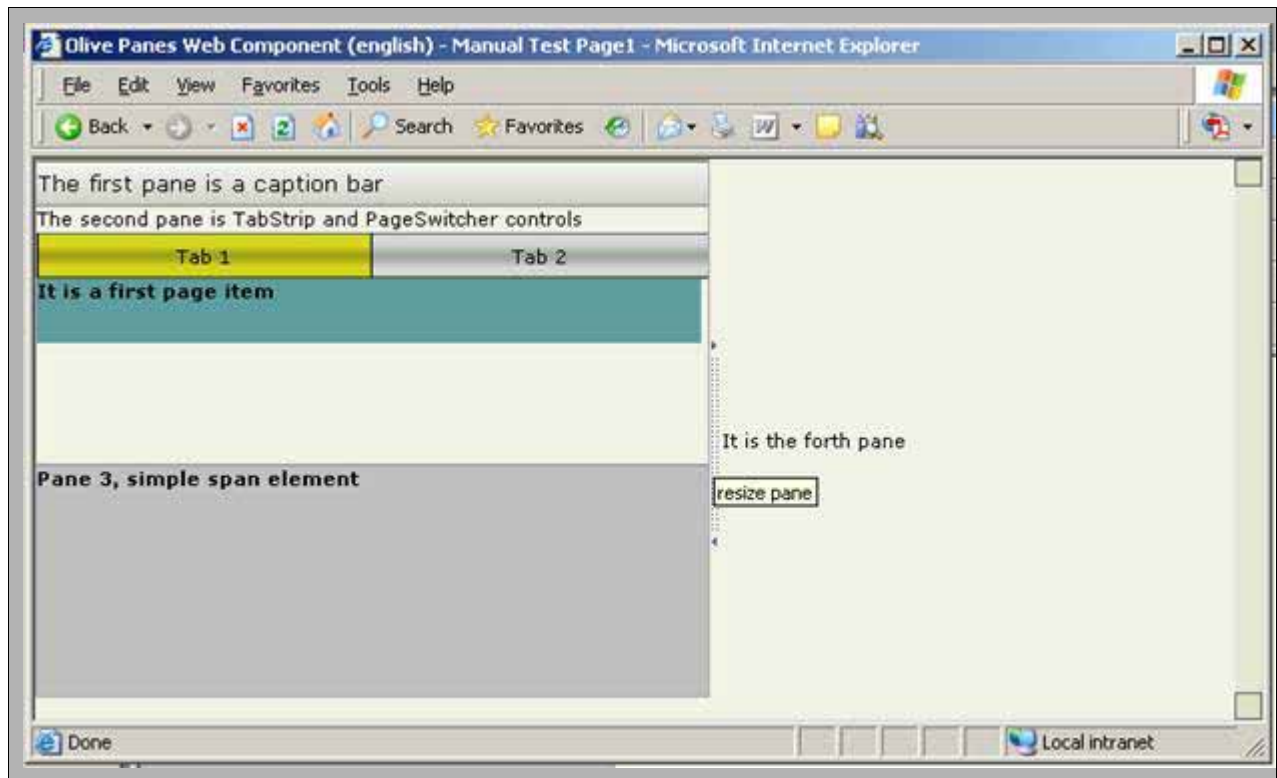
Syntax

```
<html-element  owc:control=pane
  owc:resizable=/boolean value/>
</html-element>
```

Remarks

Pane control is usually put inside of its container: pane-list control.

Preview



See Also

[Pane Attributes](#) | [Pane Class](#)

Pane HTML Attributes

Attributes inherited from [Olive.Controls.IPane](#) interface

Property	Description
owc:resizable	Specifies when the pane is re-sizable

See Also

[Pane Overview](#) | [Pane Class](#)

Pane Class Overview

The pane control functionality is implemented by Olive.Controls.Pane JavaScript class

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Pane

Implemented Interfaces

Olive.Controls.IPane

Requirements

Script Files	OwcPanes.js
--------------	-------------

See Also

[Pane Overview](#) | [Pane Methods](#)

Pane Class Methods

Methods inherited from Olive.Controls.IPane interface

Event	Description
setPanelItemSize	Sets width and height for a pane

See Also

[Pane Class](#)

pane Example

The following example illustrates using of pane control together with pane-list and pane-splitter controls.

```
<table owc:control="pane-list" owc:kind="columns" class="PaneRoot">
  <tr>
    <td owc:control="pane-list" owc:pane-size="40%">
      <!-- 1st pane, resizable -->
      <div owc:control="pane" owc:resizable="true" class="PaneTocContent">
        <table class="CaptionBar">
          <tr>
            <td>The first pane is a caption bar</td>
          </tr>
        </table>
      </div>
      <!-- 2nd pane, not resizable, default -->
      <div owc:control="pane" class="PaneTocContent">
        <span>The second pane is TabStrip and PageSwitcher controls</span>
      <!-- tab strip -->
```

```

<table owc:Control="TabStrip" id="ctrlTabs" class="TabContainer"
cellpadding="0" cellspacing="0">
  <tr>
    <td owc:control="TabItem" class="TabItem">Tab 1</td>
    <td owc:control="TabItem" class="TabItem">Tab 2</td>
  </tr>
</table>
<!-- page switcher -->
<div owc:Control="PageSwitcher" id="ctrlPageSwitcher"
class="PaneContent">
  <!-- first page -->
  <div owc:Control="PageItem" class="PageOuter Invisible">
    <div class="PageItem1">
      It is a first page item
    </div>
  </div>
  <!-- second page -->
  <div owc:Control="PageItem" class="PageOuter Invisible">
    <div class="PageItem2">
      It is a second page item
    </div>
  </div>
  <!-- 3d pane, not resizable -->
  <div owc:control="pane" owc:resizable="false" class="PaneTocContent
LastTocPane">
    <div>
      <span>Pane 3, simple span element</span>
    </div>
  </div>
</td>
  <td owc:control="pane-splitter" class="PaneColsSplitter" title="pane
splitter"></td>
  <td owc:control="pane">
    <div>
      <span>It is the forth pane</span>
    </div>

```

```
</td>
</tr>
</table>
```

See Also

[Pane Overview](#) | [Pane Class](#)

pane-list Control Overview

The pane-list control is used to form pages with complicated layouts.

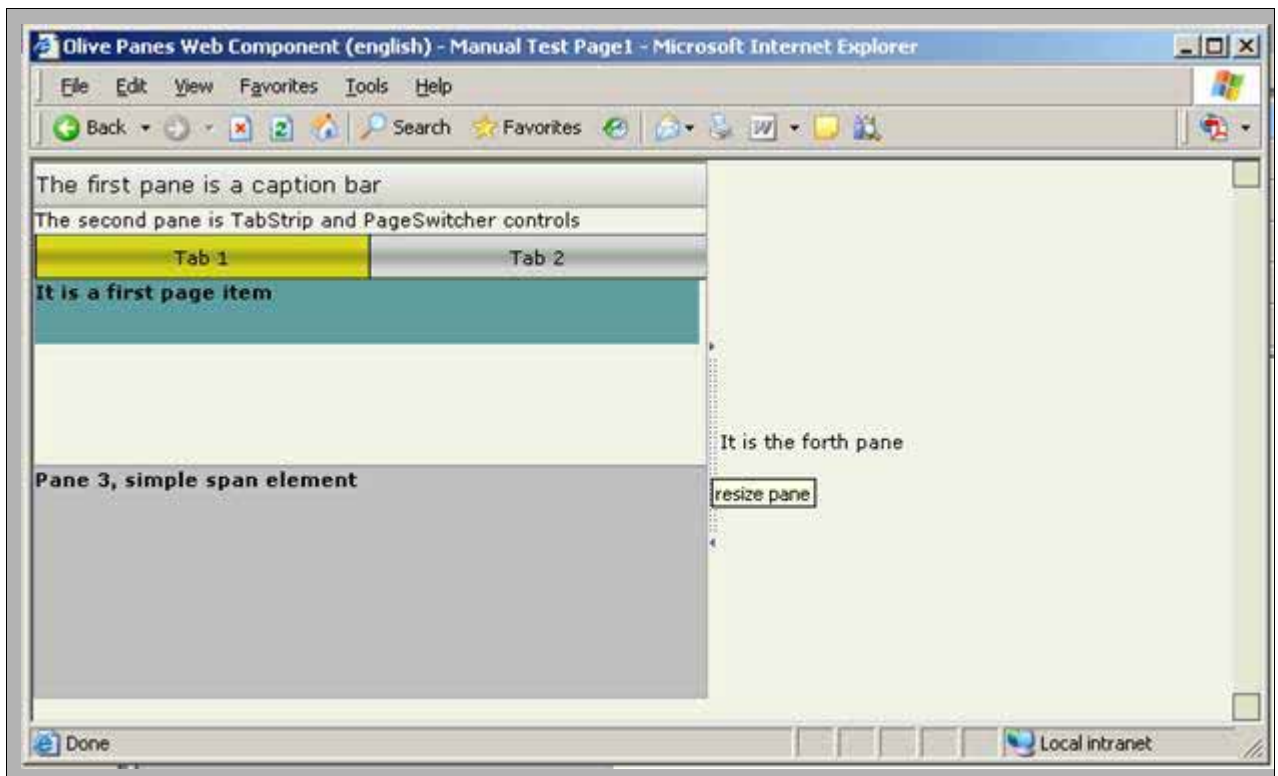
Syntax

```
<html-element owc:control=pane-list
  owc:kind=/string - kind of the pane-list;  rows or columns/>
<!-- Nested controls: pane, pane-splitter or others -->
</html-element>
```

Remarks

pane-list control is a container for [pane](#) and other controls.

Preview



See Also

[pane-list Attributes](#) | [PaneList Class](#)

pane-list HTML Attributes

Attributes inherited from [Olive.Controls.IPaneListItem](#) interface

Property	Description
owc:pane-size	Specifies a size of the pane list item

Attributes inherited from [Olive.Controls.IPaneList](#) interface

Attribute	Description
owc:kind	Specifies the kind of pane list, grouped in rows or columns

See Also

[pane-list Overview](#) | [PaneList Class](#)

PaneList Class Overview

The pane-list control functionality is implemented by Olive.Controls.PaneList JavaScript class

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.PaneList

Implemented Interfaces

Olive.Controls.IPaneList

Requirements

Script Files	OwcPanels.js
--------------	--------------

See Also

[PaneList Properties](#) | [PaneList Methods](#) | [pane-list Overview](#)

PaneList Class Methods

Methods inherited from [Olive.Controls.IPaneList](#) interface

Event	Description
isHorizontal	Defines when the pane list is horizontal

See Also

[PaneList Class](#) | [PaneList Properties](#)

PaneList Class Properties

Properties inherited from [Olive.Controls.IPaneList](#) interface

Property	Description
kind	Defines the kind of PaneList rows or columns

See Also

[PaneList Class](#) | [PaneList Methods](#)

pane-list Example

The following example illustrates use of the **pane-list** control together with **pane** and **pane-splitter** controls.

```
<table owc:control="pane-list" owc:kind="columns" class="PaneRoot"
border="0" cellpadding="0" cellspacing="0">

<tr style="height: 100%; width: 100%">

<td owc:control="pane-list" owc:pane-size="40%">

<!-- 1st pane -->

<div owc:control="pane" owc:resizable="false" class="PaneTocContent">
<table class="CaptionBar" cellpadding="0" cellspacing="0" >
<tr>
<td>The first pane is a caption bar</td>
</tr>
</table>
</div>

<!-- 2nd pane -->

<div owc:control="pane" class="PaneTocContent">
<span>The second pane is TabStrip and PageSwitcher controls</span>
<!-- tab strip -->

<table owc:Control="TabStrip" id="ctrlTabs" class="TabContainer"
cellpadding="0" cellspacing="0">

<tr>
```

```

<td owc:control="TabItem" class="TabItem">Tab 1</td>
<td owc:control="TabItem" class="TabItem">Tab 2</td>
</tr>
</table>

<!-- page switcher -->
<div owc:Control="PageSwitcher" id="ctrlPageSwitcher"
class="PaneContent">
<!-- first page -->
<div owc:Control="PageItem" class="PageOuter Invisible">
<div class="PageItem1">
It is a first page item
</div>
</div>
<!-- second page -->
<div owc:Control="PageItem" class="PageOuter Invisible">
<div class="PageItem2">
It is a second page item
</div>
</div>
</div>
</div>
<!-- 3d pane -->
<div owc:control="pane" owc:resizable="true" class="PaneTocContent
LastTocPane">
<div>
<span>Pane 3, simple span element</span>
</div>
</div>
</td>

<td owc:control="pane-splitter" class="PaneColsSplitter" title="pane
splitter"></td>

<td owc:control="pane">
<div>
<span>It is the forth pane</span>
</div>
</td>
</tr>

```

</table>

See Also

[pane-list Overview](#) | [PaneList Class](#)

pane-splitter Control Overview

The pane-splitter control is dedicated to resizing panes.

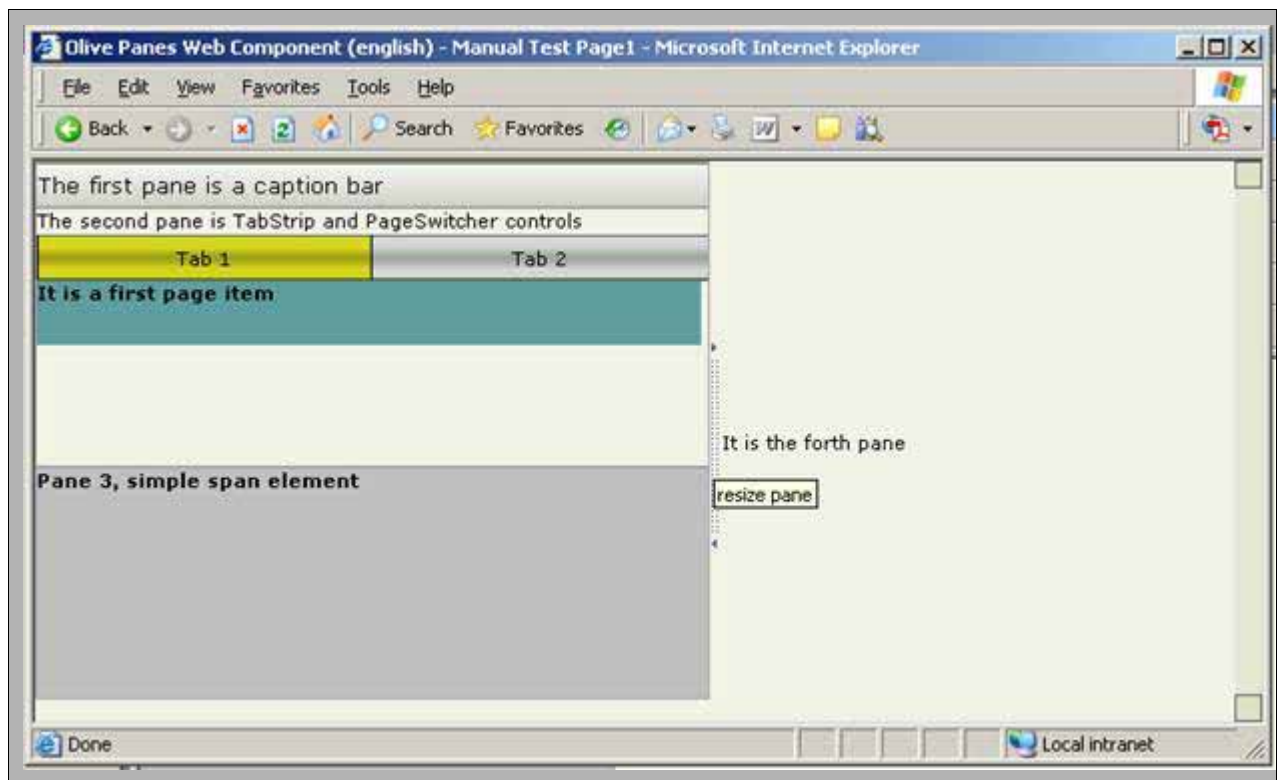
Syntax

```
<html-element owc:control=pane-splitter>  
  <!-- Nested controls: pane, pane-splitter or others -->  
</html-element>
```

Remarks

pane-splitter control is responsible for resizing pane layout and pane-list controls.

Preview



See Also

[pane-splitter Attributes](#) | [PaneSplitter Class](#)

PaneSplitter Class Overview

The pane-splitter control functionality is implemented by Olive.Controls.PaneSplitter JavaScript class

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.PaneSplitter

Implemented Interfaces

Olive.Controls.IPaneSplitter

Requirements

Script Files	OwcPanes.js
--------------	-------------

See Also

[PaneSplitter Properties](#) | [PaneSplitter Methods](#) | [pane-splitter Overview](#)

PaneSplitter Class Methods

Methods inherited from [Olive.Controls.IPaneSplitter](#) interface

Event	Description
onDragStart	ondragstart DHTML event handler for HTML element of pane-splitter control
onDragMove	ondragmove DHTML event handler for HTML element of pane-splitter control
onDragEnd	ondragend DHTML event handler for HTML element of pane-splitter control

See Also

[PaneSplitter Properties](#) | [D:\Web SDK API\MadCap\Web SDK API Reference\Content\WebSDK\PaneSplitter\Class\Methods\Methods\PaneSplitter Methods.htm](#) [PaneSplitter Class](#)

PaneSplitter Class Properties

Properties inherited from [Olive.Controls.IPaneSplitter](#) interface

Property	Description
resizablePane	False indicates that the control is not a resizable pane list item

See Also

[PaneSplitter Class](#) | [PaneSplitter Methods](#)

pane-splitter Example

The following example illustrates pane-splitter control used together with pane-list and pane controls.

```
// Vertical pane-splitter
<table owc:control="pane-list" class="PaneRoot" owc:kind="columns"
class="PaneRoot">
  <tr>
    <td>
      <!-- 1st pane -->
      <div owc:control="pane" owc:resizable="true" class="PaneContent">
        <span>The first pane</span>
      </div>
    </td>
    <!-- splitter -->
    <td owc:control="pane-splitter" class="PaneColsSplitter" title="pane
splitter">
      
    </td>
    <td>
      <!-- 2nd pane -->
      <div owc:control="pane" class="PaneContent">
        <span>The second pane</span>
      </div>
    </td>
  </tr>
</table>

// Horizontal pane-splitter
<table owc:control="pane-list" class="PaneRoot" owc:kind="rows"
class="PaneRoot">
  <tr>
    <td>
      <!-- 1st pane -->
      <div owc:control="pane" owc:resizable="true" class="PaneContentH">
        <span>The first pane</span>
```

```

</div>
</td>
</tr>
<!-- splitter -->
<tr>
<td owc:control="pane-splitter" class="PaneRowsSplitter" title="pane
splitter">

</td>
</tr>
<tr>
<td>
<!-- 2nd pane -->
<div owc:control="pane" class="PaneContentH">
<span>The second pane</span>
</div>
</td>
</tr>
</table>

```

See Also

[pane-splitter Overview](#) | [PaneSplitter Class](#)

PrintList Overview

PrintList is a container for document images that were received from the server for printing.

Syntax

```

<html-element owc:Control = PrintList >
  <owc:data type=printlist></owc:data>
</html-element>

```

Remarks

PrintList is a container for PrintListItem controls that hold images. A request for images is received using a content item in the PrintList control (owc:data). This control is required to wait and accept all images downloaded to the client side before invoking the print system menu. Some browsers need it.

Preview



See Also

PrintList Class

PrintList Class Overview

The PrintList control functionality is implemented by Olive.Controls.PrintList JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.PrintList

Implemented Interfaces

Olive.Controls.IDataBound

Requirements

Script Files	OwcPrintList.js
--------------	-----------------

See Also

[PrintList Overview](#)

PrintList Class Events

PrintList Events

Event	Description
readyToPrint	Event is fired when all images are loaded from the server

See Also

[PrintList Class](#)

readyToPrint Event

The event is activated when all images are loaded from the server.

Arguments

The event object passed to event handler is an instance of the Olive.Event class. This object doesnt have additional properties.

Remarks

This event is fired to inform the application.

Example

```
// add event handler to page object
oPage.onReadyToPrint = onReadyToPrintHandler;
// or add handler to PrintList object
oPrintList.onReadyToPrint = onReadyToPrintHandler;
// in the handler call for system print window
function onReadyToPrintHandler(oEvent)
{
    window.print();
}
```

See Also

[PrintList Events](#) | [PrintList Class](#)

PrintList Control Example

The example below describes the PrintList control.

```
<div owc:Control="PrintList">
  <owc:data type=printlist></owc:data>
</div>
```

See Also

PrintList [Overview](#)

ScoreBar Overview

The ScoreBar is an image that shows how successful a search was, on a scale of 0 to (n), like a thermometer. The ScoreBar is an abstract class that has two concrete classes that inherit from it:

scorebarentity shows the "score" from 0% to 100%, in (n) increments of (100/n), for each entity.

scorebartoc for TOC search on one entity, shows the number of hits, from 0 to min(numHits, n).

The (n+1) image files must be named {prefix}{number}.{extension}.

For example: 0.gif, 1.gif, ..., 4.gif OR score0.png, score1.png, ..., score6.png

Three ScoreBar control attributes determine the image file names:

owc:MaxNumber (=n) - The maximum value of the scorebar. There are (n+1) images, from 0 to n.

owc:UrlPrefix - The file path before the number, including the directory: for example, "/Layout/Img/Score"

owc:ImgSuffix - The file extension, not including the ".": for example, "gif"

The ScoreBar must be defined in the Layout.xml file, since it must be replicated for each item.

The "scorebarentity" control is defined at:

```
<owct:list-template for="SearchRes"><owct:xhtml-template kind="list-item">...<img owc:Control="scorebarentity"...
```

The "scorebartoc" control is defined at:

```
<owct:xhtml-template for="TocSearchResEntry" kind="tree-node" ...>...<img owc:Control="scorebartoc"...
```

The scorebartoc image will appear after the TOC row image by default, although the search results will commonly have only a scorebartoc image with no TOC row image. To disable the use of a the TOC row default image, use the nolcon attribute in the owc:NodeType element of the owc:Data element of the TOC control:

```
<div owc:Control="TOC" ...>
  <owc:Data type="TreeLayout">
    <owc:NodeType name="TocEntryNode" noIcon="true" ... />
    ...
  </owc:Data>
```

Syntax

```
scorebarentity:
<div>
  <img owc:Control="scorebarentity" owc:MaxNumber="4"
owc:UrlPrefix="Layout/ArtScoreBar/" owc:ImgSuffix="gif">
  </img>
</div>
scorebartoc:
<span>
  <img owc:Control="scorebartoc" owc:MaxNumber="4"
owc:UrlPrefix="Layout/TocScoreBar/" owc:ImgSuffix="gif">
  </img>
</span>
```

See Also

[SearchRes Overview](#)

ScoreBar HTML Attributes

ScoreBar Specific Attributes

Attribute	Description
-----------	-------------

owc:MaxNumber	Maximum value of the scorebar. There are (n+1) images, from 0 to n.
owc:UrlPrefix	File path before the number, including directory: for example, "/Layout/Img/Score"
owc:ImgSuffix	File extension, not including the ".": for example, "gif"

See Also

[ScoreBar Overview](#) | [SearchRes Overview](#)

OwcScoreBar Class Overview

The ScoreBar component functionality is implemented by OwcScoreBar JavaScript class. OwcScoreBar interface provides access to the ScoreBar component properties.

The OwcScoreBar class itself is not really used. Instead, the users application can use the following derived from it classes: **OwcScoreBarEntity** and **OwcScoreBarToc**.

The last two classes serve the Controls that are defined in the application's preferences Layout.xml file.

Inheritance Hierarchy

OwcEventsSource

OwcCmdTarget

OwcControlsContainer

OwcControl

OwcScoreBar

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

[ScoreBar Control Overview](#) | [OwcScoreBar Class Overview](#)

OwcScoreBar Class Properties

Inherited Properties

Property	Description
m_sId	Olive Web control unique ID (string)
WebApplication	Reference to the Olive Web application object

WebPage	Reference to the Olive Web page object
Parent	Reference to the parent (container) control
HtmlElement	Reference to associated HTML element
ControlState	Control state (for persistency)
ControlTypes	Array of registered Control Types
Controls	Collection of controls (array)
m_arrCommands	Registered commands array
OwcEvents	Array of supported Events
OwcEventHandlers	Array of supported Events Handlers
m_bUsePagination	Boolean flag: use or not pagination
m_nPageSize	Number of items per page
m_nPageNo	Number of current page
m_arrItemTypes	Collection of registered item types
m_oDefaultItemType	Default item type

OwcScoreBar properties

Property	Description
m_sScoreValueProperty	String "m_nSearchScore" or "m_nTocSearchHits" that denotes kind of the Score Bar
m_nMaxNumber	???
m_sUrlPrefix	???
m_sImgSuffix	???
m_bDisableContentAutoLoad	Defines when to load the Controls content when Control is initialized.

See Also

[ScoreBar Overview](#) | [OwcScoreBar Class Overview](#)

m_bDisableContentAutoLoad Property

This property defines when to load the Controls content when Control is initialized.

Syntax

Data Type	Boolean
Default value	true

Remarks

This property's default value is true: disable autoloading.

Sample Code

```
var disableContentAutoLoad= scoreBarController.m_bDisableContentAutoLoad;
```

See Also

ScoreBar Overview_ | Owcscorebar Class Overview | [Owcscorebar Class Properties](#)

m_sScoreValueProperty Property

This property denotes the kind of the ScoreBar.

Syntax

Data Type	String
Default value	empty string

Remarks

This property's string value is "m_nSearchScore" or "m_nTocSearchHits", which denotes the kind of Score Bar is used for the search results of the TOC.

Sample Code

```
var sScoreValueProperty= scoreBarController.m_sScoreValueProperty;
```

See Also

ScoreBar Overview_ | Owcscorebar Class Overview | [Owcscorebar Class Properties](#)

ScoreBar Example

The examples below describe the ScoreBar controls used in the user application.

The description of the ScoreBar class denotes that user application uses the ScoreBarEntity or ScoreBarToc Controls that are defined in the applications preferences Layout.xml file.

The following example shows using of these Controls.

```

<img owc:Control="scorebarentity" owc:MaxNumber="4"
owc:UrlPrefix="Layout/ArtScoreBar/" owc:ImgSuffix="gif">
</img>
And:
<img owc:Control="scorebartoc" owc:MaxNumber="4"
owc:UrlPrefix="Layout/TocScoreBar/" owc:ImgSuffix="gif">
</img>

```

See Also

ScoreBar Overview_ | Owcscorebar Class Overview

SearchForm Overview

The SearchForm control enables input of search parameters. Each parameter is a sub-control within the SearchForm control. The following types of sub-controls are available:

Fields: The control type "Field" (class Olive.Controls.SearchTerm) is to specify values that match document data fields. The field name is specified by the owc:FieldName attribute. Most field names are determined by search configuration files. Required fields should have owc:Required="true". The following are possible fields:

- **words** in the **text** of the document(s) - owc:FieldName="Text"
- **words** in the document' **metadata fields** of the document(s)
- **publication**
- **issue-ID**
- **date or date range** - for a range, use owc:Operators="range-start" or "range-end"
- **language** - This control can be of LangList list, instead of Field, with an owc:FieldName attribute for language metadata

Search Options: The control Type "SearchOption" is specify how the results should be searched or displayed. The following are the possible search options (see also on the [SearchOptions](#) control, attribute [owc:FieldName](#)):

- **pagesize** - how many results to display per page
- **sort-by** - which field to search by, and in which order (ascending or descending)
- **hits-only** - when showing the table of contents of a search result, whether to show only rows that have hits
- **propagate-hits** -when showing how many hits are in each TOC entry, whether to include hits in sub-entries in the total for the parent entry
- **stemming** - include words from the same stem as the search word, for example "bound" and "binding" from "bind"
- **phonic** - include words that sound like the search word, for example "site" from "sight"
- **synonyms** - include synonyms of the search word, for example "vision" from "sight"

- **fuzzy-level** - include words that differ from the search word in this number of letters (0-5)

Search in Folder

For searching within a specified folder, you can use either a `SearchInSelectedFolder` or `SearchInFolderLookup` sub-controls (both are derived from the `SearchInFolder` class).

- [`SearchInSelectedFolder`](#) - a selected element enable choosing between "" (search in all folders), "selected" (search in a folder), or "recursive" (search in a folder and its subfolders). A custom script must attach a `FolderTree` control to this control with its `setFolderTree()` method.
- [`SearchInFolderLookup`](#) - this control should be passed to a popup window in the `popupInfo` custom arguments. The popup window has a folder tree, and when the user clicks on a folder, the popup calls the `setFolder()` method of this control through the reference in the popup arguments. It should also have subcontrols of ui type "SearchInAllFolders" (boolean / checkbox), "Recursive" (boolean / checkbox), and "FolderTitle" (write-only, for displaying the title of the selected folder).

Example

```
<table id="ctrlSearchForm"  owc:control="SearchForm"
class="LibSearchTable"  cellpadding="0" cellspacing="0">

  <tbody>
    <tr>
      <td>Search  For:</td>
      <td>
        <!--  "Search for" field -->
        <input  type="text"  owc:control="Field"  owc:FieldName="Text"  />
      </td>
      <td>in</td>
      <td>
        <!--  "Search in" field -->
        <select  id="cmbSearchIn"  owc:control="SearchInSelectedFolder"
disabled="disabled">
          <option  value=""  selected="selected">All  folders</option>
          <option  value="selected">Selected folder</option>
          <option  value="recursive">Selected folder and its  sub-
folders</option>
        </select>
        <input  type="hidden"  value="1"  owc:control="SearchOption"
owc:FieldName="hits-only"  />
        <input  type="hidden"  value="1"  owc:control="SearchOption"
owc:FieldName="propagate-hits"/>
      </td>
    </tr>
  </tbody>
</table>
```

```

<td style="text-align: center; width: 70px;">
<!-- "Search" button -->
<span owc:control="CommandButton" owc:command="Search" title="Submit
search">Search</span>
</td>
<td>
<a href="javascript:void(0)" owc:control="Command"
owc:command="ShowAdvancedSearch">Advanced Search</a>
</td>
</tr>
</tbody>
</table>

```

Remarks

The SearchForm control does not perform the search, but only collects the search parameters. A [SearchRes](#) control performs the search by sending the search data to the server. It is the application's responsibility to forward the search data to the appropriate SearchRes control, usually in response to the "search" command.

Preview



The screenshot shows a web form titled "SEARCH PARAMETERS". It contains the following fields and controls:

- Search string:** A text input field.
- DC_Title:** A text input field.
- Select Date:** A section containing two sub-fields:
 - From:** A text input field.
 - To:** A text input field.
- Sort by:** A dropdown menu currently showing "Date Ascending".
- Results per page:** A dropdown menu currently showing "6".
- Search:** A button at the bottom of the form.

See Also

[SearchTerm Overview](#) | [SearchOptions Overview](#) | [SearchInFolderLookup Overview](#) | [SearchInSelectedFolder Overview](#)

SearchForm Class Overview

The SearchForm component functionality is implemented by the Olive.Controls.SearchForm JavaScript class.

The Olive.Controls.SearchForm interface provides access to the SearchForm component events, commands, and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.SearchForm

Implemented Interfaces

[Olive.IForm](#)

Requirements

Script Files	OwcSearchForm.js
--------------	------------------

See Also

[SearchForm Overview](#) | [SearchForm Methods](#) | [SearchForm Events](#) | [SearchForm Commands](#)

SearchForm Commands

SearchForm Commands

Command	Description
Search	Calls the submitSearch() method (read the search data from the control, and fire a "searchRequest" event containing the search data).

See Also

[SearchForm Overview](#) | [SearchForm Methods](#) | [SearchForm Events](#)

Search Command

This command calls the submitSearch() method (read the search data from the control, and fire a "searchRequest" event containing the search data).

Syntax

object. **launchCommand(Search)**

Where object is an expression referencing an instance of the OwcSearchForm class

Parameters

This command has no parameters

Sample Code

The following example illustrates using the Command control to launch the Search command on a specific SearchForm control.

```
<div id="divSearchForm" owc:control="SearchForm">  
  Search For: <input type="text" owc:control="Field"
```

```

owc:FieldName="Text" />
    <input type="hidden" value="1" owc:control="SearchOption"
owc:FieldName="hits-only" />
    <span owc:control="CommandButton" owc:command="Search" title="Submit
search">Search</span>
</div>

```

The following example illustrates launching the Search command on a specific SearchForm control

```

var oMySearchForm = getObj("divSearchForm");
oMySearchForm.launchCommand("Search");

```

The following example illustrates using the searchFormSubmit() method as an alternative to the Search command.

```

var oMySearchForm = getObj("divSearchForm");
oMySearchForm.searchFormSubmit();

```

See Also

[SearchForm Overview](#) | [SearchFom Class](#) | [SearchForm Methods](#) | [SearchForm Commands](#)

SearchForm Class Events

SearchForm Events

Method	Description
searchRequest	Requests to perform a search: It is automatically fired by the SearchForm control by the submitSearch() method (and by the Search command).

See Also

[SearchForm Overview](#) | [SearchForm Class](#) | [SearchForm Methods](#) | [SearchForm Commands](#)

searchRequest Event

This event requests to perform a search. It is automatically fired by the SearchForm control by the submitSearch() method (and by the Search command).

Properties

SearchParams - An object containing the search parameters. In order to perform the search, the application should forward as the argument of the loadSearchResults() method of the SearchRes control.

Example

```
function myApplication_onSearchRequest(oEvent) {  
    var oSearchRes = OwcGetControl( "divSearchRes");  
    oSearchRes.loadSearchResults(oEvent.SearchParams);  
}
```

See Also

[SearchForm Overview](#) | [SearchForm Class](#) | [SearchForm Methods](#) | [SearchForm Commands](#)

SearchForm Class Methods

SearchForm Methods

Method	Description
submitSearch()	Method updates the control data from the HTML element values. It also fires a searchRequest event with this data, so that the application can forward it to a SearchRes object. Arguments: none

See Also

[SearchForm Overview](#) | [SearchForm Events](#)

searchFormSubmit Method

This method updates the control data from the HTML element values.

It then fires a searchRequest event with this data, so that the application can forward it to a SearchRes object.

Arguments

none

Syntax

```
oForm.submitSearch()
```

See Also

[SearchForm Overview](#) | [SearchForm Methods](#) | [SearchForm Events](#)

SearchForm Example

The example below includes the following features:

- Search in a field with owc:FieldName="Text"
- Search in a field with owc:FieldName="DocType"

- SearchInFolderLookup with sub-elements SearchInAllFolders, FolderSelection, FolderTitle, and Recursive, and a command to open a popup window and select a folder
- Search in metadata fields dc:title, dc:subject, dc:description
- Search in field dc:date with owc:Operators="range-from" and "range-to", DataType="Date", and command controls to open a popup window and select a date in a calendar control
- Search options sortby and hits-only
- A hidden input element for search option propagate-hits
- A LangList control with owc:FieldName="dc:language"
- Command controls to search or cancel the search

```
<table id="ctrlAdvSearchForm" owc:control="SearchForm"
class="PaneOutlineAdvS" cellspacing="0" cellpadding="0" border="0">
  <colgroup>
    <col width="30%"></col>
    <col width="60%"></col>
    <col width="5%"></col>
  </colgroup>
  <thead>
    <tr class="PaneHeaderRow">
      <td colspan="3">Advanced Search<hr/></td>
    </tr>
  </thead>
  <tbody>
    <tr class="InputRow">
      <td class="FieldName">Search For:</td>
      <td colspan="2"><input type="text" owc:control="Field"
owc:FieldName="Text" class="FieldInput"/></td>
    </tr>
    <tr class="InputRow">
      <td class="FieldName">Include:</td>
      <!-- "Search in" specific entity type -->
      <td colspan="2"><select class="FieldInput" owc:control="Field"
owc:FieldName="DocType">
        <option value="" selected="selected">All Documents</option>
        <option value="Book">Books</option>
        <option value="Magazine">Magazines</option>
        <option value="Journal">Journals</option>
      </td>
    </tr>
  </tbody>
</table>
```

```

    <option value="Newspaper">Newspapers</option>
    <option value="Catalog">Catalogs</option>
    <option value="Technical Document">Technical Documents</option>
    <option value="Report">Reports</option>
  </select></td>
</tr>
</tbody>
<tbody id="ctrlFolderLookup" owc:control="SearchInFolderLookup">
  <tr class="InputRow">
    <td>&#160;</td>
    <td colspan="2"><select owc:ui="SearchInAllFolders"
owc:control="Value" owc:DataType="Boolean" class="FieldInput">
    <option value="true" selected="selected">All Folders</option>
    <option value="false">Specific Folder...</option>
  </select></td>
  </tr>
  <tr owc:ui="FolderSelection">
    <td>&#160;</td>
    <td colspan="2">
      <table class="NestedSearchTable" cellpadding="0" cellspacing="0">
        <colgroup>
          <col width="88%" />
          <col width="12%" />
        </colgroup>
        <tr class="InputRow">
          <td><input class="FieldInput"
readonly="readonly"
type="text"
owc:ui="FolderTitle"
owc:control="Value"/></td>
          <td><input owc:control="Command" owc:Command="ViewChooseFolder"
type="button" value="..." ID="Button1" NAME="Button1"/></td>
        </tr>
        <tr class="InputRow">
          <td colspan="2"><input class="Checkbox" type="checkbox"
owc:ui="Recursive" owc:control="Value"/>Search in sub-folders</td>
        </tr>
      </table>
    </td>
  </tr>

```

```

</td>
</tr>
</tbody>
<tbody>
<tr class="Separator">
<td colspan="3"><hr /></td>
</tr>
</tbody>
<tbody owc:SkinGenerateFor="AdvancedSearch">
<tr class="InputRow">
<td class="FieldName">Title:</td>
<td colspan="2"><input type="text" class="FieldInput"
owc:control="Field" owc:FieldName="dc:title" /></td>
</tr>
<tr class="InputRow">
<td class="FieldName">Subject:</td>
<td colspan="2"><input type="text" class="FieldInput"
owc:control="Field" owc:FieldName="dc:subject" /></td>
</tr>
<tr class="InputRow">
<td class="FieldName">Description:</td>
<td colspan="2"><input type="text" class="FieldInput"
owc:control="Field" owc:FieldName="dc:description" /></td>
</tr>
<tr class="Separator">
<td colspan="3"><hr /></td>
</tr>
<tr class="InputRow">
<td class="FieldName" colspan="3">Select Date:</td>
</tr>
<tr class="InputRow">
<td class="FieldName">From:</td>
<td class="DateCell"><input id="FromCalendarDate"
class="FieldInput"
type="text"
owc:control="Field"
owc:DataType="Date"
owc:FieldName="dc:date"

```

```

owc:Operators="range-start" /></td>
<td class="BrowseDateCell"></td>
</tr>
<tr class="InputRow">
<td class="FieldName">To:</td>
<td class="DateCell"><input id="ToCalendarDate"
class="FieldInput"
type="text"
owc:control="Field"
owc:DataType="Date"
owc:FieldName="dc:date"
owc:Operators="range-end" /></td>
<td class="BrowseDateCell"></td>
</tr>
</tbody>
<!-- "Sort search results by..." field -->
<tbody>
<tr class="InputRow">
<td class="FieldName">Sort by:</td>
<td colspan="2"><select owc:control="SearchOption"
owc:FieldName="sort-by" owc:SkinGenerateFor="AdvancedSearchSort"
class="FieldInput">
<option value="" selected="selected">Relevance</option>
<option value="WordCount;desc">Size</option>
<option value="dc:date;desc">Date (descending)</option>
<option value="dc:date;asc">Date (ascending)</option>
<option value="dc:title;asc">Title</option>
<option value="dc:creator;asc">Creator</option>
</select></td>
</tr>

```

```

<tr class="Separator">
<td colspan="3"><hr /></td>
</tr>
<tr class="InputRow">
<!-- "hits only" flag -->
<td class="FieldName" colspan="3"><input owc:control="SearchOption"
owc:FieldName="hits-only"
class="Checkbox"
type="checkbox"
checked="checked" />Show only TOC items with hits
<input type="hidden" value="1" owc:control="SearchOption"
owc:FieldName="propagate-hits" />
</td>

</tr>
<tr class="Separator">
<td colspan="3"><hr /></td>
</tr>
<tr class="InputRow">
<!-- "Search using language" field -->
<td class="FieldName">Language:</td>
<td colspan="2"><span owc:control="LangList"
owc:FieldName="dc:language" class="LangList"></span></td>
</tr>
</tbody>
<tbody>
<tr>
<!-- "Search" button -->
<td class="Buttons" align="center" colspan="3">
<span owc:control="CommandButton" owc:Command="Search" title="Submit
search">Search</span>
<span owc:control="CommandButton" owc:Command="CancelSearch"
title="Cancel search">Cancel</span>
</td>
</tr>
</tbody>
</table>

```


See Also

[SearchForm Overview](#) | [SearchForm Class Overview](#)

SearchInFolder Class Overview

The SearchInFolder class functionality is implemented by the Olive.Controls.SearchInFolder JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Field

Olive.Controls.SearchTerm

Olive.Controls.SearchInFolder

Requirements

Script Files	OwcSearchForm.js
--------------	------------------

See Also

[SearchInFolder Methods](#) | [SearchInFolder Overview](#)

SearchInFolderLookup control Overview

SearchInFolderLookup control is a control that enables choosing a folder to search and specify if to search in subfolders. The control includes some UI elements for these options:

- **SearchInAllFolders** UI element is for specifying to search in all folders
- **FolderSelection** UI element gives an option to choose a searched folder
- **FolderTitle** UI element for choosing a Folder title
- **Recursive** UI element specifies if to search in subfolders

SearchInFolderLookup control is a special control, registered in [Olive.Controls.SearchForm](#) class represented SearchForm control. So it can be used only in the SearchForm control.

Syntax

```
<table id="ctrlFolderLookup" owc:control="SearchInFolderLookup">
  <tr class="InputRow">
    <td>
      <select owc:ui="SearchInAllFolders" owc:control="Value"
owc:DataType="Boolean" class="FieldInput"
```

```

owc:onValueChanged="onChooseFolderChange">
  <option value="true" selected="selected">All Folders</option>
  <option value="false">Specific Folder...</option>
</select>
</td>
</tr>
<tr owc:ui="FolderSelection">
  <td>
    <table class="NestedSearchTable" cellpadding="0" cellspacing="0">
      <tr class="InputRow">
        <td>
          <input class="FieldInput" readonly="readonly"
            type="text" owc:ui="FolderTitle" owc:control="Value"/>
        </td>
        <td>
          <input owc:control="Command" owc:Command="ViewChooseFolder"
            type="button" value="..."/>
        </td>
      </tr>
      <tr class="InputRow">
        <td colspan="2">
          <input class="Checkbox" type="checkbox" owc:ui="Recursive"
            owc:control="Value"/>Search in sub-folders
        </td>
      </tr>
    </table>
  </td>
</tr>
</table>

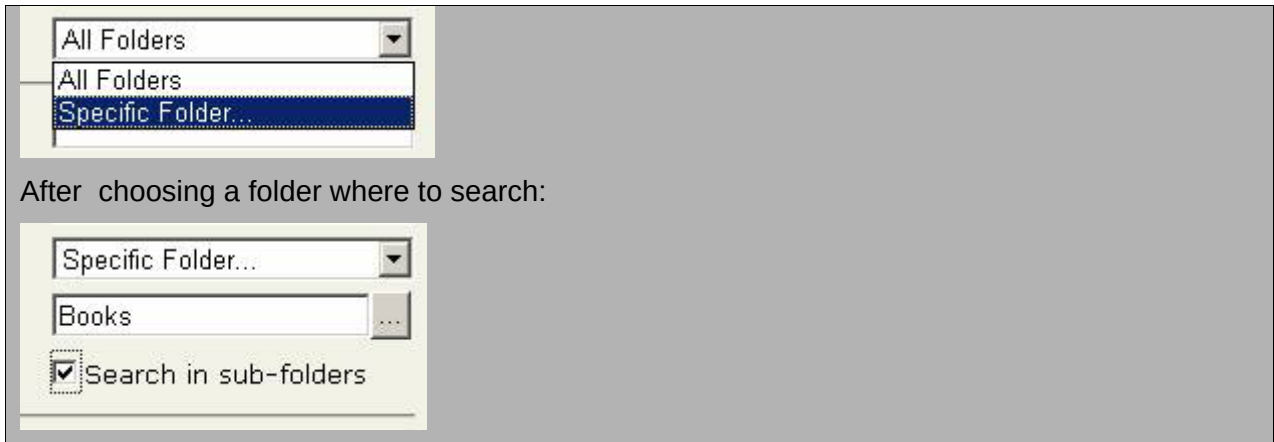
```

Preview

Before selections:



Select options:



After choosing a folder where to search:

See Also

[SearchInFolderLookup UiElements](#) | [SearchInFolderLookup Class](#) | [SearchInFolderLookup Methods](#)

SearchInFolderLookup Class Overview

The SearchInFolderLookup control functionality is implemented by `Olive.Controls.SearchInFolderLookup` JavaScript class.

Inheritance Hierarchy

`Olive.EventsSource`

`Olive.Object`

`Olive.CmdTarget`

`Olive.Controls.Control`

`Olive.Controls.Field`

`Olive.Controls.SearchInTerm`

`Olive.Controls.SearchInFolder`

`Olive.Controls.SearchInFolderLookup`

Implemented Interfaces

[Olive.IUiElements](#)

Requirements

Script Files	OwcSearchForm.js
--------------	------------------

See Also

[SearchInFolderLookup Overview](#) | [SearchInFolderLookup Methods](#)

SearchInFolderLookup Methods

SearchInFolderLookupspecific Methods

Event	Description
setFolder	Sets a chosen Folder title in the HTML element of the FolderTitle UI element of the SearchInFolderLookup control
onSearchInAllFoldersChanged	Value changed event handler for the control that shows/hides HTML element of the FolderSelection UI element

See Also

[SearchInFolderLookup Class](#)

onSearchInAllFoldersChanged Method

This method is a value changed event handler for the control that shows/hides HTML element of the FolderSelection UI element. The handler is attached to the event during control initialization.

Parameters

oEvent - An instance of Olive.Event class with parameters of the valueChanged event of the Olive.Controls.IValue interface

Example

```
oSearchInFolderLookupControl.onSearchInAllFoldersChanged(oEvent);
```

See Also

[SearchInFolderLookup Class](#) | [SearchInFolderLookup Methods](#)

setFolder Method

This method sets the chosen Folder title in the HTML element of the FolderTitle UI element of the SearchInFolderLookup control. It is used to transfer data to a search form regarding the chosen folder.

Parameters

oFolder - An instance of a folder object content item

Syntax

```
oSearchInFolderLookupControl.setFolder(oFolder);
```

Example

```
// Set a selected folder for the control
oSearchInFolderLookupControl.setFolder(oSelectedFolder);
```

See Also

[SearchInFolderLookup Class](#) | [SearchInFolderLookup Methods](#)

SearchInFolderLookup UiElements

SearchInFolderLookup-specific UiElements

Command	Description
FolderTitle	UI element that holds a chosen Folder title
SearchInAllFolders	UI element that specifies search in all folders
FolderSelection	UI element with an option to choose a searched folder
Recursive	UI element that specifies to search in subfolders

See Also

[SearchInFolderLookup Overview](#) | [SearchInFolderLookup Class](#) | [SearchInFolderLookup Methods](#)

FolderSelection UI Element

This UI element defines the HTML element that provides an option to choose a searched folder

Syntax

```
<html-element owc:ui=FolderSelection >
...
</html-element>
```

Remarks

The HTML element with this UI element opens a window choose a search folder

Example

```
<div owc:control="SearchInFolderLookup"> <div owc:ui="FolderSelection" >
... </div> </div>
```

Usage

Control	SearchInFolderLookup
---------	----------------------

Use	Optional
Default value	no

See Also

[SearchInFolderLookup Overview](#) | [SearchInFolderLookup UiElements](#)

FolderTitle UI Element

This UI element defines the HTML element that holds the chosen folder's title

Syntax

```
<html-element owc:ui=FolderTitle >
...
</html-element>
```

Remarks

The HTML element with this UI element is hidden before a user chooses the folder

Example

```
<div owc:control="SearchInFolderLookup"> <div owc:ui="FolderTitle" > ...
</div> </div>
```

Usage

Control	SearchInFolderLookup
Use	Optional
Default value	no

See Also

[SearchInFolderLookup Overview](#) | [SearchInFolderLookup UiElements](#)

Recursive UI Element

This UI element defines the HTML element that specifies search in subfolders

Syntax

```
<html-element owc:ui=Recursive >
...
</html-element>
```

Remarks

It is convenient to use a check box HTML element for this UI element

Example

```
<div owc:control="SearchInFolderLookup"> <div owc:ui="Recursive" >...  
</div> </div>
```

Usage

Control	SearchInFolderLookup
Use	Optional
Default value	no

See Also

[SearchInFolderLookup Overview](#) | [SearchInFolderLookup UiElements](#)

SearchInAllFolders UI Element

Defines the HTML element that specifies search in all folders

Syntax

```
<html-element owc:ui=SearchInAllFolders >  
...  
</html-element>
```

Remarks

The option to search in all folders is by default

Example

```
<div owc:control="SearchInFolderLookup"> <div  
owc:ui="SearchInAllFolders" > ... </div> </div>
```

Usage

Control	SearchInFolderLookup
Use	Optional
Default value	no

See Also

[SearchInFolderLookup Overview](#) | [SearchInFolderLookup UiElements](#)

SearchInSelectedFolder control Overview

SearchInSelectedFolder control is a control that enables to selecting one of the tree options:

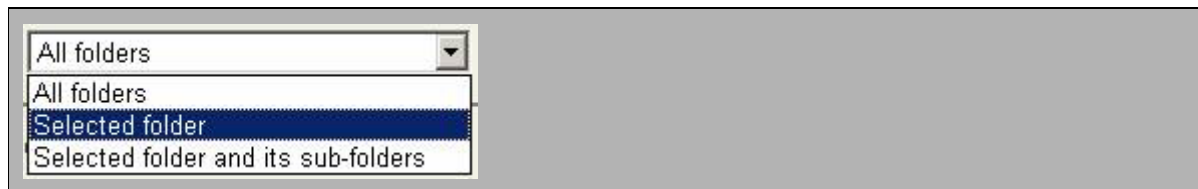
- **Search in all folders** a default mode, that corresponds to an empty value
- **Search in selected folder** a mode, that corresponds to selected value
- **Search in selected folder and its subfolders** a mode, that corresponds to recursive value

SearchInSelectedFolder control is a special control, registered in [Olive.Controls.SearchForm](#) class represented SearchForm control. So it can be used only in the SearchForm control.

Syntax

```
<select id="cmbSearchIn" owc:control="SearchInSelectedFolder">
  <option value="" selected="selected">All folders</option>
  <option value="selected">Selected folder</option>
  <option value="recursive">Selected folder and its sub-folders</option>
</select>
```

Preview



See Also

[SearchInSelectedFolder Class](#) | [SearchInSelectedFolder Methods](#)

SearchInSelectedFolder Class Overview

The SearchInSelectedFolder control functionality is implemented by Olive.Controls.SearchInSelectedFolder JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Field

Olive.Controls.SearchInTerm

Olive.Controls.SearchInFolder

Olive.Controls.SearchInSelectedFolder

Requirements

Script Files	OwcSearchForm.js
--------------	------------------

See Also

SearchInSelectedFolder Overview | SearchInSelectedFolder Methods

SearchInSelectedFolder Methods

SearchInSelectedFolderspecific Methods

Event	Description
setFolderTree	Connects a folder tree control to a search in the selected folder control

See Also

SearchInSelectedFolder Class

setFolderTree Method

This method connects a folder tree control to a search in a selected folder control

Parameters

oFolderTree - An instance of a folder tree control

Syntax

```
oSearchInSelectedFolderControl.setFolderTree(oFolder);
```

Example

```
// Set a folder tree control to the search in selected folder control
oSearchInSelectedFolderControl.setFolderTree(oSelectedFolder);
```

See Also

SearchInSelectedFolder Class | SearchInSelectedFolder Methods

SearchOption Overview

The SearchOption control is a sub-control of a [SearchForm](#) control. It enables input of additional search criteria, in addition to the Field controls that allow the user to input a string to search documents' text or metadata fields.

A SearchOption control must be attached to an HTML element that allows input, such as a select, checkbox, or text input element. Search options that are passed to the server without the user being able to see or change them can be inserted into a hidden input element.

A selection element for a boolean field should have as its value "0" or "1", not "true" or "false".

Syntax

```
<html-element owc:Control = "SearchOption" owc:FieldName="/valid field name/">  
  <html-input-element>...</html-input-element>  
</html-element>
```

Example

```
<tr>  
  <td><span class="LabelText">Sort by:</span></td>  
  <td colspan="2"><select id="selSortBy" owc:Control="SearchOption"  
owc:FieldName="sort-by">  
    <option value="" selected="">none</option>  
    <option value="IssueDateid;desc" selected="selected">Date  
Descending</option>  
    <option value="PageNo;asc">Page Number</option>  
  </select>  
</td>  
</tr>  
<tr>  
  <td width="120px" nowrap="nowrap"><span class="LabelText">Results per  
page:</span></td>  
  <td colspan="2"><select id="selPageSize" owc:Control="SearchOption"  
owc:FieldName="pageSize">  
    <option value="" selected="">none</option>  
    <option value="3">3</option>  
    <option value="8" selected="selected">8</option>  
    <option value="20">20</option>  
  </select>  
</td>  
</tr>
```

Preview

Sort by:	Date Ascending ▾
Results per page:	5 ▾

See Also

[SearchForm Overview](#) | [SearchOption Class](#) | [Data.SearchQuery Overview](#) | [Data.SearchOption Overview](#) | [ISearchFormField Overview](#)

SearchOptions HTML Attributes

SearchOptions Attributes

Attribute	Description
owc:FieldName	Specifies the search option being selected. See the detailed description of this attribute for a list of valid values.

Attributes inherited from [IFormField](#)

Attribute	Description
owc:Required	Is the field is required (boolean)? This is an optional attribute. If it is "true" and the user does not fill in the field, an alert warning will pop up saying that the required field has not been filled.

See Also

[SearchOption Class](#) | [SearchOption Overview](#)

owc:FieldName HTML Attribute

This attribute specifies which search option is being selected.

Syntax

```
owc:FieldName=/sName/
```

Remarks

The owc:FieldName attribute has eight possible values:

- **pagesize (number)** - the maximum number of search results to show on each page of results. The default is one result per page
- **sort-by** - a parsable string of /field-name/ + ";" + /direction/ ("asc"/"desc" for ascending/descending). For example, "dc:title;asc"
- **fuzzy-level (number, from 1 to 5)** - include words that differ from the search word in this number of letters. This is useful for scanned documents with scanning errors. A value greater than 3 is not useful.

- **stemming (boolean)** - include words from the same stem as the search word, for example "bound" and "binding" from "bind"
- **phonic (boolean)** - include words that sound like the search word, for example "site" from "sight"
- **synonyms (boolean)** - include synonyms of the search word, for example "vision" from "sight"
- **hits-only (boolean)** - when displaying the table of contents of the entity containing the search term, only display rows for those TOC entries that contain the search term
- **propagate-hits (boolean)** - when displaying the table of contents, with the number of search hits for each TOC entry written in each row of the TOC, if there are parent TOC entries that include child TOC entries, display the total number of search hits for the parent plus all search hits in the child entries.

Note that if a search option has a boolean value, if the associated HTML element is a select element, it should return a value of "0" or "1", not "true" or "false". If it is a checkbox, it will automatically return the appropriate boolean value.

Example

```
owc:FieldName=propagate-hits
```

Usage

Use	Mandatory
Default value	none

See Also

[SearchOption Class](#) | [SearchOption Overview](#)

SearchOption Class Overview

The SearchOption class functionality is implemented by the Olive.Controls.SearchOption JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Field

Olive.Controls.SearchOption

Requirements

Script Files	OwcSearchForm.js
--------------	------------------

See Also

[SearchOption Methods](#) | [SearchOption Overview](#)

SearchOption Class Methods

SearchOption Methods inherited from [ISearchFormField](#)

Method	Description
readFromSearchData	Reads the HTML data and update the JavaScript control.
writeToSearchData	Writes the search parameter data from the JavaScript control to the HTML SearchForm elements.

SearchOption Methods inherited from [IFormField](#)

Method	Description
getFieldName	Returns the field name (string).
isFieldRequired	Returns true if the field is required (boolean)

See Also

[SearchOption Class](#) | [SearchOption Overview](#)

SearchRes Overview

The SearchRes component executes a search and shows the search results on the page. Typically a [SearchForm](#) component is used to input the search parameters, and they are sent to the SearchRes component by the application, by means of the handler of the "searchRequest" event.

The SearchRes component inherits pagination features from the [List](#) component: the attributes [owc:UsePagination](#) and [owc:PageSize](#), and the commands `goToPage` / `prevPage` / `nextPage` / `firstPage` / `lastPage`.

The layout of the search results must be determined by the application's Layout.xml file. This information cannot be put directly into the html pages, since it must be replicated for each search-result-item.

Syntax

<pre><div id="divSearchRes" owc:Control="SearchRes" owc:UsePagination="true"> </div> <div id="divSearchRes" owc:Control="SearchRes" owc:UsePagination="true"</pre>

```
owc:PageSize="6">
  <span class="Msg_NoHits">No search performed</span>
</div>
```

See Also

[SearchRes Class](#) | [SearchForm Overview](#) | [List Overview](#)

SearchRes HTML Attributes

SearchRes Methods inherited from [List](#)

Attribute	Description
owc:UsePagination	Divides the results into pages (true) or display all the results in one page (false). Type boolean; optional; default value = false.
owc:PageSize	Number of items per page. This is only used if owc:UsePagination="true" Type number, must be greater than 0; default value = 1.

See Also

[SearchRes Overview](#) | [SearchForm Overview](#) | [List Overview](#) | [List Attributes](#)

SearchRes Class Overview

The SearchRes component functionality is implemented by the Olive.Controls.SearchRes JavaScript class.

The Olive.Controls.SearchRes interface provides access to the SearchRes component properties and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.List

Olive.Controls.SearchRes

Implemented Interfaces

Olive.IDataBound

Requirements

Script Files	OwcSearchRes.js
--------------	-----------------

See Also

[SearchRes Overview](#) | [SearchRes Methods](#) | [List Overview](#)

SearchRes Class Methods

SearchRes Methods

Method	Description
loadSearchResults(oSearchQuery)	Executes the search request.
navigateToPage()	Loads the search results content according to parameters defined in the Content Item. This method is called from the List base class to navigate to the next/previous/first/last page.

See Also

[SearchRes Overview](#) | [SearchRes Class](#)

loadSearchResults Method

This method executes the search request. The parameter oSearchQuery typically is passed by an application that saved the SearchQuery object from a SearchForm control.

Parameters

oSearchQuery - An Olive.Data.SearchQuery object that contains the search parameters.

Syntax

```
OwcSearchResControl.loadSearchResults(oSearchQuery);
```

Example

```
function myOnSearchRequest(oEvent)
{
    oEvent.cancelBubbling = true;
    var oSearchRes = OwcGetControl("divSearchRes");
    oSearchRes.loadSearchResults(oEvent.SearchParams);
}
```

See Also

[SearchRes Overview](#) | [SearchRes Class Ref267457857](#) | [SearchRes Methods](#)

navigateToPage Method

This method loads the search results page according to parameters defined in the Content Item. It is called from the List base class to navigate to the next/previous/first/last page.

Parameters

This method does not have parameters.

Syntax

```
OwcSearchResControl.navigateToPage();
```

See Also

[SearchRes Overview](#) | [SearchRes Class](#) | [SearchRes Methods](#)

OwcSearchRes Class Properties

Inherited Properties

Property	Description
----------	-------------

See Also

[SearchRes Overview](#) | [SearchRes Class](#)

SearchRes Control Example

The example below has a SearchRes control that uses pagination.

```
<div id="divSearchRes" owc:Control="SearchRes" owc:UsePagination="true"
owc:PageSize="10" >
</div>
```

This example has a SearchRes control that has an initial message before the first search has been performed.

```
<div id="divSearchRes" owc:Control="SearchRes" owc:UsePagination="true"
owc:PageSize="10" >
  <span class="Msg_NoHits">No search performed</span>
</div>
```

See Also

[SearchRes Overview](#) | [List Overview](#)

SearchTerm Overview

A SearchTerm control makes enables using an HTML input element to input a search parameter. It has the following attributes (only owc:Control and owc:FieldName are required attributes):

- **owc:Control="Field"**
- **owc:FieldName** - the name of the document metadata field which the input must match. "Text" means matching any text of a document
- **owc:Required** - whether the user must input data into this element ("true"/"false"; default is "false")
- **owc:DataType** - the data type of the input (for example, "Date")
- **owc:Format** - the format of the input (for example, date format "%m/%d/%Y")
- **owc:Operators** - operators to apply to the matching. Valid values are "range-start", "range-end", "and", "or", "not".
- **owc:Flags** (not yet implemented)

Inheritance

SearchTerm implements [ISearchFormField](#), which in turn implements [IFormField](#).

The abstract class [SearchInFolder](#), and its two derived classes [SearchInFolderLookup](#) and [SearchInSelectedFolder](#), are derived from SearchTerm.

Example

```
<span>Search For: </span>
<span>
  <input type="text" owc:control="Field" owc:FieldName="Text"
owc:Required="true" />
</span>
```

Preview

Search For:

See Also

[SearchForm Overview](#) | [SearchTerm Attributes](#) | [SearchTerm Class](#) | [ISearchFormField Overview](#)

SearchTerm HTML Attributes

Attributes inherited from [Olive.Controls.IFormField](#)

Attribute	Description
owc:FieldName	The name of the document metadata field to which the input must match. "Text" means matching any text of a document.

owc:Required	Boolean value if the user must input data into this element ("true"/"false"; default is "false")
------------------------------	--

SearchTerm-specific Attributes

Attribute	Description
owc:Operators	Operators to apply to the matching. Valid values are "range-start", "range-end", "and", "or", "not"
owc:Flags	(not yet implemented)

See Also

[SearchTerm Overview](#) | [SearchTerm Class](#) | [SearchTerm Methods](#)

SearchTerm Class Overview

The SearchTerm control functionality is implemented by Olive.Controls.SearchTerm JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Field

Olive.Controls.SearchTerm

Implemented Interfaces

Olive.Controls.ISearchFormField

Requirements

Script Files	OwcSearchForm.js
--------------	------------------

See Also

[SearchTerm Methods](#) | [SearchTerm Overview](#)

SearchTerm Methods

Inheriting Methods of [Olive.ISearchFormField](#)

Event	Description
readFromSearchData	Reads a value from search data and puts it into the HTML element of a search term control

writeToSerchData	Writes a value into search data from HTML element represented a search term control
----------------------------------	---

SearchTerm specific Methods

Event	Description
parseSearchOperators	Parses HTML attributes of search operators
parseSearchFlags	Parses HTML attributes of search flags

See Also

SearchTerm Class

parseSearchFlags Method

This method gets the HTML attribute owc:Flags, parses it and saves it in an internal field.

Parameters

No parameters

Syntax

```
oSearchTermControl.parseSearchFlags();
```

Example

```
// read HTML attribute owc:Flags
oSearchTermControl.parseSearchFlags();
```

See Also

SearchTerm Methods | SearchTerm Class

parseSearchOperators Method

This method gets the HTML attribute owc:Operators, parses it and saves it in an internal field.

Parameters

No parameters

Syntax

```
oSearchTermControl.parseSearchOperators();
```

Example

```
// read HTML attribute  owc:Operators
oSearchTermControl.parseSearchOperators();
```

See Also

[SearchTerm Methods](#) | [SearchTerm Class](#)

readFromSearchData Method

This method reads a value from the search data and enters it into the HTML element of a search term control. It returns the value set to HTML element.

It performs an operation opposite to `writeToSearchData` method.

Parameters

oDataObject - An instance of `Olive.Data.SearchQuery` data type

sFieldName - A field name string

Syntax

```
oSearchTermControl.readFromSearchData(oDataObject,  sFieldName);
```

Example

```
// read data  from a search query object and set the value into the HTML element
of the  SearchTerm control
var oValue = oSearchTermControl.readFromSearchData(oSearchQueryObject,  dc:title);
```

See Also

[SearchTerm Methods](#) | [SearchTerm Class](#)

writeToSearchData Method

This method writes a value into search data from the HTML element representing a search term control.

It performs an operation opposite to `readFromSearchData` method

Parameters

oDataObject - An instance of Data type (`Olive.Data.SearchOptions` or `Olive.Data.SearchQuery`)

sFieldName - A field name string

Syntax

```
oSearchTermControl.writeToSearchData(oDataObject,  sFieldName);
```

Example

```
// read value of the HTML element and write it into the data of search query object
var oSearchQueryObject = new Olive.Data.SearchQuery();
oSearchTermControl.writeToSearchData(oSearchQueryObject, dc:date);
```

See Also

SearchTerm Methods | SearchTerm Class

SortBy Overview

The SortBy control is a sub-control of a [List](#). It enables you to attach an HTML select element to the List, with a list of fields by which the user can sort the items in the list, by field name and direction (ascending / descending). When the user changes the displayed choice on the selected element, the new value is automatically passed to the list control and a request is sent to the server to retrieve a page of list data in the new sort order.

In principle, any HTML input element with an *onchange* event may be used, but a `<select>` element is most useful, since the required value must be one of a limited number of encoded strings of the format "{field-name};{asc/desc}".

The SortBy control is intended for use with a [DocList](#) control. The other List control, the [SearchRes](#) control, gets data from search parameters that are input in a SearchForm control, and the SearchForm control has its own mechanism for specifying the sort order. For further information, see on the SearchForm control. control, and the SearchForm control has its own mechanism for specifying the sort order. For more information, see on the [SearchForm](#) control.

The can be specified on the HTML page or in the *Layout.xml* definition of the [DocList](#) header, so that it will not appear for an empty list.

The API includes four functions:

- **getListControl:** gets which List control is associated with this SortBy control
- **setListControl:** sets which List control is associated with this SortBy control
- **updateSelectionFromList:** update the HTML display from the SDK Javascript List object
- **updateListFromSelection:** update the SDK Javascript List object from the HTML display.

These four functions are mainly for internal use. The normal data flow is done automatically by the control.

Syntax

```
<html-input-element owc:Control = "SortBy">
...
</html-input-element>
```

Example

```
<select owc:Control = "SortBy" size="1">
  <option value="dc:title;asc">Title</option>
  <option value="dc:date;desc">Date</option>
</select>
```

Preview



See Also

[SortBy Class](#) | [SortBy Methods](#) | [List Overview](#) | [SearchForm Overview](#)

SortBy Class Overview

The SortBy base class functionality is implemented by the Olive.Controls.SortBy JavaScript class.

The Olive.Controls.SortBy interface provides access to the SortBy component methods.

Inheritance Hierarchy

- Olive.EventsSource
- Olive.Object
- Olive.CmdTarget
- Olive.Controls.Control
 - Olive.Controls.SortBy

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[SortBy Overview](#) | [SortBy Methods](#) | [List Overview](#)

SortBy Class Methods

SortBy Methods

Method	Description
setListControl (oListControl)	Sets the List control that is associated with this SortBy control

getListControl()	Gets the List control that is associated with this SortBy control
updateSelectionFromList()	Updates the HTML display from the SDK JavaScript List object
updateListFromSelection()	Updates the SDK JavaScript List object from the HTML display

See Also

[SortBy Overview](#) | [List Overview](#)

getListControl

This method gets the List control that is associated with this SortBy control. For internal use.

Parameters

none

Syntax

```
oSortBy.getListControl( oListControl );
```

See Also

[SortBy Overview](#) | [List Overview](#)

setListControl

This method sets the List control that is associated with this SortBy control. For internal use.

Parameters

object - **oListControl**

The List control object that is associated with this SortBy control

Syntax

```
oSortBy.setListControl( oListControl );
```

See Also

[SortBy Overview](#) | [List Overview](#)

updateListFromSelection

This method updates the SDK JavaScript List object from the HTML display. This is called automatically when the control is parsed. The onchange HTML handler calls it.

Parameters

none

Syntax

```
oSortBy.updateListFromSelection();
```

See Also

[SortBy Overview](#) | [List Overview](#)

updateSelectionFromList

This method updates the HTML display from the SDK JavaScript List object. This is called automatically when the control is initially parsed. The List object can be previously initialized (for example: from the application preferences) when the object or page is initialized, by a call to its sortBy() method.

Parameters

none

Syntax

```
oSortBy.updateSelectionFromList();
```

See Also

[SortBy Overview](#) | [List Overview](#)

TabItem Control Overview

The TabItem control is a control representing a tab in a tab strip. For more information, see on the [TabStrip](#) control.

Syntax

```
<html-element owc:Control = TabStrip>
  <html-element owc:Control = TabItem> item 1  </html-element>
  <html-element owc:Control = TabItem> item 2  </html-element>
  ..
</html-element>
```

Remarks

TabItem controls are sub-controls in the container TabStrip control, which has an API that gives access to the individual TabItem controls.

The visual interface of each TabItem control is based on combination of CSS classes. Every TabItem has a set of CSS classes for left, body and right (see the link to [TabItem](#)

[CSS Classes](#)). There are also styles for active and non-active items and first and last items.

Follow the technique below to create a TabItem:

You need to define 3 styles for the left, body and right parts of the tab item. It is recommended to use background-image and background-repeat styles for each item. For the background-image style, define the URL to the image.

For the style of the body part, use an image with a width of 1px and set background-repeat to repeat-x or repeat-y. For the styles of the left and right parts, set the background-image to an URL with a full image and set background-repeat to no-repeat.

Preview



See Also

[TabItem Class](#) | [TabItem Css Classes](#) | [TabStrip Overview](#)

TabItem Class Overview

The TabItem component functionality is implemented by the Olive.Controls.TabItem JavaScript class. The Olive.Controls.TabItem interface provides access to the TabItem component properties and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.TabItem

Implemented Interfaces

Olive.IUIElements

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[TabItem Control Overview](#)

TabItem Class Events

TabItem Events inherited from [IState](#)

Event	Description
StateChanged	Event fired when the state of a tab-item changes between active and inactive. It is usually more useful to handle the <code>itemActivated</code> event of <code>TabStrip</code> in order to perform an action when the user selects a tab.

See Also

[TabItem Class](#) | [TabItem Methods](#) | [TabItem Overview](#)

TabItem Control CSS Classes

CSS Class	Description
OwcTabItem	CSS class for the entire tab item
TabLeft	CSS class for the left part of an item
TabBody	CSS class for the body part of an item (with a background image of 1px width)
TabRight	CSS class for the right part of an item
FirstTab	CSS class for the first tab item; can be used together with TabLeft or TabRight classes
LastTab	CSS class for the last tab item; can be used together with TabLeft or TabRight classes
ActiveTab	CSS class for the active tab item; can be used together with other predefined classes

See Also

[TabItem Control Overview](#) | [TabItem Class](#)

TabItem Example

The example below shows the TabItem control in the user application.

```
HTML:
<div id="tab0" owc:Control="TabStrip" >
  <span owc:Control="TabItem">Option 0</span>
  <span owc:Control="TabItem">Option one</span>
  <span owc:Control="TabItem">Option 2</span>
```

```

<span owc:Control="TabItem">Option last</span>
</div>
CSS Classes:
.OwcTabItem
{
    line-height: 32px;
    height: 32px;
    cursor: pointer;
    text-align: center;
    white-space: nowrap;
    font-family: Arial;
    font-size: 12pt;
    font-weight: bold;
    font-style: italic;
    display: inline-table;
}
.TabLeft
{
    width: 2px;
    height: 32px;
    background-image: url(Images/tab_Item_Edge1.gif);
    background-repeat: no-repeat;
    display: table-cell;
    vertical-align: middle;
}
.FirstTab .TabLeft
{
    width: 13px !important;
    background-image: url(Images/tab_First_Left_Inactive.gif) !important;
}
.ActiveTab .FirstTab .TabLeft
{
    width: 13px !important;
    background-image: url(Images/tab_First_Left_Active.gif) !important;
}
.TabRight
{

```

```

width: 1px;
height: 32px;
background-image: url(Images/tab_Item_Edge.gif);
background-repeat: no-repeat;
display: table-cell;
vertical-align: middle;
}
.LastTab .TabRight
{
width: 25px !important;
background-image: url(Images/tab_Last_Right_Inactive.gif) !important;
}
.ActiveTab .LastTab .TabRight
{
width: 25px !important;
background-image: url(Images/tab_Last_Right_Active.gif) !important;
}
.TabBody
{
height: 32px;
background-image: url(Images/tab_Bg_Inactive.gif);
background-repeat: repeat-x;
display: table-cell;
text-decoration: none;
padding-left: 6px;
padding-right: 6px;
vertical-align: middle;
}
.ActiveTab .TabBody
{
background-image: url(Images/tab_Bg_Active.gif) !important;
}

```

See Also

TabItem Overview | [TabStrip Overview](#)

TabStrip Control Overview

The TabStrip control is a container for [TabItem](#) controls. With it, you can create the appearance and functionality of a set of clickable tabs that activate one tab, make the other tabs appear to be inactive, and perform the action associated with the activated tab.

The TabStrip control uses a set of CSS classes for the TabStrip control and for the TabItem sub-controls to create the custom appearance of a strip of tabs. This separation of the HTML text from the CSS styling makes it easy to change the text (for internationalization, for example) without changing the style; compare the [CommandButton](#) control. In order to use styling through the pre-defined CSS classes, you must set the attribute `owc:Decorate` to "true". Otherwise, you must create your own styling for the TabStrip and TabItem sub-controls.

The TabStrip control uses the functionality of the [IActiveItem](#) interface to get and set the active item sub-element, and it fires events that enable you to respond to an active item change.

Syntax

```
<html-element owc:Control = TabStrip>
  <html-element owc:Control = TabItem> item 1  </html-element>
  <html-element owc:Control = TabItem> item 2  </html-element>
  ..
</html-element>
```

Remarks

The visual interface of the control is based on a combination of CSS classes for the tab strip as a whole, for its contents, and for the image at the end. There are also predefined CSS classes for the TabItem sub-controls.

The pre-defined CSS classes for a TabStrip are:

- OwcTab
- OwcTabContent
- OwcTabEnd

The pre-defined CSS classes for each TabItem are:

- OwcTabItem
- ActiveTab
- FirstTab
- LastTab
- TabLeft
- TabBody
- TabRight

Preview



See Also

[TabStrip Class](#) | [TabStrip Methods](#) | [TabStrip Events](#)

TabStrip HTML Attributes

TabStrip-specific Attributes

Attribute	Description
owc:Decorate	Type: boolean Specifies when to add the pre-defined CSS classes to the HTML of the control

See Also

[TabStrip Overview](#) | [TabStrip Class](#)

owc:Decorate HTML Attribute

This attribute specifies when to add the pre-defined CSS classes to the control's HTML.

Syntax

```
owc:Decorate=/boolean/
```

Remarks

If "true", define the pre-defined CSS classes of the TabStrip and TabItem controls - this TabStrip attribute also applies to the TabItem sub-controls. Set to "false" or omit to provide your own styling without using the pre-defined CSS classes.

Example

```
owc:Decorate=true
```

Usage

Use	Optional
Default value	false

See Also

[TabStrip Overview](#) | [TabStrip Attributes](#)

TabStrip Class Overview

The TabStrip component functionality is implemented by the Olive.Controls.Tab JavaScript class. The Olive.Controls.Tab interface allows access to the TabStrip component properties and methods.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Tab

Implemented Interfaces

Olive.IActiveItem

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

TabStrip Events | TabStrip Methods | TabStrip Overview

TabStrip Class Events

TabStrip Events inherited from [IActiveItem](#)

Event	Description
itemActivating	Event fired by setActiveItem() if the new sub-item is different from the old value and argument bDoNotNotify is not true. The event is fired before the active/inactive state of the old and new active items has been changed.
itemActivated	Event fired by setActiveItem() if the new sub-item is different from the old value and argument bDoNotNotify is not true. The event is fired after the active/inactive state of the old and new active items has been changed.

See Also

TabStrip Class | TabStrip Methods | TabStrip Overview

TabStrip Class Methods

TabStrip Methods inherited from [IActiveItem](#)

Method	Description
--------	-------------

getItemsCount()	Returns the number of sub-items.
getItem(itemIndex)	Returns the sub-item specified by index number itemIndex (starting from 0).
indexOfItem(oItem)	Returns the index number of sub-item oItem (starting from 0).
getActiveItemIndex()	Returns the index number of the active sub-item (starting from 0).
getActiveItem()	Returns the active sub-item.
setActiveItem(vItem, bDoNotNotify)	Sets sub-item vItem (index-number or object) to be the active sub-item. Fires events itemActivating and itemActivated unless bDoNotNotify is true.

See Also

[TabStrip Events](#) | [TabStrip Class](#) | [TabStrip Overview](#)

TabStrip Control CSS Classes

CSS Class	Description
OwcTab	CSS class for the entire TabStrip. It is recommended to define general size and appearance features here.
OwcTabContent	CSS class for content elements of a TabStrip control. It is recommended to use background-image style with an image of the width with 1px together with background-repeat style set to "repeat-x".
OwcTabEnd	CSS class for the ending of a TabStrip control. It is recommended to use background-image style with an image of the desired width together with the background-repeat style set to "no-repeat".

See Also

[TabStrip Class](#) | [TabStrip Methods](#) | [TabStrip Events](#)

Thumbnail Class Overview

The Thumbnail component functionality is implemented by Olive.Controls.Thumbnail JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Thumbnail

Implemented Interfaces

Olive.IDataBound

Requirements

Script Files	OwcBind.js
---------------------	------------

See Also

Thumbnail Overview

Thumbnail Example

The example below shows the Thumbnail control used in an application.

```
Layout.xml:
<owct:html-template kind="list-item-content" for="Document">
  <table>
    <tr valign="top">
      <td>
        <div class="DocThumbnail">
          <img owc:Control="Thumbnail" owc:ContentCmd="ViewDocument"
class="Thumbnail"></img>
        </div>
      </td>
```

See Also

Thumbnail Overview | Thumbnail Class Overview

TOC Overview

The TOC control shows the document's Table of Contents in the form of a TOC-entry nodes tree, where an entry can contain one or more entries. The typical functionality of a TOC control includes navigating through the table of contents by means of user-input nodes (clicking on "+" opens the subtree, clicking on "-" closes the subtree), and selecting a row within the tree, typically to open the document at the selected page and item, or to go to the selected page and item of a document that is already open. The TOC component inherits the *owc:MultiSelect* attribute (default value = false) from

[OwcTree](#). "True" enables selecting several nodes with control-click (for example, to add URL's of all the selected items to an email or the browser's "favorites" list).

The application's Layout.xml file determines the appearance and behavior of the TOC. It is not possible to put this information directly into the HTML pages, since it must be replicated for each TOC-item.

When the server returns the data of a TOC component, each node (row) in the TOC has a sub-component of type "TocEntry". The Layout.xml determines the layout and behavior of each TocEntry component.

For example, to open a document at the selected page and item when a node is clicked, add the following to Layout.xml:

```
<owct:tree-template for="Toc" default="true">
  <owct:tree-node-type name="TocDocNode" expand="yes">
    <owct:tree-node-layout>
      <cssClass>TocDocNode</cssClass>
      <iconUrl>Layout/Images/icon_Document.gif</iconUrl>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>TocDocNodeSelected</cssClass>
      <iconUrl>Layout/Images/icon_DocSelected.gif</iconUrl>
    </owct:tree-node-layout>
  </owct:tree-node-type>

  <owct:tree-node-type name="TocEntryNode" content-cmd="{command-
name}"xhtml-template="toc-tree-node">
    <owct:tree-node-layout>
      <cssClass>TocEntryNode</cssClass>
      <iconUrl>Layout/Images/icon_TocEntry.gif</iconUrl>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>TocEntryNodeSelected</cssClass>
    </owct:tree-node-layout>
  </owct:tree-node-type>
```

There are two kinds of TOC controls:

(1) An ordinary TOC for a document's table of contents, with subcontrol "TocDoc". This kind of TOC can be written directly as an html element.

(2) A TOC for search results. This is written in the Layout.xml template for a search result item, with a sub-control of type "TocSearchRes", and within that a command of type "Expand", which sends a second search request to the server for the TOC data (TocEntry subnodes).

Nested sub-controls

The html sub-tree of a TOC control contains html elements with the following sub-controls.

Example 1: a 2-level tree ordinary Toc with a node that contains a sub-node:

```
owc:Control="TOC"  
owc:Control="TocDoc"  
owc:ui="treeSubNodes"  
owc:ui="treeSubNodesTarget"  
owc:Control="TocEntry"  
owc:ui="treeSubNodes"  
owc:ui="treeSubNodesTarget"  
owc:Control="TocEntry"  
owc:ui="treeNodeSelectableArea"  
owc:ui="treeIcon"
```

Example 2: a 2-level tree search results Toc with a node that contains a sub-node:

```
owc:Control="SearchRes"  
owc:Control="SearchResItem"  
owc:Control="TOC" (<<< the Toc sub-control begins here)  
owc:Control="TocSearchRes"  
owc:ui="treeSubNodes" (<<< results from the second search request begin here)  
owc:ui="treeSubNodesTarget"  
owc:Control="TocEntry"  
owc:ui="treeSubNodes"  
owc:ui="treeSubNodesTarget"  
owc:Control="TocEntry"  
owc:ui="treeNodeSelectableArea"  
owc:ui="treeIcon"
```

There is one sub-control of type TocDoc or TocSearchRes in each Toc control. There can be many TocEntry controls, one for each node in the Toc tree.

Example

```
<div id="divDocToc" class="TocTree" owc:Control="TOC"  
owc:onInitialized="onTocInitialized">  
  <owc:Data type="TreeLayout">  
    <owc:NodeType name="TocDocNode"  
iconUrl="Layout/Images/icon_Document.gif"  
iconSelectedUrl="Layout/Images/icon_DocSelected.gif"  
cssClass="TocDocNode" cssClassSelected="TocDocNodeSelected" />  
    <owc:NodeType name="TocEntryNode"
```

```
iconUrl="Layout/Images/icon_TocEntry.gif"  cssClass="TocEntryNode"
cssClassSelected="TocEntryNodeSelected" />

</owc:Data>

There is no document loaded

</div>
```

See Also

[OwcToc Class](#) | [Tree Overview](#)

TOC Class Overview

The TOC component functionality is implemented by the Olive.Controls.Toc JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Tree

Olive.Controls.Toc

Implemented Interfaces

Olive.IDataBound

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[Toc Overview](#) | [Tree Overview](#)

TOC Example

This example shows the use of the Toc control.

```
<div id="divDocToc" class="TocTree" owc:Control="TOC"
owc:onInitialized="onTocInitialized">

  <owc:Data type="TreeLayout">

    <owc:NodeType name="TocDocNode"
conUrl="Layout/Images/icon_Document.gif"
iconSelectedUrl="Layout/Images/icon_DocSelected.gif"
cssClass="TocDocNode"  cssClassSelected="TocDocNodeSelected" />

    <owc:NodeType name="TocEntryNode"
iconUrl="Layout/Images/icon_TocEntry.gif"  cssClass="TocEntryNode"
cssClassSelected="TocEntryNodeSelected" />

  </owc:Data>

</div>
```

```
</owc:Data>
  There is no document loaded
</div>
```

See Also

[Toc Overview](#) | [Toc Class Overview](#)

Tree Overview

The Tree control is a base class that is not used directly, but gives tree functionality to its two derived classes, FolderTree (a tree of folders in the inventory) and TOC (the document's Table of Contents). This functionality includes:

- **Showing tree outline icons** - tracing lines, open/close icons (+/-), and possibly node images and/or texts
- **Clicking on the open/close icon** to open and close the sub-tree under a node
- **Performing a "content-command"** with the content-item that is associated with a sub-node (a folder content-item for a FolderTree, or an entity content-item for a TOC component).

The tree outline elements are called "UiElements" (user-interface elements). An HTML UiElement is not a regular SDK control: it does not inherit from Olive.Controls.Control, and is not a member of the parent's Controls property, but it does have a property owcUiParent that points to the closest ancestor that is a control.

There are two kinds of TOC components; *toc-document* and *toc-search-results*. These objects are of the same class (TOC), but with different subnodes (type TocDoc and TocSearchRes). See also on the [TOC component](#).

The public API of the Tree component has only one custom attribute, owc:MultiSelect (allow many nodes to be selected by control-shift). In addition, it is possible to add a sub-element <owc:Data type="TreeNode">, which has sub-elements <owc:NodeType>, to define the format of sub-nodes (node icons and css classes, regular and selected). See [Tree Attributes](#).

Some of the functionality of a tree is provided by the class OwcTreeNode, the base class for individual nodes of a FolderTree (FolderTreeNode) or of a Toc (TocEntryTreeNode, TocDocTreeNode, TocSearchResTreeNode). See also [TreeNode](#).

The layout of a tree is determined by the file Layout.xml. The SDK's Layout.xml default file's values can be overridden by a custom Layout.xml file in the application's server-side configuration folder.

See Also

[TOC Overview](#) | [FolderTree Overview](#)

Tree HTML Attributes and Sub-elements

Tree-specific Attributes

Attribute	Description
-----------	-------------

owc:MultiSelect	Enables the user to select more than one tree node (by shift-click). This can be used by inherited classes (FolderTree, Toc) so that the application can perform an action on several nodes. Type Boolean; optional; default value = false.
---------------------------------	---

Tree-specific Sub-elements

Element	Description
owc:Data type="TreeLayout"	This element is used as a parent element for <owc:NodeType> elements, which give image and style information for the tree nodes. Must be a sub-element of a Tree element (FolderTree or Toc).
owc:NodeType	Used for giving image and style information for a single type of Tree node. Must be a sub-element of an owc:Data type (under a Tree component).

owc:NodeType Specific Attributes [NOTE: these attributes do not have the prefix "owc:"]

Attribute	Description
name	The name of a node type returned by the server in the type attribute of owc:Data.
noIcon	Do not show any icon for this node type: neither an icon given in the owc:NodeType element, nor the default icon. Boolean; optional; default value = false
iconUrl	URL of the icon for this node-type.
iconUrlSelected	URL of the icon for this node-type if it is selected.
cssClass	Css class for this node-type if it is unselected (the style for this class should be defined in a css file).
cssClassSelected	Css class for this node-type if it is selected (the style for this class should be defined in a css file).

See Also

[Tree Overview](#) | [FolderTree Overview](#) | [Toc Overview](#)

cssClass HTML Attribute of the owc:NodeType HTML Element

CSS class for this node type if it is unselected (the style for this class should be defined in a css file).

Syntax

```
<owc:NodeType name="Folder" cssClass="normalNode"
cssClassSelected="selectedNode" />
```

Usage

Use	Required
Default value	none

See Also

[Tree Overview](#) | [Tree Attributes](#)

cssClassSelected HTML Attribute of the owc:NodeType HTML Element

This attribute is this node type's CSS class when selected (the style for this class should be defined in a css file).

Syntax

```
<owc:NodeType name="Folder" cssClass="normalNode"
cssClassSelected="selectedNode" />
```

Usage

Use	Optional
Default value	none

See Also

[Tree Overview](#) | [Tree Attributes](#)

iconUrl HTML Attribute of the owc:NodeType HTML Element

This attribute is the URL of this node type's icon.

Syntax

```
<owc:NodeType name="Folder" iconUrl="node.gif"
iconUrlSelected="nodeSelected.gif" />
```

Usage

Use	Optional
-----	----------

Default value	none
----------------------	------

See Also

[Tree Overview](#) | [Tree Attributes](#)

iconUrlSelected HTML Attribute of the owc:NodeType HTML Element

This attribute is the URL of the icon for this node type if it is selected.

Syntax

```
<owc:NodeType name="Folder" iconUrl="node.gif"
iconUrlSelected="nodeSelected.gif" />
```

Usage

Use	Optional
Default value	none

See Also

[Tree Overview](#) | [Tree Attributes](#)

name HTML Attribute of the owc:NodeType HTML Element

The name of a node type returned by the server in the type attribute of owc:Data.

Syntax

```
<owc:NodeType name="Folder" cssClass="normalNode"
cssClassSelected="selectedNode" />
```

Remarks

Valid values are TocDocNode, TocSearchResNode, TocEntryNode (the types of Tree nodes), Folder, plus any special folder node types (for example, "Inbox").

Usage

Use	Required
Default value	none

See Also

[Tree Overview](#) | [Tree Attributes](#)

noIcon HTML Attribute of the owc:NodeType HTML Element

This attribute does not show any icon for this node type: neither an icon given in the owc:NodeType element, nor the default icon.

Syntax

```
<owc:NodeType name="Folder" noIcon="true" cssClass="normalNode"
cssClassSelected="selectedNode" />
```

Remarks

If true, the node will appear as a tracing outline (for subnodes), the expand +/- box, and any images or texts specified in Layout.xml, without a fixed node icon after the expand box.

Usage

Use	Optional
Type	Boolean
Default value	false

See Also

[Tree Overview](#) | [Tree Attributes](#)

owc:MultiSelect HTML Attribute

This attribute enables the user to select more than one tree node (by shift-click). This can be used by inherited classes (FolderTree, Toc) so that the application can perform an action on several nodes.

Syntax

```
owc:MultiSelect=/boolean/
```

Remarks

If the only purpose of the tree is to click on a node to do something with the content associated with the node (display it, display its metadata, etc.), set it to "false" or omit it. If you want to be able to do something with a group of selected nodes, set it to "true".

Example

```
owc:MultiSelect=true
```

Usage

Use	Optional
------------	----------

Default value	false
----------------------	-------

See Also

[Tree Overview](#) | [Tree Attributes](#)

owc:Data HTML Sub-element

This element is used as a parent element for <owc:NodeType> elements, which give tree node image and style information.

Syntax

```
<owc:Data type="TreeLayout">
```

Remarks

This element is used as a wrapper for <owc:NodeType> elements. The type must be "TreeLayout". This type distinguishes it from the more common element <owc:Data for="/control-type-name"/>, which specifies what data to retrieve from the server. These two kinds of <owc:Data> elements are unrelated.

Usage

Use	Optional
Default value	none

See Also

[Tree Overview](#) | [Tree Attributes](#)

owc:NodeType HTML Sub-element

This element is used to give image and style information for a single Tree node type.

Syntax

```
<owc:NodeType name="Folder" cssClass="normalNode"
cssClassSelected="selectedNode" />
```

Remarks

This element gives different styles and icons for different tree node types, for example, the main document node and table of contents entry nodes in a TOC, or ordinary and special folders in a folder tree.

Usage

Use	Optional
------------	----------

Default value	none
----------------------	------

See Also

[Tree Overview](#) | [Tree Attributes](#)

Tree Class Overview

The Tree control is implemented by the Olive.Controls.Tree JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Tree

Implemented Interfaces

Olive.ISelection

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[Tree Overview](#) | [Tree Events](#) | [Tree Methods](#)

Tree Class Events

Tree Events inherited from [ISelection](#)

Event	Description
selectionChanging	Event fired if the new selected item is different from the old value. The event is fired before the active/inactive state of the old and new items has been changed.
selectionChanged	Event fired if the new selected item is different from the old value. The event is fired after the active/inactive state of the old and new items has been changed.

See Also

[Tree Overview](#) | [Tree Class](#)

Tree Class Methods

Tree Methods inherited from [ISelection](#)

Method	Description
selectionChange(vSelection, nSelectionMode)	Selects or unselects an item.
selectionClear()	Unselects all items.
isItemSelected(oItem)	Returns if item oItem is selected.
getSelectedItemsCount()	Returns the number of selected items.
getSelectedItem(nIndex)	Returns the item of index-number nIndex among all selected items.
getSelectedContentItem(nIndex)	Finds the item of index-number nIndex among all selected items, and returns its content item.

Tree-specific Methods

Method	Description
onNodeMouseDown(oEvent)	This internal function responds to a mouse down event (when a user selects a node). It can be overridden if you want to customize the response to a mouse down event. The method can be overridden
registerNodeTypes()	Registers a desirable tree node type for the Tree, can be overridden
setDefaultNodeType(vNodeType)	Sets a default node type for the tree
getNodeType(sNodeType)	Returns a node type object for the specified sNodeType.
getNodeOutlineName(dwOutlineMask)	Returns the name [to]fc [to]lx etc. from the array of 31 strings using the outline mask

See Also

[Tree Overview](#) | [Tree Class](#) | [Tree Events](#)

onNodeMouseDown

This internal function responds to a mouse down event (when a user selects a node). It can be overridden if you want to customize the response to a mouse down event.

Parameters

oTreeNode – object, the tree node that was clicked

oEvent - the event handled by the onMouseDown handler of the TreeNode class.

Remarks

This method is called internally by a TreeNode object. You can override it with a method that executes the base method in the Tree class and also does something else when the user clicks the mouse down on a node.

See Also

[Tree Overview](#) | [Tree Class](#)

registerNodeType Method

This method registers a tree node type for the Tree. It registers a default TreeNode node type, but the method can be overridden to register another/additional tree node type.

Parameters

No parameters

Syntax

```
oTree.registerNodeType();
```

See Also

[Tree Methods](#) | [Tree Class](#)

TreeNode Overview

The TreeNode component is a base class with four derived classes:

- **TocDocTreeNode** - the main sub-node in a document Table of Contents (TOC)
- **TocSearchResTreeNode** - the main sub-node of a search-results TOC
- **TocEntryNode** - the data sub-nodes of TOC
- **FolderTreeNode** - the nodes of a FolderTree

The server provides the data for tree nodes, but the layout is configured in *Layout.xml* (see "Syntax" below for an example). A content-item command, that is executed when clicking a row, is also configured in the *Layout.xml*(content-cmd="....").Member functions can also be called from an application function, and events can be handled by an application (see Methods and Events).

Since the server creates an unpredictable number of tree nodes without node IDs, the application usually calls methods on a tree node not by finding the object, but by receiving it from an event when the user clicks on a node.

Example

Layout.xml:

```
<!-- Template for document's TOC tree -->
<owct:tree-template for="Toc" default="true">
  <owct:tree-node-type name="TocDocNode" expand="yes">
    <owct:tree-node-layout>
      <cssClass>TocDocNode</cssClass>
      <iconUrl>Layout/Images/icon_Document.gif</iconUrl>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>TocDocNodeSelected</cssClass>
      <iconUrl>Layout/Images/icon_DocSelected.gif</iconUrl>
    </owct:tree-node-layout>
  </owct:tree-node-type>
  <owct:tree-node-type name="TocEntryNode" content-
cmd="ViewTocEntryContent" xhtml-template="toc-tree-node">
    <owct:tree-node-layout>
      <cssClass>TocEntryNode</cssClass>
      <iconUrl>Layout/Images/icon_TocEntry.gif</iconUrl>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>TocEntryNodeSelected</cssClass>
    </owct:tree-node-layout>
  </owct:tree-node-type>
  <owct:xhtml-template for="TocDocNode" kind="tree-node-title">
    <span class="TocEntryTitle"><owct:meta-field name="dc:title"
/></span>
  </owct:xhtml-template>
  <owct:xhtml-template for="TocEntryNode" kind="tree-node-title">
    <span class="TocEntryTitle"><owct:property name="TITLE" /></span>
  </owct:xhtml-template>
  <owct:xhtml-template for="TocEntryNode" kind="tree-node-page">
    <owct:xhtml-attribute name="class">TocEntryPageNo</owct:xhtml-
attribute>
    <owct:property name="olv-toc:page-label" />
  </owct:xhtml-template>
</owct:tree-template>
```

See Also

[Tree Overview](#)

TreeNode Class Overview

The TreeNode component functionality is implemented by the Olive.Controls.TreeNode JavaScript class.

The Olive.Controls.TreeNode interface allows access to the TreeNode component events and methods.

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.TreeNode

Implemented Interfaces

[Olive.IUiElements](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

Tree Overview | [TreeNode Overview](#) | [TreeNode Events](#) | [TreeNode Methods](#)

TreeNode Class Commands

TreeNode Commands

Event	Description
Expand	Expands or collapses the selected node

See Also

TreeNode Overview | [Tree Overview](#)

Expand Command

The Expand command expands or collapses the current node, showing or hiding the child nodes. Since the HTML for TreeNode controls is created by the server, the Expand command element must be written in Layout.xml.

Sample Code

The following example uses a Command control to launch the Expand command on a TreeNode

Layout.xml:

```
<owct:xhtml-template for="TocSearchResNode" kind="tree-node-title">
  <span class="SearchResTitle">
    <owct:meta-field name="dc:title" />
  </span>
  <owct:content>
    <div owc:Control="Command" owc:Command="Expand"
class="ExpandLink">More...</div>
  </owct:content>
</owct:xhtml-template>
```

Alternate syntax

object.launchCommand(Expand)

where object refers to a subcontrol of the TreeNode control.

Parameters

none

Remarks

In the example above, the Expand command generates a request to the server to send detailed search results for the entity (TOC search result data).

See Also

[TreeNode Overview](#) | [TreeNode Class](#)

TreeNode Class Events

TreeNode Events

Event	Description
nodeExpanding	This event is fired before a tree node is expanded. It can be used to make custom changes to a node when the user clicks to expand/collapse. Supports bubbling: true
nodeExpanded	This event is fired after a tree node is expanded. It can be used to make custom changes to a node when the user clicks to expand/collapse.

	Supports bubbling: true
treeBoxMouseDown	This event is fired when the +/- icon of a node has been clicked, after the tree node has been expanded/closed.
	Supports bubbling: true

See Also

TreeNode Overview | [Tree Overview](#)

nodeExpanded Event

This event is fired by the `nodeExpand()` method after a tree node is expanded. It is used to make custom changes to a node when the user clicks to expand/collapse.

Properties

The event object passed to the event handler is an instance of the `Olive.Event` class, with the following `expando` property:

1. `Expanding` (boolean) - if the node is being expanded (true) or collapsed (false).

Supports bubbling: true

Example

```
// Handler of the event
function Application_onNodeExpanded(oEvent)
{
    if( oEvent.Expanding )
    {
        // desirable action
    }
}
```

See Also

TreeNode Overview | [Tree Overview](#)

nodeExpanding Event

This event is fired by the `nodeExpand()` method before a tree node is expanded. It is used to make custom changes to a node when the user clicks to expand/collapse.

Properties

The event object passed to the event handler is an instance of the `Olive.Event` class, with the following `expando` property:

1. Expanding (boolean) - if the node is being expanded (true) or collapsed (false).

Supports bubbling: true

Example

```
// Handler of the event
function Application_onNodeExpanding(oEvent)
{
    if( oEvent.Expanding )
    {
        // desirable action
    }
}
```

See Also

TreeNode Overview | [Tree Overview](#)

treeBoxMouseDown Event

This event is fired when the +/- icon of a node (the "tree box") is clicked, after the tree node has been expanded/closed.

Properties

The event object passed to the event handler is an instance of the Olive.Event class, with no expando properties.

Supports bubbling: true

Example

```
// Handler of the event
function Application_onTreeBoxMouseDown(oEvent)
{
    if( oEvent.Expanding )
    {
        // desirable action
    }
}
```

See Also

TreeNode Overview | [Tree Overview](#)

OwcTreeNode Class Methods

OwcTreeNode Methods

The OwcTreeNode class has two methods to change a note's state:

- `nodeExpand` (expand or collapse the node)
- `nodeSelect` (select or unselect the node).

It also has internal methods to get information about the position or state of a node (the methods is... / get...).

Method	Description
<code>nodeExpand(nExpandMode, nDepth)</code>	Expand/collapse the node.
<code>nodeSelect(bSelect)</code>	Select or unselect the node.
<code>isNodeExpandable ()</code>	Returns a boolean, if the node is expandable.
<code>isNodeExpanded ()</code>	Returns a boolean, if the node is expanded.
<code>isNodeSelected ()</code>	Returns a boolean, if the node is selected.
<code>isNodeFirstChild ()</code>	Returns a boolean, if the node is a first child.
<code>isNodeLastChild ()</code>	Returns a boolean, if the node is a last child.
<code>getOutlineMask ()</code>	Returns a bitmap indicating the relative position of the node (first, last, root, expanded, ...). This determines what tree-tracing outline image needs to be placed before the node.
<code>getNodeIconUrl ()</code>	Returns the icon for the node (or, if the node does not have an icon, for this node type).
<code>getNodeIconSelectedUrl ()</code>	Returns the icon for the node (or, if the node does not have an icon, for this node type) when selected.

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

getNodeIconSelectedUrl Method

This method returns the node icon or the node type icon, if it does not have an icon, when it is selected. The return value is a string URL.

Parameters

none

Syntax

```
oNode.getNodeIconSelectedUrl();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

getNodeIconUrl Method

This method returns the node icon (or this node type if it does not have an icon. The return value is a string URL.

Parameters

none

Syntax

```
oNode.getNodeIconUrl();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

getOutlineMask Method

This method returns a bitmap image indicating the relative position of the node (first, last, root, expanded, ...). This determines what tree-tracing outline image needs to be placed before the node.

Parameters

none

Remarks

The return value is a bitmap. The following constants can be used to select bits:

```
var OwcTreeOutline_Expanded = 0x00000000;  
var OwcTreeOutline_Collapsed = 0x00000001;  
var OwcTreeOutline_NonExpandable = 0x00000002;  
var OwcTreeOutline_ExpandMask = 0x00000003;  
var OwcTreeOutline_RootLevel = 0x00000004;  
var OwcTreeOutline_FirstNode = 0x00000008;  
var OwcTreeOutline_LastNode = 0x00000010;  
var OwcTreeOutline_Max = 0x0000001F;
```

Syntax

```
oNode.getOutlineMask();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

isNodeExpandable Method

This method returns a "true" boolean value when the node is expandable (are there child tree nodes, or data not yet downloaded?)

Parameters

none

Syntax

```
oNode.isNodeExpandable();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

isNodeExpandable Method

This method returns a "true" boolean value when the node is expanded.

Parameters

none

Syntax

```
oNode.isNodeExpanded();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

isNodeFirstChild Method

This method returns a "true" boolean value when the node is a first child node (first child nodes have a special tracing line image)

Parameters

none

Syntax

```
oNode.isNodeFirstChild();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

isNodeLastChild Method

This method returns a "true" boolean value when the node is a last child node (last child nodes have a special tracing line image)

Parameters

none

Syntax

```
oNode.isNodeLastChild();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

isNodeSelected Method

This method returns a "true" boolean value when the node is selected

Parameters

none

Syntax

```
oNode.isNodeSelected();
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

nodeExpand Method

This method expands or collapses the node, depending on the parameter.

Parameters

Number - nExpandMode

values:

OwcTreeNodeExpand_Toggle

OwcTreeNodeExpand_Expand

OwcTreeNodeExpand_Collapse

OwcTreeNodeExpand_ExpandAll

OwcTreeNodeExpand_CollapseAll

default - OwcTreeNodeExpand_Toggle

Syntax

```
oNode.nodeExpand( nExpandMode, nDepth );
```

Example: When the user selects a node, expand it also

```
oToc.onStateChanged = myNodeHandler;
function myNodeHandler( oEvent ) {
    var oNode = oEvent.srcObject;
    if( oNode ) {
        oNode.nodeExpand( OwcTreeNodeExpand_Expand );
    }
}
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

nodeSelect Method

This method expands or collapses the node, depending on the parameter.

Parameters

bSelect - Boolean - Select (true) or unselect (false)

Syntax

```
oNode.nodeSelect( bSelect );
```

Example: When the user clicks on the +/- to expand/collapse, select the node also

```
oToc.onNodeExpandChanged = myNodeHandler;//do this when the TOC is
initialized
function myNodeHandler( oEvent ) {
    var oNode = oEvent.srcObject;
    if( oNode ) {
        oNode.nodeSelect( true );
    }
}
```

See Also

TreeNode Overview | [TreeNode Class](#) | [Tree Overview](#)

UiElemType Overview

This class' full name is Olive.UiElemType.

This class is a base class for UI element types classes used in controls. It includes general functionality to bind UI elements and update their states.

6 UI element type classes are inherited from the UiElemType base class, which are used for Olive.Controls.TabItem class represented TabItem control and Olive.Controls.TreeNodeType class:

- [Olive.UiElemType.TabItemPos](#)
- [Olive.UiElemType.TreeOutline](#)
- [Olive.UiElemType.TreeNodeSelect](#)
- [Olive.UiElemType.TreeIcon](#)
- [Olive.UiElemType.TreeBox](#)
- [Olive.UiElemType.TreeSubNodes](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[UiElemType Methods](#) | [UiElemType Properties](#)

UiElemType Methods

UiElemTypespecific Methods

Event	Description
bindUiElement	Calls bindUiElementCommon and, if defined, bindUiElementCustom methods
bindUiElementCommon	Binds UI element to parent object and applies className property, if it is defined.
bindUiElementCustom	Custom defined procedure of binding. Undefined for the base class (Olive.UiElemType)
updateUiElemState	Updates UI element state. The method is for overriding in inherited classes.

See Also

[UiElemType Overview](#) | [UiElemType Properties](#)

bindUiElement Method

This method calls the bindUiElementCommon() and bindUiElementCustom() methods if the former is defined. The method is called during HTML parsing.

Parameters

oUiElement - An HTML object with UI element for binding

oControl - An object of a control of parent control for UI element object

Syntax

```
oUiElemType.bindUiElement(oUiElement, oControl);
```

Example

```
// bind oCurHtmlElement of UI element type to oParentControl  
oUiElemType.bindUiElement(oCurHtmlElement, oParentControl);
```

See Also

[UiElemType Overview](#) | [UiElemType Methods](#)

bindUiElementCommon Method

This method is a common part of binding for all UI element types. It saves the className property, if defined, and assigns owcUiParent property. The method is called in bindUiElement() method.

Parameters

oControl - An object of a control of parent control for UI element object

oUiElement - An HTML object with UI element for binding

Syntax

```
oUiElemType.bindUiElementCommon(oControl, oUiElement);
```

Example

```
// assign properties for UI element during binding  
oUiElemType.bindUiElementCommon(oParentControl, oCurHtmlElement);
```

See Also

[UiElemType Overview](#) | [UiElemType Methods](#)

bindUiElementCustom Method

This method is a custom part of binding for all UI element types. It is overridden by inherited classes. The method is called in bindUiElement() method.

The method is undefined for the base class.

Parameters

oControl - An object of a control of parent control for UI element object

oUiElement - An HTML object with UI element for binding

Syntax

```
oUiElemType.bindUiElementCustom(oControl, oUiElement);
```

Example

```
// custom binding
oUiElemType.bindUiElementCustom(oParentControl, oCurHtmlElement);
```

See Also

[UiElemType Overview](#) | [UiElemType Methods](#)

updateUiElemState Method

This method is dedicated to update the UI element state. It should be overridden by inherited classes. The method is undefined for the base class.

Parameters

oControl - An object of a parent control for UI element object

oUiElem - An HTML object with the UI element

dwNewState - A new state to apply to UI element

dwPrevState - A previous state of the UI element

dwChangedState - A changed state of the UI element

Syntax

```
oUiElemType.updateUiElemState(oControl, oUiElem, dwNewState,
dwPrevState, dwChangedState);
```

Example

```
// update UI element state: set Visible state
oUiElemType.updateUiElemState(oParentControl, oCurHtmlElement,
Olive.IState.State.Visible, Olive.IState.State.Invisible);
```

See Also

[UiElemType Overview](#) | [UiElemType Methods](#)

UiElemType Properties

UiElemType-specified properties

Property	Description
----------	-------------

className	Holds a CSS class name defined for the UI element
owcUIParent	Hold a reference to the parent control, defined during parsing of the HTML page

See Also

[UiElemType Overview](#) | [UiElemType Methods](#)

className Property

This property holds a CSS class name defined for the UI element in the HTML page.

Syntax

Data Type	string
Default value	null

Remarks

This property is assigned during parsing an HTML page and binding the UI element.

Sample Code

```
// get a CSS class name of the UI element
var sClassName = oUiElemType.className;
```

See Also

[UiElemType Overview](#) | [UiElemType Properties](#)

owcUIParent Property

This property reference the parent control as defined while parsing the HTML page

Syntax

Data Type	SDK control object
Default value	null

Remarks

The property is assigned during parsing HTML page and binding the UI element

Sample Code

```
// get a reference to the parent control
var oParentControl = oUiElemType.owcUIParent;
```

See Also

[UiElemType Overview](#) | [UiElemType Properties](#)

UiElemType.TabItemPos Overview

The full name of the class is Olive.UiElemType.TabItemPos class.

This class represents a user interface item within a tab strip control. The class inherits a base functionality of the Olive.UiElemType class and overrides updateUiElemState method for updating a visual interface for tab items according to their position (first/last).

Inheritance Hierarchy

Olive.UiElemType

Olive.UiElemType.TabItemPos

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[UiElemType.TableItemPos Methods](#) | [UiElemType.TableItemPos Properties](#)

UiElemType.TabItemPos Methods

Methods inherited from the base class [Olive.UiElemType](#)

Event	Description
bindUiElement	Calls bindUiElementCommon, and if defined, bindUiElementCustom methods
bindUiElementCommon	Binds UI element to parent object and applies className property, if it is defined.

Overridden Methods of the base class [Olive.UiElemType](#)

Event	Description
updateUiElemState	Updates UI element state: adds or removes FirstTab and LastTab CSS classes according to the position of the tab item.

See Also

[UiElemType.TableItemPos Overview](#) | [UiElemType.TableItemPos Properties](#)

updateUiElemState Method

This method updates the UI element state of the tab items according to their position. FirstTab and LastTab CSS classes are added or removed to tab items according their position.

Parameters

oControl - An object of a parent control for UI element object

oUiElem - An HTML object with the UI element

dwNewState - A new state to apply to UI element. The parameter is unused in the method

dwPrevState - A previous state of the UI element. The parameter is unused in the method

dwChangedState - A changed state of the UI element. The parameter is unused in the method

Syntax

```
oTabItemPos.updateUiElemState(oControl, oUiElem, dwNewState,  
dwPrevState, dwChangedState);
```

Example

```
// update UI element state:  
// add FirstTab CSS class if UI element is attached to a first tab  
// item, remove the class in other case  
// add LastTab CSS class if UI element is attached to a last tab item,  
// remove the class in other case  
oTabItemPos.updateUiElemState(oTabItemControl, oHtmlUiElement);
```

See Also

[UiElemType.TableItemPos Overview](#) | [UiElemType.TableItemPos Methods](#)

UiElemType.TabItemPos Properties

Properties inherited from the base class [Olive.UiElemType](#)

Property	Description
className	Holds a CSS class name defined for the UI element in HTML page
owcUiParent	Holds a reference to the parent control, defined when parsing the HTML page

See Also

[UiElemType.TableItemPos Overview](#) | [UiElemType.TableItemPos Methods](#)

UiElemType.TreeBox Overview

The full name of this class is Olive.UiElemType.TreeBox class.

It represents a tree box icon, which expands or collapses nodes in a tree control. The class inherits a base functionality of the [Olive.UiElemType](#) class and overrides its two methods:

updateUiElemState() - Method for adding or removing CSS classes to the tree box according the node type (expandable or not expandable)

bindUiElementCustom() - Method for attaching a DHTML event handler to a click event. The class has its own CSS class treeNodeOutline, defined in the SDK.

Inheritance Hierarchy

Olive.UiElemType

Olive.UiElemType.TreeBox

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[UiElemType.TreeBox Methods](#) | [UiElemType.TreeBox Properties](#)

UiElemType.TreeBox Methods

UiElemType.TabItemPos Methods

Methods inherited from the base class [Olive.UiElemType](#)

Event	Description
bindUiElement	Calls bindUiElementCommon and, if defined, bindUiElementCustom methods
bindUiElementCommon	Binds UI element to parent object and applies className property, if it is defined

Overridden Methods of the base class [Olive.UiElemType](#)

Event	Description
updateUiElemState	Updates UI element state: adds(removes) CSS class defined for tree box icon if the tree node is expandable(non expandable)
bindUiElementCustom	Custom defined binding procedure: attaches DHTML event handlers

See Also

[UiElemType.TreeBox Overview](#) | [UiElemType.TreeBox Properties](#)

bindUiElementCustom Method

This method attaches a DHTML event handler for the click event.

Parameters

oControl - An object of a parent control for UI element object

oUiElement - An HTML object with the UI element

Syntax

```
oTreeBox.bindUiElementCustom(oControl, oUiElement);
```

Example

```
// bind UI element to oTreeNodeControl  
// attach DHTML event handler for clickevent  
oTreeBox.bindUiElementCustom(oTreeNodeControl, oHtmlUiElement);
```

See Also

[UiElemType.TreeBox Overview](#) | [UiElemType.TreeBox Methods](#)

updateUiElemState Method

The method updates UI element state of tree nodes by adding or removing a CSS classes with or without tree box. If the node is expandable, the method attaches DHTML event handler for mousedown event. If the node is non expandable, then the method detaches the event handler.

Parameters

oControl - An object of a parent control for UI element object

oUiElem - An HTML object with the UI element

dwNewState - A new state to apply to UI element. The parameter is unused in the method

dwPrevState - A previous state of the UI element. The parameter is unused in the method

dwChangedState - A changed state of the UI element. The parameter is unused in the method

Syntax

```
oTreeBox.updateUiElemState(oControl, oUiElem, dwNewState, dwPrevState,  
dwChangedState);
```

Example

```
// update UI element state:
// add/remove CSS classes with/without tree box icon
oTreeBox.updateUiElemState(oTreeNodeControl, oHtmlUiElement);
```

See Also

[UiElemType.TreeBox Overview](#) | [UiElemType.TreeBox Methods](#)

UiElemType.TreeBox Properties

UiElemType.TabItemPos Properties

Properties inherited from the base class [Olive.UiElemType](#)

Property	Description
className	Holds a CSS class name defined for the UI element in HTML page. It is set to treeNodeOutline CSS class, defined in the SDK
owcUiParent	Holds a reference to the parent control, defined when parsing the HTML page

See Also

[UiElemType.TreeBox Overview](#) | [UiElemType.TreeBox Methods](#)

className Property

This property holds the CSS class name defined for the UI element in HTML page. The property is set to **treeNodeOutline** CSS class, defined in the SDK.

Syntax

Data Type	string
Default value	treeNodeOutline

Remarks

This property is assigned during parsing the HTML page and binding the UI element

Sample Code

```
// get a CSS class name of the UI element
var sClassName = oTreeBox.className;
```


See Also

[UiElemType.TreeBox Overview](#) | [UiElemType.TreeBox Properties](#)

UiElemType.TreeIcon Overview

The full name of the class is Olive.UiElemType.TreeIcon class.

This class represents a tree node icon. It inherits a base functionality of the Olive.UiElemType class and overrides updateUiElemState method for updating an icon for a tree node.

Inheritance Hierarchy

Olive.UiElemType

Olive.UiElemType.TreeIcon

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[UiElemType.TreeIcon Methods](#) | [UiElemType.TreeIcon](#) Properties

UiElemType.TreeIcon Methods

Methods inherited from the base class [Olive.UiElemType](#)

Event	Description
bindUiElement	Calls bindUiElementCommon and, if defined, bindUiElementCustom methods
bindUiElementCommon	Binds UI element to parent object and applies className property, if it is defined

Overridden Methods of the base class [Olive.UiElemType](#)

Event	Description
updateUiElemState	Updates UI element state: adds or updates an HTML element with icon for a tree node control.

See Also

updateUiElemState Method

This method updates the UI element state of the tree node icon when a URL to the image is defined. Different icons can be defined for selected and not selected nodes.

Parameters

oControl - An object of a parent control for UI element object

oUiElem - An HTML object with the UI element

dwNewState - A new state to apply to UI element. The method checks selected/unselected state.

dwPrevState - A previous state of the UI element. The parameter is unused in the method

dwChangedState - A changed state of the UI element. The parameter is unused in the method

Syntax

```
oTreeIcon.updateUiElemState(oControl, oUiElem, dwNewState, dwPrevState, dwChangedState);
```

Example

```
// update UI element state:  
// applies selected icon if the node is selected and non selected icon otherwise  
oTreeIcon.updateUiElemState(oTreeNodeControl, oHtmlUiElement, Olive.IState.State.Selected);
```

See Also

[UiElemType.TreeIcon Methods](#) | [UiElemType.TreeIcon Overview](#)

UiElemType.TreeIcon Properties

Properties inherited from the base class [Olive.UiElemType](#)

Property	Description
className	Holds a CSS class name defined for the UI element in HTML page
owcUiParent	Hold a reference to the parent control, defined during parsing of the HTML page

See Also

[UiElemType.TreeIcon Overview](#) | [UiElemType.TreeIcon Methods](#)

UiElemType.TreeNodeSelect Overview

The full name of the class is Olive.UiElemType.TreeNodeSelect class.

It represents a tree node selectable area. The class inherits a base functionality of the Olive.UiElemTypeclass and overrides its two methods:

updateUiElemState() method for adding or removing CSS classes defined for selected nodes

bindUiElementCustom()method for attaching DHTML event handlers.

The class has its own CSS class `treeNodeSelectArea`, defined in the SDK.

Inheritance Hierarchy

Olive.UiElemType

`Olive.UiElemType.TreeNodeSelect`

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[UiElemType.TreeNodeSelect Methods](#) | [UiElemType.TreeNodeSelect](#) Properties

UiElemType.TreeNodeSelect Methods

Methods inherited from the base class [Olive.UiElemType](#)

Event	Description
bindUiElement	Calls <code>bindUiElementCommon</code> and, if defined, <code>bindUiElementCustom</code> methods
bindUiElementCommon	Binds UI element to parent object and applies <code>className</code> property, if it is defined

Overridden Methods of the base class [Olive.UiElemType](#)

Event	Description
updateUiElemState	Updates UI element state: adds or removes CSS class defined for selected tree nodes.
bindUiElementCustom	Custom defined procedure of binding: attaches DHTML event handlers.

See Also

[UiElemType.TreeNodeSelect Overview](#) | [UiElemType.TreeNodeSelect](#) Properties

bindUiElementCustom Method

This method attaches DHTML event handlers for following events:

- `mousedown`
- `mouseover`

- selectstart

Parameters

oControl - An object of a parent control for UI element object

oUiElement - An HTML object with the UI element

Syntax

```
oTreeNodeSelect.bindUiElementCustom(oControl, oUiElement);
```

Example

```
// bind UI element to oTreeNodeControl
// attach DHTML event handlers
oTreeNodeSelect.bindUiElementCustom(oTreeNodeControl, oHtmlUiElement);
```

See Also

[UiElemType.TreeNodeSelect Methods](#) | UiElemType.[TreeNodeSelect](#) Overview

updateUiElemState Method

This method updates the tree nodes' UI element state according to the selection status. It adds or removes a CSS treeNodeSelected class to the UI element or another CSS class defined for a specific tree node type.

Parameters

oControl - An object of a parent control for UI element object

oUiElem - An HTML object with the UI element

dwNewState - A new state to apply to UI element. The parameter is unused in the method

dwPrevState - A previous state of the UI element. The parameter is unused in the method

dwChangedState - A changed state of the UI element. The parameter is unused in the method

Syntax

```
oTreeNodeSelect.updateUiElemState(oControl, oUiElem, dwNewState,
dwPrevState, dwChangedState);
```

Example

```
// update UI element state:
// add/remove treeNodeSelected class for the UI element
oTreeNodeSelect.updateUiElemState(oTreeNodeControl, oHtmlUiElement);
```

See Also

[UiElemType.TreeNodeSelect Methods](#) | UiElemType.[TreeNodeSelect](#) Overview

UiElemType.TreeNodeSelect Properties

Properties inherited from the base class [Olive.UiElemType](#)

Property	Description
className	Holds a CSS class name defined for the UI element in HTML page. Is set to treeNodeSelectArea CSS class, defined in SDK
owcUiParent	Hold a reference to the parent control, defined during parsing of the HTML page

See Also

[UiElemType.TreeNodeSelect Overview](#) | UiElemType.[TreeNodeSelect Methods](#)

className Property

This property holds a CSS class name defined for the UI element in HTML page. The property is set to **treeNodeSelectArea** CSS class, defined in the SDK

Syntax

Data Type	string
Default value	treeNodeSelectArea

Remarks

The property is assigned during parsing HTML page and binding the UI element

Sample Code

```
// get a CSS class name of the UI element
var sClassName = oTreeNodeSelect.className;
```

See Also

[UiElemType.TreeNodeSelect Overview](#) | UiElemType.[TreeNodeSelect](#) Properties

UiElemType.TreeOutline Overview

The full name of the class is Olive.UiElemType.TreeOutline class.

It represents a tree node outline. The class inherits a base functionality of the Olive.UiElemType class and overrides updateUiElemState method to update the visual interface of tree nodes according to their state (expanded, collapsed, last child). The class has its own CSS class treeNodeOutline, defined in the SDK.

Inheritance Hierarchy

Olive.UiElemType

Olive.UiElemType.TreeOutline

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[UiElemType.TreeOutline Methods](#) | UiElemType.[TreeOutline](#) Properties

UiElemType.TreeOutline Methods

Methods inherited from the base class [Olive.UiElemType](#)

Event	Description
bindUiElement	Calls bindUiElementCommon and, if defined, bindUiElementCustom methods
bindUiElementCommon	Binds UI element to parent object and applies className property, if it is defined

Overridden Methods of the base class [Olive.UiElemType](#)

Event	Description
updateUiElemState	Updates UI element state: adds or removes CSS class with icon toe for expanded/collapsed and not last nodes

See Also

[UiElemType.TreeOutline Overview](#) | [UiElemType.TreeOutline Properties](#)

updateUiElemState Method

This method updates tree nodes' UI element state according to following states:

- expanded
- collapsed
- last node

It adds or removes the icon to represent a vertical line.

Parameters

oControl - An object of a parent control for UI element object

oUiElem - An HTML object with the UI element

dwNewState - A new state to apply to UI element. The parameter is unused in the method

dwPrevState - A previous state of the UI element. The parameter is unused in the method

dwChangedState - A changed state of the UI element. The parameter is unused in the method

Syntax

```
oTreeOutline.updateUiElemState(oControl, oUiElem, dwNewState, dwPrevState, dwChangedState);
```

Example

```
// update UI element state:  
// add toe icon if the UI element is for tree nodes that can be  
// expanded/collapsed and are not last nodes in a tree  
oTreeOutline.updateUiElemState(oTreeNodeControl, oHtmlUiElement);
```

See Also

[UiElemType.TreeOutline Overview](#) | [UiElemType.TreeOutline Methods](#)

UiElemType.TreeOutline Properties

Properties inherited from the base class [Olive.UiElemType](#)

Property	Description
className	Holds a CSS class name defined for the UI element in HTML page. Is set to treeNodeOutline CSS class, defined in SDK
owcUiParent	Hold a reference to the parent control, defined during parsing of the HTML page

See Also

[UiElemType.TreeOutline Overview](#) | [UiElemType.TreeOutline Methods](#)

className Property

This property holds a CSS class name defined for the UI element in HTML page. It is set to treeNodeOutline CSS class, defined in SDK

Syntax

Data Type	string
Default value	treeNodeOutline

Remarks

The property is assigned during parsing HTML page and binding the UI element

Sample Code

```
// get a CSS class name of the UI element
var sClassName = oTreeOutline.className;
```

See Also

[UiElemType.TreeOutline Overview](#) | [UiElemType.TreeOutline Properties](#)

UiElemType.TreeSubNodes Overview

The full name of the class is **Olive.UiElemType.TreeSubNodes**class.

This class represents a tree node container. The class inherits a base functionality of the Olive.UiElemType class and overrides updateUiElemState method for making subnodes visible/invisible.

Inheritance Hierarchy

Olive.UiElemType

Olive.UiElemType.TreeSubNodes

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[UiElemType.TreeSubNodes Properties](#) | [UiElemType.TreeSubNodes MethodsD:\Web SDK API\MadCap\Web SDK API Reference\Content\TreeNode Overview.htm](#)

UiElemType.TreeSubNodes Methods

Methods inherited from the base class [Olive.UiElemType](#)

Event	Description
bindUiElement	Calls bindUiElementCommon, and if defined, bindUiElementCustom methods
bindUiElementCommon	
	Binds UI element to parent object and applies className property, if it is defined.

Overridden Methods of the base class [Olive.UiElemType](#)

Event	Description
updateUiElemState	Updates UI element state: adds or removes FirstTab and LastTab CSS classes

according to the position of the tab item.

See Also

[UiElemType.TreeSubNodes Properties](#) | [UiElemType.TreeSubNodes OverviewD:\Web SDK API\MadCap\Web SDK API Reference\Content\WebSDK\TreeNode Overview.htm](#)

updateUiElemState Method

This method updates the UI element state: adds or removes treeContentHidden CSS class to make tree subnodes visible/invisible when the user expands/collapses the nodes.

Parameters

oControl - An object of a parent control for UI element object

oUiElem - An HTML object with the UI element

dwNewState - A new state to apply to UI element. The parameter is unused in the method

dwPrevState - A previous state of the UI element. The parameter is unused in the method

dwChangedState - A changed state of the UI element. The parameter is unused in the method

Syntax

```
oTreeSubNodes.updateUiElemState(oControl, oUiElem, dwNewState, dwPrevState, dwChangedState);
```

Example

```
// update UI element state:  
// add/remove treeContentHidden CSS class to the UI element  
oTreeSubNodes.updateUiElemState(oTreeNodeControl, oHtmlUiElement);
```

See Also

[UiElemType.TreeSubNodes Overview](#) | [UiElemType.TreeSubNodes MethodsD:\Web SDK API\MadCap\Web SDK API Reference\Content\WebSDK\TreeNode Overview.htm](#)

UiElemType.TreeSubNodes Properties

Properties inherited from the base class [Olive.UiElemType](#)

Property	Description
className	Holds a CSS class name defined for the UI element in HTML page

owcUiParent	Hold a reference to the parent control, defined during parsing of the HTML page
-----------------------------	---

See Also

UiElemType.TreeSubNodes Overview | UiElemType.TreeSubNodes MethodsD:\Web SDK API\MadCap\Web SDK API Reference\Content\WebSDK\TreeNode Overview.htm

Value Overview

The Value class is a base class from which all SDK value controls are inherited. A value control is attached to an HTML element that contains a simple value, such as a number, string, or date. Its API provides the basic functionality to exchange the HTML element's value with the SDK/Javascript object's value: reading and writing the value to an HTML element to input and/or display the value. It also includes methods to format, validate, and compare values, and events to report that a changed or invalid value.

PUBLIC API

ATTRIBUTES

- **owc:DataType:** object, string, boolean, number, date, year, month, day, email [address], email-list [addresses separated by ;], sort-by
- **owc:Format:** especially used for dates; for example, "%m/%d/%Y" for Date, "%B" for month name, "%A" for day name]
- **owc:HtmlValue (boolean):** is the value of the HTML element its innerHTML (true) or does it have a value property (false)

METHODS

- **getValue():** returns the internal value of the control
- **setValue(oValue, bDoNotUpdateControl):** parses and validates the value, updates the control's display, fires event valueChanged
- **getFormat():** returns the format of the control
- **setFormat(sFormat, bUpdateControl):** assigns a format to the control, updates the control's display
- **formatValue(oValue):** returns the value, formatted by the control's format setting
- **parseValue(oValue):** returns the value, parsed according to the data type and formatting
- **validateValue(sFormat, bUpdateControl):** returns boolean, true=valid, false=invalid
- **compareValue(oPrevValue, oNewValue):** tests if they are equal
- **updateControl():** assigns the internal value to the HTML element
- **updateData():** assigns the HTML value to the internal value
- **getHtmlValue(bParseValue):** reads the HTML value
- **setHtmlValue(oValue):** writes the value to the HTML element

- `reportValidationError(oValue)`: fires an event with the event properties value and error.

EVENTS

- `valueChanged`
- `validationError`

The Value control's functionality comes from the [IValue](#) interface. For details, see the documentation on the methods and events of the IValue interface.

See Also

[IValue Overview](#) | [IValue Methods](#) | [IValue Events](#) | [IComplexValue Overview](#) |

Value Class Overview

The Value control functionality is implemented by `Olive.Controls.Value` JavaScript class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Value

Implemented Interfaces

[Olive.IValue](#)

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[IValue Interface](#) | [Calendar Overview](#)

Viewer Overview

The Viewer component consists of a base class that is not used directly, but serves as a common base class for the two document viewer components, `DocViewer` (the HTML-based document viewer) and `FlashViewer` (the Flash-based document viewer). It provides the basic viewer functionality of opening and paging through documents by page number, page label, or entity. It also opens documents according to search criteria, displaying search hits and moving from hit to hit.

NOTE:

The inheritance of Viewer functionality by `FlashViewer` is incomplete. The Viewer interface is currently available only in `DocViewer`.

See Also

[DocViewer Overview](#) | [FlashViewer Overview](#) | [Viewer Class](#)

OwcViewer Class Overview

The Viewer base class functionality is implemented by the [Olive.Controls.Viewer](#) JavaScript class.

The Olive.Controls.Viewer interface provides access to Viewer methods and events.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.Control

Olive.Controls.Viewer

Implemented Interfaces

Olive.IDataBound

Requirements

Script Files	OwcBind.js
--------------	------------

See Also

[Viewer Methods](#) | [Viewer Overview](#)

OwcViewer Class Methods

OwcViewer Methods

Method	Description
displayDocumentByPageNumber(baseHref, pageNo)	Opens a document at a specified page. Arguments: string baseHref - Href path to the document. pageNo - Page number (string or number)
displayDocumentByPageNumberLabel(baseHref, pageNo, pageLabel)	Opens a document at a page that is specified by page number and page label. Arguments: string baseHref - Href path to the document. pageNo - Page number (string or number)

displayDocumentByPageNumberFromSearch (baseHref, pageNo, collName, searchStr, entityID)	string pageLabel - Label of the page Opens a document at a specified page number, searching for a specified string (in order that search hits can be highlighted). Arguments: string baseHref - Href path to the document. pageNo - Page number (string or number) string collName - The search collection to use in searching this document. string searchStr - The string to search for. string entityID - The entity in which the search is to be performed.
displayDocumentByPageNumberLabelFromSearch (baseHref, pageNo, pageLabel, collName, searchStr, entityID)	Opens a document at a specified page number and page label, searching for a specified string (in order that search hits can be highlighted). Arguments: string baseHref - Href path to the document. pageNo - Page number (string or number) string pageLabel - Label of the page string collName - The search collection to use in searching this document. string searchStr - The string to search for. string entityID - The entity in which the search is to be performed.
viewerGotoPageByNumber(pageNo)	Go to a specified page (by page number) in the current document. Argument: pageNo - Page number (string or number)
viewerGotoPageByLabel(pageLabel)	Go to a specified page (by page label) in the current document. Argument: string pageLabel - Label of the page
viewerReloadCurPageByNumber(pageNo)	Reloads a specified page (by page number) in the current document. NOTE: This function is not different from viewerGotoPageByNumber().

	Argument: pageNo - Page number (string or number)
viewerReloadCurPageByLabel(pageLabel)	Reloads a specified page (by page label) in the current document.
	NOTE: This function is not different from viewerGotoPageByLabel().
	Argument: string pageLabel - Label of the page
viewerGotoEntity(sEntityId)	Go to the specified entity in the current document, paging to a different page if necessary.
	Argument: string entityId - The id of the entity.
viewerGotoPrevHit()	Go to the previous hit in this entity (move the highlighting to the previous hit). If the previous hit is not on the selected page, the page where the previous hit is located is loaded.
	Arguments: (none)
viewerGotoNextHit()	Go to the next hit in this entity (move the highlighting to the next hit). If the next hit is not on the selected page, the page where the next hit is located is loaded.
	Arguments: (none)

See Also

[Viewer Overview](#) | [Viewer Events](#) | [DocViewer Overview](#)

displayDocumentByPageNumber Method

This method opens a document at a specified page.

Parameters

baseHref - String: Href path to the document

pageNo - Page number (string or number)

Syntax

```
displayDocumentByPageNumber( baseHref, pageNo )
```

Example

```
displayDocumentByPageNumber( "EVT/2004/02/17", 2 )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

displayDocumentByPageNumberFromSearch Method

This method opens a document at a specified page number, searching for a specified string (to highlight search hits).

Parameters

baseHref - Href path to the document

pageNo - Page number (string or number)

collName - String: The search collection to use in searching this document

searchStr - String: The string to search for

entityId - String: The entity in which the search is performed

Syntax

```
displayDocumentByPageNumberFromSearch( baseHref, pageNo, collName, searchStr, entityID )
```

Example

```
displayDocumentByPageNumberFromSearch( "EVT/2004/02/17", 2, "ETV_EP", "political", "Ar00200" )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

displayDocumentByPageNumberLabel Method

This method opens a document at a page that is specified by page number and page label.

Parameters

baseHref - String: Href path to the document

pageNo - Page number (string or number)

pageLabel - Page label (string)

Syntax

```
displayDocumentByPageNumberLabel( baseHref, pageNo, pageLabel )
```

Example

```
displayDocumentByPageNumberLabel( "EVT/2004/02/17", 2, "A2" )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

displayDocumentByPageNumberLabelFromSearch Method

Open a document at a specified page number and page label, searching for a specified string (in order that search hits can be highlighted).

Parameters

baseHref - String: Href path to the document

pageNo - Page number (string or number)

pageLabel - Label of the page

collName - String: The search collection to use in searching this document

searchStr - String: The string to search for

entityID - The entity in which the search is to be performed

Syntax

```
displayDocumentByPageNumberLabelFromSearch( baseHref, pageNo, pageLabel,  
collName, searchStr, entityID )
```

Example

```
displayDocumentByPageNumberLabelFromSearch( "EVT/2004/02/17", 2, "A2", EVT_EP",  
"political", "A00200" )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

viewerGotoEntity Method

This method goes to the specified entity in the current document, paging to a different page if necessary.

Parameters

entityID - String: The entity in which the search is to be performed

Syntax

```
viewerGotoEntity( entityID )
```


Example

```
viewerGotoEntity( "A00200" )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

viewerGotoNextHit Method

This method goes to the next hit in this entity (moves highlighting to the next hit). If the next hit is not on the selected page, it loads the page where the next hit is located.

Parameters

(none)

Syntax

```
viewerGotoNextHit()
```

Example

```
viewerGotoNextHit()
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

viewerGotoPageByLabel Method

This method goes to a specified page (by page label) in the current document.

Parameters

pageLabel - Label of the page

Syntax

```
viewerGotoPageByLabel( pageLabel )
```

Example

```
viewerGotoPageByLabel( "A2" )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

viewerGotoPageByNumber Method

This method goes to a specified page (by page number) in the current document.

Parameters

pageNo - Page number (string or number)

Syntax

```
viewerGotoPageByNumber( pageNo )
```

Example

```
viewerGotoPageByNumber( 2 )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

viewerGotoPrevHit Method

This method goes to the previous hit in this entity (moves highlighting to the previous hit). If the previous hit is not on the selected page, it loads the page where the previous hit is located.

Parameters

(none)

Syntax

```
viewerGotoPrevHit()
```

Example

```
viewerGotoPrevHit()
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

viewerReloadCurPageByLabel Method

This method reloads a page (specified by page label) in the current document.

NOTE: This method's function is the same as `viewerGotoPageByLabel()`.

Parameters

pageLabel - Label of the page

Syntax

```
viewerReloadCurPageByLabel( pageLabel )
```

Example

```
viewerReloadCurPageByLabel( "A2" )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

viewerReloadCurPageByNumber Method

This method reloads the current page (specified by page number) in the current document.

NOTE: This method's function is the as viewerGotoPageByNumber().

Parameters

pageNo - Page number (string or number)

Syntax

```
viewerReloadCurPageByNumber( pageNo )
```

Example

```
viewerReloadCurPageByNumber( 2 )
```

See Also

[Viewer Overview](#) | [Viewer Methods](#) | [DocViewer Overview](#)

OwcViewer Class Properties

OwcViewer properties

Property	Description
searchStr	Used to request hits from the server according to search pattern
collName	Used to request hits from the server according to the name of the search collection
entityID	Used to request hits from the server according to Entity ID, and create a highlighted entity
curPage	Currently loaded page; First page is double pages
numPagesDisplayed	Number of loaded pages 1 or 2
firstPage	First page of the document

lastPage	Last page of the document
baseLegalPage	Number of requested pages. When the legal page range is used (range of permitted pages), the Control gets from the server this page and the pages before and after it, as defined by the firstLegalPage and lastLegalPage properties
firstLegalPage	First permitted page in the legal page range (range of permitted pages)
lastLegalPage	Last permitted page in the legal page range (range of permitted pages)
bRetrieveHits	Flag that defines if the Control should get hits
language	Language of document
langDirection	Direction of the language
pageRes	Resolution of the page
imagesRes	Resolution of the images
firstPageWidth	Width of the page (the first page for double pages)
secondPageWidth	Width of the second page for double pages (if the second page exists)
totalPageWidth	Calculated sum of the width of the first and the second pages
pageHeight	Height of the page
loadReqParams	Object that stores the persistent scrolling parameters

See Also

[DocViewer Overview](#) | [DocViewer Class Overview](#)

baseLegalPage

This property defines the number of requested pages. When using a legal page range (range of permitted pages), the Control gets from the server this page and the pages before and after it, as defined by the firstLegalPage and lastLegalPage properties. This is an internal property.

See Also

[Viewer Overview](#) | [Viewer Class Overview](#) | [Viewer Properties](#) | [Viewer Methods](#)

bRetrieveHits

This property is a flag that defines if the Control should get hits. This is an internal property.

Type - boolean

See Also

[Viewer Overview](#) | [Viewer Class Overview](#) | [Viewer Properties](#) | [Viewer Methods](#)

collName

This property is used to request hits from server according to the search collection's name. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

curPage

This property defines the currently loaded page or the first page in a double page. This is an internal property.

See Also

[Viewer Overview](#) | [Viewer Class Overview](#) | [Viewer Properties](#) | [Viewer Methods](#)

entityID

This property is used to request hits from the server according to Entity ID, and create a highlighted entity. This is an internal property.

See Also

[Viewer Overview](#) | [Viewer Class Overview](#) | [Viewer Properties](#) | [Viewer Methods](#)

firstLegalPage

This property defines the first permitted page in a legal page range (range of permitted pages). This is an internal property.

See Also

[Viewer Overview](#) | [Viewer Class Overview](#) | [Viewer Properties](#) | [Viewer Methods](#)

firstPage

This property defines the document's first page; its value is always 1. This is an internal property.

See Also

[Viewer Overview](#) | [Viewer Class Overview](#) | [Viewer Properties](#) | [Viewer Methods](#)

firstPageWidth

This property defines page width (first page for double pages). This is an internal property.

See Also

[Viewer Overview](#) | [Viewer Class Overview](#) | [Viewer Properties](#) | [Viewer Methods](#)

imagesRes

This property keeps the image resolution as defined in the Repository (ViewPoint Warehouse). Together with the page resolution, this defines the page scale that is used for the proper hits layout. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

langDirection

This property defines the direction of the used language: "ltr" or "rtl".

Syntax

```
var langDirection = docViewerControl.langDirection;  
docViewerControl.langDirection = rtl;
```

Sample Code

```
var langDirection = docViewerControl.langDirection;  
or  
docViewerControl.langDirection = rtl;
```

Remarks

This property's default value is ltr. Its value can be changed to right-to-left page orientation for Hebrew and Arabic, and this also changes the order of double pages.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

language

This property keeps the document's language as defined in the Repository (ViewPoint Warehouse). This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

lastLegalPage

This property In case of using the legal page range (range of permitted pages) its the last permitted page. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

lastPage

This property defines the last page of document. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

loadReqParams

Object that stores the persistent scrolling parameters. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

numPagesDisplayed

This property defines the number of loaded pages: 1 or 2. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

pageHeight

This property defines page height. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

pageRes

This property defines page resolution as defined in Repository. Together with the image resolution, this defines the page scale that is used for proper hits layout. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

searchStr

This property is used to request hits from server according to a search pattern. This value is stored in the Control as the last used search pattern parameter from the Controls method. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

secondPageWidth

This property defines the width of the second page in a double page spread (if one exists, else it is zero). This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

totalPageWidth

This property calculates the sum width of the first and the second pages. This is an internal property.

See Also

[Viewer Control Overview](#) | [OwcViewer Control Class Overview](#) | [OwcViewer Control Class Properties](#) | [OwcViewer Control Class Methods](#)

WebAppBase Class Overview

The Web Application Base (WebAppBase) class is a base application class with general functionality that can be expanded and overridden in a custom application class.

The application may have only one instance of the application object. To have access to this singleton application object, define a global function OwcGetApplication in an application-specific file, which will return the application object.

For detail information about functionality, please refer to the documentation for each method, event and property in this class.

Inheritance Hierarchy

Olive.EventsSource

Olive.Object

Olive.CmdTarget

Olive.Controls.WebAppBase

Requirements

Script Files	OwcAppBase.js
---------------------	---------------

See Also

[WebAppBase Events](#) | [WebAppBase Methods](#) | [WebAppBase Properties](#)

WebAppBase Class Events

WebAppBase Events

Event	Description
applicationReady	Event fired when the application is ready: all settings are loaded, parsed and onSettingsLoaded function is called.

See Also

[WebAppBase Class](#) | [WebAppBase Properties](#) | [WebAppBase Methods](#)

applicationReady Event

This event is activated when the application is ready: all application specific settings are loaded and parsed; initial actions are made; function onSettingsLoaded is called.

Arguments

The event object passed to the event handler is an instance of the Olive.Event class

Remarks

This event is fired to inform other controls (usually main page of the application) that the application object is ready and following initializations can be performed.

Example

```
// In page loading
if (oPage.WebApplication && !oPage.WebApplication.isReady())
{
    oPage.WebApplication.attachOwcEventHandler("applicationReady",
    Page_onApplicationReady, oPage);
}
else
    oPage.initialize();
// In Page_onApplicationReady
oPage.WebApplication.detachOwcEventHandler("applicationReady",
OwcPage_onApplicationReady, oPage);
oPage.initialize();
```

See Also

[WebAppBase Class](#) | [WebAppBase Events](#)

Year Overview

The Year control reads and writes the year portion of a date/time value in an HTML element. It is designed to be a subcontrol of a [Calendar](#) control, where the year part of the date value is shown and changed through this control, and the month and day are shown and changed through [Month](#), [Day](#), or [DayTable](#) controls.

The basic functionality of parsing and validating year strings comes from the data type `Olive.Controls.DataType.Year`. The Year control also has an internal "mask" property that allows it to perform data exchange with the parent Calendar control. When the value of the Year control is changed, the "valueChanged" event updates the year part of the Calendar date/time value, and the Calendar control then updates the other date-related subcontrols that are affected by a change in year ([DayTable](#) and [Date](#)).

Possible values of the Format HTML attribute are:

`%Y` - four-digit year ("2006") - default value

`%y` - two-digit year ("06")

`%#y` - two-digit year without zero padding ("6")

Only the first format ("`%Y`") is useful for inputting values, since 2-digit input values cannot be parsed correctly ("`06`" = 1906 by default).

For an example of a Year control in a Calendar control, see [Calendar Examples](#).

Syntax

```
<html-element owc:Control = "Year" owc:Format="/format string/">
</html-element>
```

Example

```
<input id="iYear" type="text" size="4" owc:Control="Year" owc:Format="%Y"
value="2006" maxlength="4" />
<input type="button" value="Set to the following year"
onclick="setToFollowingYear()" />
function setToFollowingYear() {
    var oYear = OwcGetControl("iYear");
    if( oYear ) {
        oYear.updateData();
        var nYear = oYear.getValue();
        oYear.setValue( nYear + 1 );
    }
}
```

Preview



The preview shows a web interface with a text input field containing the number "2007" and a button next to it with the text "Set to the following year". The input field has a small border and the button has a 3D effect.

See Also

[Calendar Overview](#) | [Value Overview](#) | [Olive.Controls.DataType Overview](#)

Layout XML Overview

The Layout.xml file provides sample HTML templates to format repeated elements created by the server's data provider. These templates are used by WebSDK controls that have repeated data elements - tree controls (folder tree, TOC) and list controls (Document list, Search Results). The data provider for these controls first retrieves the data set of elements to populate the control, and then formats each element according to an HTML template, as specified in the Layout.xml file. This allows each application to format repeated data elements according to a custom format.

The WebSDK provides an internal version of Layout.xml (at /Olive/WebComponents/Layout). Values in this internal version can be overridden in the custom version of Layout.xml for the application. When an application begins, the two versions are merged, and the custom version gets priority over the internal version.

File Structure

The Layout.xml file is structured as follows:

1. Root element: owct:layout-se
2. There are 3 possible sub-elements of the root element:
 - A. **owct:tree-template** - formatting instructions for tree components (folder tree, TOC). It can have 2 sub-element types:
 - owct:tree-node-type** - formatting instructions for the tree-node type
 - owct:xhtml-template** - (as above) sample HTML code used to format data elements for this tree control type
 - B. **owct:list-template** - formatting instructions for list components (document list, search results). It can have 3 sub-element types
 - owct:list-item-type** - formatting instructions for a list-item type
 - owct:header** - defines if a list header is displayed
 - owct:xhtml-template** - (as above) sample HTML code used to format data elements for this list control type
 - C. **owct:xhtml-template** - sample HTML code to be used for formatting a data element. This element can also be embedded under an element owct:tree-template or owct:list-template.

XML Elements and Attributes

Element name	Description
owct:layout-set	Root element of the Layout.xml file. It has mandatory attributes that define namespaces owc and owct (see example below).
owct:html-template	This is a child element of owct:layout-set, owct:tree-template, or owct:list-template. Each XHTML-template element gives the HTML to format one kind of data element, such as tree-node, list-item, list-footer, as specified in the kind attribute.

	Attributes: 1. kind (mandatory) - for possible values, see below 2. for - The possible values depend on the kind attribute. For example, if kind="tree-node-title" or a similar TOC data element (tree-node-page, tree-node-attr, tree-node-score), for specifies a type of TOC node (TocDocNode, TocEntryNode, TocSearchResNode). If kind="search-term", for specifies the search parameter (Text, dc:title, olv:Folder, ...). If kind="list-empty", for specifies a reason (EmptyQuery, NoSignificantCharacters) 3. name - This can be referred to by the attribute XHTML-template of an owct:tree-node-type element. For example, the following element (a child of an owct:tree-templateelement) says that data nodes of TocSearchResNodetype (the entity-level node of a search-result) should get its HTML format from the owct:xhtml-template for which the name attribute has the value search-res-tree-node: <pre><owct:tree-node-type name="TocSearchResNode" content-cmd="ViewSearchResult" xhtml-template="search-res-tree-node"></pre> 4. default - If "true", then this template is used for the type specified in the kind attribute, unless it is overridden by another template with the same kindvalue but a specific for value. For example, in the following two elements, the first template is used for all search terms except for "olv:Folder", where the second template overrides the default. <pre><owct:xhtml-template kind="search-term" default="true">where <owct:search-term-title />... <owct:xhtml-template kind="search-term" for="olv:Folder">in</pre> Sub-elements: 1. Any HTML element 2. Any OWCT element that is converted to an HTML element, attribute, or text, such as: owct:xhtml-element owct:xhtml-attribute owct:property owct:meta-field
owct:tree-template	General layout features for a tree type; TOC or FolderTree. Attributes: 1. for - "Toc" or "FolderTree" 2. name - Can be used to provide different templates for different

	varieties of the same kind of tree. It can be used to distinguish kinds of TOC for search results, for example <code>name="SearchInDocToc"</code> (search results TOC for search-within-document) or <code>name="SearchResultToc"</code> (search results TOC for global search). The HTML definition of a component can specify a template by name, with the custom attribute <code>owc:TemplateId</code>. 3. default - If <code>true</code>, then this template will be used for the type specified in the 'for' attribute unless this template is overridden by another template with the same 'for' value but a specific 'name' value. Sub-elements: 1. <code>owct:tree-node-type</code> 2. <code>owct:xhtml-template</code>
owct:tree-node-type	Sub-element of <code>owct:tree-template</code>. Attributes: 1. name - Valid node type: <code>Folder</code>, <code>TocDocNode</code>, <code>TocEntryNode</code>, <code>TocSearchResNode</code> 2. expand="yes" – Defines if the sub-tree should be displayed in expanded form upon opening. 3. content-cmd - Command that executes when the user clicks on the node. 4. xhtml-template - Reference to the name attribute of an <code>owct:xhtml-template</code> element, giving the HTML to format this tree node type. The internal <code>Layout.xml</code> in the WebSDK defines 3 names; "no-outline-tree-node", "toc-tree-node", and "search-res-tree-node". Sub-element: 1. <code>owct:tree-node-layout</code>
owct:tree-node-layout	Sub-element of <code>owct:tree-node-type</code>, giving the layout for this type of tree nodes. Up to 2 of these elements may be used; one for normal nodes (<code>mode="unselected"</code>) and one for selected nodes (<code>mode="selected"</code>). Attribute: 1. mode="selected"- gives the layout for a node when it is selected. Sub-elements: 1. cssClass - the text value of this gives the name of the css class for this node type 2. iconUrl - the text value of this gives the icon for this node type (for example, a picture of a book or document). The icon for the 'selected' mode does not have to be given; it will be taken from the

	normal mode.
owct:tree-layout	Layout for a tree that is embedded within another XHTML-template, such as a search-results-toc tree within a search-results list item. It implements the tree-node-layout(referred to in the tree-template attribute) to build an XHTML element <code><owc:Data type="treeLayout"></code>, which is used by the client to format the tree. Attribute: 1. tree-template - Reference to value of 'name' attribute of an owct:tree-template element.
owct:tree-node	Embeds a tree within another XHTML-template, such as a search-results-toc tree within a search-results list item. Attribute: 1. tree-template - reference to the value of the 'name' attribute of an owct:tree-template element.
owct:tree-node-attr	Used in the internal Layout.xml within the HTML code of a <code><owct:xhtml-template kind="tree-node"></code>element, to add the required custom attributes to the <code><table></code> element (owc:control, ContentCmd, NodeType, ...). It is normally not necessary to override this internal use. Attributes: none Sub-elements: none
owct:tree-node-children	Used in the internal Layout.xml within the HTML of an <code><owct:xhtml-template kind="tree-node"></code>element, for adding recursive sub-nodes to the <code><td owc:ui="treeSubNodesTarget"></code>element. It is often not necessary to override this internal use. Attributes: none Sub-elements: none
owct:tree-render-template	Calls the template of the specified kind and name. It is used in the internal Layout.xml within the HTML code of an <code><owct:xhtml-template kind="tree-node"></code> element, for calling another XHTML-template, such as tree-node-title, tree-node-score, or tree-node-page types, to insert the HTML code of the title, score, or page in a TOC tree. It makes it possible to separate out common HTML code into a single smaller element owct:xhtml-template. Attributes: 1. kind 2. name Sub-elements: none

owct:list-template	Gives the general layout features for a list type (DocList or SearchRes). Attributes: 1. for - DocList or SearchRes 2. name - May provide different templates for different varieties of the same list type, such as distinguishing search results list types (for example, name="DocListSearchRes" for search results for search-within-document). The HTML definition of a component can specify a template by name, with the custom attribute owc:TemplateId. 3. default - If true, then this template will be used for the type specified in the for attribute unless this template is overridden by another template with the same 'for' value but a specific 'name' value. Sub-elements: 1. owct:list-item-type 2. owct:header 3. owct:footer 4. owct:xhtml-template
owct:header	Sub-element of owct:list-template, specifying when to display a header at the top of the list. The header's format is specified by a sibling element <owct:xhtml-template kind="list-header">, or by an independent owct:xhtml-element named like this element's 'name' attribute. Attributes: 1. display - "always", "not-empty" when the list is not empty, or "auto" when the list has more than 1 page. 2. name - the 'name' attribute of the owct:html-template, specifying the header's format. Sub-elements: none
owct:footer	Sub-element of owct:list-template that specifies when to display a footer at the bottom of the list. The footer's format is specified by the sibling <owct:xhtml-template kind="list-footer"> element or by an independent owct:xhtml-element named like this element's 'name' attribute. Attributes: 1. display - "always", "not-empty" when the list is not empty, or "auto" when the list has more than 1 page. 2. name - the 'name' attribute of the owct:html-template, specifying the footer's format.

owct:list-header	Sub-elements: none Inserts a header at the top of the list in the internal Layout.xml within the <owct:xhtml-template kind="list" default="true">element. This only occurs if the owct:list-templateelement has a owct:header sub-element. This element's attributes override the corresponding attributes of the owct:header element, although the version in the internal Layout.xml has no attributes. Attributes: 1. display - "always", "not-empty" when the list is not empty, or "auto" when the list has more than 1 page. 2. name - the 'name' attribute of the owct:html-template, specifying the header's format.
owct:list-footer	Inserts a footer at the bottom of a list in the internal Layout.xml within the <owct:xhtml-template kind="list" default="true">element, to insert a footer at the bottom of the list. This only occurs if the owct:list-template element has a owct:footer sub-element. This element's attributes override the corresponding attributes of the owct:footer element, although the version in the internal Layout.xml has no attributes. Attributes: 1. display - "always", "not-empty" when the list is not empty, or "auto" when the list has more than 1 page. 2. name - the 'name' attribute of the owct:html-template, specifying the footer's format. Sub-elements: none
owct:list-pagination-header	Element embedded in the HTML code within an <owct:xhtml-template kind="list-header">element. It inserts a series of page numbers, near the current page, that may be clicked to navigate to other pages. Attributes: 1. max-links – maximum number of page numbers to display together (default value = 10) 2. links-after-current – maximum number page numbers after the current page (default value = max-links / 2) Sub-elements: none
owct:list-page-links	Sub-element within the owct:list-pagination-headerelement, inserting actual page numbers. Other sub-elements of owct:list-pagination-header can insert links to the first, previous, next, and last pages.

	Attributes: none Sub-elements: none
owct:data	Inserts the appropriate owc:Dataelement for this control type, to pass this component's data back to the client. Used by the internal Layout.xml in the basic definitions of the various data element types. It is often not necessary to override this use.
owct:list-items	Inserts list items into a list. It is used by the internal Layout.xml in the owct:xhtml-template element (kind="list" default="true"). It is often not necessary to override this internal use.
owct:list-item-render-template	Inserts the actual html for each list-item by the internal Layout.xml. It is often not necessary to override this internal use.
owct:list-item-attr	Creates the basic attributes for each list item element by the internal Layout.xml: owc:Control, ItemType, ContentCmd, ... It is often not necessary to override this internal use.
owct:property	Inserts the property value (a data field in the xml data for this item). Attribute: 1. name - the property name. This can be an attribute or sub-node of the data-xmlelement, or a reserved name (listed below). Reserved names: - olv-tree:title - olv-folder:title - olv-folder:index - olv-list:results-count - olv-list:results-from - olv-list:results-to - olv-list:results-this-page - olv-list:page-count - olv-list:page-no - olv-list:page-size - olv-list:page-link-no - olv-list:page-link-first - olv-list:page-link-last - olv-list:page-link-index-first - olv-list:page-link-index-last - olv-list:index

	olv-search:search-string olv-search:total-searched olv-search-res:hits-count olv-search-res:content-hits-count olv-search-res:metadata-hits-count olv-search-res:first-hit-page olv-search-res:last-hit-page olv-search-res:hits-count olv-search-res:hits-count olv-search-res:toc-entries-count olv:is-recursive [for search query] olv-toc:page-label olv-request-param:{name of request parameter}
owct:meta-field	Inserts the value of the item's meta-data field. Attribute: 1. name - metadata field's name. May be a reserved name (listed below). Reserved names: olv:folder
owct:field	[Not yet implemented.]
owct:field-title	[Not yet implemented.]
owct:field-value	[Not yet implemented.]
owct:res-string	Returns this name's localized text from the locale files. Attribute: 1. name - name of the localized text Sub-elements: none
owct:version	Returns the SDK version, such as "1.0". Attributes: none Sub-elements: none
owct:xhtml-element	Creates an HTML element, with a specified name, in this place in the output HTML code. It may be used to dynamically control the output HTML element's name. Attribute:

owct:xhtml-attribute	1. name - the name of the element Sub-elements: Any sub-elements become sub-elements of the created element. Creates an attribute of the parent HTML element, with the specified name, and with the value specified by executing the sub-elements. Attribute: 1. name - the attribute's name Sub-elements: Any executable template elements that result in a string value for the attribute
owct:if-compare	Executes the sub-elements only if a condition is fulfilled. Attributes: 1. field - some field or property that is used as the left operand 2. condition - "=" or "!=" can be used for any comparison. If comparison="numerical" or comparison="strlength", the condition can also be ">" or "<" 3. operand - a value for the right operand. If the right operand is a field or property, 'to-field' should be used instead of 'operand' 4. to-field - a field or property that is be used as the right operand, when 'operand' is not specified 5. comparison - can be "numerical", "strength", or not specified. Sub-elements: any XHTML elements that are conditionally executed.
owct:if-exists	Executes the sub-elements only if a property exists. Attribute: 1. field - some property that is tested to verify it exists. Sub-elements: Any XHTML elements that are conditionally executed.
owct:if-not-exists	Executes the sub-elements only if a property does not exist. Attribute: 1. field - some property that is tested to verify it exists. Sub-elements: Any XHTML elements that are conditionally executed if the property does not exist.
owct:if-complex	Joins more than 1 element owct:if-compare, with the operator "and" or "or".

owct:condition	Attributes: none Sub-element: owct:condition has owct:if-compare sub-elements Provides an owct:if-complexelement to the operator.
	Attribute: 1. operator - "and" or "or" Sub-elements: owct:if-compare
owct:search-query	Inserts a string with the search parameters. It should be embedded in the XHTML for an owct:xhtml-templateelement with kind="list-header" or "list-footer". The string layout is specified by the <owct:xhtml-template kind="search-term">element, with owct:search-term-value, owct:search-term-title, and owct:search-term-operator sub-elements. The XSL loops through all search parameters, applies the formatting specified by the appropriate XHTML-template of kind="search-term", and inserts the value, title, and/or operator of each parameter according to owct:search-term-value, owct:search-term-title, and owct:search-term-operator, the sub-elements of owct:xhtml-template kind="search-term"element. Attributes: none Sub-elements: none
owct: search-term-title	The search parameter's title. It may have an external value according to OlvApplication.config.xml <Field TITLE="...."/>.
	Attributes: none Sub-elements: none
owct: search-term-value	The search parameter's value. It may have the date formatted according to OlvApplication.config.xml, <Metadata DateOutputFormat="....">.
	Attributes: none Sub-elements: none
owct:search-term-operator	The search operator title. It may have an external value according to OlvApplication.config.xml <Operator TITLE="...."/>. Attributes: none Sub-elements: none
owct:snippet	Inserts a snippet image of the entity at this point [not yet implemented].

owct:render-template	Inserts the results of executing the template, as specified by the attributes 'kind', 'name', and/or 'for'. Attributes: 1. kind 2. name 3. for
-----------------------------	---

Attributes

Below are the possible values that may be used for the kind attribute of the owct:xhtml-template.

Attribute value	Description
tree-node	HTML structure for each tree-node. It is often not necessary to override the version that is defined in the internal Layout.xml
tree-node-title	Text displayed in the node row
tree-node-page	Page of this node in the TOC
tree-node-score	This node's score in the search results
tree-node-attr	Any HTML attributes that should be added to each TOC node
list	Basic HTML structure for a list (a div containing the list items, with an optional header and footer). It is often not necessary to override the version that is defined in the internal Layout.xml
list-header	List header
list-footer	List footer
list-item	List item
list-item-content	Contents of a list-item
list-item-attr	Any HTML attributes that should be added to each list item
list-empty	HTML code that should be inserted when a list is empty (no list items)
list-pagination-header	HTML template to paginate links at the top of a list (may include links to go to the first, last, previous, or next page, and the element owct:list-page-links to page numbers adjacent to the current page)
search-term	HTML code to display each search parameter in the header or footer of the search results list

Syntax

```
<?xml version="1.0" encoding="utf-8" ?>
<owct:layout-set xmlns:owc="http://www.olivesoftware.com/schema/owc.xsd"
  xmlns:owct="http://www.olivesoftware.com/schema/owct.xsd">
  <owct:xhtml-template kind="tree-node" default="true">
    <!-- add html elements here with embedded owct template elements-->
  </owct:xhtml-template>

  <owct:xhtml-template kind="tree-node" name="search-res-tree-node">
    <!-- add html elements here with embedded owct template elements-->
  </owct:xhtml-template>

  <owct:xhtml-template kind="list" default="true">
    <owct:list-header />
    <div>
      <owct:data />
      <owct:list-items />
    </div>
    <owct:list-footer />
  </owct:xhtml-template>

  <owct:xhtml-template kind="list-item" default="true">
    <div>
      <owct:list-item-attr />
      <owct:list-item-render-template kind="list-item-content" />
      <owct:data />
    </div>
  </owct:xhtml-template>

  <!-- other (non-default) templates -->
</owct:layout-set>
```

Layout.xml File

Below is an example of a typical Layout.xml file.

```
<?xml version="1.0" encoding="utf-8"?>
```

```

<owct:layout-set
xmlns:owc="http://www.olivesoftware.com/schema/owc.xsd"
xmlns:owct="http://www.olivesoftware.com/schema/owct.xsd">

  <!-- Template for folder tree -->

  <owct:tree-template for="FolderTree">

    <owct:tree-node-type name="Folder" content-cmd="ViewFolderContent">

      <owct:tree-node-layout>

        <cssClass>folderNode</cssClass>

        <iconUrl>Layout/folder.gif</iconUrl>

      </owct:tree-node-layout>

      <owct:tree-node-layout mode="selected">

        <cssClass>folderNodeSelected</cssClass>

      </owct:tree-node-layout>

    </owct:tree-node-type>

    <owct:xhtml-template default="true" kind="tree-node-title">

      <span class="folderTitle"><owct:property name="olv-
folder:title"/></span>

    </owct:xhtml-template>

  </owct:tree-template>

  <!-- Template for document's TOC tree -->

  <owct:tree-template for="Toc" default="true">

    <owct:tree-node-type name="TocDocNode" expand="yes">

      <owct:tree-node-layout>

        <cssClass>TocDocNode</cssClass>

        <iconUrl>Layout/Images/icon_Document.gif</iconUrl>

      </owct:tree-node-layout>

      <owct:tree-node-layout mode="selected">

        <cssClass>TocDocNodeSelected</cssClass>

        <iconUrl>Layout/Images/icon_DocSelected.gif</iconUrl>

      </owct:tree-node-layout>

    </owct:tree-node-type>

    <owct:tree-node-type name="TocEntryNode" content-
cmd="ViewTocEntryContent" xhtml-template="toc-tree-node">

      <owct:tree-node-layout>

        <cssClass>TocEntryNode</cssClass>

        <iconUrl>Layout/Images/icon_TocEntry.gif</iconUrl>

      </owct:tree-node-layout>

      <owct:tree-node-layout mode="selected">

        <cssClass>TocEntryNodeSelected</cssClass>

```

```

        </owct:tree-node-layout>
    </owct:tree-node-type>
    <owct:xhtml-template for="TocDocNode" kind="tree-node-title">
        <span class="TocEntryTitle">
            <owct:xhtml-attribute name="title"><owct:property
name="LogicTitle"/></owct:xhtml-attribute>
            <owct:property name="LogicTitle"/>
        </span>
    </owct:xhtml-template>
    <owct:xhtml-template for="TocEntryNode" kind="tree-node-title">
        <span class="TocEntryTitle">
            <owct:xhtml-attribute name="title"><owct:property
name="TITLE"/></owct:xhtml-attribute>
            <owct:property name="TITLE"/>
        </span>
    </owct:xhtml-template>
    <owct:xhtml-template for="TocEntryNode" kind="tree-node-page">
        <owct:xhtml-attribute name="class">TocEntryPageNo</owct:xhtml-
attribute>
        <owct:property name="olv-toc:page-label"/>
    </owct:xhtml-template>
</owct:tree-template>
<!-- Template for "search in document" TOC tree -->
<owct:tree-template for="Toc" name="SearchInDocToc">
    <owct:tree-node-type name="TocSearchResNode" content-
cmd="ViewSearchResult" xhtml-template="search-res-tree-node">
        <owct:tree-node-layout>
            <cssClass>SearchResNode</cssClass>
            <iconUrl>Layout/Images/icon_Document.gif</iconUrl>
        </owct:tree-node-layout>
        <owct:tree-node-layout mode="selected">
            <cssClass>SearchResNodeSelected</cssClass>
            <iconUrl>Layout/Images/icon_DocSelected.gif</iconUrl>
        </owct:tree-node-layout>
    </owct:tree-node-type>
    <owct:tree-node-type name="TocEntryNode" content-
cmd="ViewTocEntryContent" expand="all" xhtml-template="toc-tree-node">
        <owct:tree-node-layout>
            <cssClass>TocEntryNode</cssClass>

```



```

</owct:tree-node-layout>
<owct:tree-node-layout mode="selected">
  <cssClass>TocEntryNodeSelected</cssClass>
</owct:tree-node-layout>
</owct:tree-node-type>
<owct:xhtml-template for="TocSearchResNode" kind="tree-node-attr">
  <owct:if-compare field="olv-list:results-count" condition="="
operand="1" comparison="numerical">
    <owct:xhtml-attribute name="owc:AutoLoad">true</owct:xhtml-
attribute>
    <owct:xhtml-attribute name="owc:Expand">1</owct:xhtml-attribute>
  </owct:if-compare>
</owct:xhtml-template>
<owct:xhtml-template for="TocSearchResNode" kind="tree-node-title">
  <span class="SearchResTitle">
    <owct:xhtml-attribute name="title"><owct:property
name="LogicTitle"/></owct:xhtml-attribute>
    <owct:property name="LogicTitle"/>
  </span>
  <!-- add number of hits when there is no toc-entries -->
  <owct:if-compare field="olv-search-res:toc-entries-count"
condition="=" operand="0" comparison="numerical">
    <span>
      <owct:if-compare field="olv-search-res:content-hits-count"
condition=">" operand="0" comparison="numerical"><span dir="ltr"
class="TocEntryHits"> (<owct:property name="olv-search-res:content-
hits-count"/> Hit<owct:if-compare field="olv-search-res:content-hits-
count" condition=">" operand="1" comparison="numerical">s</owct:if-
compare>) </span></owct:if-compare>
    </span>
  </owct:if-compare>
</owct:xhtml-template>
<owct:xhtml-template for="TocSearchResNode" kind="tree-node-score">
  <owct:xhtml-attribute name="class">Score</owct:xhtml-attribute>
  <owct:if-compare field="olv-list:results-count" condition=">"
operand="1" comparison="numerical">
    <img owc:Control="scorebarentity" owc:MaxNumber="4"
owc:UrlPrefix="Layout/ArtScoreBar/" owc:ImgSuffix="gif"></img>
    <span class="ScoreText">
      <owct:property name="Score"/>%
    </span>
  </owct:if-compare>
</owct:xhtml-template>

```

```

</owct:if-compare>
<owct:xhtml-attribute name="class">Score</owct:xhtml-attribute>
<!-- release space when there is only 1 result -->
<owct:if-compare field="olv-list:results-count" condition="="
operand="1" comparison="numerical">
    <owct:xhtml-attribute name="class">EmptyTd</owct:xhtml-attribute>
</owct:if-compare>
</owct:xhtml-template>
<owct:xhtml-template for="TocEntryNode" kind="tree-node-title">
    <span><img owc:Control="scorebartoc" owc:MaxNumber="4"
owc:UrlPrefix="Layout/TocScoreBar/" owc:ImgSuffix="gif"></img></span>
    <owct:if-compare field="HITS" condition=">" operand="0"
comparison="numerical"><span dir="ltr"
class="TocEntryHits"> (<owct:property name="HITS"/> Hit<owct:if-compare
field="HITS" condition=">" operand="1"
comparison="numerical">s</owct:if-compare>) </span></owct:if-compare>
    <span class="TocEntryTitle">
        <owct:xhtml-attribute name="title"><owct:property
name="TITLE"/></owct:xhtml-attribute>
        <owct:property name="TITLE"/>
    </span>
</owct:xhtml-template>
<owct:xhtml-template for="TocSearchResNode" kind="tree-node-page">
    <!-- add p. and page-label for search-res-toc when there is no toc
entries -->
    <owct:if-compare field="olv-search-res:toc-entries-count"
condition="=" operand="0" comparison="numerical">
        <owct:xhtml-attribute name="class">TocEntryPageNo</owct:xhtml-
attribute>
        p.<owct:property name="olv-search-res:page-label"/>
    </owct:if-compare>
    <!-- release space when there are toc entries -->
    <owct:if-compare field="olv-search-res:toc-entries-count"
condition="!=" operand="0" comparison="numerical">
        <owct:xhtml-attribute name="class">EmptyTd</owct:xhtml-attribute>
    </owct:if-compare>
</owct:xhtml-template>
<owct:xhtml-template for="TocEntryNode" kind="tree-node-page">
    <owct:xhtml-attribute name="class">TocEntryPageNo</owct:xhtml-
attribute>
    <owct:property name="olv-toc:page-label"/>

```

```

</owct:xhtml-template>
</owct:tree-template>
<!-- Template for TOC tree in search results (general) -->
<owct:tree-template for="Toc" name="SearchResultToc">
  <owct:tree-node-type name="TocSearchResNode" content-
cmd="ViewEntity" xhtml-template="no-outline-tree-node">
    <owct:tree-node-layout>
      <cssClass>SearchResNode</cssClass>
      <iconUrl>Layout/Images/icon_Document.gif</iconUrl>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>SearchResNodeSelected</cssClass>
      <iconUrl>Layout/Images/icon_DocSelected.gif</iconUrl>
    </owct:tree-node-layout>
  </owct:tree-node-type>
  <owct:tree-node-type name="TocEntryNode" content-
cmd="ViewTocEntryContent" expand="all" xhtml-template="toc-tree-node">
    <owct:tree-node-layout>
      <cssClass>TocEntryNode</cssClass>
    </owct:tree-node-layout>
    <owct:tree-node-layout mode="selected">
      <cssClass>TocEntryNodeSelected</cssClass>
    </owct:tree-node-layout>
  </owct:tree-node-type>
<owct:xhtml-template for="TocSearchResNode" kind="tree-node-title">
  <span>
    <span class="SearchResTitle">
      <owct:xhtml-attribute name="title"><owct:property
name="LogicTitle"/></owct:xhtml-attribute>
      <owct:property name="LogicTitle"/>
    </span>
    <owct:if-compare field="olv-search-res:content-hits-count"
condition="&gt;" operand="0" comparison="numerical">
      <span dir="ltr" class="TocEntryHits">
        (<owct:property name="olv-search-res:content-hits-
count"/> Hit<owct:if-compare field="olv-search-res:content-hits-count"
condition="&gt;" operand="1" comparison="numerical">s</owct:if-
compare>)
      </span>
    </owct:if-compare>
  </span>
</owct:xhtml-template>
</owct:tree-template>

```

```

<owct:if-complex>
  <owct:condition operator="or">
    <owct:if-compare field="olv-search-res:metadata-hits-
count" condition=">" operand="0" comparison="numerical"/>
    <owct:if-compare field="olv-search-res:hits-count"
condition="=" operand="0" comparison="numerical"/>
  </owct:condition>
  <owct:content>
    <span class="TocEntryHits" title="There are hits in
metadata">(*)</span>
  </owct:content>
</owct:if-complex>
</span>
<owct:if-complex>
  <owct:condition operator="and">
    <owct:if-compare field="olv-search-res:metadata-hits-count"
condition=">" operand="0" comparison="numerical"/>
    <owct:if-compare field="olv-search-res:content-hits-count"
condition="=" operand="0" comparison="numerical"/>
  </owct:condition>
  <owct:content>
    <div>Hits were only found in the document's metadata.</div>
  </owct:content>
</owct:if-complex>
  owct:if-compare field="olv-search-res:toc-entries-count"
condition=">" operand="0" comparison="numerical">
    <owct:if-complex>
      <owct:condition operator="or">
        <owct:if-compare field="olv-search-res:content-hits-count"
condition=">" operand="0" comparison="numerical"/>
        <owct:if-compare field="olv-request-param:hits-only"
condition="!=" operand="1" comparison="numerical"/>
      </owct:condition>
      <owct:content>
        <div owc:Control="Command" owc:Command="Expand"
class="ExpandLink">More...</div>
      </owct:content>
    </owct:if-complex>
  </owct:if-compare>
</owct:xhtml-template>

```

```

<owct:xhtml-template for="TocEntryNode" kind="tree-node-title">
  <img owc:Control="scorebartoc" owc:MaxNumber="4"
owc:UrlPrefix="Layout/TocScoreBar/" owc:ImgSuffix="gif"></img>
  <owct:if-compare field="HITS" condition=">" operand="0"
comparison="numerical"><span dir="ltr"
class="TocEntryHits"> (<owct:property name="HITS"/> Hit<owct:if-compare
field="HITS" condition=">" operand="1"
comparison="numerical">s</owct:if-compare>) </span></owct:if-compare>
  <span class="TocEntryTitle">
    <owct:xhtml-attribute name="title"><owct:property
name="TITLE"/></owct:xhtml-attribute>
    <owct:property name="TITLE"/>
  </span>
</owct:xhtml-template>
<owct:xhtml-template for="TocSearchResNode" kind="tree-node-page">
  <!-- add page-label for search-res-toc when there is no toc
entries -->
  <owct:if-compare field="olv-search-res:toc-entries-count"
condition="=" operand="0" comparison="numerical">
    <owct:xhtml-attribute name="class">TocEntryPageNo</owct:xhtml-
attribute>
    <owct:property name="olv-search-res:page-label"/>
  </owct:if-compare>
  <!-- release space when there are toc entries -->
  <owct:if-compare field="olv-search-res:toc-entries-count"
condition="!=" operand="0" comparison="numerical">
    <owct:xhtml-attribute name="class">EmptyTd</owct:xhtml-attribute>
  </owct:if-compare>
</owct:xhtml-template>
<owct:xhtml-template for="TocEntryNode" kind="tree-node-page">
  <owct:xhtml-attribute name="class">TocEntryPageNo</owct:xhtml-
attribute>
  <owct:property name="olv-toc:page-label"/>
</owct:xhtml-template>
</owct:tree-template>
<!-- Document list template -->
<owct:list-template for="DocList">
  <owct:header display="not-empty"/>
  <owct:list-item-type name="Document"/>
  <!-- XHTML templates -->
  <owct:xhtml-template kind="list-empty">

```

```

    <span><owct:res-string name="EmptyFolder"/></span>
</owct:xhtml-template>
<owct:xhtml-template kind="list-header">
    <div class="DocListHeader">
        <span class="DocListHeaderContents">Documents
            <span class="headerResNumbers_dl"><owct:property name="olv-
list:results-from"/>-<owct:property name="olv-list:results-
to"/></span> of
            <span class="headerResNumbers_dl"><owct:property name="olv-
list:results-count"/></span>
            <span title="Go to previous page" class="OwcPrevPageButton
NavigButton" owc:Control="Command" owc:Command="PrevPage"> </span><span
title="Go to next page" class="OwcNextPageButton NavigButton"
owc:Control="Command" owc:Command="NextPage"> </span><!--IE must have
no white space between lines--> in folder
            <span class="headerResNumbers_dl"><owct:property name="olv-
folder:title"/></span>
            <span class="NavigButtonEmpty"> </span>
            Sort By:
            <select id="sortSel" owc:Control="SortBy" class="sortSelClass"
size="1" name="sortSel">
                <option value="LogicTitle;asc">Title</option>
                <option value="SRT_DATE;desc">Date</option>
            </select>
        </span>
    </div>
    <div class="headerUnderscore"></div>
</owct:xhtml-template>
<owct:xhtml-template kind="list-item-content" for="Document">
    <table>
        <tr valign="top">
            <td>
                <div class="DocThumbnail">
                    <img owc:Control="Thumbnail"
owc:ContentCmd="ViewDocument" class="Thumbnail"></img>
                </div>
            </td>
            <td>
                <table border="0">
                    <colgroup>
                        <col width="120px"/><!--for Safari-->

```

```

        <col/>
    </colgroup>
    <tbody>
        <owct:if-exists field="LogicTitle">
            <tr>
                <td class="MetaTitle">Title:</td>
                <td class="MetaTOC">
                    <owct:property
name="LogicTitle"/>
                </td>
            </tr>
        </owct:if-exists>
        <owct:if-exists field="dc:description">
            <tr>
                <td
class="MetaTitle">Description:</td>
                <td class="MetaTOC">
                    <owct:meta-field
name="dc:description"/>
                </td>
            </tr>
        </owct:if-exists>
        <owct:if-exists field="pageCount">
            <tr>
                <td class="MetaTitle">Number of
Pages:</td>
                <td class="MetaTOC">
                    <owct:property
name="pageCount"/>
                </td>
            </tr>
        </owct:if-exists>
        <owct:if-exists field="dc:publisher">
            <tr>
                <td
class="MetaTitle">Publisher:</td>
                <td class="MetaTOC">
                    <owct:meta-field
name="dc:publisher"/>
                </td>
            </tr>
        </owct:if-exists>
    </tbody>

```

```

        </tr>
    </owct:if-exists>
    <owct:if-exists field="dc:date">
        <tr>
            <td class="MetaTitle">Date:</td>
            <td class="MetaTOC">
                <owct:meta-field
name="dc:date" Type="Date" SrvFormat="dd/mm/yyyy" OutputFormat="MMMM
dd, YYYY"/>
            </td>
        </tr>
    </owct:if-exists>
</tbody>
</table>
</td>
</tr>
</table>
<hr/>
</owct:xhtml-template>
</owct:list-template>
<!-- Template for search results list (general) -->
<owct:list-template for="SearchRes">
    <owct:list-item-type name="SearchResult"/>
    <owct:header display="always"/>
    <owct:xhtml-template kind="list-header">
        <owct:if-compare field="olv-list:page-count" condition="&gt;"
operand="0" comparison="numerical">
            <div class="SearchResHeaderBg">
                <div class="SearchResHeader">
                    <span class="SearchQueryHeading">Results Page: </span>
                    <owct:list-pagination-header max-links="10" links-after-
current="5"/>
                </div>
            </div>
        </owct:if-compare>
        <div class="SearchResQueryHeader">
            <div class="SearchResQueryText">
                <span class="SearchQueryHeading">Search: </span>
                <owct:search-query/>.
            </div>
        </div>
    </owct:xhtml-template>
</owct:list-template>

```



```

        </div>
        <hr/>
    </div>
</owct:xhtml-template>
<owct:xhtml-template kind="list-item-attr" default="true">
    <owct:xhtml-attribute name="class">SearchResult</owct:xhtml-
attribute>
</owct:xhtml-template>
<owct:xhtml-template kind="list-item-content" default="true">
    <table border="0" width="100%" style="table-layout: fixed">
        <colgroup>
            <col width="80px"/>
            <col width="340px"/>
            <col width="450px"/>
        </colgroup>
        <tr valign="top">
            <td width="67">
                <div class="DocThumbnail">
                    <img owc:Control="Thumbnail"
owc:ContentCmd="ViewEntity" class="Thumbnail"></img>
                </div>
                <div class="TextPage" noWrap="true">Page <owct:property
name="olv-search-res:page-label"/></div>
            </td>
            <td>
                <table border="0" width="100%" style="table-layout:
fixed">
                    <colgroup>
                        <col width="120px"/><!--for Safari-->
                        <col/>
                    </colgroup>
                    <tr>
                        <td class="MetaTitle">Rank: </td>
                        <td class="MetaTOC">
                            <div>
                                <img owc:Control="scorebarentity"
owc:MaxNumber="4" owc:UrlPrefix="Layout/ArtScoreBar/"
owc:ImgSuffix="gif"></img>
                                <span
class="ScoreText"><owct:property name="Score"/>%</span>

```

```

        </div>
    </td>
</tr>
<tbody>
    <owct:if-exists field="LogicTitle">
        <tr>
            <td class="MetaTitle">Title:</td>
            <td class="MetaTOC">
                <owct:property
name="LogicTitle"/>
            </td>
        </tr>
    </owct:if-exists>
    <owct:if-exists field="dc:description">
        <tr>
            <td
class="MetaTitle">Description:</td>
            <td class="MetaTOC">
                <owct:meta-field
name="dc:description"/>
            </td>
        </tr>
    </owct:if-exists>
    <owct:if-exists field="dc:publisher">
        <tr>
            <td
class="MetaTitle">Publisher:</td>
            <td class="MetaTOC">
                <owct:meta-field
name="dc:publisher"/>
            </td>
        </tr>
    </owct:if-exists>
    <owct:if-exists field="olv:folder">
        <tr>
            <td class="MetaTitle">Folder
Path:</td>
            <td class="MetaTOC">
                <owct:meta-field
name="olv:folder"/>

```

```

        </td>
    </tr>
</owct:if-exists>
<owct:if-exists field="dc:rights">
    <tr>
        <td class="MetaTitle">Rights:</td>
        <td class="MetaTOC">
            <owct:meta-field
name="dc:rights"/>
        </td>
    </tr>
</owct:if-exists>
<owct:if-exists field="dc:date">
    <tr>
        <td class="MetaTitle">Date:</td>
        <td class="MetaTOC">
            <owct:meta-field
name="dc:date" Type="Date" SrvFormat="yyyymmdd" OutputFormat="MMMM dd,
YYYY"/>
        </td>
    </tr>
</owct:if-exists>
<owct:if-exists field="PagesCount">
    <tr>
        <td class="MetaTitle">Number of
Pages:</td>
        <td class="MetaTOC">
            <owct:meta-field
name="PagesCount"/>
        </td>
    </tr>
</owct:if-exists>
</tbody>
</table>
</td>
<td>
    <div class="TocTreeOuter">
        <div owc:Control="TOC" owc:AutoLoad="false"
owc:TemplateId="SearchResultToc">

```

```

        <owct:tree-layout tree-
template="SearchResultToc"/>
        <owct:tree-node tree-
template="SearchResultToc"/>
    </div>
</div>
</td>
</tr>
</table>
<hr/>
</owct:xhtml-template>
</owct:list-template>
<!-- Template for search results (search in document) -->
<owct:list-template for="SearchRes" name="DocListSearchRes">
    <owct:list-item-type name="SearchResult"/>
    <owct:xhtml-template kind="list-header">
        <div class="SearchResHeader">
            Results Page: <owct:list-pagination-header max-links="5" links-
after-current="2"/>
        </div>
    </owct:xhtml-template>
    <owct:xhtml-template kind="list-item-content" default="true">
        <div class="TocTreeOuter">
            <div owc:control="TOC" owc:AutoLoad="false"
owc:TemplateId="SearchInDocToc">
                <owct:tree-layout tree-template="SearchInDocToc"/>
                <owct:tree-node tree-template="SearchInDocToc"/>
            </div>
        </div>
        <owct:if-compare field="olv-list:index" condition="&lt;" to-
field="olv-list:results-to" comparison="numerical">
            <hr/>
        </owct:if-compare>
    </owct:xhtml-template>
</owct:list-template>
<!-- Common XHTML templates -->
<owct:xhtml-template kind="list-empty" for="SearchRes">
    <span class="Msg_NoHits"><owct:res-string
name="EmptySearchRes"/></span>

```

```

</owct:xhtml-template>
<owct:xhtml-template kind="list-empty" for="EmptyQuery">
  <span class="Msg_NoHits"><owct:res-string
name="SEARCH_EMPTYSTR"/></span>
</owct:xhtml-template>
<owct:xhtml-template kind="list-empty" for="NoSignificantCharacters">
  <span class="Msg_NoHits"><owct:res-string
name="SEARCH_2SIGNCHARSTR"/></span>
</owct:xhtml-template>
<!-- Common page link XHTML templates -->
<owct:xhtml-template kind="list-pagination-header">
  <span class="PageLink_Header">
    <!-- Link to the previous page -->
    <owct:if-compare field="olv-list:page-no" condition="&gt;"
operand="1" comparison="numerical">
      <span class="PageLink_PrevPage" owc:control="command"
owc:command="PrevPage">Previous</span>
    </owct:if-compare>
    <!-- Link to the first page -->
    <owct:if-compare field="olv-list:page-link-first" condition="&gt;"
operand="1" comparison="numerical">
      <span class="PageLink_FirstPage" owc:control="command"
owc:command="FirstPage">1...</span>
    </owct:if-compare>
    <!-- "Goto page" links -->
    <owct:list-page-links/>
    <!-- Link to the last page -->
    <owct:if-compare field="olv-list:page-link-last" condition="&lt;"
to-field="olv-list:page-count" comparison="numerical">
      <span class="PageLink_LastPage" owc:control="command"
owc:command="LastPage">
        ...<owct:property name="olv-list:page-count"/>
      </span>
    </owct:if-compare>
    <!-- Link to the next page -->
    <owct:if-compare field="olv-list:page-no" condition="&lt;" to-
field="olv-list:page-count" comparison="numerical">
      <span class="PageLink_NextPage" owc:control="command"
owc:command="NextPage">Next</span>
    </owct:if-compare>
  </span>
</owct:xhtml-template>

```

```

</owct:xhtml-template>
<owct:xhtml-template kind="list-page-link">
  <owct:if-compare field="olv-list:page-no" condition="=" to-
field="olv-list:page-link-no" comparison="numerical">
    <span class="PageLink_CurPage"><owct:property name="olv-list:page-
link-no"/></span>
  </owct:if-compare>
  <owct:if-compare field="olv-list:page-no" condition="!=" to-
field="olv-list:page-link-no" comparison="numerical">
    <span class="PageLink_GotoPage" owc:control="command"
owc:command="GoToPage">
      <owct:xhtml-attribute name="owc:CommandParams">
        <owct:property name="olv-list:page-link-no"/>
      </owct:xhtml-attribute>
      <owct:property name="olv-list:page-link-no"/>
    </span>
  </owct:if-compare>
</owct:xhtml-template>
<owct:xhtml-template kind="search-term" default="true">where
<owct:search-term-title/> includes <span
class="SearchString">'<owct:search-term-value/>'</span>
</owct:xhtml-template>
<owct:xhtml-template kind="search-term" for="dc:date">
  <owct:search-term-operator/> <span
class="SearchString">'<owct:search-term-value/>'</span>
</owct:xhtml-template>
<owct:xhtml-template kind="search-term" for="Text">text <span
class="SearchString">'<owct:search-term-value/>'</span>
</owct:xhtml-template>
<owct:xhtml-template kind="search-term" for="DocType">include only:
<span class="SearchString">'<owct:search-term-value/>'</span>
</owct:xhtml-template>
<owct:xhtml-template kind="search-term" for="dc:language">where
<owct:search-term-title/> is <span class="SearchString">'<owct:search-
term-value/>'</span>
</owct:xhtml-template>
<owct:xhtml-template kind="search-term" for="olv:Folder">in <span
class="SearchString">'<owct:search-term-value/>'</span><owct:if-compare
field="olv:is-recursive" condition="=" operand="1"
comparison="numerical"> and its subfolders</owct:if-compare>
</owct:xhtml-template>
</owct:layout-set>

```

Client Side Coding Standards

The following coding standards should be enforced:

- Use an external CSS file instead of hard coding the style in the HTML file.
- All the HTML tags should be written in lower case.
- The HTML pages should conform to XHTML standard, and should be checked with available validation tools on regular basis. Every HTML tag should be closed, using explicit opening and closing tags.
- Instead of using an img tag use a CSS that contains the `img` tag inside.
- Use digital text instead of using text that is part of the picture. Doing so is crucial issue for customization readiness. For example when there is a need to change the text, it does not require the graphical designer intervention.
- Use explicit size in pixels in table's size instead of using percents, because different browsers interpret the percents differently.
- Use conditional comment as an elegant way for supporting CSS or javascript code that is browser specific.
- Folders names – it is recommended to use the following sub directories names:

scripts – for all JavaScript files.

Images – for all images.

Styles – for CSS files.

- Store JavaScript code in external files and not as part of the HTML file. It helps in portability. It means that it should be avoided to use the *script* tag that is not accompanied with the `src` attribute.
- Script files should be fully documented. Remember that the code is transmitted to the client so the comments should be brief.
- Global JavaScript variables should be encapsulated inside global objects.
- Global handler functions should be encapsulated inside global objects.